

GRIFFITH COLLEGE LIMERICK

Software Development Project

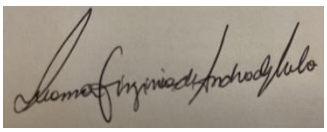
Thesis Cover Sheet

Student name:	**Luanna Virginia de Andrade Melo**
Student number:	**3103635**
Faculty:	**Engineering, Computing & IT**
Course:	**Bachelor of Science in Computing Science (Honours) Degree** **Stage/year: 4**
Subject:	**Software Development Project (SDP)**
Study Mode:	Full time **x** Part-time
Lecturer Name:	**Sonia Zheleva**
Assignment Title:	***Project Thesis***
No. of pages:	**257**
No. of words:	**62596**
Date due:	**23.05.2025**
Date submitted:	**17.05.2025**

Plagiarism disclaimer:

I understand that plagiarism is a serious offence and have read and understood the college policy on plagiarism. I also understand that I may receive a mark of zero if I have not identified and properly attributed sources which have been used, referred to, or have in any way influenced the preparation of this assignment, or if I have knowingly allowed others to plagiarise my work in this way.

I hereby certify that this assignment is my own work, based on my personal study and/or research, and that I have acknowledged all material and sources used in its preparation. I also certify that the assignment has not previously been submitted for assessment and that I have not copied in part or whole or otherwise plagiarised the work of anyone else, including other students.

Signed: 

Date: 23.05.2025



GRIFFITH COLLEGE LIMERICK

Rise Together: A Support Platform for Cancer Patients and Survivors.

A thesis submitted to Faculty of Computing, IT & Engineering, Griffith College Limerick,

in partial fulfilment of the requirements for the award of Bachelor of Science (Honors) Degree in Computing Science

by

Luanna Virginia de Andrade Melo

3103635

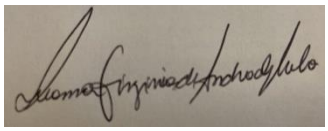
May 2025

Supervisor: **Sonia Zheleva**

External Examiner: **Robert Bordianu**

Disclaimer

I hereby certify that this material, which I now submit for assessment on the programme of study leading to the Degree of Bachelor of Science in Computing (Honours) at Griffith College Limerick, is entirely my own work and has not been submitted for assessment for an academic purpose at this or any other academic institution other than in partial fulfilment of the requirements of that stated above.

Signed:  **Date:** **23.05.2025**

Acknowledgements

First and foremost, I would like to express my deepest gratitude to my mother, Lindinalva Alves de Andrade, who, despite being miles away, has always been a pillar of love, strength, and unwavering support throughout this journey. Her presence in my life, even from afar, has been a constant source of motivation, and I carry her words and encouragement in everything I do. I am also profoundly thankful to my friends and my boyfriend, whose patience, encouragement, and companionship have been invaluable during the most challenging moments. Their belief in me has helped me keep moving forward, and I am grateful for their support. A special thanks goes to the lecturers at Griffith College for sharing their knowledge and expertise and for their commitment to my growth, both academically and personally. Their dedication has played a significant role in shaping me into the person I am today.

I would like to express my heartfelt thanks to my mentor, Sonia Zheleva, for her guidance and technical support throughout the development of this project. Her motivation and practical knowledge insights were crucial to the success of this work, and I will always be grateful for the time and effort she dedicated to helping me. I must also acknowledge my English professors, whose dedication to teaching helped me not only develop my academic skills but also enabled me to complete this thesis in a second language. Their support has been indispensable in overcoming the challenges of writing in a language that is not my own. To my doctor, who has been by my side with kind words and constant encouragement. Finally, I would like to extend my gratitude to everyone who, in one way or another, has been part of this journey. Your support, whether large or small, has made all the difference, and I will always carry it with me.

Abstract

Cancer patients go through many difficult moments during their treatment and recovery, not only physically, but also emotionally and mentally. Many feel lonely, anxious, or overwhelmed, especially when they don’t have someone close who understands what they are going through. While medicine continues to improve, emotional support is often missing. The Rise Together project was created to help change that by building a mobile app that connects patients with cancer survivors who volunteer to offer emotional support and help with simple everyday tasks like going to appointments, doing groceries, or even walking the dog.

This project was inspired by personal experience. As someone who has gone through cancer, I know how important it is to have someone by your side who truly understands. A person who has lived the same situation can bring peace, comfort, and hope when everything feels too heavy. Feeling understood can make all the difference. The app was developed using Android Studio and Google Firebase, combining cloud services such as Firestore for database storage and Firebase Authentication for secure login. The main features were based on research conducted with the app’s intended users, including cancer patients, survivors, and doctors. The goal was to build a solution that is simple, functional, and respectful of users' privacy.

Patients can be matched with mentors based on cancer type, gender, and availability. Once matched, they can send assistance requests, receive reminders, and share feedback after each interaction with the volunteer that performed the chosen task. The results from testing were very positive. People said they found the app easy to use and helpful in real life. Rise Together shows that technology can be used to bring people closer, offering not just help, but also hope, from someone who understands.

List of Figures:

Figure 1: Cancer Survivors with Distress 17
Figure 2: Investment in telehealth 19
Figure 3: LivingWith MobileApp 21
Figure 4: My Cancer Coach Ap 22
Figure 5: CareClinic App features 23
Figure 6: Screenshot of the CareZone App features' (Carezone.com) 244
Figure 7: Number of apps, available in leading app stores 2024. Statista, (Ceci, 2024). 566
Figure 8: Digital Health Apps by Category (2017), (Colgan, 2019). 577
Figure 9: Mobile Operating System Market Share Worldwide (StatCounter 2024). 588
Figure 10:Mobile Operation System Market Shared in Ireland, (StatCounter Global Stats, 2024). 588
Figure 11: Global Android vs iOS Market Share, 2024 (Buck, 2024). 6060
Figure 12: Versions of Android (Nucleus AI, 2024). 61
Figure 13: Mobile Android Version Market Share in Ireland - October 2024 (StatCounter Global Stats, 2024b) 611
Figure 14: The Becca app's main interface. (Google Play, 2024a) 622
Figure 15: The Becca app's main interface. (Google Play, 2024a) 633
Figure 16: The MySunrise app's main interface. (Google Play, 2024c) 644
Figure 17: Belong Beating Cancer Together. app's SignUp Screen. (Google Play, 2024b) 666
Figure 18: Belong Beating Cancer Together. app's main interface. (Google Play, 2024b) 666
Figure 19: Classes and Objects in Object Oriented Programming (TechTarget, 2024). 71
Figure 20: Waterfall Development Model (Itransition, 2024). 744
Figure 21: Phases of the Incremental model (Itransition, 2024) 755
Figure 22: Software Development Life Cycle in the Spiral model (Itransition, 2024). 766
Figure 23: Process model for Rapid Application Development (Itransition, 2024). 777
Figure 24: Agile development methodology (GeeksforGeeks, 2024) 788
Figure 25: Scrum Development Methodology (Itransition, 2024) 799
Figure 26: An example of a Kanban board (Itransition, 2024). 800
Figure 27: Lean Software Development (Itransition, 2024). 800
Figure 28: Planning /feedback loops in Extreme Programming (Itransition, 2024). 81
Figure 29: Steps in the DSDM cycle (Itransition, 2024). 8282
Figure 30: UML Diagram Type (Visual Paradigm, 2024) 833
Figure 31: Use Case Diagram (Visual Paradigm, 2024). 844
Figure 32: <<Extend>> relationship (Visual Paradigm, 2024). 855
Figure 33: <<Include>> relationship (Visual Paradigm, 2024). 855
Figure 34: Generalization relationship (Visual Paradigm, 2024). 855
Figure 35: The dotted vertical line represents an Object's Lifeline (JavaTpoint, 2019a). 877
Figure 36: Representation of an Actor's Activation bar, depicting the role of an entity interacting with the system (JavaTpoint, 2019a). 888
Figure 37: Representation of a Call Message (JavaTpoint, 2019a). 888
Figure 38: Representation of a Self-Call Message (JavaTpoint, 2019a). 888
Figure 39: Representation of a Recursive Self-Call Message, depicted by a looped arrow on the same Object's lifeline (JavaTpoint, 2019a). 888
Figure 40: Representation of a Create Message (JavaTpoint, 2019a). 899
Figure 41: Representation of a Destroy Message (JavaTpoint, 2019a). 899
Figure 42: Representation of a Duration Message (JavaTpoint, 2019a). 899
Figure 43: Representation of an Object in UML, showing the object's name and attributes in a rectangular box (Visual Paradigm, 2024). 911
Figure 44: UML representation of two objects, "Apple" and "Mango," as instances of the "Fruit" class, displaying their respective attributes, such as colour and quantity (Visual Paradigm, 2024). 922
Figure 45: Application's actual Splash screen, when the user launches the app 1044
Figure 46: Snippet of Code for the Application's Splash screen. 1044
Figure 47: Wireframe of Application Screen 1 - Application Home Screen 1055
Figure 48: Snippet of Code for Application Screen 1 - Application Home Screen 1055
Figure 49: Snapshot of the Actual Application Screen 1 - Application Home Screen 1066
Figure 50: Wireframe of the Application Screen 3.1 - Doctor's Login Screen 1077
Figure 51: Snippet of Code for Screen 3.1 - Doctor's Login Screen 1077
Figure 52: Snapshot of the Actual Application Screen 3.1 - Doctor's Login Screen 1088
Figure 53: Wireframe of Application Screen 1.2 -Reset Password Screen 1099
Figure 54: Snippet of Code for Screen 1.2 - Reset Password Screen 1099

Figure 55: Snapshot of the Actual Application Screen 1.2 - Reset Password Screen.....1100
Figure 56: Snapshot of the email received with Reset Password Link, sent to the email address, provided by the User at registration point. 110
Figure 57: Wireframe for the Application Screen 3.2 - Doctor Dashboard Home Screen 1122
Figure 58: Snippet of Code for Screen 3.2 - Doctor Dashboard Home Screen 1122
Figure 59: Snapshot of the Actual Application Screen 3.2 - Doctor Dashboard Home Screen.....1133
Figure 60: Wireframe of Application Screen 3.3A Register Patient in Treatment Screen. 1144
Figure 61: Snippet of Code for Application Screen 3.3A - Register Patient in Treatment Screen.....1144
Figure 62: Snapshots of the Actual Application Screen 3.3A - Register Patient in Treatment Screen - 1155
Figure 63: Snapshots of the Actual Application Screen 3.3A - Register Patient in Treatment Screen - 1155
Figure 64: Snapshots of the Actual Application Screen 3.3A - Register Patient in Treatment Screen - 1166
Figure 65: Snapshots of the Actual Application Screen 3.3A - Register Patient in Treatment Screen - 1166
Figure 66: Snapshots of the Actual Application Screen 3.3A - Register Patient in Treatment Screen - 1177
Figure 67: Wireframe of Application Screen 3.3B - Register Volunteering Cancer Survivor Screen1188
Figure 68: Snippet of Code for Application Screen 3.3B -Register Volunteering Cancer Survivor Screen 1188
Figure 69: Snapshots of the Actual Application Screen 3.3B - Register Volunteering Cancer Survivor Screen 1199
Figure 70: Snapshots of the email sent by the doctor for the newly registered volunteer to change their password.....12020
Figure 71: Wireframe of Application Screen 2.1 - Volunteering Cancer Survivor Login Screen.....1211
Figure 72: Snippet of Code for Application Screen 2.1 - Volunteering Cancer Survivor Login Screen 1211
Figure 73: Snapshots of the Actual Application Screen 2.1 - Volunteering Cancer Survivor Login Screen 1222
Figure 74: Wireframe of Application Screen 1.2 -Reset Password Screen 1233
Figure 75: Snapshot of the code for Screen 1.2 - Reset Password Screen.....1245
Figure 76: Snapshot of the Actual Application Screen 1.2 - Reset Password Screen.....1245
Figure 77: Reset Password Link, sent to the email address provided by the user on Application Screen 1255
Figure 78: Wireframe of Application Screen 2.2 - Volunteer's Home (Dashboard) Screen 1266
Figure 79: Snippet of Code for Application Screen 2.2 - Volunteer's Home (Dashboard) Screen 1267
Figure 80: Snapshot of the Actual Application Screen 2.2 - Volunteer's Home (Dashboard) Screen1277
Figure 81: Wireframe of Application Screen 2.2A - Create Volunteer's Profile Record.....1289
Figure 82: Snippet of Code for Application Screen 2.2A - Create Volunteer's Profile Record.....1299
Figure 83: Snapshots of the Actual Application Screen 2.2A - Volunteer Profile Record Creation Process.....13030
Figure 84: Wireframe of Application Window 2.2A1 - Create Profile Calendar Schedule 1322
Figure 85: Snippet of Code for Application Window 2.2A1 - Dynamic Field Generation and Data Saving Logic.....1322
Figure 86: Snapshots of the Actual Application Window 2.2A1 - Calendar Schedule Selection and Confirmation 1333
Figure 87: Wireframe of Application Screen 2.2B - View Patient's Requests for Required Assistance, showing the dropdown menu for week selection, list of requests, and navigation buttons. 1344
Figure 88: Snippet of Code for Application Screen 2.2B - View Patient's Requests for Required Assistance.....1344
Figure 89: Snapshots of the Actual Application Screen - View Patient's Requests for Required Assistance.....135
Figure 90: Wireframe of Application Window 2.2B1 - "View Selected Patient`s Request for Assistance & Send Notification of Acceptance/Rejection".....13636
Figure 91: Snippet of Code for Window 2.2B1 "View Selected Patient`s Request for Assistance & Send Notification of Acceptance/Rejection".....1366
Figure 92: Snapshot of the Actual Application Window 2.2B1 "View Selected Patient`s Request for Assistance & Send Notification of Acceptance/Rejection".....1377

Figure 93: Wireframe of Application Window 2.2B2a - “Send Notification of Acceptance of Request for Assistance”.....1388
Figure 94: Snippet of Code for Window 2.2B2a - “Send Notification of Acceptance of Request for Assistance”. 1388
Figure 95: Snapshot of the Actual Application Window 2.2B2a - “Send Notification of Acceptance of Request for Assistance”.....1399
Figure 96: Wireframe of Application Window 2.2B2b: Send Notification of Rejection of Request for Assistance.....14040
Figure 97: Snippet of Code for Window 2.2B2b: Send Notification of Rejection of Request for Assistance.....1400
Figure 98: Snapshot of the Actual Application Window 2.2B2b: Send Notification of Rejection of Request for Assistance 1411
Figure 99: Wireframe of Application Screen 2.2C “View List of Feedback and Ratings for Requested Tasks”.....1422
Figure 100: Snippet of Code for Application Screen 2.2C “View List of Feedback and Ratings for Requested Tasks” 142
Figure 101: Snapshot of the Actual Application Screen 2.2C “View List of Feedback and Ratings for Requested Tasks”. 1433
Figure 102: Wireframe of Application Window 2.2C1 "Display Feedback & Ratings Details for Assistance of Selected Task".....1444
Figure 103: Snippet of Code for Window 2.2C1 "Display Feedback & Ratings Details for Assistance of Selected Task" 1444
Figure 104: Snapshot of the Actual Application Window 2.2C1 "Display Feedback & Ratings Details for Assistance of Selected Task" 1455
Figure 105: Wireframe of the Application Screen 1.1 - Patient Login Screen 1466
Figure 106: Snippet of Code for Screen 1.1 - Patient Login Screen.....1466
Figure 107: Snapshots of the Actual Application Screen 1.1 - Patient Login Screen.....1477
Figure 108: Wireframe of Application Screen 1.2 - Reset Password Screen 1488
Figure 109: Snippet of Code for Screen 1.2 - Email Reset Link 1488
Figure 110: Snapshot of the Actual Application Screen 1.2 - Password Reset Screen.....1499
Figure 111: Snapshot of the email with the received link from screen 1.2 - Password Reset screen .1499
Figure 112: Wireframe of Application Screen 1.3: Patient in Treatment Home (Dashboard Screen) 1500
Figure 113: Snippet of Code for Application Screen 1.3 - Navigation and Session Control.....1511
Figure 114: Snapshot of the Actual Application Screen 1.3 - Patient Dashboard Interface 1511
Figure 115: Wireframe of Application Screen 1.4 - Patient Profile with Required Assistance Tasks 1522
Figure 116: Snippet of Code for Screen 1.4 - Saving Task and Schedule Preferences 1533
Figure 117: Snapshots of the Actual Application Screen 1.4 - Patient Profile with Required Assistance Tasks.....1533
Figure 118: Wireframe of Application Screen 1.5: List of Volunteers for Required Assistance Task 1544
Figure 119: Snippet of Code for Screen 1.5 1555
Figure 120: Snapshot of the Actual Application Screen 1.5 - Matched Volunteers by Task, Cancer Type, and Gender. 1555
Figure 121: Wireframe of Application Window 1.5A: “View Details of Selected Volunteer”. 1566
Figure 122: Snippet of Code for Window 1.5A: “View Details of Selected Volunteer”.....1577
Figure 123: Snapshots of the Actual Application Window 1.5A: “View Details of Selected Volunteer”. 157
Figure 124: Wireframe of Application Window 1.5B - Send Request for Assistance Task.....158
Figure 125: Snippet of Code for Window 1.5B.....159
Figure 126: Snapshot of the Actual Application Window 1.5B - Request Submission.....159
Figure 127: Wireframe of Application Screen 1.6 - List of Notifications for Requests Acceptance/Rejections.....160
Figure 128: Snippet of Code for Screen 1.6 - List of Notifications for Requests Acceptance/Rejections. 161
Figure 129: Snapshot of the Actual Application Screen 1.6 - List of Notifications for Requests Acceptance/Rejections.....161
Figure 130: Wireframe of Window 1.6A: “Display Notification of Acceptance of Requested Task Assistance.” 162
Figure 131: Snippet of Code for Window 1.6A: “Display Notification of Acceptance of Requested Task Assistance.” 163

Figure 132: Snapshot of the Actual Application Window 1.6A: “Display Notification of Acceptance of Requested Task Assistance”. 1633
Figure 133: Wireframe of Window 1.6B: Display Notification of Rejection of Requested Task Assistance 1644
Figure 134: Snippet of Code for Window 1.6B: Display Notification of Rejection of Requested Task Assistance 165
Figure 135: Snapshot of the Actual Application Window 1.6B: Display Notification of Rejection of Requested Task Assistance. 1655
Figure 136: Wireframe of Application Screen 1.7: Provide Feedback and Ratings for Obtained Assistance with a Requested Task 1666
Figure 137: Snippet of Code for Screen 1.7: Provide Feedback and Ratings for Obtained Assistance with a Requested Task 167
Figure 138: Snapshot of the Actual Application Screen 1.7: Provide Feedback and Ratings for Obtained Assistance with a Requested Task 1677
Figure 139: Wireframe of Window 1.7A: Provide Feedback and Ratings for Obtained Assistance with Selected Task 169
Figure 140: Snippet of Code for Window 1.7A: Provide Feedback and Ratings for Obtained Assistance with Selected Task 169
Figure 141: Snapshot of the Actual Application Window 1.7A: Provide Feedback and Ratings for Obtained Assistance with Selected Task 170
Figure 142: Gradle.kts file and dependencies 1722
Figure 143: Screenshot of the Rise Together connecting Application’s real-time Database for notifications. 1744
Figure 144: Screenshot of the Rise Together connecting Application’s real-time Database for notifications. -2 175
Figure 145: Table (Collection) Doctor with UNIQUE ID, encrypted for each record. 176
Figure 146: Doctors' Collection with UNIQUE encrypted IDs. 1766
Figure 147: Patient details for an individual Patient ID. 177
Figure 148: DATA Selection for a specific Volunteer ID record. 1777
Figure 149: User Tokens for Volunteer`s REQUEST collection with UNIQUE ID, encrypted for each notification record. 178
Figure 150: Volunteer Profile ID - to bring all the information and details on the screen. 1788
Figure 151: Feedback - to bring all the information and details to the screen through their ID (Patient and Volunteer) taken from the task provided. 179
Figure 152: Usernames (Emails) and IDs created and encrypted by Firebase for each User at Registration Point 179
Figure 153: Login Authentication Method in the Google Firebase 180
Figure 154: Snippet of the database storage with the pictures stored in Google Firebase 1811
Figure 155: The Two-tier Architecture of the Project’s Application 182
Figure 156: The Use of Android Studio (Ladybug 2024.2.2 Version), in building the Curren Application. 1822
Figure 157: The Use of Google’s Firebase and Cloud Firestore in building the Backend of the current Application. 1833
Figure 158: Types of Software Testing by GeeksforGeeks (2025) 1855
Figure 159: Black Box Testing by: (GeeksforGeeks, 2024a). 1877
Figure 160: White Box Testing (GeeksforGeeks, 2024b) 189
Figure 161: Grey Box Testing (GeeksforGeeks, 2024c) 1911

List of Tables:

Table 1: Project Risks, Measurement, Monitor & Mitigation (RMMM) Matrix 30
Table 2: Risks Table 31
Table 3: Worldwide Smartphone Shipment OS Market Share Forecast, 2017-2023 (ResearchGate, 2024) 59
Table 4: Fixed Errors during the Testing Process with Testing Cases 1944
Table 5: Functional Testing Process with Testing Cases 1955
Table 6: Performance Testing Process 2055
Table 7: Usability Testing Process with Testing Cases for the User Interface 2066
Table 8: Security Testing Process with Testing Cases 207

Contents

Acknowledgements iv
List of Figures: vi
List of Tables: x
Chapter 1: Introduction 16
1.1 Problem Statement 16
1.2 Background Information on the Topic 18
1.2.1 Historical Perspective: 18
1.2.3 Current Perspective: 18
1.2.4 Already Existing Solutions 20
1.2.5 Proposed Solution to the Problem 24
1.3 Project Goal 25
1.4 Project Objectives 25
1.5 Project Description 26
1.6 Project Milestones, Deliverables and Timeframe 29
Project & Milestones: 29
1.8 Project Research & Development Methodology 32
1.9 Evaluation Criteria 32
1.10 General Population Survey Questionnaire: 33
1.10.1 Analysis of General Population Survey Questionnaire Results 33
1.11 Doctors Structured Interview 38
1.12 Chapter Summary 41
Chapter 2: Review of the Literature on the Topic 43
2.1 The Impact of Cancer on Patients' Mental and Physical Well-being 43
2.1.1 The Role of Peer Support and Volunteer Programs in Cancer Recovery 45
2.1.2 The Impact of Cancer on Mental Health and the Importance of Supportive Services 46
2.1.3 Physical Challenges and Practical Assistance 49
2.1.4 Social Aspects of Well-being for Cancer Patients 50
2.2 Data Privacy Laws in Ireland 51
2.3 Privacy and Security in Firebase 52

2.4 Well-being and Quality of Life for Users 53
2.5 Evolution of Mobile Applications and Their Role in Digital Health Support . 55
2.6 Use of the Internet and Mobile Applications 56
2.7 Android Operating System 57
2.8 Android Versions and Statistics 60
2.9 Case Studies on Similar Mobile Applications 61
2.9.1 Becca - Breast Cancer Support 62
2.9.2 MySunrise 63
2.9.3 BELONG Beating Cancer Together. 65
2.10 The Proposed for Development Application 67
2.11 Chapter Summary 69
Chapter 3 Project Application Analysis Phase 70
3.1 Object Oriented Environment 70
3.1.1 Encapsulation 71
3.1.2 Inheritance 71
3.1.3 Polymorphism 72
3.1.4 Abstraction 72
3.1.5 Benefits of OOP 72
3.1.6 Criticism of OOP 73
3.2 Development Methodologies 73
3.2.1 Waterfall Model 74
3.2.2 Incremental/Iterative Methodology 75
3.2.3 Boehm's Spiral Methodology 75
3.2.4 Rapid Application Development (RAD) 76
3.2.5 Agile Methodology 77
3.2.6 Dynamic Systems Development Methodology (DSDM) 81
3.3 Object Oriented Analysis with UML Diagrams 82
3.3.1 Use Case Diagram 83
3.3.2 Sequence Diagram 87
3.3.3 Object Class Diagram 91
3.4 Alternative Design Solutions` 98
3.4.1 Low-end Solution 98
3.4.2 Mid-Range Solution 98
3.4.3 High-end Solution 99

3.5 Decisions Made for the Current Project 99

3.6 Chapter Summary 99

CHAPTER FOUR: APPLICATION DESIGN AND CODING PHASE 101

4.1 Design and Coding of Graphical User Interface (GUI) Screens 101

4.1.1 Application Screen 1: *Splash* Screen and a *Home* Screen 103

4.1.2 Doctor Interface: Application Screen 3.1 -Login Screen 106

4.1.3 Application Screen 1.2: Re-set Password Screen 108

4.1.4: Application Screen 3.1: Doctor Home (Dashboard) Screen 110

4.1.5 Application Screen 3.3A - Register Patient in Treatment Screen 113

4.1.6 Application Screen 3.3B - Register Volunteering Cancer Survivor 117

4.1.7 Application Screen 2.1 - Volunteering Cancer Survivor *Login* Screen 120

4.1.8 Application Screen 2.2 - Volunteer's Home (Dashboard) Screen 125

4.1.9 Application Screen 2.2A - Create Volunteer's Profile Record 127

4.1.10 Application Pop-up Window 2.2A1 - Create Profile Calendar Schedule for the Selected Assistance Task 130

4.1.11 Application Screen 2.2B - View Patient's Requests for Required Assistance 133

4.1.12 Application Window 2.2B1 - View Selected Patient Request & Send Notification 135

4.1.13 Application Window 2.2B2a - Send Notification of Acceptance of Request for Assistance 137

4.1.14 Application Window 2.2B2b - Send Notification of Rejection of Request for Assistance 139

4.1.15 Application Screen 2.2C - View List of Feedback and Ratings for Requested Tasks 141

4.1.16 Window 2.2C1 - Display Feedback & Ratings Details for Selected Assistance Task 143

4.1.17 Application Screen 1.1 - Patient Login Screen 145

4.1.18 Application Screen 1.2 - Reset Password Screen 147

4.1.20 Application Screen 1.3 - Patient in Treatment Home (Dashboard) Screen 149

4.1.21 Application Screen 1.4 - Create Patient Profile with Required Assistance Tasks 152

4.1.22 Application Screen 1.5 - View List of Volunteers for Required Assistance Task 154

4.1.23 Application Window 1.5A - View Details of Selected Volunteer & Place Request 155

4.1.24 Application Window 1.5B - Send Request for Selected Assistance Task 158

4.1.25 Application Screen 1.6 - View List of Notifications for Requested Tasks 160

4.1.26 Application Window 1.6A - Display Notification of Acceptance of Requested Task Assistance 162

4.1.27 Application Window 1.6B - Display Notification of Rejection of Requested Task Assistance 164

4.1.28 Application Screen 1.7 - Provide Feedback and Ratings for Obtained Assistance 166

4.1.29 Application Window 1.7A: Provide Feedback and Ratings for Obtained Assistance with Selected Task 168

4.2 Application's Database Design and Connectivity 170

Main Characteristics of Google's Firebase Cloud Firestore: 171

o Firebase Authentication 171

o Firebase Cloud Storage 171

o Firebase Cloud Functions 171
4.2.1 Implementation Path 172
4.2.3 Connectivity 173
How do the notifications work in this application. 173
4.3.1 Software Architecture 180
4.4 Chapter Summary: 183
Chapter 5: Application Testing Phase 184
5.1 Testing Methods 185
5.2 Testing Methods 186
5.2.1 Black Box Testing 186
5.2.2. Techniques of Black Box Testing: 186
5.2.2 White Box Testing 188
5.2.3 Grey Box Testing 190
5.3 Chosen Test Method for the Current Project's Application 191
5.3.1 Initial Errors Encountered During the Application Coding Process 192
5.4 Application Testing Process with Test Cases 193
5.4.1 Functional Testing 195
This employs Black box testing methodology, focusing on evaluating software functional requirements. 195
5.4.2 Performance Testing: 205
5.4.3 Usability Testing: 206
5.4.4 Security Testing: 207
5.5 Chapter summary 207
CHAPTER SIX: Application Evaluation, Results Analysis, Project Achievements, Conclusions, and Scope for Future Work 209
6.1 Application Evaluation 209
6.2 Analysis of the Evaluation Survey Results 209
6.3 Project Achievements 212
6.4 Conclusion 213
6.5 Scope for Future Work 213
References 214

APPENDIX A1: Project's *Gantt Chart* 223
APPENDIX A2: Graphical Representation of the General Population Survey Results 224
APPENDIX A3: Structured Interview Questions for Doctors. 232
Appendix A4: Project's Use Case Diagram 242
Appendix A5: Project's Sequence Diagram 243
Appendix A6: Project's Object Class Diagram 244
Appendix A7: Project's Three Alternative Design Solutions 245
Appendix A8: Graphical Representation of the Evaluation Survey Results 246
Appendix B: User Manual for Rise Together Cancer Support Mobile Application 255

Chapter 1: Introduction

1.1 Problem Statement

Cancer is a group of diseases, characterized by the uncontrolled growth and spread of abnormal cells. This abnormal growth can invade surrounding tissues and, if not controlled, may spread to other parts of the body, leading to serious health complications. Despite advances in treatment, the complexity of cancer's development and its impact on patients' lives remains a significant global health challenge, requiring ongoing research, better treatment approaches, and supportive care to improve survival rates and quality of life (American Cancer Society, 2024a).

Cancer remains a major cause of death globally, accounting for around 10 million deaths in 2020 alone (Ferlay, 2021). Advances in medical research have significantly improved treatment options and survival rates for cancer patients, but challenges persist throughout the cancer treatment journey. Patients face emotional distress, social isolation, and difficulty in accessing the necessary care. Even after treatment, many survivors continue to struggle with psychological and physical challenges, including long-term fatigue and mental health issues (Ferlay, 2021).

During active treatment, patients must endure psychological and emotional stress caused by chemotherapy, radiation, and other therapies. Administrative burdens, such as frequent hospital visits, further complicate their lives. These difficulties were exacerbated by the COVID-19 pandemic, which led to treatment schedule disruptions and reduced access to support networks, leaving many patients feeling vulnerable and alone (Doroshow, 2020).

In their publication, the American Psychological Association highlights the psychological toll that early-onset cancer takes on patients, particularly those who are younger. These include loneliness, depression, and anxiety, all of which can get worse in the absence of strong support networks. It has been demonstrated that peer support, particularly from survivors, lessens these consequences by providing hope, practical guidance, and shared experiences (Medaris, 2024).

However, ensuring effective patient-volunteer matching process poses significant challenges. Matching based on shared experiences improves the efficacy of the relationship, but logistical issues and privacy concerns add complexity to these programs (Meyer, Coroiu and Körner, 2014).

Fatigue remains a prevalent and distressing side effect of cancer treatment, often cited as the most burdensome symptom. Research has shown that over 70% of patients experience fatigue, with many abandoning their treatment plans due to its impact. The physical and emotional toll of fatigue, especially when combined with other stressors, can lead to a significant reduction in quality of life (Brennan and Moynihan, 2005). Addressing these challenges requires comprehensive solutions that include psychosocial support alongside medical care (Brennan and Moynihan, 2005).

The integration of Artificial Intelligence (AI) into healthcare has the potential to revolutionize medical services by improving patient outcomes, enhancing clinical decision-making, and automating administrative tasks. AI can predict patient health trajectories, guide treatments, and optimize hospital operations, making healthcare more efficient. Despite its potential,

significant barriers remain that hinder the widespread adoption of AI technologies. These challenges include data privacy concerns, biases in AI models due to non-representative datasets, and difficulties in scaling AI solutions across diverse healthcare settings (Howard, 2020).

Additionally, while many healthcare leaders recognize the benefits of AI, there is uncertainty regarding its ethical use and the liability associated with AI-driven decisions, which can further slow down implementation. Addressing these issues requires improving data access, establishing best practices, and ensuring interdisciplinary collaboration to develop AI tools that can be safely and effectively integrated into healthcare systems (Willige, 2021).

In a National Health Interview Survey, (2022) participants were asked to rate how often they had experienced certain emotions over the previous 30 days, such as feeling hopeless, like everything was an effort, feeling worthless, nervous, restless, or fidgety, or feeling so depressed that nothing could make them feel better. The response choices were given numerical values ranging from 0 to 4, with the range being "none of the time" to "all of the time." The separate item scores were added to provide a total symptom score, which may range from 0 to 24. According to established criteria used in prior studies, participants scoring 13 or higher were considered to be experiencing clinically significant psychological distress. Notably, cancer survivors who reported more severe psychological distress were more likely to indicate that they found mental health services unaffordable, compared to those who experienced less distress. Specifically, 15.15% of those with higher distress levels reported challenges affording mental health services, compared to only 5.63% of those with less distress (OR: 0.90; 95% CI: 0.85-0.97, $p < 0.01$). The survey asked respondents whether they had ever delayed, skipped, or altered mental health services due to cost. The response options were simple "yes" or "no." This self-reported data was collected, using the Kessler 6 questionnaire, which measures psychological distress across several dimensions (Kranenburg and Shen, 2023). For more details, please see Figure 1, which visualizes the percentage of cancer survivors, facing difficulties in affording mental health services based on their level of psychological distress (National Libraty of Medicine, 2023).

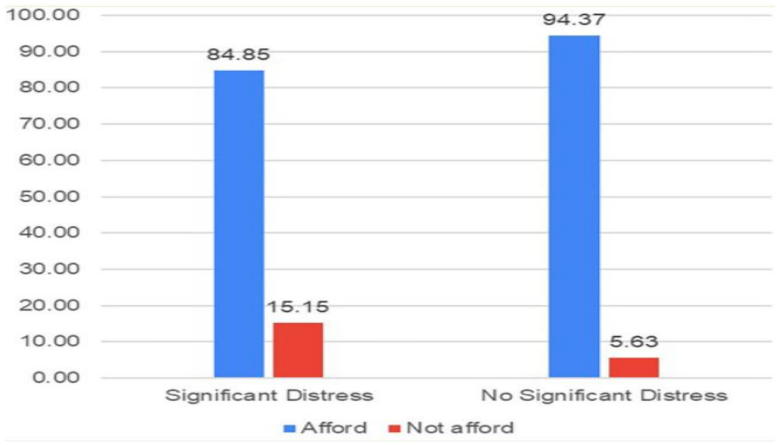


Figure 1: Cancer Survivors with Distress

Cancer survivors with severe psychological distress reported more difficulty affording mental health services than those with less distress. (National Health Interview Survey, 2022) (National Library of Medicine, 2023).

1.2 Background Information on the Topic

1.2.1 Historical Perspective:

Cancer has long been a significant health challenge, with mortality rates consistently placing it among the top causes of death globally. The development of treatments such as chemotherapy, radiation, and surgery has improved outcomes over the past several decades, yet the experience of undergoing these treatments remains physically and emotionally taxing for patients. Historically, cancer patients had little access to peer support, relying solely on medical professionals for guidance. However, the rise of volunteer and peer mentorship programs in recent years has provided cancer patients with additional resources for coping with their illness. Throughout history, social support has played a critical role in recovery from major illnesses, including cancer. Ancient healing practices often incorporated both medical treatment and community-based support, recognizing the holistic needs of the patient. Modern peer support programs, such as those led by the American Cancer Society's "*Reach to Recovery*" initiative, have roots in these traditional approaches, emphasizing the emotional and practical support, that survivors can offer to those currently undergoing treatment (American Cancer Society, 2024b).

The challenges of maintaining volunteer support programs, particularly in terms of managing the logistical complexities of patient-volunteer matching, have persisted. As far back as 2007, studies highlighted the difficulties in creating effective peer support networks that could be tailored to individual patient needs (Meyer, Coroiu and Körner, 2014). Programs like Cancer Hope Network and Imerman Angels have developed sophisticated systems for matching patients with survivors based on their specific cancer experiences, but issues such as privacy concerns and emotional readiness of volunteers remain unresolved (Van Puffelen et al., 2023).

1.2.3 Current Perspective:

In the context of the COVID-19 pandemic, cancer patients faced unprecedented challenges, including delays in diagnosis and treatment, restricted access to healthcare, and heightened anxiety. These disruptions resulted in worsened treatment outcomes for many patients, as they were forced to manage their health in isolation, without the usual support systems (van Puffelen, *et al*., 2023). Even post-pandemic, the backlog in treatment schedules continues to affect patients, and the long-term psychological impact of the pandemic on cancer survivors is only beginning to be fully understood (Afshar, *et al*., 2021).

Peer support has proven to be particularly valuable during and after the pandemic, as cancer patients benefit from connecting with survivors who can share their experiences. Survivorship care plans, which include emotional and psychological support alongside medical follow-up, are now considered essential to long-term recovery. However, finding suitable volunteer matches remains a significant hurdle. Studies have shown that volunteers, who have experienced similar cancer types or stages of treatment, are more effective in providing support, but matching patients and volunteers, based on these criteria, is challenging. Finding suitable volunteer matches remains a significant hurdle. The process of aligning patients with volunteers, who have experienced similar cancer types or stages of treatment, is essential for the success of peer support programs. Studies have consistently shown that volunteers, who share comparable cancer experiences, are more effective in providing meaningful and empathetic support. Patients feel a deeper connection and are more likely to trust advice and emotional guidance from those, who have walked a similar path. Despite the proven benefits,

matching patients and volunteers, based on these specific criteria, remains challenging (Kiemen, Czornik and Weis, 2023).

Fatigue, one of the most debilitating symptoms of cancer treatment, has been widely reported by patients as a leading cause of treatment abandonment. Despite its prevalence, healthcare providers often overlook this issue in their conversations with patients. Research indicates that radiotherapy-induced fatigue can persist for months, or even years after treatment, making it a chronic condition for many survivors (Brennan and Moynihan, 2005). The impact of fatigue extends beyond physical symptoms, contributing to mental health challenges, such as depression and anxiety, and complicating the recovery process (Fitch, *et al*., 2022).

Efforts to address these challenges have resulted in the development of comprehensive support programs, that integrate emotional care with medical treatment. However, logistical, privacy, and volunteer readiness issues continue to impede the full effectiveness of these initiatives. Ongoing research into peer support and survivorship care is essential to improving the quality of life for cancer patients, both during and after treatment (American Cancer Society, 2024b).

Significant developments are still influencing cancer treatment in 2024, with an emphasis on improving patient outcomes and care. One significant invention, that has improved cancer detection, diagnosis, and therapy, is the application of artificial intelligence (AI). One of the most important problems in cancer treatment is early and accurate detection, which is being addressed by the growing use of AI-based technologies to enable early diagnosis and give patients more individualised care. It is anticipated that this technology would transform the way physicians track treatment outcomes and customise treatments for each patient, increasing survival rates and quality of life (American Association for Cancer Research, 2024).

By 2023, according to IDC, almost two-thirds of people are anticipated to receive healthcare via digital platforms, as the COVID-19 pandemic accelerated the adoption of digital health solutions and shifted healthcare from in-person consultations to telemedicine (Miliard, 2020). Increasing healthcare resilience and crisis preparedness have been ranked as top goals by senior healthcare officials from 14 nations, including the US, Germany, and India. To improve patient care and close access gaps, particularly in underprivileged areas, telemedicine and digital healthcare delivery via apps, phone calls, and video consultations are currently considered crucial. As a result, 64% of healthcare executives are already making significant investments in digital health technology; but, when asked how much they plan to spend in three years, that percentage reduces to 40%. This could be because of respondents' expectations that strong foundations will have been established by that point, or ongoing uncertainty regarding healthcare funding after the pandemic (please, see Figure 2) (Willige, 2021).



Figure 2: Investment in telehealth

People may expect telehealth foundations to have been laid three years from now, which would require less spending (Royal Phillips, 2021).

1.2.4 Already Existing Solutions

The project's planned application aims to fill important gaps in cancer patients' emotional support, community involvement, and overall well-being. Clinical treatment and physical health are frequently the first things that come to mind when we think about cancer care. True well-being, on the other hand, includes social, mental, and emotional support all of which are critical to enhancing the quality of life for cancer patients. A robust support system and emotional help are essential for assisting patients and carers in overcoming the difficulties associated with having cancer.

To better understand the gaps in existing cancer care solutions, a review of similar applications, including *LivingWith*, *My Cancer Coach*, *CareClinic*, and *CareZone*, was conducted and analysed. These applications highlight the need for a more comprehensive approach to cancer support, however, there are few gaps still existing in terms of helping with physical tasks, integrating health tracking with emotional support, peer connection, and community-driven collaboration. None of the reviewed applications fully address the combination of emotional, practical, and social needs that the proposed for this project application aims to meet.

Case Studies on Similar, Already Existing Applications:

- *LivingWith App* is developed by Pfizer Oncology and assists cancer patients with managing their treatment and connecting with their support networks (please, see Figure 1.3). However, there is no help with everyday domestic tasks and emotional support. It provides tools, such as symptom tracking, health data sharing with caregivers, and task management. Despite its focus on practical aspects, the app lacks features such as peer-to-peer support groups and gamified wellness activities, which could foster emotional engagement and provide a stronger sense of community.

Included Features:

- Patients can record their daily symptoms and mood swings with the use of symptom and mood tracking.
- Health data sharing: Patients have the option to offer their carers or medical team access to their health information.
- Task management: People can ask family members for assistance with routine duties like driving or food shopping.
- Community and event sharing: Users can join events pertaining to cancer care and establish connections with friends and family using the app.

Not included Features:

- No volunteer network: Outside of the user's personal contacts, the app does not have a network of volunteers that can offer support.
- Lack of peer support groups: Although patients can communicate with friends and family, there isn't a specific peer-to-peer support tool that allows cancer patients to talk to others going through comparable therapies about their experiences.

- No gamified wellness challenges: Although the app emphasises task management, it lacks features that would gamify wellness or recovery activities to promote participation.

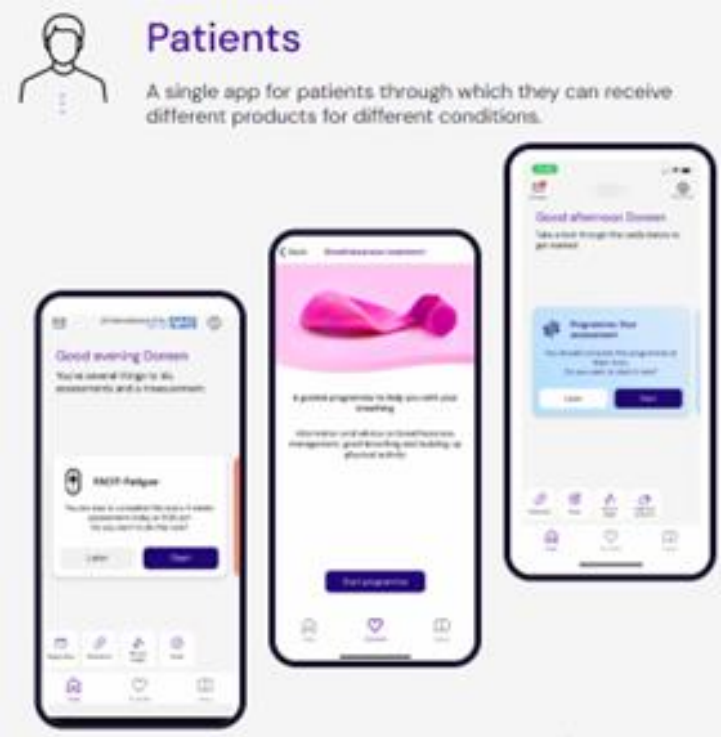


Figure 3: LivingWith MobileApp

- *My Cancer Coach App* is created by Genomic Health and focuses on educating patients about their diagnosis and treatment options, such as personalized treatment plans, symptom tracking, and reminders for medications (please, see Figure 4). While it provides valuable tools for managing physical health, it does not emphasize emotional support or peer-based connections, leaving a significant gap in holistic patient care.

Included Features:

- Personalized treatment plans, based on cancer type.
- Access to educational videos and articles, tailored to a patient's diagnosis.
- Symptom tracking and reminders for medications.
- Tools to help patients prepare for doctor appointments.

Not Included Features:

- No community or peer support features.
- Does not offer volunteer matching for assistance with daily tasks or emotional support.

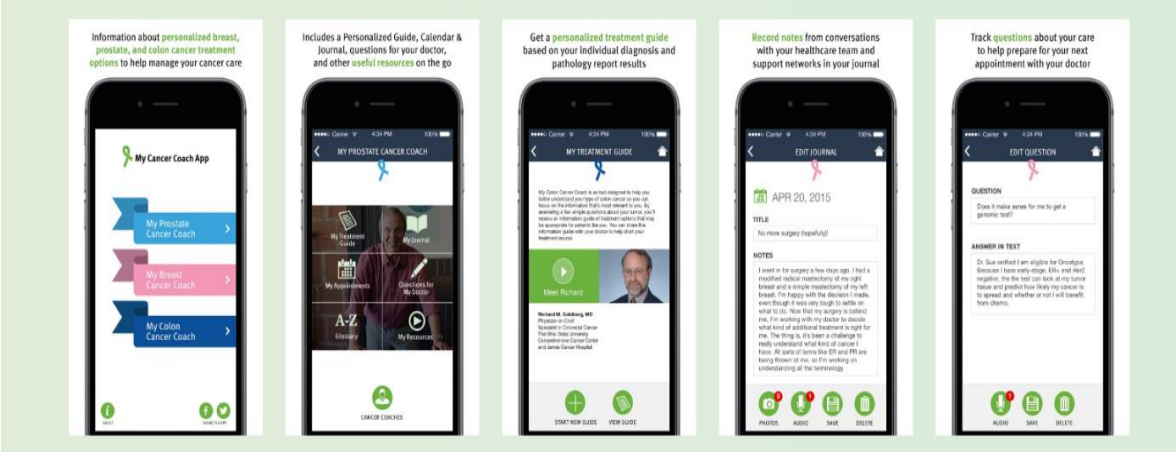


Figure 4: My Cancer Coach Ap

- Lacks features for emotional and mental health tracking, focusing mainly on the physical aspects of cancer management.

➢ *CareClinic App* is a more complete health management tool, designed to help users keep an eye on their medicines, symptoms, and overall health (please, see Figure 5). It contains functions for daily health logs, safe data exchange, and medication management. However, it mostly focusses on monitoring physical health and offers limited resources for emotional support and peer interactions, two crucial components.

- Symptom and medication tracking.
- Daily health logs and wellness reports.
- Secure storage for health data.
- Care team sharing, allowing patients to communicate their health data directly with healthcare providers.

Not Included Features:

- No emotional support features, or peer support groups.
- Limited focus on community-driven engagement and volunteer involvement.
- No direct interaction between patients for experience-sharing, or group support.

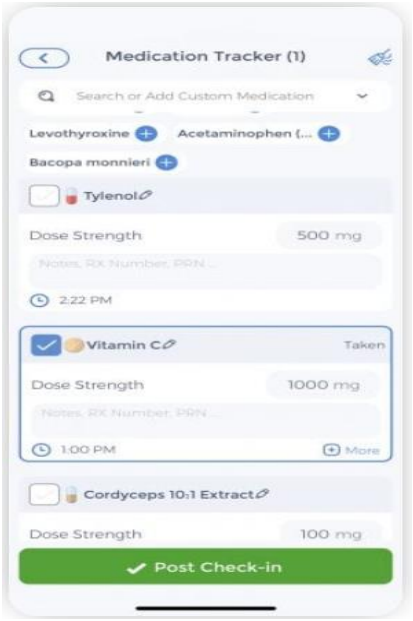


Figure 5: CareClinic App features

- *CareZone App* - with a focus on medication organisation, symptom tracking, and sharing health information with family members and carers, the application is a complete health management software (please, see Figure 6). Features, like journal entries to track symptoms, prescription reminders, and safe storage of medical records, are all included in the app. Additionally, *CareZone* makes it simple to manage complicated medication schedules, by enabling users to scan prescriptions straight into the program.

  However, despite the app's practical management benefits, its lack of peer support groups, community-driven cooperation, and mental health tracking features, leaves patients without means to address their emotional and social needs.

- Reminders and tracking for medications.
- Tracking symptoms, using safe notes.
- Health records securely stored.
- Collaboration among carers, granting family members access to medical records.

Not Included Features:

- Peer support, or emotional care features.
- No volunteer matching, or community-driven collaboration.
- Limited focus on emotional well-being, or connecting patients with others, facing similar health challenges.

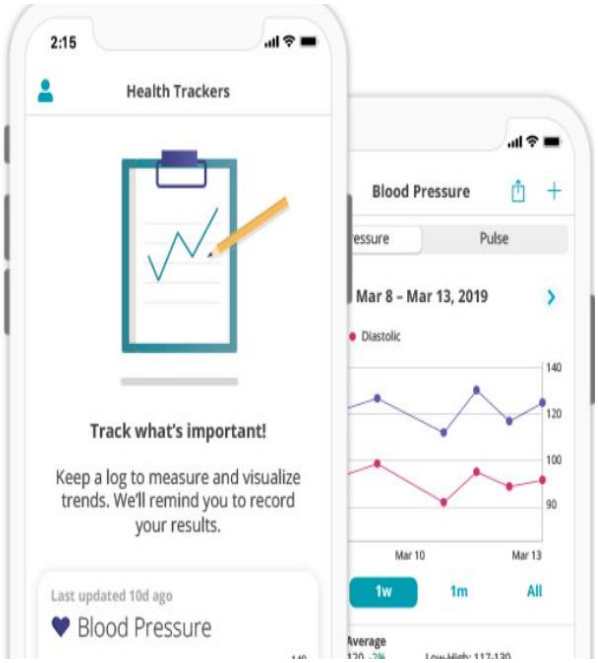


Figure 6: Screenshot of the CareZone App features' (Carezone.com)

The disadvantages of the currently existing on the market applications highlight the need for a platform, which would combine practical assistance, emotional support, and community interaction into a single, all-inclusive solution.

1.2.5 Proposed Solution to the Problem

Based on the analysis of the research conducted, the proposed for development *Rise Together* mobile application in the current Project, intends to address all the issues that have been brought to light from the experiences of cancer patients, survivors, and carers. To help with enhancing the quality of life for cancer patients and with creation of supportive community for them, the app will serve as a safe and caring platform that blends practical help, emotional support, and wellness activities. Through the *Rise Together* app, patients will be able to connect with carefully screened cancer survivors as mentors, providing both emotional support and useful assistance with everyday domestic activities, like meal preparation, transportation, gardening. By sharing their personal stories and offering sympathetic company throughout treatment and recuperation, these mentors can awaken optimism. The platform will be supervised by medical experts who will make sure that all interactions are suitable, safe, and advantageous to both mentors and patients.

"*Cancer care requires a shift in focus from clinical treatment alone to the inclusion of emotional and practical support, empowering patients to live with dignity, hope, and resilience*." (American Cancer Society, 2024b).

The current Project aims, through comprehensive research and development, to create a suitable platform that addresses the emotional and logistical burdens faced by cancer patients. The Rise Together app aspires to enhance the well-being of patients, alleviate their isolation, and enable cancer survivors to give back to their community by offering meaningful support. The Rise Together application will integrate wellness tools such as guided meditation, sound therapy, and light exercises to help patients manage their emotional distress, reduce stress, and

improve mental clarity. Additionally, private chat groups will create safe spaces for patients and survivors to connect and share their journeys, fostering a strong sense of community.

1.3 Project Goal

The goal of the proposed Project is to create a secure, compassionate digital platform, where cancer patients can connect with survivors, who have experienced similar journeys. This platform would allow patients to receive not only emotional support, but also practical assistance from volunteer mentors. These volunteers, who are carefully screened through their oncologists, can help with daily physical tasks, such as transportation, meal preparation, or even assistance with household chores, ensuring patients have the practical support they need during treatment and recovery.

In addition to the physical support, patients can engage in meaningful conversations and receive advice on how to navigate the complexities of cancer treatment. The proposed for development *Rise Together* mobile application will also prioritize user privacy and data security, ensuring that all personal and medical information is protected, giving users peace of mind as they engage with the platform. By offering a space, where cancer patients can receive both practical help and emotional guidance by cancer survivor volunteers, *Rise Together* application aims to improve the overall well-being of its users. The Project would foster a cyclical model of support, allowing survivors to give back by mentoring others, thus creating a continuous loop of assistance and encouragement. In essence, the Project seeks to help empower cancer patients, reduce their sense of isolation, and provide comprehensive support, both practical and emotional, through a secure and well-organized digital platform.

Project Scope:

The *Rise Together* application would be specifically designed to support cancer patients by cancer survivors, with a focus on fostering strong community of peer mentorship and volunteer assistance. The system will be developed as an *Android* mobile application, catering initially to cancer patients in Ireland, with potential for broader adaptation. The platform will provide both practical and emotional support, connecting patients with vetted cancer survivor volunteers, who can assist with tasks, like transportation, daily errands, and home care, while also offering emotional guidance and support through shared experiences.

The scope of the Project includes the creation of a functional prototype for *Android* OS that ensures user privacy and security. It will be integrated with local doctors, allowing them to verify and connect patients with the appropriate volunteers. The prototype will be thoroughly tested to ensure it meets the privacy, security, and usability standards, required for handling sensitive patient information.

This Project is limited to patients, survivors, and doctors in Ireland, but the long-term vision is to expand its use to other regions, and potentially other Operating Systems.

1.4 Project Objectives

The *Rise Together* Project aims to develop a platform, focused on mental health support and mentorship for cancer patients by cancer survivors, and its objectives are as follows:

- Conduct research on the mental health challenges, faced by cancer patients and survivors, and the benefits of peer support in their emotional well-being.
- Research Case studies on existing platforms, that connect patients with vetted volunteers for functionality comparison.
- Complete the Project Proposal.
- Conduct *Primary* research on cancer patient support networks, using a Mixed-Research method:
    - *Quantitative* research: Design and distribute a *Survey Questionnaire* to gather information from cancer patients and survivors on the effectiveness of peer mentorship in improving their mental health.
    - *Qualitative* research: Conduct Structured Interview with healthcare professionals to gather insights on the impact of mental health support from doctors and volunteering mentors.
- Analyse Survey and Interview results to evaluate Project's feasibility and the functional requirements of the application. Complete Chapter 1 of the Thesis.
- Perform a comprehensive Literature review, as Secondary research, including Case Studies on similar, existing applications. Complete Chapter 2 of the Thesis.
- Structure the System requirements in a Conceptual model for the application, focusing on secure communication between patients and doctors. Complete Chapter 3.
- Design and Code the User Interface screens and Data structures and ensure the implementation of security measures to protect sensitive health information. Complete Chapter 4.
- Research on Testing methods and select appropriate one to perform thorough Testing on the application prototype. Complete Chapter 5.
- Conduct User Evaluation, analyse the feedback, and apply necessary improvements. Provide Conclusions and Scope for Future Work and Complete Chapter 6.
- Prepare Project Presentation slides.
- Submit the Finalised Thesis, present the Project, and Demonstrate the developed Application

1.5 Project Description

Cancer patients often face emotional and physical challenges during treatment, many of which can lead to feelings of isolation and helplessness. The *Rise Together* app is designed to address these challenges by providing a secure and trusted platform where cancer patients can connect with carefully vetted volunteers. These volunteers, most of whom are cancer survivors, offer both practical help such as transportation to medical appointments and assistance with daily tasks and emotional support.

The central role of doctors in this process ensures that all interactions are safe and beneficial. Doctors serve as the gatekeepers, registering patients and selecting appropriate volunteers after a thorough vetting process. This guarantees that the volunteers are trustworthy and able to provide the right kind of assistance, whether it be helping with physical tasks or offering emotional companionship. The involvement of doctors also adds an extra layer of security, as all volunteer activities are closely monitored. Once logged into the app, volunteers can create a profile and select the types of services they can provide, such as transportation, meal preparation, or companionship. They can also update their availability, which is then automatically shared with patients who are seeking support. After each interaction, patients are

prompted to provide feedback and rate their experience, helping to ensure continuous improvement in the quality of support provided.

Rise Together also would include mental health support, allowing patients to join private chat groups where they can discuss their emotional well-being with others, facing similar challenges. In addition, the app would offer access to mindfulness resources, such as meditation exercises and audiotherapy sessions, further aiding in the recovery process. The platform would not only serve patients, but will also give survivors the opportunity to give back to the community, creating a continuous cycle of support. By engaging in volunteer work, survivors can find new meaning in their recovery journey, which contributes to their own well-being as well.

Key Features:

- User-friendly Interface: The app provides an intuitive, easy-to-navigate design for all users—patients, doctors, and volunteers.
- Simple Navigation: Every user, whether a patient, doctor, or volunteer, can quickly find the features they need to access.
- Scalability and Reliability: The application is built to support multiple users with real-time updates on patient needs and volunteer availability.
- Data Security and Privacy: All sensitive personal and medical data is encrypted to ensure compliance with privacy guaranteeing that all information is handled securely.

Functionalities:

***Registration and Profile Creation*:**

- For Doctors: Doctors are registered offline by the application administrator, to ensure that only legitimate healthcare professionals are part of the system. After secure login, they can access patient data and emergency alerts.

- For Patients: Doctors register their Cancer Patients on the platform, providing them with initial password. After login process of First-time Patient users, they can re-set their password and create own profile, including relevant medical information (e.g., treatment history, allergies) and personal preferences, allowing easily access to the support they need.

- For Volunteering Survivors: Doctors register their vetted Cancer Survivors, willing to volunteer as Mentors on the platform. After successful Login process of First-time Volunteer users, they can re-set their password, and create own profile, indicating the types of tasks they would like to provide assistance with (e.g., transportation, meal preparation, house & garden chores) and their availability for these tasks.

Doctor Interface:

- Doctors securely log in, after being verified by app administrator, ensuring only authorized professionals can access the platform.
- Doctors approve and monitor volunteers, ensuring they are appropriate for supporting patients, based on medical conditions.
- Doctors can view and oversee the tasks volunteers choose to assist with, ensuring the services provided are suitable for patients' health conditions.
- Important health-related updates or reminders can be sent by Doctors to both patients and volunteers.

Volunteer Interface:

- Volunteers create profiles, indicating their availability and the services they offer, such as transportation, companionship, etc.
- Volunteers can view Patients' Requests for assistance tasks and respond with *Acceptance* or *Rejection* of the request, based on their availability.
- A dedicated screen allows Volunteers to stay organized with *Accepted* upcoming tasks for which they receive Reminders.

- After completing the tasks, Volunteers can view Feedback & Rating, provided by the requesting Patient.

Patient Interface:

- Patients securely log in and create own profile with medical details and preferences for support.
- Patients can browse Volunteer profiles and choose those, best suited for their needs (e.g., transportation, home chores, etc. tasks with Volunteer's time availability).
- Request for Assistance task(s), such as grocery shopping or pet care, can be sent by the Patient to the selected Volunteer.
- Once the Volunteer view the Request, the Patient receives a Notification of Acceptance or Rejection of their Request.
- After each assistance task provided by the Volunteer, the Patient can Rate the Volunteer and provide a Feedback.

1.6 Project Milestones, Deliverables and Timeframe

Project & Milestones:

The Project milestones will include the following:

- Research potential topics and gather ideas for suitable Project.
- Conduct initial Literature review for the Project Proposal draft.
- Revise the Proposal, based on Supervisor's feedback.
- Identify samples of suitable participants for the Surveys and Structured Interviews.
- Develop and distribute the Survey questionnaire and Structured Interview questions.
- Analyse Survey and Interview results and complete Thesis Chapter 1.
- Conduct a comprehensive Literature review to complete Thesis Chapter 2.
- Perform System Analysis and structure system requirements in a Conceptual model; complete Thesis Chapter 3.
- Submit the *First draft* of the Thesis (Chapters 1, 2, and 3).
- Design and Code the application, ensuring compatibility with relevant devices; complete Thesis Chapter 4.
- Choose appropriate Testing methods and conduct comprehensive application Testing; complete Thesis Chapter 5.
- Conduct User Evaluation, analyse results, reflect on project outcomes, provide Conclusions and Scope for Future work; complete Thesis Chapter 6.
- Submit the *Second draft* of the Thesis (Chapters 4, 5, and 6).
- Prepare PowerPoint Presentation slides.
- Submit the finalised Thesis, present the Project, and demonstrate the developed Application.

***Project Deliverables*:**

The project deliverables will include:

- Project Proposal draft.
- Completed Project Proposal.
- Thesis First draft.
- Thesis Second draft.
- The Completed Thesis document.
- Application Source Code.
- Project Presentation slides.

***Project Timeframe*:**

Table 1: Project Risks, Measurement, Monitor & Mitigation (RMMM) Matrix

#	Milestones	Deliverables	Duration
1	Brainstorm to find an appropriate project topic;	Project topic chosen along with a suitable research approach;	1 week (weeks of 24.09.25 and 1.10.25)
2	Conduct Literature Review on the chosen topic for Project Proposal;	Draft of the Project Proposal completed;	1 week (weeks of 8.10 and 15.10.25)
3	Revise the Proposal based on Supervisor's feedback;	Finalized Project Proposal;	3 days week (from 22.10 till 25.10.24)
4	Select target audience samples and prepare Survey and Structured Interview questions	Survey and Interview questions ready for distribution	1week (26.10-2.11.24)
5	Send out Survey and Structured Interview questions to the target groups	Analysis of Survey and Interview responses completed; Chapter 1 finalized	2.5 weeks (3.11.-18.11.24)
6	Perform comprehensive Literature Review and write Thesis Chapter 2	Chapter 2 of the Thesis completed	2 weeks (overlapping Task 5) (12.11.-25.11.24)
7	Carry out System Analysis, build a Conceptual Model, and write Thesis Chapter 3; submit first draft of Thesis	Chapter 3 completed; First Draft of Thesis submitted	3 weeks (26.11.-17.12.24!)
8	Learn Android Studio & Google Firebase; Improve coding skills	Achieve proficiency in coding screens and functionalities	20 weeks (overlapping Tasks 1-7) (and including

			the weeks of 24.12 and 31.12.2024, 7.01.25, 14.01.25, 21.01.and 28.01.2025!)
9	Design and code interface screens, database structures, and required functionalities; Write Thesis Chapter 4	Design and coding phase completed, Chapter 4 finalized	8 weeks (starting Semester 2 in Week of 4.02.25-till week of 25.03.25)
10	Select appropriate testing methods and conduct testing on the application	Application fully tested; Chapter 5 of the Thesis completed	1 week (week of 1.04. and 8.04.25)
11	Conduct User Evaluation, analyze feedback, assess project success, provide conclusion and future work scope in Chapter 6	Chapter 6 completed	1 week (week of 15.04 and 22.04.25)
12	Submit second version of the Thesis	Chapters 4, 5, and 6 submitted	1 week (week of 29.04.25)
13	Create PowerPoint Presentation for the Project	Slides for the presentation completed	3 days (6.05.-8.05.2025)
14	Submit the final Thesis and deliver the Project Presentation	Thesis submitted and Project presented	1 day

Table 2: Risks Table

No.	Risk Description	Impact	Action
1	Time to Learn Android Development application programming languages between Compose or XML	Critical	Allocate time early to learn Android development and use online tutorials to build necessary skills.
2	Managing Project Deadlines	Critical	Plan tasks in detail from the start, use time management tools, and stick to a schedule to ensure the project is completed on time.
3	Data Loss	Critical	Regularly back up all data on Cloud storage and external devices to prevent loss in case of a system failure.

The Gantt Chart of the Project is presented in.

1.8 Project Research & Development Methodology

The *Rise Together* Project will employ a *Mixed Research* Methodology, integrating both *Qualitative* and *Quantitative* techniques to gain a comprehensive understanding of the challenges, faced by cancer patients and survivors in accessing support services

Quantitative Research:
This approach uses discernible patterns and connections to present the research challenge. It entails formulating of precise, quantifiable, and testable purpose statements or hypotheses. Numerical data from a sizable sample of study participants is obtained and analysed to identify patterns and trends (Creswell, 2012). For the proposed Project, a Survey Questionnaire will be designed and distributed to cancer patients, carers, and survivors to gather their experiences and opinions on the proposed for development application and identify patterns and connections, pertinent to the features required in the *Rise Together* application, which data will be objectively analysed.

Qualitative Research:
Through focus groups and interviews, the Qualitative research delves deeper into the experiences of the study participants. A more focused and smaller group of study participants provides data, with an emphasis on their individual experiences. Thematic analysis of this data is conducted, considering the viewpoints of the participants, as well as the researcher's personal reflexivity (Creswell, 2012). Using the *Qualitative* strategy with *Structured Interviews* will make it possible to fully comprehend the social and emotional demands of the study participants for the proposed Project, which will influence the features and functionalities of the app for the different user types.

Combined Research Methods
The goal of the *Rise Together* Project is to develop a comprehensive understanding of the demands and difficulties, experienced by cancer patients and survivors, integrating both *quantitative* and *qualitative* data collection techniques. To improve the application's efficacy and relevance, Survey with cancer patients and survivors, and Structured Interviews healthcare professionals, will be conducted to acquire their perspectives.

A *Survey Questionnaire* will be designed and distributed to a sample of Limerick-based cancer patients and their caregivers and some vetted cancer survivors. Additionally, *Structured Interview* questions will be designed and sent to local medical professionals. Google Forms tool will be used to gather participants' insights and feedback on the proposed for development application.

1.9 Evaluation Criteria

The developed such an application will be made available on the Cloud, and an online *Evaluation Survey* will be conducted to gather feedback from target users, including cancer patients, survivors, and healthcare professionals.

The *Evaluation Survey* will focus on the following key factors:

- Accessibility → Evaluates the application's availability, compatibility with *Android* devices, and the ability to maintain access in the future.
- Navigation → Evaluates the ease and simplicity of reaching specific content, through use of menus, to efficiently move through the application
- Content → Assesses whether the information presented meets user expectations and aligns with the application's intent across different topics.
- Design → Analyses the Graphical User Interface (GUI) of the application, considering the effectiveness of color schemes, font choices, sizes, and overall content readability.
- Security → Reviews the measures in place to protect personal information against threats and breaches, ensuring robust security protocols.
- Feedback Collection → The app allows users to provide feedback about the support received from voluntaries.

1.10 General Population Survey Questionnaire:

A Survey Questionnaire was designed and distributed to cancer patients currently in treatment, to cancer survivors, volunteering as mentors, and to individuals interested in emotional and practical support systems in November'2024. The purpose of the Survey was to collect information on the types of assistance needed by cancer patients, the interest in receiving mentorship from survivors, wellness activity preferences, and the use of mobile applications for such support.

To obtain important information about the unique requirements of cancer patients, the reasons behind the willingness of cancer survivors to mentor others, and the general interest in utilising a mobile application, created specifically for this purpose, the *Survey* included **22** questions. To ensure the app's effectiveness and usability, it also sought to find out what features and functionalities consumers would find desirable. Based on the Survey's results, a user-centred solution will be developed.

1.10.1 Analysis of General Population Survey Questionnaire Results

There were **21** respondents to the Survey in total (please, see *Appendix A2*), where 35% were female and 65% male (Question 11) (Questions to be answered by all Survey Participants).

To **Q1**:"*Are you currently a cancer survivor, a cancer patient currently undergoing a treatment, or neither?*", there were 21 responses in total. 61.9% of the respondents were currently *in cancer treatment*, 33.3% were a *Cancer Survivor*, and 4.8% (accounting for one person) was *neither*, therefore was asked to leave the Survey (such group was not included in the research). The fact that most respondents were undergoing treatment, emphasises how pertinent the survey is to comprehend their needs in both practical and emotional support.

Q2 to **Q6** were targeting specifically *patients, currently in cancer treatment*.

To **Q2**:"*What type of assistance and support do you find most helpful in your daily life with cancer? (Please, select **all** that applies.*)", there were 13 responses in total. *Emotional support* was the most selected option, with 92.3% of the respondents choosing it. This

was followed by *Help with household tasks* (69.2%), *Transportation for scheduled appointments* (46.2%), *Help with pet care* (38.5%), and *Help with kids* (30.8%). These results underscore the importance of providing both, emotional, as well as domestic & logistic assistance to cancer patients during their course of treatment.

To **Q3**:"*How would you feel about receiving support from someone, who has gone through a similar cancer experience and would volunteer to assist others?*", 13 responses were obtained. A significant 61.5% of respondents reported feeling *Very comfortable*, and the remaining 38.5% answered the question selecting option *Comfortable*. **None** of the respondents (0%) has selected option *Not comfortable at all* as an answer, which results demonstrated strong openness and receptivity to peer-based support networks.

To **Q4**:"*How often would you prefer a volunteer to check on your well-being*?", there were 13 responses received. 38.5% of the respondents preferred *daily* checks, 30.8% preferred *twice a week*, and 23.1% preferred *once a week*. Only 7.7% preferred to be contacted *only when they needed*. These results indicate a preference for frequent and consistent interaction with volunteers to support their well-being.

To **Q5**:"*Which wellness activities are most important to you? (Please, select **all** that applies)*", obtained were 13 responses in total. *Guided meditation* was the top choice, with 92.3% selecting it, followed by *Sound therapy* (53.8%), *Deep breathing* exercises (46.2%), and *Yoga, Pilates, Tai Chi, or light exercises* (23.1%). These results reflect a strong interest in activities, that promote mental and emotional relaxation. *Guided meditation* was overwhelmingly the top choice among respondents, which reflects the widespread recognition of meditation as a tool to manage stress, anxiety, and mental fatigue, as common challenges for cancer patients. *Sound therapy*, ranked second, uses soothing sounds, or music and frequencies, such as (432 Hz - Deep Healing Music for The Body & Soul - DNA Repair, Relaxation Music, Meditation Music) to create a calming environment, and is particularly effective for reducing stress and improving emotional well-being. Its popularity suggests that respondents are drawn to alternative or non-invasive methods, to manage their emotional health.

To **Q6**:"*How important to you is having an emotional support*?", there were 13 responses received. A vast majority (92.3%) rated emotional support as *Very important*, while 7.7% rated it as *Important*. **No** respondents (0%) have indicated that emotional support was *Not too important*, which results emphasized the critical role emotional support plays in their treatment and recovery.

Questions **Q7** to **Q10** incl. were to be answered by **Cancer Survivor Patients, volunteering as Mentors** to Cancer Patients in Treatment.

To **Q7**:"*What motivates you to volunteer as a Mentor to Cancer patients in treatment?*", there were 7 responses in total. Majority of the respondents (71.4%) were motivated by their *Personal experience with cancer*, while 28.6% expressed a *Willingness to help others beat cancer*. This result shows that personal experiences play a significant role in driving mentorship.

To **Q8**: "*Being a Cancer Survivor, what type of support would you like to offer to patients in current cancer treatment? (Please, select **all** that applies)*", received were 7 responses. *Emotional support* was the most popular, with 85.7% of the respondents selecting it.

This was followed by *Help with pet care* (71.4%), *Transportation for appointments* (57.1%), and 42.9% each for *Help with household tasks* and *Help with kids*. The results reflect a broad willingness among mentors to provide both emotional and practical assistance.

To **Q9**:"*How often would you be available to provide such support?*", obtained were 7 responses. 28.6% of the respondents each preferring *once a week* or *once a month*, and 14.3% each available *daily,* or *twice a week,* or *every second week.* This variation highlights the need for flexibility in scheduling volunteer support,

To **Q10**: "*In your opinion as a Cancer Survivor, what type of emotional support is most needed by patients in treatment*?", 7 responses were gathered. *Sharing own personal stories* and *understanding what uplifts the person* were tied at 42.9% each as answers, while 14.3% of the respondents selected *regular chats* option as an answer to that question. This suggests that mentors view storytelling and offering encouragement as the most impactful forms of emotional support.

Questions **Q11** to **Q20** incl. were to be answered by **All** Survey Participants.

To **Q11**: "*What is your gender?*", of the 20 responses received, 65% of the respondents identified themselves as *male* and 35% as *female*. The presence of both genders ensured diverse perspectives on the proposed application's relevance and usability.

To **Q12**: "*What is your age group?*", 20 responses were obtained with the following results:
18-25 age group - 10% of the respondents
26-35 age group: 15% of the respondents
36-45 age group: 25% of the respondents
46-55 age group: 45% of the respondents
56-65 age group: 15% of the respondents
66-75 age group: 0% of the respondents
75+ age group: 0% of the respondents

Most respondents (45%, accounting for nine persons) were aged 46-55, followed by 25% in the 36-45 age group. This aligns with the typical demographic, affected by cancer, where middle-aged adults are often the most impacted. The smaller representation of younger (18-25, 26-35) and older age groups suggests that the app should primarily focus on the needs of middle-aged users, while remaining inclusive of all ages.

To **Q13**: "Do you have a smartphone or tablet?", **all** 20 respondents (100%) answered with affirmative "*Yes*", ensuring that the proposed mobile app would be accessible to the entire target audience.

To **Q14**: "*How often do you use mobile applications on your device?*", there were 20 responses gathered. Majority of the respondents (80%) reported using apps *very often*, and the remaining 20% reported using them *often*. None of the respondents (0%) indicated to *rarely* or *never use* apps on their devices, demonstrating high familiarity with mobile app technology.

To **Q15**: "*Are you familiar with apps, designed to help your overall improvement and well-being, while in cancer treatment/recovery?*", of the 20 responses received, option "Yes" was selected by 20% (accounting for 2 respondents), while option "No" - by 80% (accounting for 18 respondents).

Majority of the respondents (80%) were not familiar with well-being apps, tailored for cancer treatment and recovery. This points to a knowledge gap and indicates a big chance for the suggested application to fill this unmet requirement. Once the app is developed, it could also need to be promoted through awareness campaigns and educational initiatives.

To **Q16**: "*If you have answered "Yes" to Q15, please state the name of such an application*", there were 2 responses obtained, as follows:

My Cancer Couch

My Wellbeing

While a few respondents were aware of existing apps, the limited variety of responses suggests that there are relatively few well-known options for cancer patients. This reinforces the importance of creating an app, that could fill this niche with comprehensive and practical features.

To **Q17**: "*If you have answered "Yes" to Q15, please state what were the features that you most liked in such an application*?", the 2 respondents familiar with well-being apps, helping cancer patients, highlighted the following two key features:

Appointments and reminders to the calendar

Easy navigation and features

The features mentioned emphasize the importance of usability and practical functionality, such as scheduling tools and an intuitive interface. These preferences should guide the development of the proposed app, ensuring it prioritizes simplicity and useful features to meet the needs of its users effectively.

To **Q18**: "*Would you be interested in using a specially designed Android Mobile application, as a platform to provide emotional and practical support from Cancer survivors, volunteering their time and experience as Mentors to patients, currently in cancer treatment and recovery?*", there were 20 responses obtained. A significant 95% of the respondents answered with affirmative "*Yes*", while only 5% answered "*No*". This overwhelming interest underscores the potential demand for the proposed app among the target audience.

To **Q19**: "*If you have answered "Yes" or "Maybe" to Q18, what **features** would you like such an application to have? (Please, select **all** that applies)*", the results obtained from the 18 respondents were as follows:

Privacy and security of personal data was the most requested feature by 80% of the respondents, followed by *Good design* and *Easy navigation* (65% each). These preferences indicate that user trust in the application and its usability are critical factors for the app's success.

To **Q20**: "*If you have answered "Yes" or "Maybe" to Q18, what **functionalities** would you like such an application to have? (Please, select **all** that applies)"*, obtained were 18 responses for Patients in Cancer Treatment, and 8 - for Vetted Cancer Survivors.

For Cancer Patients:

- 88.9% wanted *online registration and personal profile creation*
- 72.2% wanted *Secure Login with option to Re-set Password*
- 66.7% each wanted to *View List of prospective Mentors* and *Select suitable mentor & Send Request for Assistance Task* and then *Receive Notification of Acceptance/Rejection of their Request*.
- 55.6% wanted to *Provide feedback and ratings* for the obtained support.
- Additionally, 50% expressed interest in *Group chats for emotional support and to finalise details in case of Request acceptance*.

For Mentors:

- 75% each prioritized *Offline registration after vetting* with option to *Re-set Password, Personal profile creation* and *Sending Notifications for Patients Requests' Acceptance/Rejection, based on availability*.
- 62.5% wanted to *View requests from Cancer Patients for Assistance*
- 37.5% wanted to *View Feedback & Ratings* from a Patient in Treatment for the support provided
- Additionally, 50% expressed interest in *Group chats to finalise details in case of Request acceptance*.

To **Q21**: "*Would you recommend such an app, when being developed, to other Cancer Patients to use?*", obtained were 20responses. **All** respondents (100%) answered with affirmative "*Yes*", demonstrating confidence in the app's usefulness and potential to make a positive impact.

To **Q22**: "*In your opinion, would such an application, when developed, help to enhance Patients' personal well-being and speed-up their recovery process?*", **all** 20 respondents (100%) answered with affirmative "*Yes*", indicating unanimous confidence in the app's potential to positively impact patients' lives.

1.11 Doctors Structured Interview

Doctor Oncologists, involved in cancer treatment, were given a *Structured Interview*, consisting of **24** questions (please, see *Appendix A3*). The interview was conducted in November'2024 to investigate the viability and perceived worth of proposed for development mobile application, that would use cancer survivors' mentorship to offer cancer patients both practical and emotional support. Features, like wellness advice, emotional support, and help with everyday chores, would fall under this category. Experts' perspective on patient demands, present difficulties, and app features, that could enhance patient wellbeing and healing results, were also approached by the questions.

Three doctors responded to the *Structured Interview*:

The first three questions gathered personal information, such as: **Q1**: *Please state your Name and Surname*, **Q2**: *Please state the name of the organization you work for*, and **Q3**: *Please state your position at this organization*.

The responses obtained were as follows:

- Dr Eric Masterson, Orthopaedic Surgeon at Bon Secours Limerick.
- Dr Alexander Fraser, Consultant Rheumatologist at HSE.
- Dr. Emmet Kerin, Doctor at Gentle Procedures Ireland.

This initial information establishes the respondents' professional background and the healthcare organizations they represent, providing context for their insights, shared throughout the interview.

To **Q4**:"*What is your level of involvement in cancer patient care?*", two of the respondents (66.7%) stated they were "*Involved*", while one respondent (33.3%) reported being "*Minimally involved*." None of the respondents selected "*Highly involved*" or "*Moderately involved*" options.

To **Q5**: "*In your professional opinion, how helpful would a peer mentorship system between cancer survivors and patients in treatment be?*", two of the respondents (66.7%) believed it would be "*Very helpful*," while one respondent (33.3%) felt it would be "*Moderately helpful*". No respondents selected "*Slightly helpful*" or "*Not helpful at all*" options as their answer.

To **Q6**:"*Would you feel comfortable approving cancer survivors, that you have treated, as mentor volunteers (after being thoroughly vetted), to your current cancer patients in treatment?*", two of the three respondents (66.7%) answered "*Yes, without reservations*", while one respondent (33.3%) answered "*Yes, but with certain conditions*". No one (0%) selected option "*No*" as an answer.

To **Q7**: "*In your opinion, what types of support would be most important to assist with cancer patients in treatment? (Please, select **all** that applies.)*", the three respondents selected multiple options, with 66.7% each identifying *Emotional support, Household tasks, Transportation,* and *Pet care* as key support types. Additionally, 33.3% of respondents selected each "*All of the above*" and "*Companionship*" options.

To **Q8**:"*What wellness activities would you recommend to your patients? (Please, select **all** that applies)*", **all** respondents (100%) recommended *Yoga or light exercises* as a wellness activity, while two respondents (66.7%) suggested *Guided meditation*. One respondent (33.3%) mentioned *Sound therapy (audio therapy)* as a viable option.

To **Q9**: "*How often would you recommend cancer patients to engage in mindfulness, or audio therapy activities (such as meditation, or light exercises)?",* two respondents (66.7%) recommended "*A few times a week*", while one respondent (33.3%) suggested "*As needed*". None of the respondents (0%) selected "*Daily*" or "*Weekly*".

To **Q10**: "*In your professional opinion, what type of emotional support has the most positive impact on patients during treatment? (Please, select **all** that applies)*", **all** three respondents (100%) agreed that *One-on-one mentorship* has the most positive impact. Additionally, two respondents (66.7%) also highlighted *Professional counselling* as impactful. None of the respondents (0%) selected "*Group support sessions*" or "*Virtual emotional check-ins*" options.

To **Q11**: "*In your opinion, how effective an emotional support via digital means (e.g., use of specially designed mobile application) would be for cancer patients?*", **all** three respondents (100%) rated it as "*Moderately effective*" None (0%) selected "*Very effective*", "*Slightly effective*", or "*Not effective at all*" options.

To **Q12**: "*In your professional opinion, how important a peer support (mentorship relationship between cancer survivors and patients currently in treatment) is in improving patient's outcomes?*", **all** three respondents (100%) stated it is "*Very important*". None (0%) selected "*Moderately important*", "*Slightly important*", or "*Not important*" options.

To **Q13**: "*In your opinion, what are the biggest challenges that patients in treatment face, apart from the medical treatment itself? (Please, select **all** that applies)"*, respondents identified *Emotional support*, *Managing daily tasks (e.g., cleaning, shopping),* and *Transportation to/from medical appointments* as key challenges, with each option selected by one respondent (33.3%). Additionally, one respondent (33.3%) selected "*All of the above*" option.

To **Q14**: "*In your opinion, how important a volunteer system for practical tasks would be for your patients?*" two respondents (66.7%) stated it would be "*Essential*", while one respondent (33.3%) considered it "*Moderately important*". None (0%) selected "*Slightly important*" or "*Not important at all*" options.

To **Q15**: "*Would you encourage cancer patients in treatment to use a specially designed app, that connects them to cancer survivors as volunteering mentors for support?*", **all** three respondents (100%) answered with affirmative "*Yes*". None (0%) selected "*Maybe*" or "*No*" options.

To **Q16**: "*Are you familiar with apps, designed to help cancer patients' overall improvement and well-being, while in cancer treatment/recovery?*", **all** respondents (100%) answered "*No*" and were directed to **skip** to **Q19**.

Q17 and **Q18** were follow-up questions for those, who answered "*Yes*" to Q16. Since there were 0% "*Yes*" answers to that question, there were no responses (0%) obtained for Q17 and Q18.

To **Q19**: "*Would you be interested in using (and recommending your cancer patients to use) a specially designed Android Mobile application, as a platform to provide emotional and practical support from Cancer survivors, volunteering their time and experience as Mentors to patients, currently in cancer treatment and recovery?*", **all** three respondents (100%) answered with affirmative "*Yes*". None (0%) selected "*Maybe*" or "*No*" options.

To **Q20**: "If you have answered "*Yes*" or "*Maybe*" to Q19, what **features** would you like such an application to have? (Please, select **all** that applies)", respondents identified several key features as follows:
Easy navigation was selected by all respondents (100%). Other features, such as *Good design, Robust functionality without Software glitches* and *Security of personal information* each were selected by 66.7% of respondents.

To **Q21**: "*If you have answered "Yes" or "Maybe" to Q19, what **functionalities** would you like such an application to have? (Please, select **all** that applies)*", **all** respondents (100%) emphasised how crucial it is for physicians, patients, and volunteer survivors to register and create profiles on the app.
All 3 respondents (100%) agreed that *Doctors* should be able to *safely register **both**, patients* and *survivors* on the app. This would ensure authenticity through *offline registration* and a rigorous verification process that guards against *fraud* and *online impersonation*. Furthermore, it was determined that *secure two-factor authentication* was essential for safeguarding private patient and mentor information. Further to that, established was:

For Doctors:

- 100% of respondents selected each *Offline Registration by the app administrator*, and *Secure Login with 2-Factor authentication*, enabling only legitimate healthcare professionals to be part of the system.
- 66.7% selected functionality *After Garda Vetting process, Doctors approve cancer survivors as volunteering mentors in a List, by matching their type of cancer with the type and stage of cancer of patients currently in treatment*
- 66.7% selected functionality *Doctors provide **initial login link** to Volunteering Mentors from the List, to prevent **fraud at online registration***
- 33.3% selected functionality *Doctors can view and oversee the tasks that the Volunteers choose to assist with*
- 33.3% selected functionality *Doctors can send important health-related updates or reminders to both, patients and volunteers*

***For Patients in Cancer Treatment*:**

- **All** respondents (100%) agreed that patients should have the ability to *Create a personal profile outlining medical information* and *tasks requiring assistance (e.g., household chores, transportation, gardening).*
- *View List of prospective Mentors (filtered by gender, cancer type, and type of assistance provided)* - 33.3%
- *Select a suitable Mentor and send Request for Assistance with a Task, based on Mentor's availability* -100%.
- *Receive Notifications of Request Acceptance or Rejection from the Mentor* - 66.7%
- *In case of Acceptance - option to finalise details over chat/phone call* - 66.7%
- *Provide Feedback & Rating for the obtained Mentor's support* - 33.3%
- *Participate in a Group Chat with other Patients in Treatment and Cancer Survivors* - 33.3%

For vetted Cancer Survivors, volunteering as Mentors:

- *Offline Registration by the App's Administrator after vetting process & confirmation by the Oncology Doctor to prevent online impersonation and fraud* - 33.3%
- *Creation of Personal Profile (outlining medical diagnosis and current condition, preference of tasks to assist with, time availability to help with listed tasks, etc)* -33.3%
- *View Requests from Patients in Cancer Treatment for assistance with chosen task for preferred day and time (according to Mentor's availability, stated in the profile)* and *Respond with Notification to them* -33.3%

To **Q22**: "*Would you recommend such an app, when being developed, to your Cancer Patients to use?*", **all** three respondents (100%) answered with affirmative "*Yes*". None (0%) selected "*Maybe*" or "*No*" options.

To **Q23**: "*In your opinion, would such an application, when developed, help to enhance Patients' personal well-being and speed-up their recovery process?*", there were two responses obtained and **both** respondents (100%) answered with affirmative "*Yes*". None selected "*Maybe*" or "*No*" option.

To **Q24**: "*In your professional opinion, what might prevent patients from using such an app?*", from the 3 responses obtained, two respondents (66.7%) selected *Lack of Technology skills*, and one respondent (33.3%) selected *Privacy concerns*, with none of them selecting *App complexity* option as an answer.

1.12 Chapter Summary

Cancer is a life-changing condition that profoundly affects the emotional, physical, and mental well-being of individuals. Patients undergoing cancer treatment often face significant challenges, including feelings of isolation, emotional distress, and practical difficulties in managing their daily lives. Similarly, *cancer survivors* often have valuable experiences to share but may lack a structured way to support others. This situation emphasises the necessity of creative methods to deal with these issues and offer significant assistance. By providing a platform, where both physical and emotional support can be provided efficiently, the suggested

for development mobile application seeks to close the gap between cancer *patients* and *survivors*. The findings of the *Primary* study showed how much need there is for practical assistance, like domestic chores or transportation, as well as emotional support and guided wellness activities. The sustainability of a peer-based support system was further supported by the significant willingness of cancer survivors to donate their time and experiences.
By include features, like mentor matching, wellness resources, safe data processing, and user-friendly interfaces, the app's creation complies with these requirements. By doing that, it would help to improve patients' quality of life and establish a network of support for both patients and survivors.

Thorough *Secondary* research through a *Literature review* on subjects, including wellness practices, digital technologies for cancer care, and emotional support networks, will be presented in the next chapter. Additionally, pertinent Case Studies on current healthcare applications, data privacy laws, and the deployment of mobile technology in patient-centred care, will all be examined.

Chapter 2: Review of the Literature on the Topic

2.1 The Impact of Cancer on Patients' Mental and Physical Well-being

For this project, support for cancer patients will be divided into key aspects: emotional well-being, practical assistance, peer support, and digital health integration. Each component plays a crucial role in a patient's overall support system. Rather than serving as separate solutions, these elements work together to offer a holistic support experience for cancer patients, enhancing their journey to recovery. By integrating all aspects, the project aims to create a comprehensive platform to help improve both, the quality of care, and the patient outcomes.

Patients with cancer have emotional and physical challenges during diagnosis, treatment, and recovery, which has a substantial impact on their mental and physical health. Their quality of life is impacted by a complex experience, that combines physical side effects from therapies, like chemotherapy, with psychological load, like loneliness and anxiety.

Anxiety is one of the main psychological symptoms of cancer, resulting from worries about the prognosis, fear of treatment, and confusion about the diagnosis. Patients who receive a cancer diagnosis and undergo treatment, frequently experience severe psychological problems, such as increased anxiety, depression, and social isolation. As they deal with the physical hardship of extensive therapies and the uncertainty surrounding their diagnosis, many patients endure mental distress, according to the American Cancer Society. Relationships, sleep, and even the capacity to follow treatment plans, can all be negatively impacted by anxiety (American Cancer Society, 2024c). This effect highlights the necessity of providing cancer patients with comprehensive mental health support in addition to medical care. Younger individuals exhibit especially high anxiety levels because of life disruptions (American Cancer Society, 2024d). Anxiety can affect social interactions, sleep patterns, and treatment compliance, among other areas of life (American Cancer Society, 2024b).

These issues show how important the project's proposed holistic support system—which addresses both bodily and mental needs, is. The platform aims to help improve patients' quality of life and overall adherence to treatment plans, by emphasising emotional support.

One of the most popular cancer therapies, chemotherapy, has a major negative impact on patients' physical health. Chemotherapy medications target cells that divide quickly, including healthy cells in the bone marrow, hair follicles, and digestive tract in addition to cancer cells. This worsens a patient's quality of life by causing adverse consequences as fatigue, nausea, and immunosuppression (Ferlay, et al., 2021). With over 70% of chemotherapy patients reporting persistent exhaustion, fatigue is a problem that impairs mental toughness, appetite, and mobility (Brennan and Cameron, 2005).

Cancer treatments, particularly chemotherapy and radiation, place significant physical strain on patients. These treatments often result in fatigue, pain, and other long-term physical challenges. Fatigue is one of the most reported symptoms, with studies suggesting that 25% to 99% of patients experience persistent fatigue that affects their daily functioning (National Library of Medicine, 2023). This exhaustion is not simply a result of physical exertion but can be compounded by the emotional stress that cancer treatments impose (Bower 2015).

Treatment-related physical side effects, including weight fluctuations, skin changes, and hair loss, might affect a patient's self-esteem and lead to social isolation. Peripheral neuropathy brought on by chemotherapy, for instance, can impair movement and make it difficult to perform basic activities like walking or gripping objects. One important aspect influencing cancer patients' independence and quality of life is their physical decline (American Cancer Society, 2024d). Additionally, because some patients feel overwhelmed by the severity of their symptoms, the combination of physical discomfort and psychological anxiety can result in poor adherence to treatment measures.

Long-term emotional suffering results from cancer patients with high distress levels delaying or avoiding mental health services because of cost and accessibility concerns, according to research (National Library of Medicine, 2023). To lessen this, medical professionals stress the value of accessible, reasonably priced mental health therapies as a component of comprehensive cancer therapy, balancing the psychological and physical effects.

Another significant issue is depression, which is frequently brought on by the weariness and physical changes that come with cancer therapy. This mental illness can have a detrimental effect on a patient's quality of life and view on their recuperation. Up to 25% of cancer patients report having depressive symptoms, according to research, underscoring the necessity of supportive mental health therapies, in addition to conventional treatments (World Health Organisation, 2023). Coping strategies, including counselling or peer support groups, have also been demonstrated to be helpful in reducing feelings of loneliness and provide emotional comfort (Butow, et al., 2023).

The definition of social isolation, first put forth by Berkman and Syme in 1979, is a condition marked by the lack of support and social networks. The subjective and perceived status of individual experiences brought on by insufficient social support was then highlighted by Lien-Gieschen (2009), adding that the notion of social isolation needs to encompass subjective emotional states, like loneliness. In academic literature from a variety of fields, including sociology, medicine, and nursing, the phrase "social isolation" has been used. However, a consensus on its definition has not yet been reached amongst and even within different disciplines because of the disparate research topics and objectives. Different definitions of social isolation have been proposed by researchers. Others saw social isolation as multifaceted, including both subjective and objective characteristics of isolation, and others saw it as a unidimensional notion that only referred to the absence of social integration (Wang, et al., 2024).

Cancer sufferers' physical and emotional difficulties were made worse by the COVID-19 pandemic. Many cancer treatments were postponed or stopped due to the diversion of healthcare resources, which made patients feel more anxious, afraid, and uncertain (Afshar, et al., 2023). Emotional support is much more important because isolation and severe social distancing measures exacerbated emotions of loneliness and mental suffering. To address the psychological and physical effects of the disease, the pandemic highlighted the necessity of integrated mental health support within cancer care systems.

Relationship changes are another common experience for cancer sufferers. Self-esteem may suffer because of the change in relationships, as friends and relatives may take on caring responsibilities. As they juggle helping their loved one with their own emotional struggles, partners of cancer patients may also feel psychologically strained, which can disrupt relationships (World Health Organisation, 2023). To help patients and carers deal with these interpersonal difficulties, support groups and mental health counselling are advised, highlighting the need of having a network of people who are there to support them during treatment.

The multifaceted challenges, faced by cancer patients, ranging from emotional distress and anxiety to physical exhaustion and social isolation highlight the critical need for comprehensive, supportive care systems. Emotional well-being is an essential component that collectively enhance the patient’s journey from diagnosis through recovery. Anxiety, which is prevalent among cancer patients, can disrupt social interactions, sleep quality, and treatment adherence, particularly in younger patients, who experience significant life disruptions. Physical effects from treatments like chemotherapy further impact quality of life through symptoms such as fatigue, pain, and peripheral neuropathy, which can lead to decreased mobility and independence.

These problems can be lessened by integrating peer programs and mental health support into cancer treatment, particularly by giving patients access to efficient volunteer support from patients who survived cancer. This all-encompassing strategy guarantees that patients receive mental and emotional support, in addition to physical treatment, which enhances overall results and treatment compliance. By combining these components, the project hopes to create a strong, encouraging environment that attends to cancer patients' physical and emotional needs, ultimately helping to promote a more successful recovery and enhancing their quality of life.

2.1.1 The Role of Peer Support and Volunteer Programs in Cancer Recovery

Effective support systems play a vital role in helping patients manage the impact of cancer on their mental and physical well-being. Peer support programs and counselling services are essential for addressing the psychological needs of cancer patients. Supportive care that includes mental health services, peer support, and family involvement has been shown to improve patient outcomes and overall quality of life (National Comprehensive Cancer Network, 2023a). Structured support systems can help patients share experiences, gain insights from others who have faced similar challenges, and feel less isolated (NCCN Guidelines for Patients, 2023b).

The platform for this initiative will include peer support programs that pair patients with people who have gone through similar things, lowering feelings of loneliness and improving skills through mutual support and understanding. By strengthening emotional resilience and creating a feeling of community among survivors, volunteering can greatly help in the healing process from cancer. These programs give patient`s the chance to interact with peers who have gone through similar things, which decrease the sense of loneliness that is frequently connected to the cancer journey. Volunteering helps survivors heal emotionally and psychologically by giving them a sense of purpose and motivation in addition to helping a larger cause. Workshops, mentoring, and group activities that strengthen coping mechanisms and promoting a sense of agency are frequently included in the structured help provided by these programs. Additionally, studies suggest that the social connections present in these settings can enhance mental health results, which in turn can lead to a more thorough healing process. Stressing the value of volunteer programs draws attention to their part in creating a network of support that is essential for a full recovery as well as in promoting personal healing.

Volunteers are vital pillars of support in many cancer recovery programs, offering people overcoming health obstacles both practical help and emotional support. Volunteers provide vital needs that patients might find hard to express by taking on a variety of responsibilities, such as providing company or assistance with everyday chores. According to research, virtual communities can empower members by encouraging better coping mechanisms and emotional involvement throughout their health journey, indicating that support from the community improves personal wellbeing (Illanes, 2024). Additionally, peer support creates a specific relationship that helps lessen the sense of loneliness that cancer patients frequently suffer, particularly when it is based on shared experiences. But even while these volunteers offer priceless support, it's imperative to make sure they have the tools and training they need to avoid fatigue and continue to provide long-term support (Yao, 2024). As a result, the incorporation of organised volunteer activities promoting a cooperative society committed to improving lives in addition to helping people heal.

Incorporating volunteer and peer support initiatives is essential to improving cancer patients' recovery paths. Online peer groups provide a special platform for connection and shared experiences as people work through the challenges of their illnesses, enabling members to offer emotional and informational support without regard to location (Yao, 2024). These kinds of settings help people feel like they belong, which is important for those dealing with serious

health issues. According to qualitative research of patients with chronic illnesses, patient involvement with virtual platforms emphasises the spaces' transformative potential in encouraging changes in health-related behaviours and improving general well-being (Illanes, 2024). Ultimately, the psychosocial responsibilities related to cancer recovery can be considerably reduced by implementing a comprehensive strategy that incorporates various networks of support.

Many cancer support services moved online during COVID-19, because in-person volunteer and peer support programs were severely limited. Despite their physical isolation, patients could connect with others and get emotional support through virtual peer support, which has developed as a valuable alternative (Afshar, 2023). These online relationships demonstrated the flexibility and resiliency of peer networks during trying times, by reducing emotions of isolation and loneliness.

Resilience, both mental and physical, plays a significant role in how patients manage the long-term impact of cancer. Many survivors report developing a heightened sense of resilience, which helps them adapt to post-treatment life. Programs, focusing on building resilience through peer support, lifestyle modification, and physical rehabilitation, have shown to enhance quality of life for survivors, helping them to manage both physical and psychological symptoms effectively (Nacional Cancer Institute, 2022).

Peer support and volunteer programs play a vital part in cancer recovery because they give patients a sense of community and emotional support. According to research, people who participate in peer support networks frequently report feeling less depressed and anxious, which improves their general wellness. Patients can get support, useful guidance, and coping mechanisms that enhance emotional resilience by interacting with people who have gone through similar experiences with cancer. By enabling survivors to give back to the community, volunteer programs further improve this healing process and provide a sense of fulfilment and purpose that may be extremely therapeutic. Additionally, participating in these activities frequently promoting social ties that decrease the sense of loneliness that many cancers patients experience.

2.1.2 The Impact of Cancer on Mental Health and the Importance of Supportive Services

Receiving a cancer diagnosis is a life-changing event, that presents people with significant mental health issues in addition to physical ones. Feelings of shock, anxiety, uncertainty, and grief are frequently experienced after receiving a cancer diagnosis. Treatment is often a drawn-out, difficult process with many side effects that affect both the body and the mind. Cancer has a complex psychological toll that includes social isolation, anxiety, and sadness. Patients' relationships, quality of life, and even their capacity to follow treatment plans are all impacted by this mental health stress. Supportive services, especially peer assistance, have become crucial parts of comprehensive cancer care as these mental health issues gain more recognition. The efficacy of digital peer support systems designed especially for cancer patients has not been thoroughly studied, even though traditional support programs are well-established. By developing a digital support system that combines peer support, mental help, and instructional materials for helping with physical tasks, the current project aims to fill this gap.

Cancer patients can experience a wide range of mental health issues at any point during their cancer journey, from diagnosis and treatment to remission and survivorship. Among the most typical psychological reactions to a cancer diagnosis and the treatment that follows are anxiety and depression. Between 15% to 25% of cancer patients report having depression, and even more are reporting to having severe levels of anxiety, according to the American Cancer Society (2024b). The physical effects of cancer and its treatment are frequently enlaced with

these emotional reactions, resulting in a vicious cycle as much as psychological discomfort is made worse by physical symptoms and vice versa (Stein, 2020).

Fears about the prognosis, the results of treatment, and the potential for recurrence are common causes of anxiety among cancer patients. Many people have existential worries about mortality and the future along with the first shock of receiving a diagnosis. A patient's social interactions, sleep patterns, and day-to-day functioning can all be negatively impacted by this worry. Younger cancer patients, in particular, deal with a special set of pressures, since they can be going through pivotal times in their personal, professional, or interpersonal growth. These people's experiences with cancer not only interfere with their day-to-day lives but also raise the possibility of changing their long-term goals and ambitions, which exacerbates their worry and sadness (American Cancer Society, 2024b).

Another serious mental health issue is depression, which is frequently made worse by the psychological and physical effects of cancer therapies like radiation and chemotherapy. Cancer patients who are depressed may experience pessimism, lack of interest in once-enjoyed hobbies, and ongoing unhappiness. When physical symptoms like chronic exhaustion, pain, or nausea make it difficult for patients to obtain relief or keep a good attitude, the burden of depression is especially great. Furthermore, patients' self-esteem may be harmed by medication side effects such as weight fluctuations, hair loss, and other obvious changes, which can result in social disengagement and a feeling of loneliness. Untreated depression has been linked to worse health outcomes for cancer patients, according to research, since it can lower motivation to follow treatment regimens, visits to the doctor, or adopt a healthy lifestyle (World Health Organization, 2023).

Additionally, because cancer and its treatment frequently interfere with everyday social activities, social isolation is a significant problem for cancer patients. A feeling of distance may result from friends and relatives not fully comprehending the patient's condition. Patients' energy levels are also lowered by the physical toll of treatment, which makes it challenging for them to continue regular social interactions. Because social isolation frequently results in emotions of loneliness, which can exacerbate symptoms of anxiety and depression, it can be very harmful. Later researchers focused on the emotional impact of perceived isolation on mental health, as defined by Berkman and Syme (1979), who defined social isolation as the absence of social networks and support. For cancer patients, this sensation is particularly powerful, and they may believe that their friends and Family members are unable to empathise with the anxiety, suffering, and uncertainty they experience. To address the emotional and social aspects of the cancer experience, supportive services that go beyond medical care are necessary mainly for people that know the situation before help others in the same situation.

The significance of supportive services that offer psychological and emotional respite is highlighted by the effects of mental health issues on cancer patients. A variety of services are included in supportive care, such as instructional materials, support groups, and psychiatric counselling and therapy. These services assist patients in improving their quality of life, developing resilience, and managing their emotions. To help patients process their feelings and create coping mechanisms, traditional supportive services like counselling and therapy are essential. However, stigma, availability, and expense are some of the obstacles that frequently keep people from seeking or getting the proper care for mental health issues. Financial worries cause over 15% of cancer patients to postpone or forego accessing mental health therapies, according to studies, indicating a serious care gap for this population (National Library of Medicine, 2023).

Due in large part to heightened social isolation and restricted access to mental health care, the COVID-19 exacerbated mental health problems for cancer patients. Because routine care and support networks were disrupted by lockdowns and limited hospital access, many patients experienced increased anxiety and sadness during this time (National Comprehensive Cancer

Network, 2023a). This demonstrated how crucial it is to provide accessible and adaptable supporting services, such as online counselling and telehealth, to meet mental health needs in challenging circumstances.

A useful and easily available part of supportive treatment for cancer patients in recent years has been peer support. In contrast to professional therapy, peer support groups pair patients with people who have gone through comparable diagnoses and treatment paths so they can exchange stories, worries, and coping mechanisms. According to Meyer and Katz (2007), this type of support is distinct because it is based on shared experience, which offers a degree of empathy and comprehension that can be challenging to duplicate in conventional therapeutic interactions. One-on-one mentorship, group support sessions, and online communities are just a few of the ways that peer support may be provided, making it a flexible solution that can be customised to match the unique requirements and preferences of each patient.

Peer assistance provides a special remedy for cancer sufferers' mental health issues. It offers a sense of understanding, validation, and fellowship that is very consoling for anyone going through such a traumatic event. Peer support, in contrast to typical mental health services, is founded on the common human experience of coping with and overcoming cancer rather than on official counselling. Because they are connecting with someone who has had comparable worries, difficulties, and anxieties, patients feel genuinely understood in this shared experience (American Psychological Association, 2024).

Peer support can help cancer patients feel less alone and more hopeful, which has been found to considerably lessen their anxiety and depression symptoms. Cancer patients who took part in peer support programs reported better mental health and a higher quality of life than those who did not, according to a National Comprehensive Cancer Network study from 2023 (National Comprehensive Cancer Network, 2023b). The emotional connection that develops when patients tell their stories and hear firsthand accounts of survival and resiliency is thought to be the cause of this improvement. For cancer patients, it can be tremendously inspiring to know that they are not travelling alone and that others have overcome comparable obstacles. In the other hand, giving patients a safe space to vent their frustrations and anxieties can help them deal with the psychological effects of cancer and manage better.

Survivors can provide current patients with useful techniques and information, which is one of the main advantages of peer support. Survivors might offer advice on coping with side effects, maintaining motivation, or handling changes in body image brought by treatment, for instance. Since this kind of advice comes from someone who has gone through similar difficulties themselves, it is frequently more approachable and practical than recommendations from medical experts. Furthermore, peer support can promote treatment adherence by encouraging patients to remain optimistic and continue to their treatment regimens, even throughout challenging stages of recuperation (National Cancer Institute, 2022). Support from someone who genuinely understands their situation might change how patients perceive their course of therapy, changing their perspective from one of survival to one of resilience.

Building a sense of community among cancer patients and survivors is another important function of peer support. By encouraging social interactions, this community helps cancer sufferers feel less alone. Through mentorship programs, online forums, or group gatherings, peer support helps patients build deep connections with people who have gone through similar things. By offering patients shared resources, emotional support, and a forum for knowledge sharing, this support system can act as a lifeline (Given, Sherwood, & Given, 2017). The opportunity to connect with people going through similar things helps many people feel less alone, recover emotionally, and feel better overall.

Peer support is crucial for fostering the mental and emotional fortitude needed to manage the long-term effects of cancer. Survivors who take part in peer support programs frequently report feeling more resilient, which helps them adjust to life after treatment. Resilience-building

programs that emphasise shared experiences, lifestyle changes, and physical and emotional rehabilitation have been shown to improve the quality of life for survivors and current patients (Bower, 2015). This resilience benefits patients in overcoming mental difficulties like worry and sadness as well as helping them better manage physical symptoms like pain and exhaustion. Being a peer mentor and giving back to the community gives many survivors a new feeling of fulfilment and purpose. Serving others and volunteering promotes personal development and a feeling of achievement, both of which can help in one's own emotional healing. Both mentors and mentees gain from the ongoing support loop that is produced by this reciprocal paradigm, in which survivors serve as mentors to current patients. The mentor acquires a better comprehension of their own journey and the influence they may have on others, all the while providing the mentee with priceless support and direction that could help them get through the most trying times of their treatment.

Cancer patients' mental health issues, such as social isolation, anxiety, and despair, underscore the critical need for supportive services beyond medical care. By matching patients with others who have lived through the cancer journey themselves, peer support provides a special and efficient way to deal with these issues. Peer support, as opposed to conventional therapy, helps cancer patients develop a sense of belonging, empathy, and fortitude that can be extremely therapeutic.

Being a peer mentor and giving back to the community gives many survivors a new feeling of fulfilment and purpose. Serving others and volunteering promotes personal development and a feeling of achievement, both of which can aid in one's own emotional healing. Both mentors and mentees gain from the ongoing support loop created by this reciprocal paradigm, in which survivors serve as mentors to current patients (Meyer & Katz, 2007). The mentee receives priceless support and direction that may help them get through the most trying times of their treatment journey, while the mentor develops a greater awareness of their own path and the influence they can have on others.

2.1.3 Physical Challenges and Practical Assistance

Fatigue is one of the most prevalent and incapacitating physical symptoms that cancer patients must deal with. Cancer patients frequently experience fatigue that is more intense and long-lasting than ordinary fatigue, which affects their quality of life and everyday activities. The review indicates that several variables, such as the cancer itself, the side effects of therapies like chemotherapy and radiation, and the general mental and physical stress of managing a chronic illness, might contribute to cancer-related fatigue. Patients' mental health may be further impacted by fatigue, which can make it difficult for them to continue with daily tasks and cause them to feel frustrated and powerless (National Comprehensive Cancer Network, 2023a).

Helping cancer patients deal with these difficulties requires practical support, such as help with everyday tasks, housework, and transportation. For instance, reliable transportation ensures patients can follow their treatment plans, which is essential for enhancing health results. Treatment may be interrupted because of missed appointments, which could decrease the efficacy. Help around the house is similarly important because it frees up patients' energy for self-care and other necessities rather than wearing them out with physical labour. Research shows that practical support improves quality of life and encourages adherence to treatment, which in turn improves clinical outcomes for cancer patients (Given, Sherwood, & Given, 2017).

The platform for this project seeks to link patients with neighbourhood volunteers for helpful support, facilitating their access to assistance with household chores, transportation, and other critical everyday necessities.

The research emphasises the importance of established support networks, which offer services and resources to lessen the logistical and physical difficulties faced by cancer patients. For example, volunteer programs are available in many towns and healthcare systems to help with errands, cleaning, and meal preparation. By allowing patients to concentrate on their recuperation, these services can lower stress levels and enhance general health and wellbeing (Boucher, et al., 2018).

Support from someone who has gone through the difficulties of cancer treatment themselves may offer contemporary patients a special kind of understanding and motivation. Cancer survivors, as opposed to ordinary carers, can provide direct knowledge about how to deal with exhaustion and accept helpful help. Because they are being supported by someone who genuinely knows their path, patients may be more motivated to follow their treatment and care regimens because of this lived experience, which also creates a strong sense of connection.

In the cancer care continuum, carers are essential and frequently do a variety of tasks beyond providing direct physical support. Helping with everyday tasks, scheduling medical appointments, handling prescriptions, and offering emotional and spiritual support are all examples of caregiving responsibilities. However, because they usually put the patient's needs ahead of their own, these responsibilities can be taxing on the carers themselves. This self-neglect can result in high levels of psychological and physical stress, as well as carer burnout, which has an impact on the health of the patient and the carer (National Cancer Institute, 2022).

Carers frequently face emotional and psychological difficulties as they attempt to juggle their caring responsibilities with other obligations in their lives. For example, many carers struggle to keep a job, take care of their children, or take care of their own health. As carers may feel overburdened by their new duties, role shifts within families can often lead to stress. For instance, the dynamics of a relationship may change if a spouse must transition from being a partner to the primary carer (Badr and Acitelli, 2019).

Cancer patients' physical problems, like exhaustion and the need for help with everyday chores, worsened during COVID-19. Many patients found it difficult to take care of themselves and go to doctor's appointments since they were unable to get in-person help because of pandemic restrictions. This reaffirmed the necessity of all-encompassing emergency-ready support networks, such as dependable telemedicine access and distant volunteer support for hands-on help (Pollock, et al., 2020).

Support from a person who has conquered the same illness can be a potent symbol of resiliency for cancer sufferers. Survivors who support patients can use their personal experiences to provide not only useful advice but also emotional support that is firmly based in compassion and understanding. Because they are receiving guidance from someone who has been in their position, patients may find this peer support especially encouraging, giving them hope and encouragement to persevere despite treatment's hurdles, and avoid overburdening anyone, as multiple volunteers can divide the tasks among themselves.

2.1.4 Social Aspects of Well-being for Cancer Patients

Accordingly, with (National Cancer Institute, 2022) given that cancer patients frequently experience significant interruptions in their social and personal lives, social support is essential to the healing process. Strong social ties are essential since the physical difficulties of treatment exacerbate the psychological toll of loneliness, anxiety, and despair. Strong social support networks have been linked to better mental health, better physical outcomes, and longer survival rates for patients. (Fabbietti,2023)

By offering a virtual community where cancer patients can create deep social ties, the proposed platform seeks to lessen feelings of isolation and promote a sense of community.

Cancer patients' health outcomes have been demonstrated to be negatively impacted by social isolation, as individuals who lack strong social ties may suffer from increased stress, compromised immune responses, and reduced treatment compliance (American Cancer Society, 2024). During difficult times, friends, family, and community networks provide both practical and emotional assistance. But for many cancer patients, the COVID-19 epidemic made their isolation worse by reducing in-person encounters and taxing established support networks (National Cancer Institute, 2022).

These challenges were exacerbated during the COVID-19 pandemic, as social distancing policies and overburdened healthcare systems made it difficult for many cancer patients to receive in-person help with everyday duties. The pandemic highlighted the importance of reliable volunteer support networks that could assist patients with essential tasks, especially when access to traditional support networks was limited (Afshar, et al., 2023). Many patients found virtual support networks to be a lifeline during the COVID-19 pandemic, providing a means of preserving emotional support and social connections despite physical isolation. These online resources demonstrated the adaptability and resilience of peer networks, offering essential support in unprecedented times (Afshar, et al., 2023).

Poorer mental and physical health effects can result from social isolation. To improve emotional resilience and general well-being, this initiative provides a platform that enables patients to quickly interact with mentors and peers who are sympathetic to their circumstances. Peer support allows patients to connect with people who have had similar difficulties, which offers a special kind of empathy and understanding. Peers provide a kind of support based on common experiences, in contrast to family and friends who might find it difficult to completely understand the cancer experience.

Social isolation has been shown to have a detrimental effect on cancer patients' health outcomes. According to the American Cancer Society (2024c), those who don't have social support are more likely to suffer from increased stress, compromised immune systems, and decreased treatment compliance. Although they might not be able to handle all the practical demands of a patient receiving intense treatments, family and friends frequently offer early assistance. This disparity emphasises the necessity of volunteer-run support initiatives that provide cancer patients with useful help so they can continue to be independent and concentrate on their recuperation.

Peer support helps patients deal with the psychological effects of cancer by lowering feelings of loneliness and fostering emotional resilience. By developing an app that pairs patients with mentors who have gone through similar things, our project seeks to capitalise on these advantages and build a caring and supportive community. This project/app's planned features are intended to create a social well-being by providing a safe online community for cancer patients and survivors by establishing a complete social support platform, this app addresses the loneliness typically felt by cancer patients, providing emotional comfort and practical resources to improve quality of life and promote recovery.

2.2 Data Privacy Laws in Ireland

Comprehensive data protection legislation in Ireland protects people's right to privacy regarding their personal information. These regulations guarantee that businesses or other third parties manage personal information sensibly, protecting user privacy and guaranteeing data security. Information gathering, storing, analysing, sharing, and deletion are all examples of processing personal data (Data Protection Commission, 2019a).

The General Data Protection Regulation (GDPR) and the Data Protection Act 2018 are the two main pieces of legislation that control data privacy in Ireland. These rules provide out standards for the management of personal data to safeguard people's privacy. The Data Protection Act

2018 amends earlier laws from 1988 and 2003 to conform to the EU-wide GDPR, as stated by Citizens Information (2020). The European data privacy directive, which had been in effect since 1995, was superseded by the GDPR, which went into effect in May 2018 (Citizens Information, 2018).
The GDPR is based on seven core principles that specify how data must be handled:

Lawfulness, Fairness, and Transparency: Businesses must gather and handle data in a way that is both lawful and equitable, making sure that people are aware of how their data is being used.
Purpose Limitation: Information should only be gathered for specific, well-defined, and justifiable purposes; it should not be used needlessly or for purposes other than those for which it was created.
Data minimisation: To avoid storing too much information, only data that is absolutely required for a particular purpose should be gathered and processed.
Accuracy: Information must be maintained up to date and accurate; any inaccurate or out-of-date data must be removed or changed right away.
Limitation on Storage: Personal information should only be kept for as long as is required to fulfil its intended function, after which it should be safely.
Integrity and Confidentiality: Sufficient safeguards must be in place to protect data from loss, damage, and unauthorised access.
Accountability: Documented procedures and policies are required to show that an organisation complies with all data protection standards (Data Protection Commission, 2019b).

The suggested application will make sure that user data is only gathered with their express agreement to adhere to these values. Prior to the registration process, users must accept the terms and conditions. To maintain the accuracy of the data, the site will also enable users to update their personal information. Additionally, users will have the option to delete their accounts, which will remove all their data from the system forever and cannot be recovered.
Personal data and special category personal data are the two primary categories of data under the GDPR. Most of the data that the application gathers, including email addresses and usernames, is classified as personal data. On the other hand, health-related data, including data about physical and mental health, is categorised as special category data. Before such sensitive data can be used, the user must give their express consent (Citizens Information, 2018).
Only with users' express consent will sensitive data particularly that pertaining to medical conditions be processed. In certain situations, such as when data is utilised for law enforcement activities like looking into criminal conduct, GDPR principles could not be applicable. In these situations, the GDPR is superseded by the Law Enforcement Directive (LED) (Data Protection Commission, 2019a).

2.3 Privacy and Security in Firebase

Several privacy and security criteria, such as ISO and SOC certifications, are met by Google's Firebase service. Google is dedicated to assisting developers in complying with the GDPR's regulations, guaranteeing the security of end-user data (Firebase Support, 2020).
The developers of applications built with Firebase take on the role of Data Controllers, meaning they oversee deciding how and why personal data is processed. Conversely, Google serves as a data processor, managing the developers' data processing and storage. Application developers

are responsible for ensuring that data protection laws are fully applied, and that users' privacy rights are upheld in their capacity as Data Controllers (Firebase Support, 2020).
Security Rules, which serve as barriers between stored data and any risks, are offered by Firebase. Developers can configure these rules for any Firebase product, including Firestore, Realtime Database, and Cloud Storage. According to predetermined criteria, the rules guarantee that only authorised users can access data (Firebase Documentation, 2020a). Here's how developers can successfully create and oversee these security guidelines:
Discover the Language of Firebase Security Rules: Because the rules are formatted in JSON and written with a syntax akin to JavaScript, developers can design intricate access controls.
Put Firebase Authentication into Practice: By including authentication, developers may include an additional degree of user verification, enabling the application to safely identify users prior to authorising access to data.
Describe Data Structures Considering Security: Security systems are impacted by the organisation of data; for instance, rules that are applied at a parent node will also apply to all of its child nodes, allowing access down the hierarchy. Access Security Rules via Console or CLI: Firebase's console or command-line interface gives developers the ability to view and modify security parameters.
Establish Basic Security Rules: To begin, developers should establish rules for typical situations, including limiting content access according to user roles or giving user groups particular permissions.
Rules for Testing Security: Before implementing their rules, developers can test their functionality in various scenarios using the Firebase Rules Playground.
Deploy Rules to Production: The Firebase console or command-line tools can be used to deploy the rules to the live environment following testing (Firebase Documentation, 2020b).
Firebase protects data in transit and at rest with encryption. HTTPS ensures that data is encrypted while being transmitted between the application and the database. Furthermore, only authorised staff members with legitimate business needs are permitted access to Google's systems thanks to internal security procedures that limit access to sensitive data. Two-factor authentication required sign-ins with Google credentials, and thorough data access logging are examples of access control mechanisms (Firebase Documentation, 2020b).

2.4 Well-being and Quality of Life for Users

Enhancing the general well-being and quality of life of consumers of digital health platforms has gained more attention in recent years. These platforms aim to improve users' emotional and social well-being by fostering a supportive atmosphere in addition to attending to their medical needs. A comprehensive strategy is necessary, especially for people dealing with difficult medical diseases like cancer, which can have a big influence on social, familial, and personal lives.
Emotional support, mental health tracking, and community integration are among the essential characteristics of digital health applications that prioritise user well-being. These factors work together to support patients in maintaining a high standard of living during their medical care and recuperation. Peer support groups, psychiatric counselling, mindfulness exercises, and content personalisation options are common tools provided by these platforms, enabling users to customise their experience according to their own needs and preferences. Health apps can offer features that encourage patient autonomy and empowerment during their recuperation, much as how businesses have started to offer flexible work schedules to enhance work-life balance. Giving users the option to select their own activity plans or track their success in terms

of their physical and emotional well-being, for example, can increase user engagement and platform adherence.

Research indicates that enhancing patient adherence and treatment results throughout treatment requires social and psychological support. These platforms can help patients feel less alone by creating a sense of community (National Comprehensive Cancer Network, 2023b). Platforms that provide emotional and mental health support can also help users deal with anxiety and sadness, which are normal reactions to significant medical problems, according to study by the American Cancer Society (2024b).

Furthermore, because mindfulness programs and relaxation techniques help people focus better, feel less stressed, and be more satisfied with their treatment experience overall, they are being incorporated into health applications more and more (Brammer, 2020). These techniques have been shown to improve mental health by encouraging composure and assisting users in managing the unknowns and difficulties associated with long-term therapies.

Last but not least, digital platforms frequently allow users to set goals and evaluate their progress, allowing them to keep an eye on their own health indicators like physical activity, sleep quality, and the management of symptoms. In addition to enhancing the user experience, this individualised approach promotes active participation in the healing process, giving consumers a sense of empowerment and control over their health journey.

2.5 Evolution of Mobile Applications and Their Role in Digital Health Support

Long before cell phones were common, the idea of downloadable software, or "apps," was conceived. Steve Jobs presented an innovative concept similar with App Store during the 1983 International Design Conference in Aspen. He likened it to a "record store" where customers could read and purchase recordings, and eventually connect to a phone connection to access and download software from a catalogue. This concept laid the foundation for the current app ecosystem and was proposed 24 years prior to the 2007 release of the first iPhone (Roemmele, 2017).

A mobile application, sometimes known as an "app," is a program made specifically to operate on mobile devices that enables users to access services, such as those related to health and wellness, connect with others, do tasks, or consume media (Richmond, 2019). Early Personal Digital Assistants (PDAs) are where mobile applications first appeared. One of the earliest gadgets with integrated programs like a text editor, calculator, and contact database was the Psion Organiser, which was manufactured by a London-based business and introduced in the middle of the 1980s. These integrated programs established the foundation for modern smartphone applications, even though they did not support phone calls (Richmond, 2019).

Apple's Newton PDA, which debuted in 1993, stood out for allowing users to access third-party apps, such as an email client and a web browser. Because it signalled the start of mobile devices' ability to increase functionality beyond pre-installed software, this development was noteworthy. Snake, the first app with an entertainment focus, was released shortly after for the Nokia 6110, showcasing the potential of mobile applications to offer captivating and interactive experiences (Richmond, 2019). Phones got more advanced as mobile technology developed in the early 2000s, with better screens, more processing power, and the capacity to perform activities that got more complicated. As a result of these advancements, new mobile operating systems were developed, including Apple's proprietary iOS for the iPhone and iPad, Symbian OS for companies like Nokia, Sony Ericsson, and Motorola, and Android OS for Sony, LG, and HTC. These operating systems made it possible for developers to produce a wide range of applications, such as those centred on fitness, health, and wellbeing, which eventually became essential to the mobile app ecosystem (Richmond, 2019).

With the opening of the Apple App Store in 2008, the real app revolution got underway. The tool Store had 500 apps in its first three days, with 10 million downloads, including early versions of PhoneSaber and Evernote, a well-known personal organiser tool that established the benchmark for productivity and self-management (Strain, 2015). The popularity of the App Store showed how much demand there is for mobile apps that promote daily activities as well as individual wellbeing.

Google introduced the Android Market (now the Google Play Store) at about the same time, giving Android users access to games, apps, movies, and TV series. At first, e-books were available through the Google eBookstore (established in 2011), and music downloads were handled independently through Google Music (opened in 2010). Google combined these platforms in 2012 to create the Google Play Store, which at launch included about 450,000 cloud-based apps (Callaham, 2017). Users found it easier to incorporate digital health tools into their daily lives because of this consolidation, which made a variety of applications, including fitness and health apps, easier to access.

Mobile applications are now essential for supporting wellness and healthcare. From the ease of a mobile device, consumers can use digital tools for managing mental health, tracking physical activity, connecting with peer support groups, and even consulting healthcare specialists thanks to the millions of apps available across several app stores. Innovative approaches to health and well-being have been made possible by the development of mobile apps, which help users manage their diseases, adopt better lifestyle choices, and live better lives.

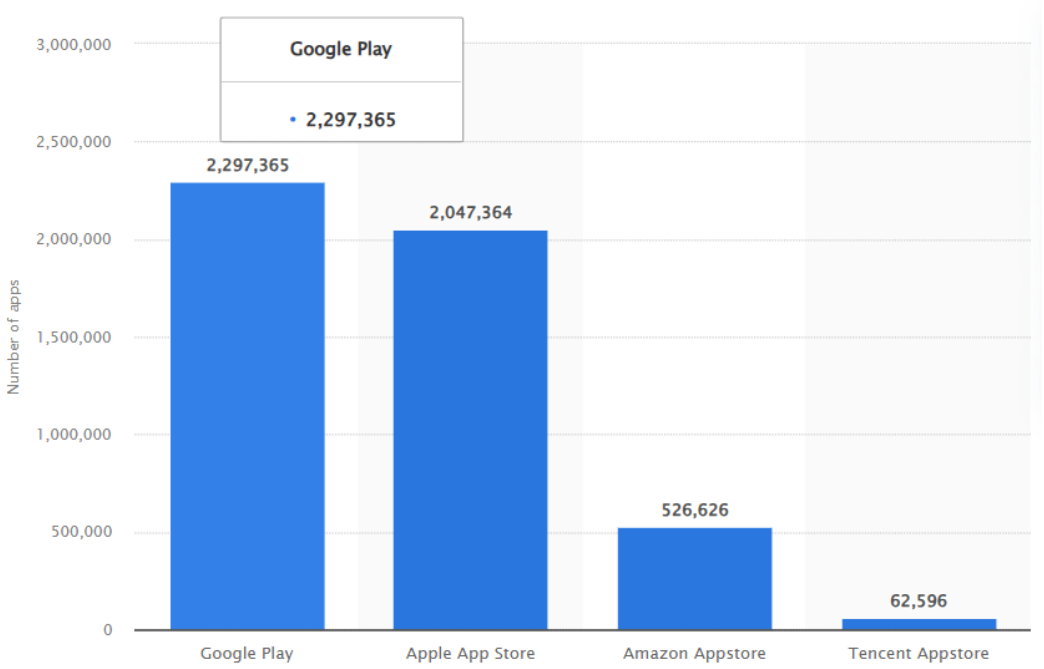


Figure 7: Number of apps, available in leading app stores 2024. Statista, (Ceci, 2024).

As presented on Figure 7, as of August 2024, Android users had access to 2.3 million apps on Google Play, making it the largest app store globally. The Apple App Store followed with around two million apps available for iOS. While app counts fluctuate as both stores remove low-quality apps, the overall number of available apps has grown over time. App Pricing and Revenue: Apps generate revenue through download fees, in-app purchases for premium features, and advertising. As of January 2024, the average price of an app on the Apple App Store was $0.73. Despite having fewer apps, the Apple App Store is more successful in monetization. In Q3 2022, consumer spending on the App Store reached $21.2 billion, compared to Google Play's $10.4 billion. Other App Stores: Although Apple and Google dominate the market, other app stores exist. The Amazon Appstore offers around 526,000 Android apps, with popular categories like gaming, education, and utilities. In China, third-party app stores, like the Tencent Appstore, provide over 63,000 mobile apps, tailored for the local market (Ceci, 2024).

2.6 Use of the Internet and Mobile Applications

People's methods of accessing online content have changed because of the significant decrease in the cost of smartphone access over time. Because of their ease of use and intuitive design, mobile applications have essentially supplanted traditional websites. These days, even websites are frequently made to be mobile-friendly.

The prevalence of mobile devices over other devices is demonstrated by the fact that 88% of people in 2019 used their phones to access the Internet when they were not at home or at work (Central Statistics Office, 2019a). Furthermore, 60% of respondents looked for health-related material online within three months of the poll, and 90% of respondents were between the ages of 16 and 29. Nearly 70% of respondents were active on social networks (Central Statistics Office, 2019b).

With its social networking elements, the suggested application, which is classified under the Health and Wellbeing category, enables users to communicate and discuss issues. Engagement requires this social component.

According to a Google and Ipsos MediaCT survey, 25% of smartphone users download apps they find through search engines, and 40% find new apps through app stores (Tiongson, 2015). But 25% of installed apps are rarely utilised, thus continuous interaction is essential. The software for this project is built around interactive challenges rather than static content to keep users interested. According to the study, if an app had discounts, 24% if it had exclusive material, 21% if friends or family used it, and 16% if it had new features, 30% of users would return to an app they had abandoned.

According to the World Health Organisation (2022), mHealth is the practice of medicine and public health that is helped by mobile technologies, including PDAs, cell phones, patient monitoring devices, and other wireless gadgets. Additionally, the value of these technologies, according to IQVIA, usually stems from their capacity to convey information via text messaging, the web, or the internet to monitor human health metrics continuously or to more clearly display health data. As the healthcare sector has embraced new technology in recent years, the market for mHealth apps has expanded quickly. The global market for mHealth apps was estimated to be worth $8 billion in 2018 and is projected to reach $111.1 billion by 2025. As illustrated in Figure 8, 16% of people use apps specifically to seek information about diseases. This indicates a significant demand for targeted health information through mobile applications, underscoring the importance of providing accessible and reliable resources for individuals managing specific health conditions (Colgan, 2019).

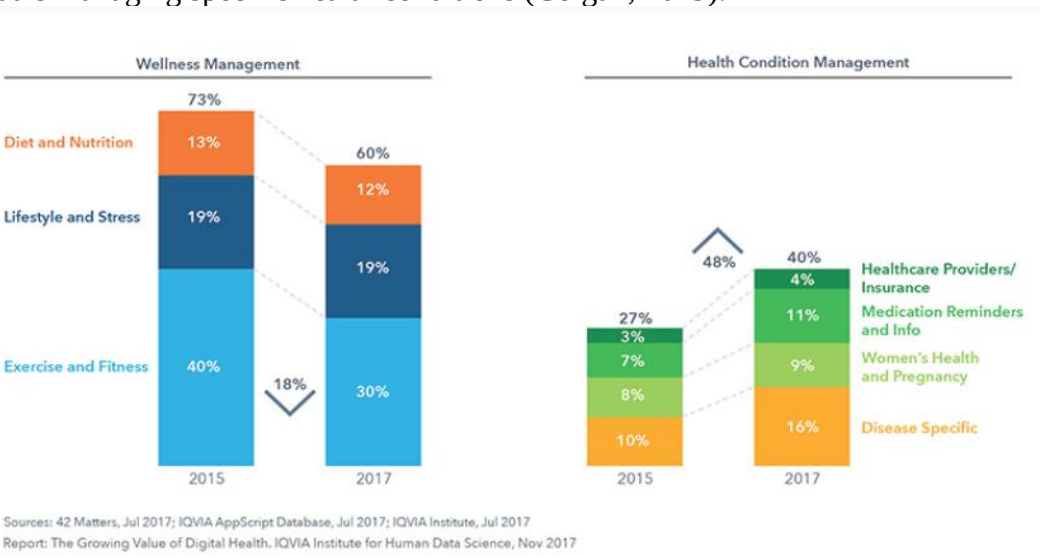


Figure 8: Digital Health Apps by Category (2017), (Colgan, 2019).

2.7 Android Operating System

Android Inc. was founded in 2003 in Palo Alto, California, with the primary goal of creating an operating system for digital cameras. However, as the demand for digital cameras declined, the company focused on developing a mobile phone operating system (Callaham, 2020). When Google acquired Android Inc. in 2005, the decision to make Android an open-source platform (based on the Linux kernel) was essential. This allowed third-party makers to utilise the operating system for free, which greatly appealed to phone manufacturers around the world. The flexibility of Android to function on a range of devices, from high-end smartphones to more affordable models that cost less than $100, would boost its adaptability and widespread adoption (Callaham, 2020).

The first beta version of Android (version 1.0) was made available to developers in November 2007, and the first Android smartphone, the T-Mobile G1 (HTC Dream), was released in

September 2008. This initial release contained popular Google apps including Maps, YouTube, a pre-version of the Chrome browser, and the Android Market (the precursor to the Google Play Store). Over time, Android has advanced rapidly the most recent significant update, Android 11, was made available in September 2020. Android is now the most popular mobile operating system in the world, overcoming Symbian, BlackBerry OS, Palm OS, webOS, and Windows Phone. Its open-source nature and compatibility with a variety of devices have been largely credited with its success, positioning it as the top platform in the global smartphone industry (Callaham, 2020).
With a 71.17% market share, Android is by far the most widely used mobile operating system worldwide, according to the most recent figures. iOS is the second most popular operating system with 28.33% of the market. The notion that there is minimal competition from other operating systems in the mobile OS sector, which is a duopoly, is supported by the fact that Android and iOS together account for nearly all mobile devices.

The smaller competitors are Linux (0.01%), Unknown OS (0.11%), KaiOS (common on feature phones), and Samsung's proprietary OS (0.32%). The enormous dominance of iOS and Android is highlighted by the fact that these minor OS options have a very small effect on the global market share (StatCounter Global Stats, 2024). Figure 9 visually represents the market share distribution, further illustrating how Android and iOS overshadow all other mobile operating systems combined.



Figure 9: Mobile Operating System Market Share Worldwide (StatCounter 2024).

With 58.67% of people selecting Android smartphones, Android has the biggest market share for mobile operating systems in Ireland as of October 2024. With 41.03% of the market, iOS comes in second, suggesting that Irish consumers' preferences for the two main operating systems are more evenly distributed than the global average, where Android leads by a larger margin. There is very little use of the other operating systems in Ireland. Only 0.01% of the market is made up of Windows, Series 40, and Linux, compared to 0.25% for Samsung's proprietary OS. This distribution indicates that, like the global market, the Irish market is primarily divided between iOS and Android, with little use of other operating systems (StatCounter Global Stats, 2024a). As can be seen on Figure 10, the market share distribution in Ireland emphasizes the dominance of Android and iOS in the region.



Figure 10:Mobile Operation System Market Shared in Ireland, (StatCounter Global Stats, 2024).

The technology industry underwent a transformation with the release of the Android operating system, which dominated the worldwide smartphone market and opened new prospects. But this quick uptake has also put the platform at serious risk from an unchecked and ongoing increase in malware that preys on Android, taking advantage of its open-source nature and widespread use (ResearchGate, 2024).

The global smartphone OS market share forecast for 2017-2023 is shown in Table 2.1. With a significant market share that increased from 85.1% in 2017 to 87.1% in 2023, Android is clearly the most popular mobile operating system. iOS, on the other hand, has a significantly smaller but steady market share, falling just from 14.7% in 2017 to 12.9% in 2023. Once strong rivals, other operating systems have all but vanished from the market, with a combined share of just 0.2% in 2017 and 0.0% starting in 2018. With Android solidly leading the worldwide smartphone market, this pattern highlights the efficient duopoly between the two operating systems.

Table 3: Worldwide Smartphone Shipment OS Market Share Forecast, 2017-2023 (ResearchGate, 2024).

Year	2017	2018	2019	2020	2021	2022	2023
Android	85.1%	85.1%	86.7%	86.6%	86.9%	87.0%	87.1%
iOS	14.7%	14.9%	13.3%	13.4%	13.1%	13.0%	12.9%
Others	0.2%	0.0%	0.0%	0.0%	0.0%	0.0%	0.0%
TOTAL	100.0%	100.0%	100.0%	100.0%	100.0%	100.0%	100.0%

Worldwide Smartphone Shipment OS Market Share Forecast [18]

The two major participants in the global mobile operating system market in 2024 are Android and iOS, which together represent more than 99% of all smartphone users globally (Buck, 2024). With a global market share of almost 70%, or about 4.2 billion devices, Android dominates the market, while iOS accounts for around 29.3%, or about 1.74 billion devices. In regions like Asia, Africa, and South America, where Android devices are more popular due to their price and large range of manufacturers, this global preference for Android is most noticeable. In contrast, iOS is the most popular operating system in Oceania, Europe, and North America. Regional Differences: iOS is the most widely used operating system in the US, with a 61% market share. While the UK market is more evenly divided between iOS and Android, similar trends are seen in Canada and several regions of Europe. With market shares of 95% and 75%, respectively, Android continues to hold a significant advantage in nations like China and India. In contrast to the premium pricing of iPhones in higher-income nations, the pricing strategies and market penetration of Android handsets in emerging economies are reflected in the glaring preference gap. (Statcounter GlobalStats, 2024).

App Store Competition: Apple's App Store makes a lot more money than Android's, even though Android has a larger user base. Consumers spent $21.2 billion in the App Store and $10.4 billion in the Google Play Store in the third quarter of 2022. Because of this disparity, iOS users have more purchasing power, which makes it a desirable platform for app developers that want to make money off their apps. With iOS accounting for 77% of all subscription revenue in the mobile app market, iOS consumers also tend to spend more on non-gaming and subscription-based apps (Buck, 2024).

Tablet Market Share: With 52% of the global tablet market, iOS has a little advantage over Android, which has 47%. Android tablets have gradually gained momentum, despite Apple's initial dominance in this market with the iPad. This is especially true when major smartphone makers like Samsung and Lenovo entered the tablet industry.

Historical Trends: Around 2012, Android surpassed iOS in terms of worldwide market share, and it has since maintained its position as the most popular mobile operating system. Android's global market share surpassed 70% by 2017, and it has remained there with minimal change. However, iOS is still more widely used in the US, where it has consistently outperformed Android in terms of market share since the early 2010s.

Implications for App Development: According to the statistics, launching on both iOS and Android is the best way for developers to reach the largest audience when choosing between

platforms. While iOS offers superior monetisation prospects, particularly for paid or subscription-based apps, Android gives access to a wider worldwide market. Businesses can reach almost 99% of the market by creating apps for both platforms, albeit doing so natively may result in higher expenses. Cross-platform app releases are now more accessible thanks to alternative solutions like MobiLoud, which enable businesses to turn webpages into iOS and Android apps at a reduced development cost (Buck, 2024).

By 2024, Android will have a 69.9% global market share among smartphone users, while iOS will have a 29.3% share. This notable disparity is indicative of Android's broad use, particularly in areas where consumers favour devices that are versatile and reasonably priced. In the meantime, iOS continues to keep a firm hold in regions like North America and some parts of Europe where consumers favour high-end handsets (Buck, 2024).

Figure 11 illustrates this distribution, highlighting the clear lead of Android over iOS in the global smartphone operating system market.

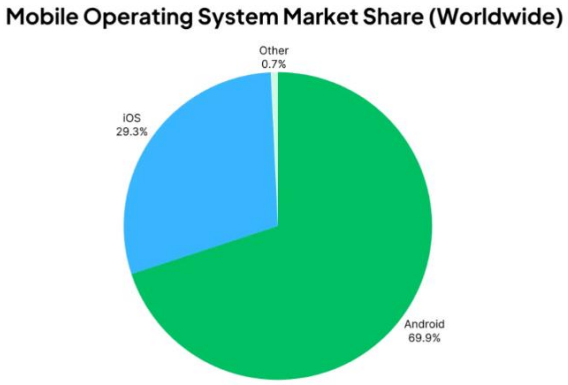


Figure 11: Global Android vs iOS Market Share, 2024 (Buck, 2024).

2.8 Android Versions and Statistics

Google created the Android operating system, which has become the most popular smartphone OS globally. With around 3 billion smartphones in use worldwide as of 2024, Android holds an astounding 80% market dominance. The Android app market brought in nearly $180 billion in 2023 alone, demonstrating its substantial economic influence and contribution to the development of the digital world. According to recent data, Android 11 has a 20.0% market share, whereas Android 13 has the largest market share of any Android version at 24.2%. This pattern shows that consumers are embracing newer OS versions more frequently, which is indicative of a good user reaction to Google's ongoing innovation and updates. Nearly 97% of the 2.67 million apps in the Google Play Store are free, demonstrating the platform's accessibility and variety. The fact that Android users spend an average of 3.5 hours a day on their devices shows how important the platform is to day-to-day living. Android is a flexible option for people all over the world because of its open-source nature, which supports a thriving ecosystem that includes not only smartphones but also tablets, wearables, and TVs. With a market share of more than 34%, Samsung is still the market leader for Android, followed by Xiaomi and Oppo. According to sales data from 2022, 1.57 billion Android smartphones were sold, indicating a steady increase in consumer adoption. Notably, Android is still quite dominant in emerging economies, with over 85% of the market in nations like Brazil, India, and Indonesia. There are still issues because 97% of smartphone malware attacks target Android devices. This emphasises how crucial security measures are to preserving user confidence and device integrity (Elad, 2024).

In summary, the Android operating system's economic contributions, user interaction data, and growth trajectory highlight its critical role in the mobile technology market. Maintaining user

pleasure and meeting changing market demands require regular upgrades and security improvements, especially with the advent of Android 13.
To make each update more enjoyable and memorable for consumers, the Android Operating System codenamed its versions after desserts. In alphabetical order, each iteration was named after a dessert, starting with Cupcake, Doughnut, Eclair, and ending with Pie. Each update felt friendlier and more distinctive, because of the dessert-themed naming. This custom persisted until Android 9 (Pie); Google began using a numerical naming approach with Android 10. Certain dessert titles were difficult to recognise or comprehend in many languages and cultures, therefore this alteration was made to make the naming simpler for a worldwide audience (Nucleus AI, 2024).
Figure 12 illustrates the different versions of Android, showcasing the progression of Android versions over time and marking version 10 as a significant point in the timeline.



Figure 12: Versions of Android (Nucleus AI, 2024).

The Android versions that are in use in Ireland as of October 2024 vary greatly. With a 34.9% market share, Android 14 is in the lead, followed by Android 13 with 13.92%. Significant shares are still held by older versions such as Android 8.0 Oreo (10.41%), Android 6.0 Marshmallow (10.31%), and Android 5.0 Lollipop (9.8%), as seeing in figure 13.



Figure 13: Mobile Android Version Market Share in Ireland - October 2024 (StatCounter Global Stats, 2024b).

This variation draws attention to the difficulties, faced by app developers that need to make sure their apps work with different Android versions to reach a large audience. It is crucial to modify app features for various Android versions, because older versions also provide higher security threats.

2.9 Case Studies on Similar Mobile Applications
Analysing current applications in the market is crucial when creating a mobile application that supports volunteers, cancer patients, and promotes wellbeing. Features, user interaction tactics, and opportunities for development are all aided by the current study. A comparison to the suggested for development "Rise Together" app is provided in the next section, along with Case Studies of noteworthy applications in this field.

2.9.1 Becca - Breast Cancer Support

The Becca app has over 10,000 downloads and is rated suitable for all ages. The app was first released on 5 May 2017, and its current version, 1.33.2, was last updated on 20 August 2024. Becca is offered by Breast Cancer Now, the largest breast cancer charity in the UK, which provides research and support for individuals affected by breast cancer. The app is free to download and use, requiring only an Internet connection to access its content. The Becca app is specifically designed for individuals living with or beyond primary breast cancer, providing them with a range of supportive resources and real-life stories from others who have experienced similar journeys. The app includes curated content, covering a variety of topics relevant to cancer recovery, such as managing side effects, coping with menopausal symptoms, dealing with fatigue, and guidance on diet, exercise, and body image. Each week, users gain access to new "cards" containing hints, tips, information, and blog posts from others who have been through breast cancer, providing insights and practical advice. Users can save their favorite cards for future reference, allowing them to revisit helpful information whenever needed. Additionally, Becca includes the option to reach out to specialist breast care nurses, giving users access to personalized support from professionals (BreastCancerNow.org).

When the application is first installed, users create an account (with email and password), the app shows few options to click and read more like the first feature is a card about stress and anxiety and an option “tap to discover”. Becca collates its information from various reputable sources, including qualified dietitians, nurses, and medical professionals, as well as contributions from the breast cancer community. The app also suggests podcasts, recipes, makeup tutorials, exercise routines, and articles to support users’ physical and emotional health. Testimonials within the app highlight its impact, with users like Kim Hulme stating, “Becca has really helped me to start living in the moment,” and July Honey sharing, “Becca directed me to relevant and trustworthy sites … with good advice and ideas to help me cope.” On the main interface, the user can start learning the basics of mental health, Inspirations diet and exercise, family and relationship, (please, see Figure 14 & 15). The links shows tips for exercising like walking, yoga, Pilates, tai hi, dancing it includes breathing exercises and body and mind connection to teach how they can improve bad feelings.

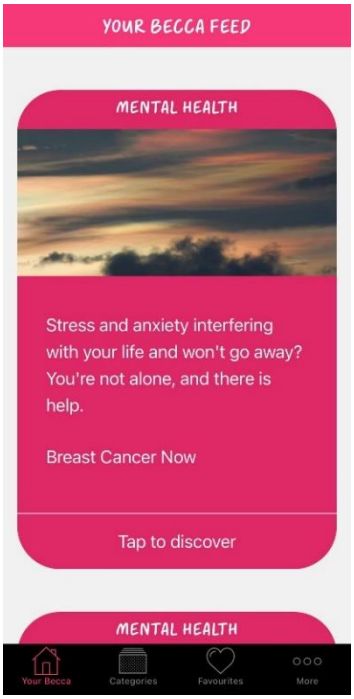


Figure 14: The Becca app’s main interface. (Google Play, 2024a)

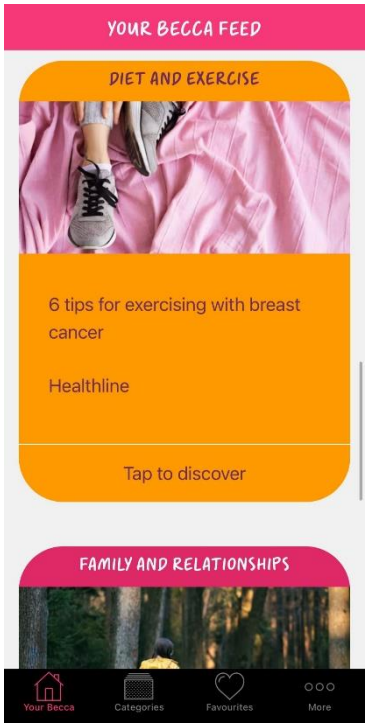


Figure 15: The Becca app's main interface. (Google Play, 2024a)

However, *Becca* lacks interactive community features, such as forums or chat options for direct peer-to-peer support, and it does not offer a volunteer network to provide additional practical or emotional support. Users, who seek community connection, may find this a limitation, as the app primarily focuses on delivering informative content rather than fostering interpersonal engagement. Overall, *Becca* app serves as a reliable resource for individuals, looking to rebuild their lives post-treatment, offering consistent, trusted information and support focused on health, lifestyle, and managing the long-term effects of breast cancer.

2.9.2 MySunrise

The app has over 3,000 downloads. The app was first released in 2018 as a pilot at the *Royal Cornwall Hospital* and has since expanded to all five NHS Cancer Centres in Devon and Cornwall, with support from the *Peninsula Cancer Alliance*. The current version was last updated on 27 April 2023, and the app is free to download and use, requiring only an internet connection for full functionality. *MySunrise* app is designed to support cancer patients and their families throughout their treatment journey, providing them with detailed information, tracking tools, and resources to manage their health and wellbeing. The app includes comprehensive resources related to cancer treatment, such as explanations of treatment options, potential side effects, supportive medications, and guidance on navigating treatment locations within cancer centres. Users can track their wellbeing at every stage of their cancer journey, with tools and advice to help them manage symptoms and side effects effectively. The app also includes information on when to reach out to specialists for further support, helping patients feel more in control of their treatment experience. Additionally, MySunrise offers multilingual support, with resources available in Polish, Cantonese, Arabic, and Gujarati, and video subtitles available in Mandarin, Polish, Brazilian Portuguese, and French. British Sign Language (BSL) videos are also included, making the app accessible to a wider range of users (MySunrise.co.uk).

Upon first installation, users can explore MySunrise's categorized content, which allows them to quickly access relevant information based on their treatment needs. The app collates information from trusted sources, including NHS cancer centers, healthcare professionals, and

the Peninsula Cancer Alliance. This ensures that users receive accurate, up-to-date information to support them during their cancer treatment. The MySunrise app is compatible with a variety of devices. For Apple, it requires iOS 13.0 or later for iPhones, iPadOS 13.0 or later for iPads, and macOS 11.0 or later for Macs with Apple M1 chips or newer. For Android, it supports any device running Android 5.0 or later. Currently, *MySunrise* is not available on Windows computers.

When the application is first installed, users can select the location and check the places according with the disease and automatically the app will redirect for the main page according with the type of patient without need any information about email or password, the app shows few options to click and read more like Cancer services (centres and teams), Treatment Pathway (Diagnosis), Symptom Support and Patient Feedback also a field to Contact Centre.

On the main interface, the user can see all the options included on the app and after click find out more (please, see Figure 16).



Figure 16: The MySunrise app's main interface. (Google Play, 2024c)

The vision behind MySunrise is to empower patients by providing them with the information, support, and resources they need to navigate their treatment with confidence. By addressing both the practical and emotional needs of cancer patients, the app aims to reduce anxiety and give patients a greater sense of control. However, MySunrise lacks interactive community features, such as forums or peer-to-peer support options, which could help patients connect with others going through similar experiences. Additionally, the app does not offer a volunteer network for additional support. These limitations mean that users seeking a more interactive, community-based experience may find this app less suited to their needs. Overall, the My Sunrise app is a valuable resource for cancer patients looking for structured, reliable information and tools to help manage their treatment and wellbeing. Its multilingual and accessible design, coupled with a focus on comprehensive treatment information and wellbeing tracking, makes it a strong support tool for navigating the cancer journey.

2.9.3 BELONG Beating Cancer Together.

Belong Beating Cancer Together app has over 100,000 downloads and is rated 12+. The app was first released in 2016 and has since become the world's largest social and professional network for cancer patients and caregivers. The app is free to download and use, offering a wide range of support and navigation tools for individuals affected by cancer, and it requires an internet connection for full functionality. The Belong Beating Cancer Together app is designed to provide comprehensive support for cancer patients, offering a combination of social, informational, and organizational resources to help them manage their treatment journey. The app includes support groups where users can connect with other patients and caregivers, access to medical professionals for advice, and tools to manage their medical records, such as a digital binder to store and organize important documents. One of the app's standout features is *Dave*, the AI Cancer Mentor, an artificial intelligence-powered conversational mentor that provides personalized, oncology-specific information. *Dave* offers empathetic and precise responses to cancer-related questions, supporting users with relevant, real-time advice throughout their treatment journey. This AI mentorship is designed to enhance patient education and engagement, helping users feel informed and always supported.
Additionally, the *Belong* app provides a clinical trial matching service called *Tara*, which uses AI to match patients with relevant clinical trials based on their medical profile. This feature helps patients stay informed about potential new treatment options and provides them with access to emerging therapies. Upon first installation, users can join cancer-specific support groups where they can connect anonymously with peers and discuss their experiences in a private, supportive environment. The app also includes access to medical professionals who can provide guidance and answer questions, creating a blend of community and professional support that helps users feel connected and well-informed.
The app's digital binder allows users to keep track of their medical documents and records in one place, making it easier to organize and access important health information. The app also offers hyper-personalized content recommendations based on the user's preferences and treatment stage, using AI and machine learning to tailor the experience to everyone's needs. The Belong Beating Cancer Together app adheres to strict privacy and data protection standards, including HIPAA, GDPR, and ISO certifications, ensuring that users' data is secure, and their privacy is maintained. The vision behind Belong, the company behind the app, is to improve the quality of life and quality of care for cancer patients worldwide. By leveraging technology, data, and AI, the app aims to empower patients and provide them with the support they need to navigate their healthcare journey with confidence.
After downloading the app and completing the Signup Form with details about the cancer type and subtype, the app will guide the user to a personalized *Home* screen. This *Home* screen will provide tailored content, resources, and community connections, that match a specific interests and needs based on the cancer type (please, see Figure 17 for SignUp screen & Figure 18).

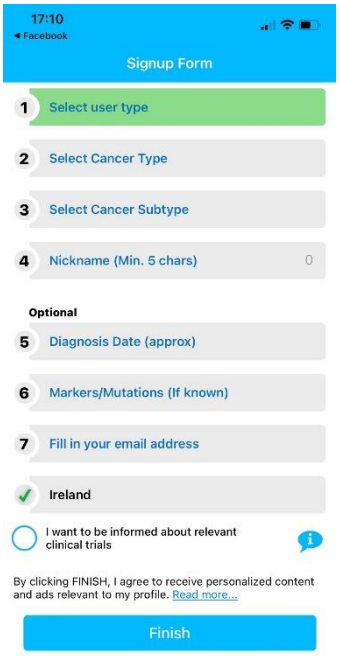


Figure 17: Belong Beating Cancer Together. app's SignUp Screen. (Google Play, 2024b)

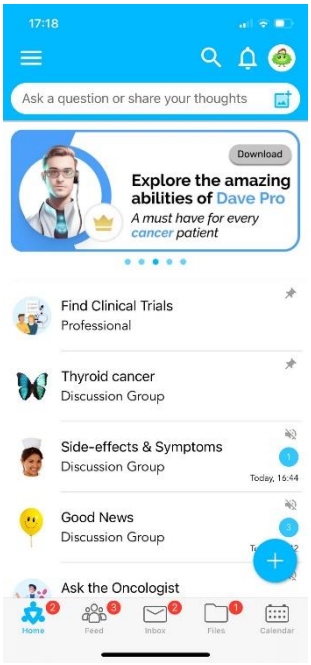


Figure 18: Belong Beating Cancer Together. app's main interface. (Google Play, 2024b)

Belong app offers a robust community and a wide number of support tools, but consumers who want a more straightforward, efficient app experience, can find it overwhelming. Although it may need some initial orientation to use properly, the numerous data-driven functionalities and the incorporation of powerful AI elements make it extremely instructive. All things considered, the *Belong Beating Cancer Together* app is an effective resource for cancer patients and their careers, providing a special blend of access to clinical trials, community support, and AI mentoring. There is no help per support or specific volunteer jobs.

2.10 The Proposed for Development Application

The existing applications in the cancer support space, such as *Becca*, *MySunrise*, and *Belong Beating Cancer Together*, each adopt unique approaches to patient engagement, information delivery, and community support. However, they have certain limitations in terms of personalization, interactivity, and sustained motivation, which the proposed Rise Together app seeks to address.

Breast Cancer Now created the *Becca* app, which is very educational and offers breast cancer survivors trustworthy information and helpful guidance. It does not, however, offer an organised path for people to follow, and does not have participatory community aspects. Although Becca provides articles and advice on subjects, like body image, fitness, and managing side effects, the app mainly depends on users finding pertinent information on their own, which may eventually lower engagement, because there isn't a guided, interactive experience to keep users motivated.

MySunrise app serves mostly as a reference library, rather than a dynamic, participatory platform, despite its value in providing thorough treatment-related information and wellness tracking. Its emphasis on structured information is helpful for monitoring side effects and comprehending treatment paths, but it lacks peer-to-peer interaction and community-driven elements. Its capacity to provide emotional support, which is frequently a vital component of the cancer healing process, may be limited by this lack of interaction.

Conversely, *Belong - Beating Cancer Together* is a powerful community-based software, that offers clinical trial matching, structured support groups, and AI-powered assistance (like *Dave*, the AI Cancer Mentor). Although it is excellent at connecting patients and giving them personalised, up-to-date information, some users may find its large feature set and data-centric design overwhelming. Furthermore, the absence of gamification, or motivational challenges in the app may make it more difficult to maintain users' continuous engagement, because the experience may come across as passive, rather than actively promoting participation.

The ***Rise Together*** app is proposed to be developed as a secure, compassionate platform, specifically designed to support cancer patients by connecting them with carefully vetted cancer survivors, who have undergone similar experiences, as volunteering mentors. Currently focused on cancer patients in Ireland, the app is intended to provide both, emotional and practical support, empowering users, and reducing feelings of isolation throughout the treatment and recovery process. The *Rise Together* app is to be developed as an *Android* application, with a focus on data security and privacy, ensuring that users' sensitive information is always protected.

The primary goal of the *Rise Together* app is to facilitate meaningful connections between cancer patients and volunteering mentors, who can assist not only with emotional support, but also with everyday tasks. Through a cyclical support model, survivors can "help it forward" by mentoring others, creating a continuous loop of assistance and encouragement within the cancer community. This approach fosters a supportive environment, that enhances the well-being of both patients and volunteers, as survivors find new purpose in giving back.

Basically, the main features is practical assistance from vetted volunteers, unlike other cancer support apps that primarily offer informational or community features. *Rise Together* app is unique in *providing practical, real-life support*. Volunteers, who are vetted by Garda, and screened and approved by healthcare professionals, offer help with daily physical tasks, such as transportation, meal preparation and other household chores, and gardening. This hands-on assistance helps alleviate some of the physical burdens, that cancer patients face during treatment, enhancing their quality of life.

Emotional Support and Mentorship: In addition to practical help, *Rise Together* fosters emotional support through peer mentorship. Patients can engage in meaningful conversations

with volunteers, who understand their journey, offering advice on coping with the complexities of cancer treatment. The app intends to include private chat groups, where patients can connect with others, facing similar challenges, sharing experiences and encouraging each other.
Doctor-Guided Volunteer Approval: To ensure a safe and reliable support system, doctors play an active role in verifying and approving volunteers on the platform. Doctors register patients and carefully select volunteers, based on their suitability for providing specific assistance as needed. This adds a layer of security and trust, as doctors monitor interactions, to ensure that patients receive appropriate and beneficial support.
Mental Health Resources: Recognizing the importance of mental health in the recovery process, *Rise Together* intends to include access to mindfulness resources, such as meditation exercises and audio therapy sessions. These resources will be designed to help patients manage stress, anxiety, and other emotional challenges, associated with cancer treatment, contributing to a more holistic approach to well-being.
Feedback and Continuous Improvement: After each interaction, patients are prompted to rate their experience and provide feedback on the volunteer's support. This feedback loop ensures continuous improvement in the quality of care, provided by volunteers, while also helping doctors monitor the effectiveness of the support being offered.

User Interfaces
Doctor Interface: Doctors are securely registered by System administrators and can access the platform to approve and oversee volunteers. They register both Patients, and Volunteers offline, to prevent any fraud with online impersonation. They can monitor volunteer tasks to ensure suitability and send important health-related updates or reminders to both patients and volunteers, maintaining an active role in the support process.
Volunteer Interface: Volunteers, after secure login, can create own profiles, indicating the types of support they can offer, such as transportation or companionship, as well as their weekly availability. They can view patient requests and select tasks they are able to assist with, confirming their participation through the app. After completing each task, volunteers receive feedback from patients, helping them improve their service.
Patient Interface: Patients, after secure login, can create own profiles, including relevant medical information and personal preferences for assistance tasks they need. They can browse volunteer profiles and select those best suited to their needs. Patients can request tasks, such as transportation, or home assistance, from a chosen volunteer, receiving notifications as the volunteer confirms or reject request, based on their availability. Patients can rate and provide feedback on their experience after each interaction, ensuring accountability and continuous improvement.
Privacy and Security: Rise Together app prioritizes data security and privacy, employing encryption and adhering to strict data protection regulations, to safeguard users' sensitive medical and personal information. Only authorized users, such as patients, verified doctors, and approved and vetted volunteers, can access specific parts of the app, ensuring that all interactions occur within a secure and trusted environment.
Comparison to Existing Applications: Compared to existing cancer support apps, like Becca, MySunrise, and Belong Beating Cancer Together, the *Rise Together* app provides a unique combination of practical and emotional support. While Becca and MySunrise focus primarily on information and resources, and Belong emphasizes community and AI-driven support, Rise Together fills a critical gap by offering hands-on, real-life assistance, along with peer mentorship. The involvement of doctors in volunteer approval adds a level of reliability and personalization, that is not available in other apps, ensuring that users receive the support they need in a safe and controlled environment.

In summary, the *Rise Together* app is a comprehensive support platform that addresses the unique needs of cancer patients through a secure and structured system. By providing both practical help from vetted volunteers, and emotional support through peer mentorship, *Rise Together* empowers patients to navigate the challenges of cancer with confidence and reduces the burden of isolation. The app's cyclical model of support, in which survivors mentor others, creates a continuous loop of compassion and assistance, fostering a strong community of resilience and solidarity. *Rise Together* stands out as an essential tool for cancer support, offering a holistic approach that combines hands-on assistance, emotional care, and secure data management to improve the overall well-being of its users.

2.11 Chapter Summary

Many organisations have prioritised mental and emotional support for people, since COVID-19 pandemic, highlighting the importance of comprehensive well-being. As a result, platforms that prioritise physical assistance, mental health, and social support have grown in popularity, especially among vulnerable groups, like cancer patients. The proposed for development *Rise Together* app seeks to address the special needs of cancer patients, by creating a secure, nurturing environment, where they can engage with volunteering mentors, who already had similar experiences.

Rise Together is a special resource for cancer sufferers, since it blends emotional support with practical, hands-on help, unlike other support platforms that mostly concentrate on information or online community forums. Verified volunteers can offer crucial practical assistance, such as transportation and meal preparation, as well as emotional support through collaborations. *Rise Together* can lessen loneliness and offer all-encompassing assistance that enhances its members' mental and physical health, because of this dual strategy. A well-structured and reliable system for cancer support will be offered by the platform's essential features, which include secure communication, privacy-focused design, and user-friendly interfaces for patients, physicians, and volunteers.

Furthermore, the cyclical mentorship approach, in which survivors support others in return, fosters a never-ending cycle of resilience and support among cancer patients. *Rise Together* seeks to enable cancer patients to face their treatment path with more confidence and support by emphasising safe, caring peer mentorship and useful volunteer help.

Next chapter will discuss the Analysis Phase of the application's Development process.

Chapter 3 Project Application Analysis Phase

Discussed in the previous chapters was thorough research, conducted to understand the feasibility and importance of developing a mobile application to connect on one platform cancer patients undergoing treatment with vetted cancer survivors. The findings emphasized that such a platform is not only essential but is critical to help improving the quality of life for cancer patients by addressing a wide range of challenges, including emotional, logistical, and practical support throughout their treatment journey. Furthermore, the platform must prioritize robust security measures to ensure the privacy of every user during such vulnerable period. Cancer patients face intensified emotional and physical challenges, making it imperative that the application provides a secure and trustworthy environment, where they can seek and receive help without fear of breaches of their private information. By addressing their needs of support and safeguarding their personal data, the platform could truly empower patients and promote a sense of stability and dignity during a challenging chapter in their lives.
Given the application's sensitive context and the diverse needs of its users, both patients and volunteering survivors, it is crucial to adopt a development methodology that emphasizes adaptability, usability, and stakeholder involvement. This chapter explores widely used Software Development methodologies, analyses their advantages and disadvantages, and recommends the most suitable one for the project application. This decision will be based on the application’s objectives, such as ease of use, accessibility, security, and scalability.

3.1 Object Oriented Environment

Instead of concentrating only on functions and logic, the popular programming paradigm, known as Object-Oriented Programming (OOP), arranges software design around objects and data. According to this paradigm, an entity having distinct characteristics and behaviours is called an object. OOP focusses on the objects that developers want to work with, rather than the logic needed to handle data. Large, complicated software systems, that need frequent upgrades or maintenance, simulation programs, or mobile applications, are particularly well-suited for this strategy (TechTarget, 2024)
OOP's modular design is one of its main advantages; it allows projects to be broken down into smaller, more manageable groupings, which promotes collaborative development. Efficiency, reusability, and scalability are further benefits. Data modelling is the first step of OOP, during which all the objects a developer wishes to work with are located and their connections established. These objects could be abstract entities, like widgets or minor software components, or they could be real-world items, like a person with name and address. Classes, which act as templates that specify the characteristics and actions (methods) of objects, are the cornerstone of Object-Oriented Programming. The behaviour of objects is determined by certain logic sequences, called methods. Messages are clearly defined interfaces, through which objects communicate with one another. A *human* individual, denoted by attributes, like *name* and *address*, could be represented by an instance of the class Human, such as Jane Doe, residing at a given address in this example (please, see Figure 3.1). The object in this example is a human being, with name, address, and other characteristics (TechTarget, 2024).

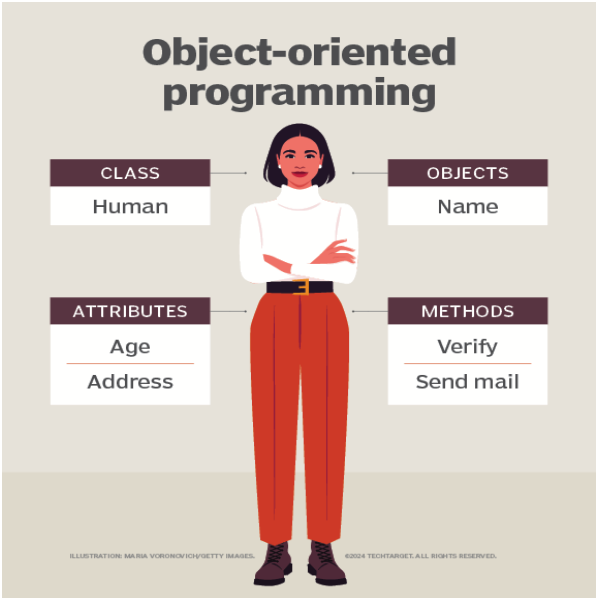


Figure 19: Classes and Objects in Object Oriented Programming (TechTarget, 2024).

OOP provides several important advantages, by emphasising objects and their interactions, and includes increased *modularity*, better security through *encapsulation*, and quicker development time through *code reuse* and *inheritance*. OOP will continue to be a fundamental component of contemporary Software Development, thanks to these characteristics.

3.1.1 Encapsulation

One of the core ideas of Object-Oriented Programming is *encapsulation*, which is the process of combining an object's *methods* (functions) and *attributes* (data) into a single unit, while limiting access to its internal workings. This idea, where only specific information is made available via a public interface, is frequently referred to as data concealing, or *encapsulation*. Only *public* methods allow external entities to communicate with an object; its internal implementation and state are kept *private* within its class. Private information, such as a student's name and grade, for example, cannot be accessed or changed directly from outside the classroom. Instead, regulated access is offered by functions like setGrade() and getName(). This keeps the object safe and guards against accidental modifications, or corrupted data. Because *encapsulation* separates an object's internal operations from its external interactions, it makes troubleshooting and maintenance easier (TechTarget, 2024).

3.1.2 Inheritance

Another fundamental component of OOP is *inheritance*, which enables a hierarchy to be formed by allowing one class to reuse the properties and functions of another. The class being inherited from, is known as the *superclass* (or *parent* class), while the class that inherits, is known as the *subclass* (or *child* class). This reduces redundancy and increases development efficiency, by allowing *reuse* of existing code. An “is a” relationship is represented by inheritance. For example, both dogs and cats are considered animals. Both Dog and Cat as subclasses inherit common characteristics and behaviours of the Animal superclass, such as blink() or eating(), however each of them is having their own specific and distinct characteristics, such as a bark() method for Dog subclass, or a meow() method for Cat subclass. This principle supports the creation of subclasses, that extend or override the behavior of their superclasses. For example, a Car class can serve as a superclass for a SportsCar subclass. While

the SportsCar inherits attributes, like speed and methods like drive () from the Car superclass, it can also introduce new features, such as a turboBoost () method. *Inheritance* enhances code readability, minimizes development time, and ensures consistent logic across related objects. However, it requires careful data analysis to avoid issues, such as fragile base classes, where changes to a superclass inadvertently affect its subclasses (TechTarget, 2024).

3.1.3 Polymorphism

Objects can assume various forms because of property, known as *polymorphism*, which comes from the Greek words "poly" (many) and "morphos" (forms). This idea permits a method or interface to act differently, based on the object that invokes it. Code is made simpler by *polymorphism*, which allows a single interface to handle many kinds of objects.
There are two types of polymorphism*:*
Static binding, or *compile-time polymorphism*, is accomplished by *method overloading*, in which several methods inside a class have the same name but distinct argument lists. For instance, the add() function in a Calculator class might be able to handle two floating-point numbers (add(float a, float b)) or two integers (add(int a, int b)).
Method overriding, in which a subclass offers a particular implementation for a method, already defined in its superclass, is how *run-time polymorphism* (dynamic binding) is accomplished. The Animal superclass, for instance, might define a method called speak(). This function might be overridden by a Dog subclass to return "barking," and by a Cat subclass to return "meowing." In software development, *polymorphism* makes flexibility and scalability possible. It eliminates the need for repeated coding, streamlines system design, and facilitates the expansion and maintenance of programs by enabling a single interface to manage several behaviours (TechTarget, 2024).

3.1.4 Abstraction

The goal of *abstraction* is to obscure unnecessary implementation details, while merely exposing an object's essential properties. This facilitates object interaction and allows developers to focus on high-level design, by relieving them of the burden of low-level details. A Car object, for example, provides methods like start() and stop(), but the user cannot see how the engine operates within. In the same way, in a banking application, an Account object can expose methods like deposit() and withdraw(), while hiding the underlying logic that modifies the account's balance. It is common practice to utilise abstract classes or interfaces to implement abstraction. An abstract class defines methods that subclasses can implement, but it cannot be created on its own. For example, the abstract class Shape may implement a method named compute Area() differently than subclasses like Circle and Rectangle. This idea makes it simpler to update or change a system, without unduly affecting other components and enhances code maintainability (TechTarget, 2024).

3.1.5 Benefits of OOP

Object-Oriented Programming provides several advantages that make it a popular choice for modern software development (TechTarget, 2024):

- ***Modularity***: By allowing objects to be self-contained, *encapsulation* makes debugging and cooperative development easier.
- ***Reusability***: By minimising repetition and conserving time, *inheritance* enables developers to reuse pre-existing code.
- ***Scalability***: Software can more easily be modified to meet evolving needs, thanks to *polymorphism* and *abstraction*.

- ***Flexibility***: A single function or interface can interact with numerous object types because of *polymorphism*.
- ***Security***: *Abstraction* and *encapsulation* shield private information and lower the possibility of inadvertent mistakes.
- ***Maintenance Ease***: Updates and revisions are made easier by *modular* and *reusable* code.
- ***Efficiency***: OOP lowers development costs and boosts productivity by using *libraries* and *reusing* existing code.

3.1.6 Criticism of OOP

OOP has its drawbacks, despite its obvious benefits. A prevalent criticism of OOP is that it overemphasises data, while undervaluing computation and methods. Inheritance can also result in problems, such as fragile base classes, where modifications to a superclass have unexpected effects on its subclasses. Alternative methods, that could be more suited for tasks are provided by other programming paradigms, such as declarative, modular, and functional programming. OOP is nevertheless a popular and flexible programming paradigm, despite these objections, especially for intricate, team-based, and scalable software projects. Developers may produce reliable, modular, and effective programs that are simpler to create, maintain, and grow over time by utilising the concepts of encapsulation, inheritance, polymorphism, and abstraction (TechTarget, 2024).

3.2 Development Methodologies

To determine the most suitable methodology, organizations must consider multiple factors and weigh the advantages and disadvantages of each approach. Consulting experienced advisors, who understand the strengths and limitations of different methodologies, can also help. An organised framework, created to plan, coordinate, and oversee the Software Development process, is known as a *Software Development Methodology*. Its main goal is to minimise risks and guarantee that the software is delivered in accordance with the project's specifications, within the expected timeframe and budget. By outlining precise phases, roles, responsibilities, activities, communication standards, and deliverables, each methodology offers a unique way to manage the Software Development Life Cycle (SDLC). Because of this diversity, companies can choose the approach that best suits their unique requirements and project objectives. In Software Development, industry best practices and knowledge can be used to recommend the most effective approach for a given initiative (Itransition, 2024).
Agile has been widely adopted since its inception in 2001 because of its adaptability to change and capacity to provide clients with ongoing value. The 17th State of Agile Report from Digital.ai states that 71% of businesses currently use Agile in their software development procedures. Sequential approaches, however, are still useful in fields that demand a methodical and predictable approach. These are especially helpful in situations where iterative rollouts are not feasible because of physical infrastructure constraints, or where the costs and hazards of uncertainty or rework are intolerable. It's interesting to note that technological developments like artificial intelligence (AI)-driven data simulations and digital twins that mimic a software's operating environment are making it possible to use Agile frameworks in sectors that have historically relied on sequential approaches (Itransition, 2024).

3.2.1 Waterfall Model

One of the oldest and most well-known approaches to Software Development is the *Waterfall* model. It proceeds in discrete stages, each of which must be finished before going on to the next, in a linear-sequential fashion. For small to medium-sized projects, this methodical approach offers improved control and simpler management. Its rigor also guarantees accurate documentation and helps in adhering to defined budgets and schedules (Bocij, *et al*., 2008).
The Waterfall methodology is based on the Systems Development Life Cycle (SDLC), which typically includes the following stages (please, see Figure 3.2):

- Initiation
- Feasibility Study
- Requirements Analysis
- System Design
- System Build
- Implementation

Itransition (2024) asserts that the scope, needs, and constraints of a project should guide the selection of Software Development approaches. The Waterfall development model follows a linear workflow, going through several stages with distinct goals from top to bottom. Since it is forbidden to alter or add to the project scope, once the development process has started, all requirements must be gathered during the planning phase. Since developers must complete each step before moving on to the next, the software is implemented as result at the end of the project (Itransition, 2024).

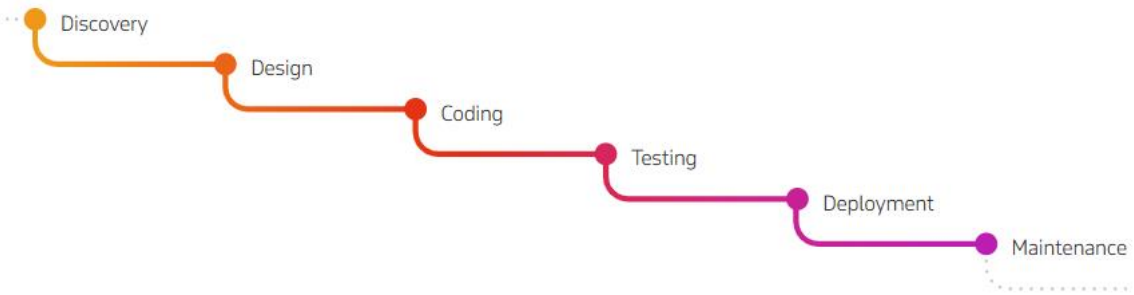


Figure 20: Waterfall Development Model (Itransition, 2024).

Waterfall Development Model is ideal for projects with clear, constant goals and specifications and projects where finance and schedule management are essential. Nash (2003) asserts that the conventional *Waterfall* approach is not widely utilised nowadays, because it performs poorly when specifications change quickly and frequently. But throughout the project, the level of risk largely stays constant, until it hits the testing stage, during which issues could be found. Since it is challenging to obtain all requirements at the outset, the Waterfall approach is not advised for complicated and lengthy projects. It is also quite rigid and poorly manages unforeseen risks. According to Mullins (2000), extending systems to address new problems accounted for 70% of all modifications made to software systems overall, with 42% of these adjustments being due to changes in user needs.

3.2.2 Incremental/Iterative Methodology

With the *Incremental/Iterative* model, the development process is broken down into smaller, autonomous parts, known as *increments*. A particular set of features, which represents a subset of the system's overall capabilities, is the main goal of each increment. Depending on the size of the project, these iterations may continue for a few weeks or months. When the initial increment of the program is delivered, end users have access to a fully functional part of it, and other increments are produced until the entire system is completed. Early problem diagnosis made possible by this method, greatly lowers the possibility of expensive mistakes later. But for the incremental model to be effective, careful and detailed planning of each increment is essential, to ensure consistency and value for the end-users (please, see Figure 21) (Itransition, 2024).

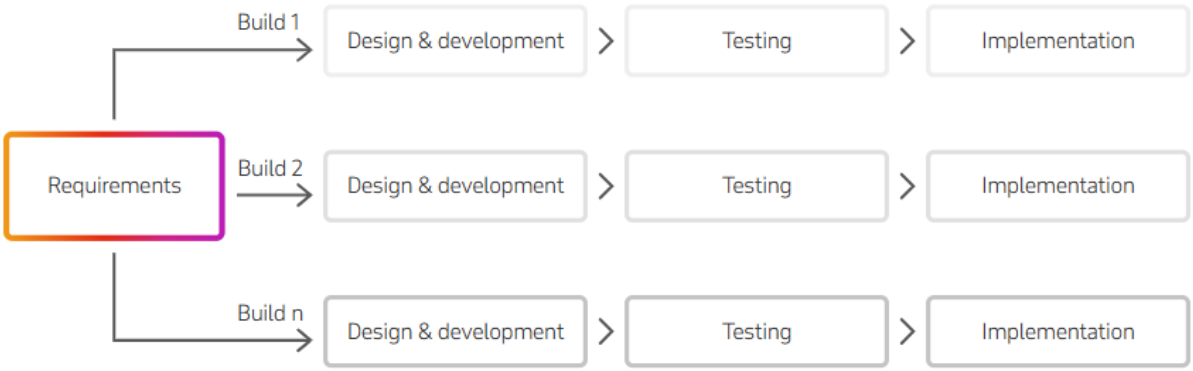


Figure 21: Phases of the Incremental model (Itransition, 2024)

Incremental models are suitable for Projects with lengthy development timelines, that prioritize delivering value as early as possible, as well as for situations, where the product needs to be launched quickly to meet market demands, or for systems that allow for easy modularization.

3.2.3 Boehm's Spiral Methodology

Compared to the conventional *Waterfall* paradigm, the *Spiral* model offers more flexibility in responding to shifting requirements, by combining *sequential* and *iterative* approaches. Each cycle, or *spiral* in the development process is designed to gradually improve and refine the final product. Each cycle, which might span several months, starts with *planning* and *objective identification* for the project. A phase of *risk analysis* and *management* follows which proactively addresses potential errors and obstacles, to assist minimise them. Software is developed in each cycle, using the specifications acquired in the previous iteration, ensuring that development is continuously aligned with evolving needs. The program is *assessed* at the conclusion of each cycle, to see if it satisfies the specified requirements. Stakeholders can use this review to determine whether to move on to the next iteration, change the project's course, or even end it. Because the *Spiral* model is iterative, it can be challenging to estimate how many cycles will be needed to complete the project, which creates uncertainty about the timeframe and budget. The Spiral approach is best suited for circumstances when flexibility is essential, such as large, complex projects with changing requirements. Additionally, it works

effectively for projects where strict *risk management* is essential to reduce any problems during the system development (please, see Figure 22) (Itransition, 2024).

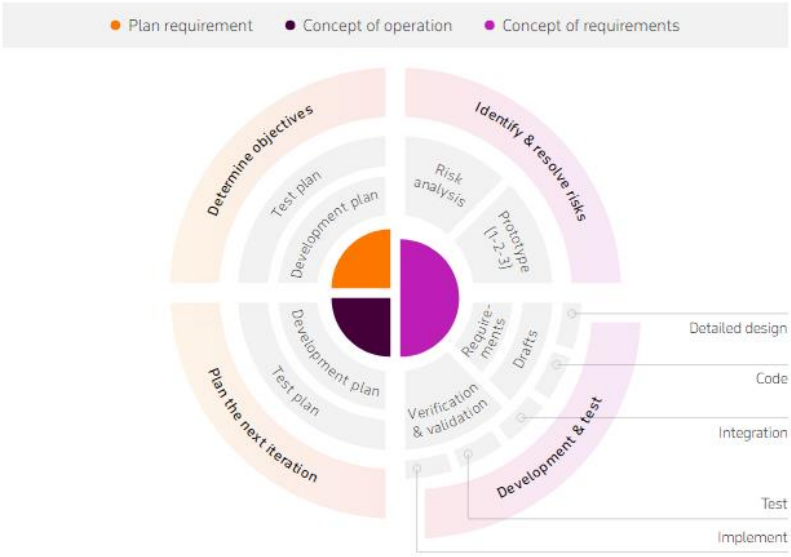


Figure 22: Software Development Life Cycle in the Spiral model (Itransition, 2024).

Spiral model is suitable for complex and large-scale projects, that are subject to uncertainty or evolving requirements, and for projects where effective risk management is critical to ensure the successful delivery of the product.

3.2.4 Rapid Application Development (RAD)

Short development cycles and the timely delivery of usable software are the main goals of the *Rapid Application Development* (RAD) paradigm. It focusses on iterative development, in which the project is divided into smaller, more manageable modules, that are created simultaneously by many teams. This method of parallel development speeds up the entire process, enabling quicker outcomes. The way that RAD collects requirements is one of its main characteristics. The model gathers feedback through focus groups, prototyping, and early user testing, rather than waiting for a complete set of specifications. Because of the rapid client input provided by this iterative method, developers can make necessary adjustments to the project before the major development phase starts. Additionally, RAD encourages the system-wide reuse of software components, which can expedite development and improve consistency. To make the most of RAD, careful upfront planning is required, including precise scheduling, coordination, and detailed requirements elaboration. The involvement of a highly skilled team of analysts and architects is critical to ensure that the project's scope, dependencies, and design are well-defined from the start. Projects where requirements can be determined early on, enabling effective planning and execution, are particularly well-suited for the RAD paradigm. It is also perfect for modular systems, which enable rapid iteration of individual components and parallel development (please, see Figure 23) (Itransition, 2024).

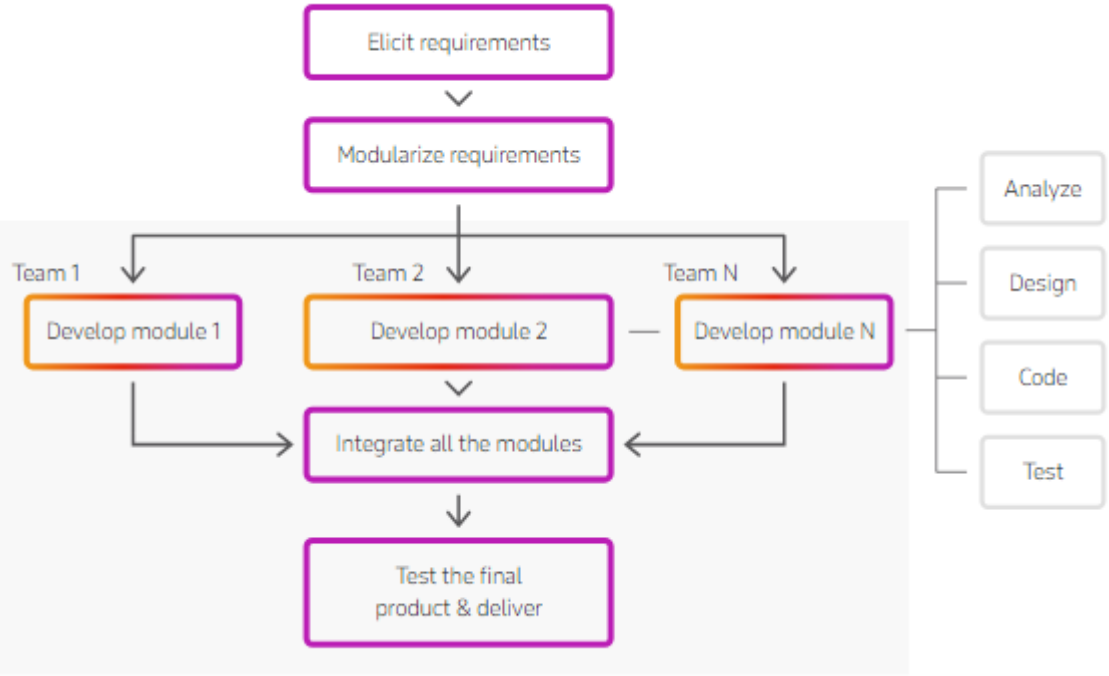


Figure 23:Process model for Rapid Application Development (Itransition, 2024).

RAD is suitable for Projects, where requirements can be clearly defined upfront and systems that can be modularized for parallel development and rapid iteration.

3.2.5 Agile Methodology

The *Agile* methodology is a contemporary approach to Software Development and Project Management, that places an emphasis on adaptability, teamwork, and client satisfaction. *Agile*, in contrast to traditional approaches, places an emphasis on incremental and iterative development, enabling teams to swiftly deliver small, functional project components and adjust to changing requirements. *Agile* is used by well-known businesses like Google, Facebook, and Amazon, because of its flexibility and emphasis on ongoing development. By dividing projects into smaller, more manageable components, known as iterations or *sprints*, the methodology takes an iterative approach. This makes it possible to get ongoing feedback from customers to make sure the finished product satisfies changing needs. The Agile Manifesto, developed in 2001, is a key component of Agile. It contains twelve principles and four fundamental values that prioritise people, interactions, practical solutions, and adaptability over inflexible procedures (Beck, 2001). Furthermore, Agile frameworks, such as *Extreme Programming* (XP), *Scrum*, *Kanban*, and *Lean*, offer defined methods for applying the concept in a variety of contexts. Projects with customer-focused goals, ambiguous or quickly changing requirements, and a need for creativity and adaptability, are best suited for an *Agile* methodology. Small to medium-sized teams, working in hectic settings where constant iteration and cooperation are essential, are the ideal candidates for it (please, see Figure 24) (GeeksforGeeks, 2024).

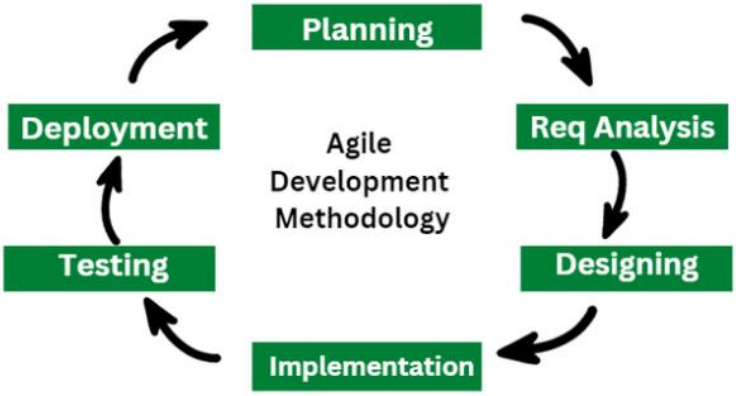


Figure 24: Agile development methodology (GeeksforGeeks, 2024)

Agile methodology's advantages include shorter time-to-market through the quick delivery of functional components, flexibility and adaptation to changing needs, and frequent customer feedback that guarantees user happiness (GeeksforGeeks, 2024). Additionally, Agile promotes a culture of continuous improvement, open communication, and teamwork (Highsmith and Cockburn, 2005).
Agile, however, has its disadvantages. Critical decisions during development must be made by a team of highly qualified and experienced individuals (Highsmith and Cockburn, 2005). Additionally, because Agile is iterative, it can be difficult to forecast project budgets and schedules, particularly for large-scale projects with changing requirements. For stakeholders that depend on thorough records, a lack of documentation might cause problems even while it increases productivity (GeeksforGeeks, 2024).
A range of frameworks, each suited to certain project requirements and team dynamics, are provided by the *Agile* methodology. Among these, Scrum, Kanban, Lean, and Extreme Programming (XP) are notable strategies that businesses can use to improve adaptability, teamwork, and productivity. *Kanban* places more emphasis on visualising workflows, to manage shifting priorities and streamline operations than *Scrum*, which emphasises incremental delivery through organised sprints. *Extreme Programming* (XP) places a higher priority on technical excellence and frequent releases to adjust to changing customer needs, whereas *Lean* development seeks to remove waste and maximise value delivery. When combined, these frameworks offer flexible tools that may be tailored for various project kinds and settings (Itransition, 2024):

Scrum: This incremental and iterative methodology starts the Software Development process with short planning meetings for *sprints*, which are fixed by iterations. Usually lasting two to four weeks, each sprint allows the development team to concentrate on tasks to produce a functioning increment of the product. The team collaborates and meets frequently, including for daily stand-ups, to monitor progress and resolve any problems that may come up. Each sprint concludes with the team reviewing its accomplishments, evaluating any difficulties, and planning the tasks for the following one. The project remains in line with the changing requirements and expectations, thanks to this ongoing feedback loop. Smaller teams, typically with up to 14 members that collaborate closely on a single project, benefit greatly from *Scrum* project management. Scrum's collaborative nature guarantees that every team member is informed and involved in the development process, which makes it ideal for settings, where adaptability, communication, and little steps forward are crucial (Itransition, 2024).

Divided into squads, assigned to specific projects, each team within the squad plays a crucial role in managing the overall project. The Product Owner (P.O.) oversees the project backlog, prioritizing and managing user stories, to ensure alignment with business goals. The Agile Consultant acts as a facilitator, engaging with the team, organizing regular meetings, and ensuring there are no blockers hindering progress toward the next delivery. Additionally, they guide the team on Agile best practices and foster collaboration. Within the squad, developers, testers, and UX designers work closely together, combining their expertise to ensure a smooth workflow and the successful delivery of tasks. This cohesive teamwork ensures that each task is completed effectively, resulting in high-quality project outcomes at the end of every iteration (please, see Figure 25).

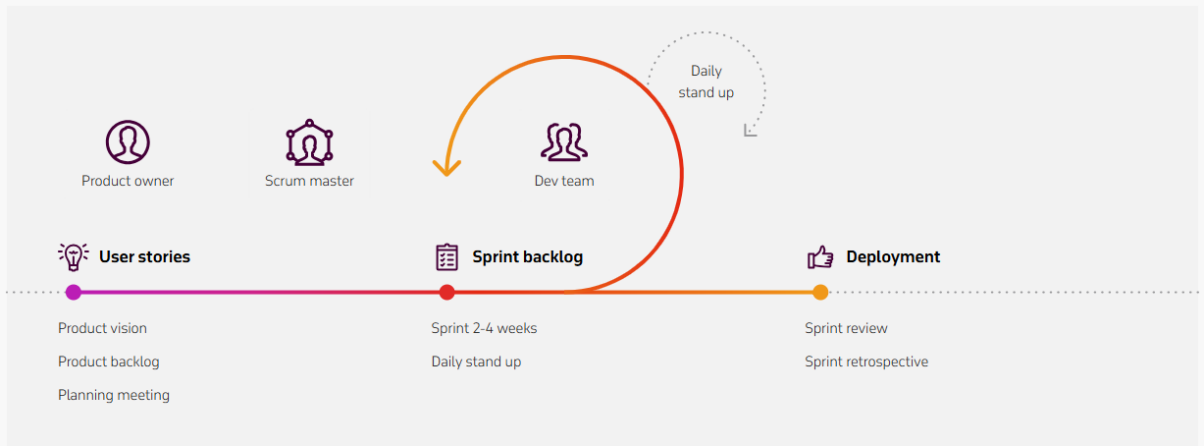


Figure 25:Scrum Development Methodology (Itransition, 2024)

Scrum is suitable for projects with dynamic requirements and a continuous delivery approach and support and maintenance projects that require ongoing adjustments and updates.

Kanban: The project management methodology, known as *Kanban*, places a strong emphasis on total transparency, streamlined processes, and the best possible task distribution for every developer. It makes it simple for the team to monitor progress, by using a *visual board* to display tasks and activities. The board is updated in real time, as tasks are completed, giving users instant access to the project's status. This makes it easier to identify any obstacles, modify the project plan as needed, and quickly rearrange tasks in order of importance when project priorities change. Due to its adaptability, *Kanban* can be used in conjunction with other approaches, including *Scrum*, to offer a flexible approach to task management. Projects with dynamic needs, where the scope and priorities may change regularly, are best suited for *Kanban*. In support and maintenance projects, when constant supply and modifications are frequently required to satisfy user requirements, or for system performance problems, it is also best suited (please, see Figure 26) (Itransition, 2024).

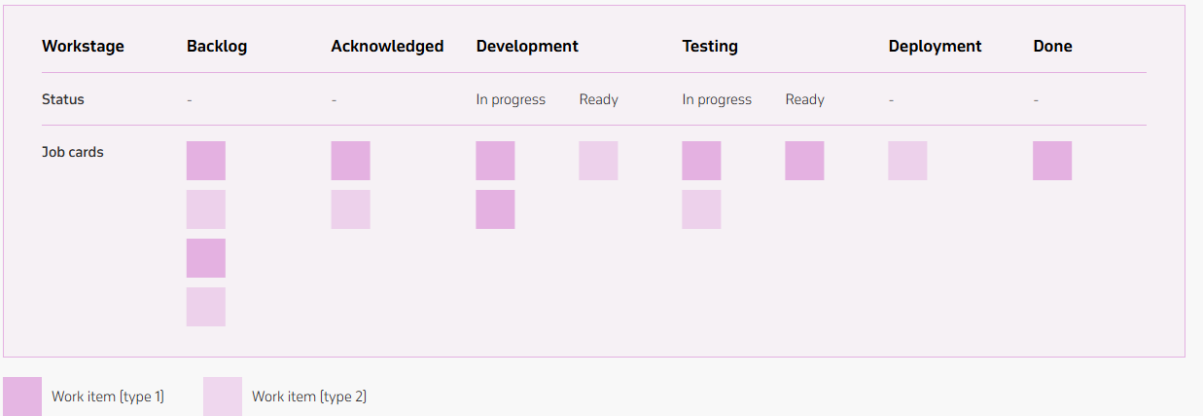


Figure 26: An example of a Kanban board (Itransition, 2024).

Lean: Because they both place a strong focus on adaptability and collaboration, *Agile* and *Lean* development are commonly employed together. *Lean* attempts to decrease inefficiencies throughout the development process and provide flexible, affordable Software by eliminating non-value-adding procedures, like job handoffs and reworks. It encourages continuous improvement, driven by user feedback and incremental changes. *Lean* teams prioritise getting thorough, trustworthy information, so they can make educated decisions, rather than assuming anything. If teams have a clear understanding of the project's potential and challenges, they may independently prioritise tasks, select the optimal technology, and adjust procedures, as necessary. The success of *Lean* initiatives depends on the team's autonomy, self-discipline, and ability to make decisions quickly and based on evidence (please, see Figure 27) (Itransition, 2024).

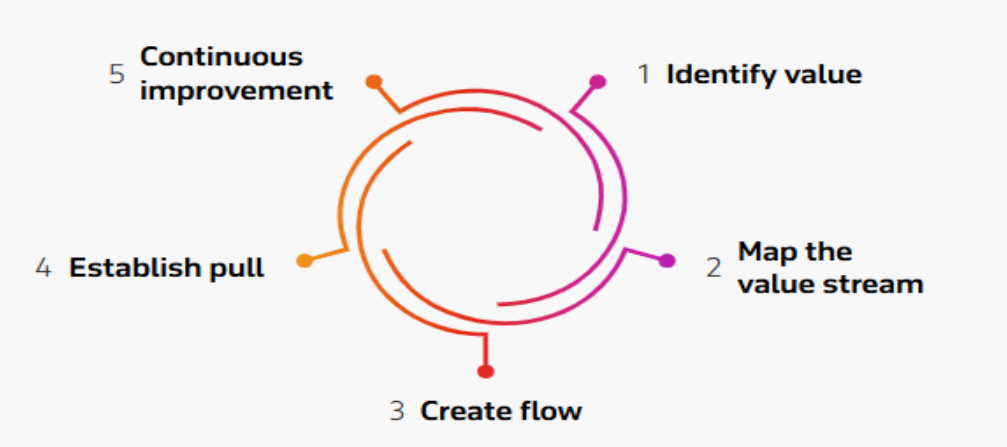


Figure 27: Lean Software Development (Itransition, 2024).

Extreme Programming (XP): Is a collection of Software Engineering techniques, intended to generate high-calibre Software rapidly, while accommodating the evolving needs of the client. *XP* places a strong emphasis on principles, including respect for the project team and their contributions, simplicity, good communication, and ongoing end-user input, to inform project priorities. Development teams are given specific guidelines by the technique, including pairing developers to review code in real-time, building tests before writing code, and only accepting code that passes tests. To collect input, find bugs early, and continuously evaluate the software's performance in production, it also promotes frequent software releases in tiny increments. However, because they demand a lot of work from developers, including active involvement in frequent meetings, XP approaches can be taxing (Itransition, 2024) (please, see Figure 28).



Figure 28: Planning /feedback loops in Extreme Programming (Itransition, 2024).

Extreme programming is suitable for Complex projects with fixed deadlines, and unstable or unclear requirements.

3.2.6 Dynamic Systems Development Methodology (DSDM)

The *Dynamic Systems Development Method* (DSDM) recognizes that needs can change or vary over time, making it impossible to provide a perfect solution on the first try. In accordance with the *Pareto Principle*, the approach recommends investing only 20% of the total effort required to develop the entire solution and delivering 80% of the most important features, based on the project details that are now available, ensuring alignment with business goals and meeting delivery deadlines. DSDM comprises well-defined team responsibilities and organized project management techniques. Along with full documentation, it also includes methods, like time boxing, MoSCoW prioritization, and aspects of attended workshops, that are sometimes missing in other *Agile* frameworks (Itransition, 2024) (please, see Figure 29).



Figure 29: Steps in the DSDM cycle (Itransition, 2024).

DSDM is suitable for Time-sensitive projects, where rapid delivery and quick time-to-market are essential.

3.3 Object Oriented Analysis with UML Diagrams

A standardized Modeling language, called UML (Unified Modeling Language) provides a means of graphically representing the system behaviour and structure. It was created in response to the demand for an Object-Oriented design modeling notation (Fowler, 2004). Each of the two categories of structural and behavioural UML diagrams provides a unique viewpoint on the system being modeled. Class diagrams and component diagrams are examples of structural diagrams that illustrate interactions between various components and represent the static characteristics of a system. On the other hand, behavioural diagrams, such as Activity and Sequence diagrams, depict changes in system behaviour over time (Object Management Group, 2017).

UML's primary goal is to facilitate complicated system design, by offering a standard visual language. Booch, Rumbaugh, and Jacobson (1999) claim that UML was created to ensure consistency and foster communication between developers and stakeholders, by bringing together several modelling languages that were available for Object-Oriented Software Development. It is a vital tool for system design and documentation, due to its broad use and adaptability. Classes, objects, inheritance, and polymorphism are just a few of the object-oriented ideas that are incorporated into UML. UML diagrams are used to model the behaviour and structure of software systems, and these ideas are essential for their development (Ambler, 2004).

Teams can also model system behaviours and pinpoint important item interactions with UML diagrams, which can be especially helpful. There are numerous software programs, available for making UML diagrams. These tools include integrated development environments (IDEs) with sophisticated modelling features, as well as basic diagramming programs. According to Henderson-Sellers (2005), some well-liked tools are Lucidchart, Microsoft Visio, and enterprise-level programs, like IBM Rational Software Architect. There are usually multiple processes, involved in the creation of UML diagrams. Initially, the purpose and scope of the diagram should be clearly defined, followed by the identification of the key elements and their relationships. Once the elements are identified, the appropriate type of diagram is chosen, and the relationships between elements are depicted. It is also important to follow best practices to

ensure the clarity and effectiveness of the diagrams. These practices include keeping diagrams simple, consistent, and focused on the key aspects of the system (Fowler, 2004).
One of the best practices for creating UML diagrams is to keep representations simple and clear. Excessive complexity in diagrams can mislead stakeholders and reduce the model's usefulness. Furthermore, as the system develops, diagrams should be updated often to reflect modifications in functionality and design (Object Management Group, 2017). Knowing when to employ which kinds of diagrams, based on the stage of development and the specific requirements of the system, is another requirement for using UML. *Agile* development environments benefit greatly from UML, since it allows for flexible design and quick iterations. Agile principles can be applied to UML despite its hierarchical nature, enabling rapid system change visualisation and communication (Ambler, 2004). Because of its adaptability, UML is a potent tool in *Agile* Software Development, allowing teams to see how the system's architecture and design are changing without impeding development.
Nonetheless, there are typical problems with UML modelling, like controlling diagram complexity, and making sure all stakeholders can understand them. UML diagrams can be more complicated, as systems get more complicated, and maintaining their clarity can take more work. However, despite these difficulties, there are significantly more advantages to utilising UML than disadvantages, including enhanced documentation, better communication, and a better comprehension of system architecture (Henderson-Sellers, 2005). In summary, UML diagrams offer a crucial framework for complex system modelling, facilitating the visualisation of the system's behavioural and structural elements. When used appropriately and in accordance with best practices, UML can greatly improve teamwork, communication, and the general calibre of system design (please, see Figure 30).

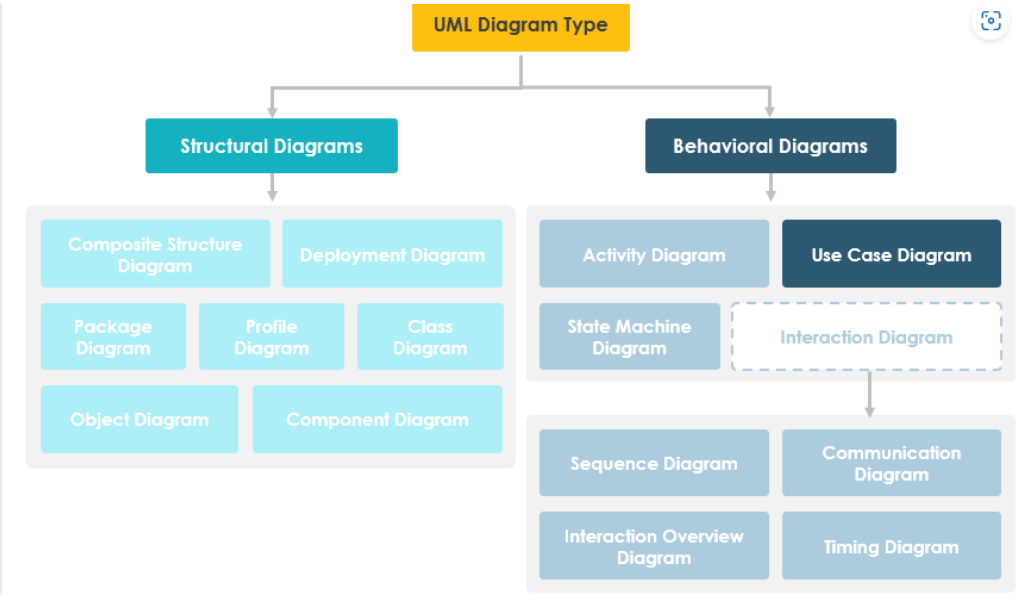


Figure 30: UML Diagram Type (Visual Paradigm, 2024).

3.3.1 Use Case Diagram

A Use Case diagram is a graphic representation of a system's functional requirements in UML (Unified Modelling Language). It demonstrates how the system communicates with its actors, who may be people or other systems, through several Use Cases. A Use Case describes a particular action or process that the system does in response to a user or another system,

according to Jacobson (1992). These diagrams are commonly used early in the system development process, to assist identifying system requirements and to provide a high-level overview of the system capabilities. Use Cases answer the question of "what" the system should do, rather than "how" it accomplishes it, and they illustrate the intended behavior of the system (Fowler, 2004). Use Case diagrams highlight the key interactions between users (or actors) and the system, rather than implementation specifics. They act as a basis for additional complete system documentation and diagrams (Jacobson, 1992) (please, see Figure 31, presenting an example of User Case diagram).

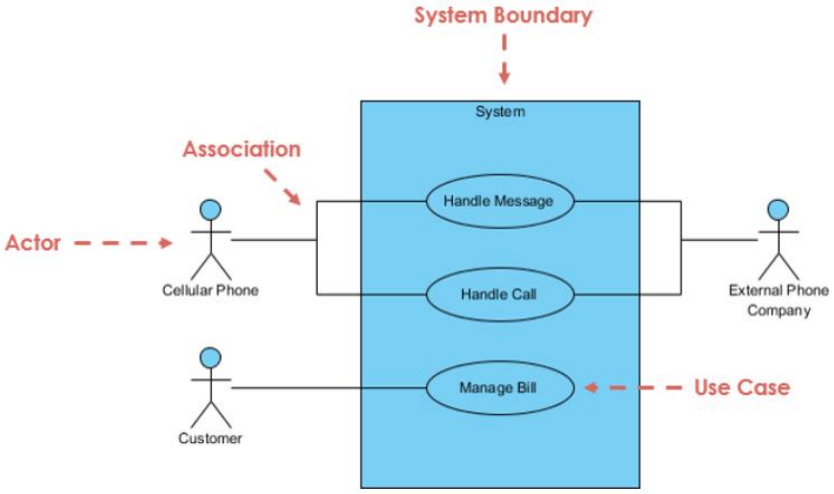


Figure 31: Use Case Diagram (Visual Paradigm, 2024).

Ivar Jacobson first suggested Use Case modelling in 1986. Jacobson first introduced the concept in his work on Object-Oriented Software Engineering, which focused on determining the functional requirements of a system. His 1992 book Object-Oriented Software Engineering: A Use Case-Driven Approach helped popularize the process for creating functional requirements, particularly in the context of software development (Jacobson, 1992).
Use Case diagram's main objective is to define, document, and verify System's requirements. They are created early in the System Development process, and are frequently employed for the following reasons:

- ✓ Specify the context of a System: They define the System boundaries and interactions with actors.
- ✓ Capture System requirements: Use Cases outline the expected behaviour of the system.
- ✓ Validate the System architecture: They help ensure that all System interactions are accounted for.
- ✓ Drive implementation and Test Case generation: Use Cases provide the foundation for both Development and Testing (Cockburn, 2001).

A Use Case diagram includes the following notations:
Ator: Represents a user or another system that communicates with the system. An Actor may interact with several Use Cases and be linked to multiple roles (Fowler, 2004).
Use Case: This is an example of a System function or procedure, that fulfils an Actor's specific objective. According to Jacobson (1992), Use Cases are usually named, using a verb-noun combination, that describes the behaviour.

Communication Link: Participation is indicated by a solid line, that connects an Actor to a Use Case, illustrating how the two interact.
System Boundary: According to Fowler (2004), the System Boundary helps to differentiate between internal system functionality and external interactions and shows the extent of the system being modelled.
Use Case Relationships in UML: There are several types of relationships between Use Cases, which help define dependencies and reuse between them:
<<Extend>>: This relationship shows that one Use Case can extend the behaviour of another Use Case, typically in optional scenarios (please, see Figure 32).

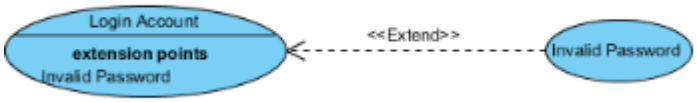


Figure 32: <<Extend>> relationship (Visual Paradigm, 2024).

<<Include>>: When a Use Case requires the behavior of another Use Case, this relationship models that dependency, helping to avoid redundancy (please, see Figure 33).



Figure 33: <<Include>> relationship (Visual Paradigm, 2024).

Generalization: This relationship indicates that a child use case inherits from a parent use case, often extending or modifying its behaviour (please, see Figure 34).

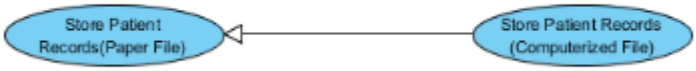


Figure 34: Generalization relationship (Visual Paradigm, 2024).

How to Identify Actors:
Identifying actors is often the first step in creating a Use Case diagram. Actors can be identified by asking questions such as:

- ✓ Who uses the system?
- ✓ Who interacts with the system to achieve specific goals?
- ✓ What external systems interact with the system?
- ✓ Who provides or receives information from the system? (Schneider and Winters, 1998)

How to Identify Use Cases:
Once Actors are identified, the next step is to determine the Use Cases. This involves understanding the goals and objectives that actors want to achieve through the system. Questions to identify Use Cases might include:

- ✓ What functions do actors want from the system?
- ✓ What information does the system store, and who interacts with it?
- ✓ Are there any external events that the system needs to respond to? (Schneider and Winters, 1998).

Use Case Levels of Detail:
The intricacy of the System determines how detailed Use Case diagrams should be. While some diagrams give a more thorough explanation of certain activities, others are made at a higher level, concentrating on the overall behaviour of the system. For stakeholders to communicate effectively, the proper level of granularity must be achieved (Cockburn, 2001).

Description of the Use Case Diagram for the Proposed Project
Please, refer to *Appendix A4* for the Sequence diagram (High-End solution), which illustrates how the system users' (Actors') activities flow sequentially.
The Patient receiving cancer treatment, the Volunteering survivor, and the Doctor are the three main actors, outside the System Boundary. To carry out functionalities, like task creation, profile management, feedback sharing, and registration, these Actors communicate with the System.

Registering as a user and logging in: Both, Patients and Volunteers, must be registered as first-time users, by entering personal information, such as name, gender, and cancer type. Accurate information about each user is ensured in the system through the registration process. To guarantee legitimacy, the Doctor position requires offline Patient and Volunteer registration approval and verification. Users can use their username and password to log in, after registering. A safe procedure within the system allows users to **re-set** their passwords, if they forget them.

Profile Creation and Management: Both, Volunteers and Patients, must build their profiles after logging in. Patients disclose the kind of help they require (e.g., transportation, emotional support, domestic tasks). To match them with the appropriate volunteers, this information is essential. Volunteers indicate their availability, the tasks they are willing to help with. Users can edit their profiles at any moment, and the system will update their new profile records to better match tasks.

Creating and Managing Assistance Tasks: Patients can use the system to create assistance requests, by entering job specifics, such the kind of help needed, the time frame, and the urgency. Following the creation of a request, the system automatically sends the request to the relevant qualified volunteer selected, according to predetermined parameters such task category, gender, and cancer type. Volunteers can accept or decline requests, based on their availability, after being informed of pending requests. The system notifies the relevant patient and modifies the timetable when a volunteer accepts a duty.

Task Scheduling and Notifications: Patients and Volunteers can arrange the time of support tasks, using the system's task Calendar Scheduling feature. Both parties **get reminders** of impending responsibilities. To make sure volunteers aren't overworked, doctors may see their schedules. This function guarantees high-quality service delivery and helps in workload management.

Feedback and Ratings: After an assistance task is completed, Patients can provide feedback and rate the volunteer. This feedback is stored in the system, and can be accessed by other actors, such as Doctors, to monitor the quality of assistance provided. Feedback is also visible to other patients seeking help, promoting accountability among volunteers.

Role of the Doctor: The Doctor acts as an overseer, ensuring smooth operation of the system. They verify and approve Volunteers during the Registration process, monitor feedback

provided by patients, and review volunteer performance. This ensures the system maintains high standards of care and quality.

3.3.2 Sequence Diagram

Also known as an *event* diagram, a Sequence diagram shows how messages move through a System. By representing the communication between two or more object lifelines, as a time-ordered series of events, it is used to visualize dynamic settings. Throughout the System's operation, these lifelines engage in message exchanges. In UML, the message flow is represented by horizontal arrows left to right between the object's lifelines, from top to bottom of the diagram, whereas an object's lifeline is represented by a dotted vertical line. Iterations and branches are also shown in the sequence diagram (JavaTpoint, 2019a).

A Sequence diagram's objective.

- To simulate the high-level interactions between a system's active elements.
- To represent object-to-object interactions in a cooperation, that fulfils a Use Case.

Lifeline:

A lifeline is located at the top of the Sequence diagram and signifies a specific object.

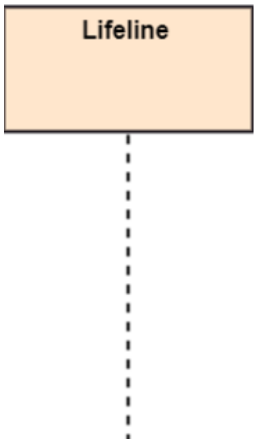


Figure 35:The dotted vertical line represents an Object's Lifeline (JavaTpoint, 2019a).

Actor:

An Actor is an entity outside the System Boundary, that plays a specific role and interacts with the System. It represents roles that may include human users, external systems, or other entities. An actor may represent *multiple* roles, and each *role* may be performed by several actors.

Activation:

An activation bar is shown as a thin rectangle on a lifeline, indicating the period during which an operation is being executed by an object. The top of the rectangle marks the start of the operation, and the bottom represents its end.

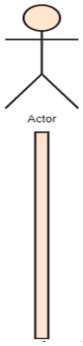


Figure 36: Representation of an Actor's Activation bar, depicting the role of an entity interacting with the system (JavaTpoint, 2019a).

Messages: Messages represent the interactions between objects and are displayed as horizontal arrows in sequential order along the lifelines. They form the core of the Sequence diagram. The main types of messages are:

Call Message: Represents communication, whereby the target lifeline invokes an operation.

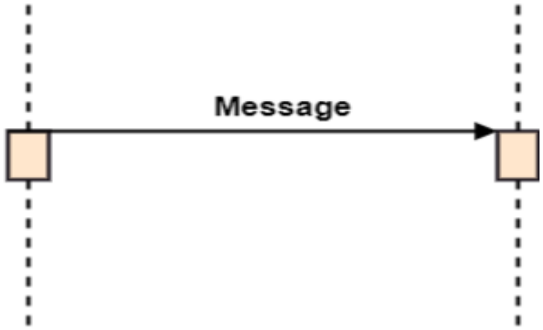


Figure 37: Representation of a Call Message (JavaTpoint, 2019a).

Self-Call Message: Indicates communication within the same Object's lifeline

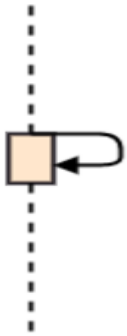


Figure 38: Representation of a Self-Call Message (JavaTpoint, 2019a).

Recursive Self-Call Message: a special type of self-message, used for recursive calls

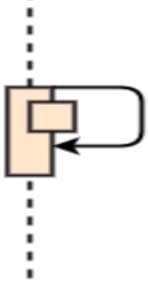


Figure 39: Representation of a Recursive Self-Call Message, depicted by a looped arrow on the same Object's lifeline (JavaTpoint, 2019a).

Create Message: indicates communication, that instantiates the target Object's lifeline, typically shown as a dashed arrow, pointing to the created Object's lifeline

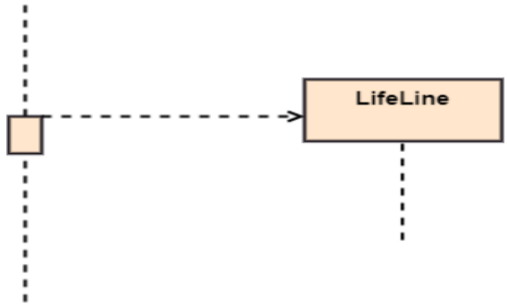


Figure 40: Representation of a Create Message (JavaTpoint, 2019a).

Destroy Message: indicates a request to terminate the lifecycle of the target Object's lifeline, typically depicted with a dashed arrow, ending in an 'X'

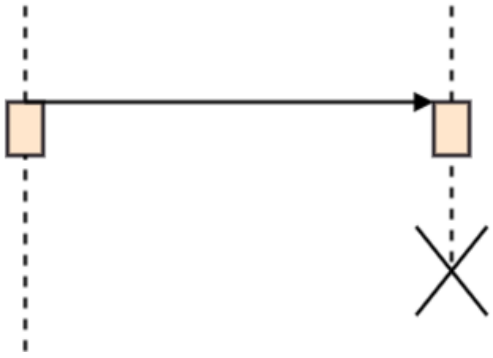


Figure 41: Representation of a Destroy Message (JavaTpoint, 2019a).

Duration Message: illustrates the passage of time associated with a message in the System, being modelled.

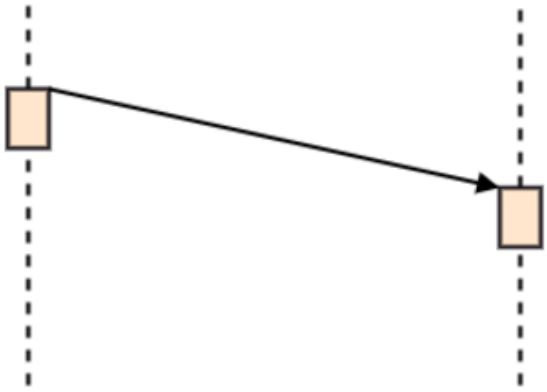


Figure 42:Representation of a Duration Message (JavaTpoint, 2019a).

Description of the Use Case Diagram for the Proposed Project

The Project's Sequence diagram (High-End solution) may be found in *Appendix A5*.

Registration of Volunteers and Patients: The first step in the procedure is for Volunteers to be registered. The **Volunteer Survivor Actor** sends the *Request Register Approved Survivor as Volunteer* message to the System Interface, which forwards the message to the System

Control. System Control receives the request and adds the volunteer's information to a new record in the persistent object Volunteering Cancer Survivor Profile. Only authorised survivors are enrolled as volunteers, thanks to this procedure.

Similarly, the **Patient in Cancer Treatment Actor** sends *Request Register Cancer Patient in Treatment* message to the System Interface, which is passed to the System Control. A new patient record is created in the Cancer Patient Profile persistent entity, storing relevant information such as gender, cancer type, and assistance tasks needed.
Both Actors can subsequently **Log in,** by sending the *Request Login* message, which the: System Interface forwards to the System Control and then to the User persistent entity for credential verification. If the credentials are correct, the user is *logged in*; otherwise, they are prompted to *re-set* their password.
Profile Creation and Update: After logging in, both Patient and Volunteer Actors can create or update their profiles. **Patient Actor** sends the *Request Create/Update Personal Category Profile* message to the: System Interface, which forwards it to the: System Control. The necessary information, including the kind of help tasks requested, is *updated* in the Cancer Patient Profile persistent entity. Through the System Boundary and System Control, the **Volunteer Actor** can also interact with the Volunteering Cancer Survivor Profile entity to *change* their *preferences* and *availability*.

Doctors can also *view* both *Patient* and *Volunteer Profiles* by sending requests to the system. For example, the message *Display Volunteering Survivor's Profile* with *Tasks Calendar Schedule for Doctor to View* ensures transparency and allows monitoring of the process.

Assistance Task Matching and Scheduling: Once Profiles are set, the system facilitates the matching of Volunteers to Patients: **Patient** Actor send a *Request Volunteering Survivor for Assistance* message to the: System Interface, which passes it to the System Control. The System Control retrieves a list of potential matches from the: Volunteering Cancer Survivor Profile entity. The patient's gender, type of cancer, and needed Assistance task category are taken into consideration, when compiling the List. This guarantees that suggestions are made for only the most qualified volunteers. After that, **Patient** Actor submits *Select Volunteer* message to choose a particular volunteer. To make sure the volunteer's availability matches the job request, the System Control *gets* the volunteer's information and schedule. The System Control sends message *Create new Assistance Task Request record* to the: Request Form for Assistance Task persistent object, if a volunteer is *selected*.

Notification for Task Request Acceptance/Rejection: After the Assistance Task Request is *created*, Notifications are sent by the **Volunteer** Actor, who chooses whether or not to *accept* the Request received, based on availability. In accordance with this, the System Control sends message *Modify Request's status* to the: Request Form for Assistance Task persistent object, a message to *Create New Record of Notification for Acceptance/Rejection of the Request* to the Notification Record persistent object, and in case of Request acceptance - a message *Create Reminder for Accepted Assistance Task* to the Reminder Record persistent object, which *Reminders* are *displayed* by the System Interface to the **Volunteer** and the **Patient** Actors to *view*.

Feedback and Ratings: The system makes it easier to get feedback after the support task is completed The **Cancer Patient** Actor submits a *Rating and Feedback for the Assistance obtained* message. The System Interface received the message and forwards it to the System Control. The Feedback and Ratings persistent entity receives a message to *Create a New record*

of Ratings and Feedback. Other patients, volunteers, and physicians can *view* this *Feedback, displayed* by the System Interface, which encourages trust and accountability.

Advanced Features in the Sequence Diagram: Other scalable features are also included in the sequence diagram: **Volunteer Scheduling**: By using the system to change their schedules, volunteers improve task matching and flexibility. **Doctor Monitoring**: Doctors can review patient and volunteer profiles, as well as Feedback, ensuring the system maintains a high standard of assistance. **Assistance History**: Patients and Doctors can access detailed records of completed tasks, feedback, and ratings for analysis and improvement. This sequence diagram captures the full lifecycle of the assistance task management process, starting from registration, profile creation, task request, and matching, to task completion, notification, and feedback. The interactions between actors and the system ensure that tasks are efficiently assigned, monitored, and evaluated, creating a user-friendly experience, tailored to the needs of cancer patients and volunteers.

3.3.3 Object Class Diagram

Class diagrams serve as the foundation for the direct source of Object diagrams. They offer a *static* perspective on an Object-Oriented system at a certain point in time, by representing a particular instance of a class diagram. Despite their similarities, class diagrams and object diagrams differ primarily in their purpose. While Object diagrams show a specific and tangible example of the system in action, Class diagrams provide an abstract picture of a system (please, see Figure 43). The notation for Object diagrams follows UML principles, highlighting the instances of objects in classes and their relationships.

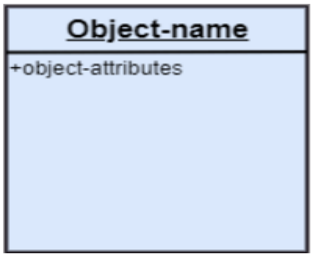


Figure 43: Representation of an Object in UML, showing the object's name and attributes in a rectangular box (Visual Paradigm, 2024).

Purpose of an Object Diagram: An Object diagram and a Class diagram serve very similar purposes. Object diagrams concentrate on a single instance at a precise point in time, whereas Class diagrams offer an abstract picture of the system, including its classes and interactions. Object diagrams provide the primary function of providing a static image of the system, while reflecting the real, tangible behaviour of the system (JavaTpoint, 2024) (please, see an example of an Object diagram on Figure 44).

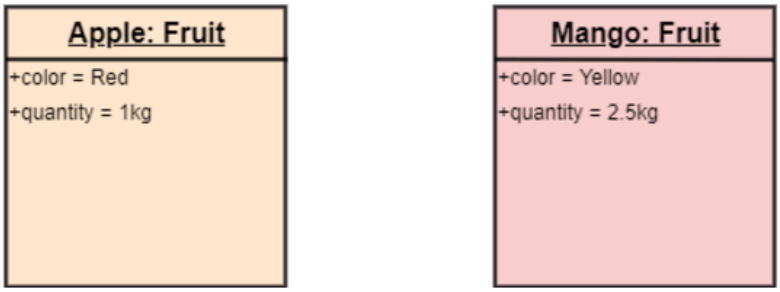


Figure 44: UML representation of two objects, "Apple" and "Mango," as instances of the "Fruit" class, displaying their respective attributes, such as colour and quantity (Visual Paradigm, 2024).

The key purposes of Object diagrams include:

- ✓ Supporting forward and reverse engineering.
- ✓ Understanding the practical behavior of objects and their relationships.
- ✓ Providing a static view of a system.
- ✓ Representing a specific instance of a system.

How to Create an Object Diagram?

To create an effective object diagram, the following steps should be followed:

- ✓ Identify all the objects in the system before starting the diagram.
- ✓ Understand the relationships between the objects before drawing the diagram.
- ✓ Ensure that the association relationships between the entities have already been defined.
- ✓ Assign meaningful and descriptive names to the objects to represent their functionality.
- ✓ Analyze and understand the behavior of each object to ensure the diagram accurately reflects their functionality.

Applications of Object Diagrams

Object diagrams can be applied in various scenarios, such as:

- ✓ Prototyping a system.
- ✓ Modeling complex data structures.
- ✓ Analysing the system from a practical perspective.
- ✓ Reverse engineering.

Class Diagram vs. Object Diagram

When modelling a system, Class diagrams and Object diagrams have various uses. With an emphasis on the connections and interactions at the class level, a Class diagram illustrates the static structure of a system. On the other hand, by displaying instances of classes at a given time, an Object diagram offers insight into the system's behaviour in real time. Since Class diagrams are primarily concerned with the system blueprint, they do not incorporate dynamic changes or the data values and properties of individual objects. However, Object diagrams provide a picture of the system's state during runtime, by capturing dynamic changes, such as specific data values, and instance characteristics (JavaTpoint, 2019b).
Class Diagram - Represents the static view of a system, does not include dynamic changes, does not include data values of attributes, and focuses on class-level behaviour.
Object Diagram - Represents the real-time behaviour of a system, captures dynamic changes, includes data values of attributes of object instances, and shows objects as instances of classes.

Description of the Use Case Diagram for the Proposed Project
For the Project's Object Class diagram, please see *Appendix A6*. The following Object Classes are presented:

Doctor Oncologist:
This class stores information about Oncologists and provides methods to create and update records.

Attributes:
-Doctor ID (Primary Key)
-Doctor Name
-Doctor Surname
-Doctor Mobile Number
-Doctor Clinic/Office Address
-Doctor Specialty Type
- Doctor Username (Email Address)
-Doctor Password
Methods:
+Create New Record for Doctor: Adds a new doctor to the system.
+Verify Login Details: Authenticates the doctor’s login credentials.
+Re-set Password: Updates the doctor’s password.

Volunteer Cancer Survivor:
This class represents Volunteering Cancer Survivors, who assist Patients with their required tasks.
Attributes:
-Volunteer ID (P. K.)
-Doctor ID (FK)
-Volunteer Name
-Volunteer Surname
-Volunteer Gender
-Volunteer Age
-Cancer Type ID (F.K.)
-Volunteer Mobile No

-Volunteer Username (email)
-Volunteer Password
Methods:
+Create New Volunteer Record: Adds a new volunteer to the system.
+Verify Login Details: Authenticates the volunteer's login credentials.
+Re-set Password: Updates the volunteer's password.

Volunteer Assistance Category Task Profile:
Represents the categories of assistance that volunteers can provide.
Attributes:
-Volunteer Category Task Profile ID (P.K.)
-Volunteer ID (F.K)
-Volunteer Gender
-Volunteer Picture
-Assistance Task Category ID (F.K)
-Assistance Task within Category ID (F.K)
-Total Hours Assistance Task/Week Availability

Methods:
+Create New Record of Volunteer Assistance Category Task Profile: Adds a new record of assistance task category to the Volunteer's Profile.
+Update Volunteer Assistance Task Profile Record: Updates the profile for a Task category.

Volunteer Assistance Task Calendar Schedule:
Stores Volunteer's availability schedules for Assistance Tasks to Cancer Patients.
Attributes:
-Volunteer Task Profile Calendar Schedule ID (P.K.)
-Volunteer ID (F.K.)
-Volunteer Category Task Profile ID (F.K)
-Assistance Task Category ID (F.K)
-Assistance Task within Category ID (F.K)
-Day1 of the Week Availability
-Day 1A.M. Time Slot
-Day 1PM.Time Slot
-Day2 of the Week Availability
-Day2 A.M Time Slot
-Day2 P.M Time Slot
Methods:
+Create New Record of Volunteer Assistance Task Calendar Schedule: Adds or updates availability.
+Update Calendar Schedule Record: Modifies an existing schedule.

Cancer Type:
Represents the type of cancer, associated with patients and assistance tasks.
Attributes:
-Cancer Type ID (Primary Key)
-Cancer Type Name
-Cancer Type Description
Methods:
+Get Cancer Type Info: Fetches information about a specific type of cancer.

Cancer Patient in Treatment:
Represents Patients, undergoing cancer treatment.
Attributes:
-Cancer Patient ID (P.K.)
-Doctor ID (F.K.)
-Cancer Patient Name
-Cancer Patient Gender
-Cancer Patient Age
-Cancer Patient Mobile Number
-Cancer Type ID (F.K.)
-Cancer Patient Username (email)
-Patient Password
Methods:
+Create New Record of Cancer Patient: Adds a new patient to the system.
+Verify Login Details: Authenticates patient login credentials.
+Re-set Password: Updates the patient's password.

Cancer Patient Profile with Required Assistance Tasks:
Stores the patient's assistance requirements.
Attributes:
-Cancer Patient`s Required Category Task Profile ID (P.K.)
-Cancer Patient (F.K.)
-Cancer Type ID (F.K.)
-Cancer Patient Gender
-Assistance Task Category ID (F.K.)
-Assistance Task within Category ID (F.K)
-No of Days Assistance Task Required/Week
Methods:
+Create New Record of Patient's Required Assistance Category Task Profile: Adds or updates the patient's assistance requirements.
+Update Profile Record: Modifies an existing profile.

Assistance Tasks Category:
Tracks assistance tasks categories.
Attributes:
-Assistance Tasks Category ID (P.K)
-Assistance Tasks Category Name
-Assistance Tasks Category Description
Methods:
+Create Assistance Task Category
+Get Assistance Tasks Category

Assistance Task within a Category:
Tracks Assistance task within specific Task Category.
Attributes:
-Assistance Task ID (P.K)
-Assistance Tasks Category ID (F.K.)
-Assistance Task Name
-Assistance Task Description

Methods:
+Create Assistance Task
+Get Assistance Tasks Within a Category

Cancer Patient's Assistance Task Request Form:
Tracks assistance task requests made by cancer patients.
Attributes:
-Cancer Patient Assistance Task Request Form ID (P.K.)
-Cancer Patient ID (F.K.)
-Cancer Patient Required Category Task Profile ID (F.K)
-Volunteer ID (F.K.)
-Volunteer Assistance Task Category Profile (F.K.)
-Assistance Task Category ID (F.K)
-Assistance Task within Category ID (F.K)
-Volunteer Task Profile Calendar Schedule ID (F.K)
-Week Commencing Date
-Day1 of the Week Selected? (Yes/No)
-Day 1A.M Time Slot Selected? (Yes/No)
-Day 1PM.Time Slot Selected? (Yes/No)
-Day2 of the Week Selected? (Yes/No)
-Day2 A.M Time Slot Selected? (Yes/No)
-Day2 P.M Time Slot Selected? (Yes/No)
-Assistance Task Request Form Status (default=created)
-Assistance Task Request Text
-Assistance Task Request Date (System Date)
-Assistance Task Request Time (System Time)
Methods:
+Create New Record of Cancer Patient's Assistance Task Request Form: Adds a new task request.
+Update Status of Request Form: Modifies the status of a request.

Notification of Patient's Assistance Task Request Form Acceptance/Rejection:
Notifies patients about the status of their task requests.
Attributes:
-Notification of Patient's Assistance Task Request ID (P.K.)
-Cancer Patient ID (F.K.)
-Cancer Patient Assistance Task Request Form ID (F.K.)
-Volunteer ID (F.K.)
-Volunteer Assistance Task Category Profile ID (F.K)
-Volunteer Calendar Profile Schedule ID (F.K)
-Assistance Task Category ID (F.K.)
-Assistance Task within Category ID (F.K)
-Week Commencing Date
-Request Accepted? (Yes/No)
-Notification Text
-Notification Date (System Data)
-Notification Time (System Time)
Methods:
+Create New Record of Notification: Adds a new notification about task request status Acceptance/Rejection.

Scheduled Reminder for Patient's Accepted Assistance Task Request:
Sets reminders for patients about their scheduled assistance tasks.
Attributes:
-Reminder for Patient`s Accepted Assistance Task Requested ID (P.K.)
-Notification of Patient`s Assistance Task Request Form ID (F.K)
-Cancer Patient ID (FK)
-Volunteer ID (FK)
-Volunteer Assistance Task Category Profile ID (F.K)
-Assistance Task within Category ID (FK)
-Request Accepted (=Yes)
-Reminder Text
-Reminder Calculated Schedule Date
-Reminder Set Time
-Reminder Date Created (System Date)
-Reminder Time Created (System Date)
Methods:
+Create New Record of Reminder for Patient`s Assistance Task Request
+Calculate Schedule Reminder Date
+Set Reminder Time

Feedback & Ratings for Volunteer's Provided Assistance:
Captures patient feedback and ratings for volunteers' assistance.
Attributes:
-Patient`s Feedback & Ratings for Volunteer`s Provided Assistance ID (P.K.)
-Volunteer ID (F.K)
-Cancer Patient ID (F.K)
-Notification of Patient`s Assistance Task Request ID (F.K)
-Week Commencing Date
-Feedback Text
-Rating Stars (1-5)
-Date Created (System Date)
-Time Created (System Time)
Methods:
+Create New Record of Patient`s Feedback & Ratings for a Volunteer`s Assistance Provided.

3.4 Alternative Design Solutions`

Three different Alternative Design Solutions are discussed in this section, each with specific functionalities, and scalability (NASA, 2023). The Project's Use Case Diagram with bounds drawn around Use Cases to show these three solutions, is presented in *Appendix A7*.

3.4.1 Low-end Solution

This design delivers *basic* functionalities, needed to connect patients undergoing cancer treatment with suitable volunteer survivors. It is the simplest implementation, focusing on the fundamental objectives of the application.

Key Features:

- User Authentication: Registration, login, and password re-set.
- Profile Management: Patients and Volunteers can create/update profiles (assistance tasks needed/offered).
- Patients view List of Volunteering survivors, matching Patient's criteria for required Assistance tasks, as per Profile.
- Task Requests: Patients send Request for Assistance task to a selected Volunteer.

Limitations:

Focused solely on basic interactions between patients and volunteers.
Lacks advanced engagement features (e.g., task scheduling or notifications).
May not motivate continued usage or cater to complex needs.

3.4.2 Mid-Range Solution

This solution includes **all** *basic functionalities* of the *Low-end Solution*, with essential features to enhance usability and user engagement. It introduces scheduling, and notifications. This design builds upon the Low-end Solution with additional features, helping at improving usability, engagement, and automation.

Key Features:

- Two-factor Authentication: Enhances security during login for all users.
- Profile Matching: Automatically matches volunteers and patients based on cancer type, gender, and required task categories.
- Notifications: Volunteers view Patients Requests and send Notification of Acceptance/Rejection, based on availability. Change of Task Request status (from created and pending to accepted or rejected).

Advantages:
Improves coordination and engagement through scheduling and notifications.
Matches tasks more effectively, reducing delays or mismatches.
Adds functionality without significantly increasing the development complexity.
Limitations:
Dependent on accurate profile data for matching.
Increased complexity, compared to the Low-end Solution.

3.4.3 High-end Solution

All features of the *Low-end* and *Mid-range* solutions are included in this solution, along with some more *sophisticated* features. It seeks to increase the application's functionality and offer a more personalised and engaging experience.
Key Features:

- Smart Task Scheduling: Patients and Volunteers both receive *automatic reminders* for accepted Requests with scheduled and pending assistance tasks, a day before Task is required.
- Provide/Receive Feedback & Rating about the assistance obtained, visible to patients and doctors.

Future work on the app may include a *Mental Health Support*: Patients can connect with mental health support staff via in-app *chat* for additional emotional assistance.
Advantages:
Provides a comprehensive, feature-rich user experience.
Encourages volunteer participation through ratings and recognition.
May include in the future an emotional support for patients, improving holistic care.
Limitations:
High development complexity and time requirements.

3.5 Decisions Made for the Current Project

Chosen Alternative: *Mid-range Solution* and *RAD* as development methodology is the most realistic and practical for the initial prototype development, due to its balanced functionality, engagement, and feasibility. It effectively enhances user experience, without overwhelming development timelines or resources. However, if time permits, the *High-end Solution* will be developed with *Pending Tasks Reminders* and *Feedback & Ratings* system.

3.6 Chapter Summary

This chapter included a thorough analysis of the Object-Oriented environment and its core concepts, including inheritance, abstraction, polymorphism, and encapsulation. It also analysed different software development methodologies, such as Waterfall, Agile, and Rapid Application Development (RAD), to determine the optimal one for the proposed project. The chapter also explained how the Unified Modelling Language's (UML) Use Case, Sequence, and Object Class diagrams were used to present a Conceptual Model of the proposed for development application. As discussed in this chapter, the proposed application is an Object-Oriented system, designed to connect cancer patients undergoing treatment with vetted volunteer survivors. The use of Object-Oriented Programming ensures a modular, scalable, and

maintainable system, with code reuse and robust security measures, being the key advantages of this approach.

As an appropriate software development methodology, the *Rapid Application Development* (RAD) method was chosen, due to its suitability for the project's goals and short timeline. RAD focuses on short development cycles, frequent user feedback, and iterative improvements, making it an excellent choice for developing the prototype application efficiently. The RAD methodology also allows for parallel development of modular components, ensuring faster results, while accommodating user-driven adjustments during the process.

After evaluating the Three Alternative Design strategies, the *Mid-range* Solution was selected as the most realistic and feasible choice for the initial prototype. This approach balances essential functionalities, such as profile management, task scheduling, and notifications, while avoiding the complexity of the High-end solution. The Mid-range Solution ensures a reliable connection between patients and volunteers, while maintaining a loop for further iterations of the application. Additionally, it introduces basic task scheduling and matching capabilities, providing users with essential features to meet their needs. However, if time permits, the *High-End* solution will be attempted with development of *Pending Tasks Reminders* to both Patients and Volunteers, and *Feedback and Ratings* for Assistance provided by the Volunteers.

The next chapter will present the Interface and Database Development Process of the proposed application. This will include storyboarding process with screen wireframes design and coding and screenshots to detail how the user interface and database structure were developed to support the application's functionality.

CHAPTER FOUR: APPLICATION DESIGN AND CODING PHASE

4.1 Design and Coding of Graphical User Interface (GUI) Screens

All standard electronics devices, such as computers, tablets, and smartphones, have a visual interface, called a Graphical User Interface (GUI) (GeekforGeeks, 2024). This section of Chapter four explains the creation and refining of the visual look of the mobile app from a user's perspective. When considering usability, it is essential to consider the needs of the three user groups for this project. As previously mentioned, the application was developed for those, who are currently fighting cancer, the recently recovered persons, volunteering their support, and for medical professionals to administer security. They each have access to tools and functions, tailored to their roles in the app. Navigating through information on a mobile screen, posed challenges in presenting data effectively. Because there is less room on a mobile device screen, for features to be displayed for user interaction, careful planning and accurate decision-making are crucial for app's usability. In addition, it's critical to consider that these devices have smaller capacities for battery, memory, and storage, compared to laptops. Therefore, when designing an interface for smooth, intuitive, simple, and user-accessible digital experiences all at once, it is crucial to make sure the application can give the user all the tools they need, while fostering a warm and inviting environment.
User Interaction and Navigation Design
By providing accessibility and ease of use to a broad population, the interface prioritizes:

- *Consistency*: A uniform design and design language throughout all the screens leads to user familiarity and minimizes the learning curve.
- *Simplicity*: Each screen has a distinct primary action with no unnecessary complexity.
- *Recognition instead of recall*: Icons, labels, and buttons adhere to accepted mobile UI norms, reducing the need to read instructions and quickly touch. This is very important for mobile usability. Thinking about touch gestures in the UI components, such as touch-based interaction, it is at the heart of the experience, relying on typical gestures, such as tap to choose options and to perform actions.
- *Swipe*: navigating between content, changing sections, and scrolling to browse through lengthy pagination or lists.

These interactions make users remain goal-oriented, providing a responsive and smooth experience through the app.

The Drawbacks of GUI
Creating attractive and functional Graphical User Interfaces (GUIs) can strain the Operating System's resources, because they are complexly designed, and require significant processing power and memory capacity. Designing a GUI, that strikes the balance between aesthetics and usability, is a challenging and time-consuming task. Although GUIs allow users to interact with machines, they can limit the flexibility that experienced users seek, especially those accustomed to command-line interfaces. Ultimately, the choice between GUIs and command-line interfaces depends on complexity and efficiency.

Development Environment

This project uses the Android Developer environment, which has many advantages in the development, ease of use, and design. Developer of the current project opted for *Android* as an Operating System for this application and successfully configured *Android Studio,* with its Integrated Development Kit (IDK) and Emulator to facilitate the development tasks efficiently. Android Studio is known for its adaptability and robust features, that help create mobile applications. *Google Firebase* was chosen as the Database Management System for the application's backend, for its efficiency in data management tasks, seamless integration capabilities, and embedded security measures. The initial setup involved downloading the *configuration file* and incorporating *dependencies* into the application build script (*Gradle*). *Firebase* offers services, like user *authentication*, *Firestore* for storing data in the Cloud, and *Firebase Storage* for managing uploaded files. Towards the end of the chapter, these aspects will be explained in more depth, as they are crucial to the functionality and organization of the application's data.

Current Project's Application Interfaces

The current application of the project has three types of users: doctors, their cancer patients, and volunteers (individuals who have recovered from cancer). The GUI design will incorporate distinct features, suitable for each user group:

Doctors: Doctors will access patient and volunteer profiles through the app. They are responsible for connecting cancer patients with suitable volunteers, during the Registration process. For safety reasons, they must register patients and volunteers, and check that volunteers have completed Garda vetting and are in good mental health.

Volunteers: Volunteers can support patients, currently undergoing cancer treatment. To qualify, a volunteer must be fully recovered from cancer and match the patient's cancer type and gender. Volunteers can select a category, and tasks related to their experience. Volunteers must register a category and task inside the category they can assist with, and put availability days and times per week, so that after patients view their profiles, they can select one and send request for assistance with preferred task. The relevant volunteer receives the request and may accept or reject the request, depending on their weekly availability.

Patients: Patients can view available volunteers, based on the selected task category. They can also check the volunteer's schedule, including preferred days and available times (AM/PM). Once a volunteer (depending on the type of cancer and gender) receives request for assistance with a task, the volunteer can confirm or decline the request. If accepted, a notification is sent back to the patient, who can contact the volunteer via phone to give their address and finalise the details for the support task. After completing the task, the patient is encouraged to provide feedback about the volunteer's support, which the volunteer can view.

4.1.1 Application Screen 1: *Splash* Screen and a *Home* Screen

A Home Screen, or a Splash screen, is often the first interface consumers see, when using computer software, websites, and mobile applications. This screen serves as the main entry point, greeting and familiarizing users with the app. Depending on their role, patients, volunteers, and doctors can access specific features and functions. Each role has access to resources and data specific to its needs.

Home screen design for the current application:

After launching the *Rise Together* app, users are landed on a clean and welcoming Home screen with three distinct role options: *Patient*, V*olunteer*, and *Doctor*. Each function is represented by a clearly identified and visually distinct *button*, to ensure easy recognition and interaction for each type of user. Once a role is selected, the user is taken directly to a dedicated Login screen, tailored to their specific user type. This role works in the backend integration and ensures users have a customized experience for each role, offering relevant features and content, according to their needs.

***Visual Presentation*:**

Figure 45 presents the actual Application Splash screen when the user launches the app.
Figure 46 presents a snippet of code for the actual Splash screen.
Figure 47 shows the wireframe for the application's Home Screen.
A snippet of the code for the application's Home Screen is presented on Figure 48, and the actual application's Home Screen is presented on Figure 49



Figure 45: Application's actual Splash screen, when the user launches the app

SplashActivity.kt

```kotlin
package com.example.risetogether

import android.content.Intent
import android.os.Bundle
import android.os.Handler
import android.os.Looper
import androidx.activity.ComponentActivity
import androidx.core.splashscreen.SplashScreen.Companion.installSplashScreen

class SplashActivity : ComponentActivity() {
    override fun onCreate(savedInstanceState: Bundle?) {
        super.onCreate(savedInstanceState)

        // Splash Screen
        installSplashScreen()

        // Wait to 2 seconds and after that launch the MainActivity
        Handler(Looper.getMainLooper()).postDelayed({
            startActivity(Intent( packageContext: this, MainActivity::class.java))
            finish() // close the screen
        }, delayMillis: 2000)
    }
}
```

ether2 > app > src > main > java > com > example > risetogether > SplashActivity

Figure 46: Snippet of Code for the Application's Splash screen.

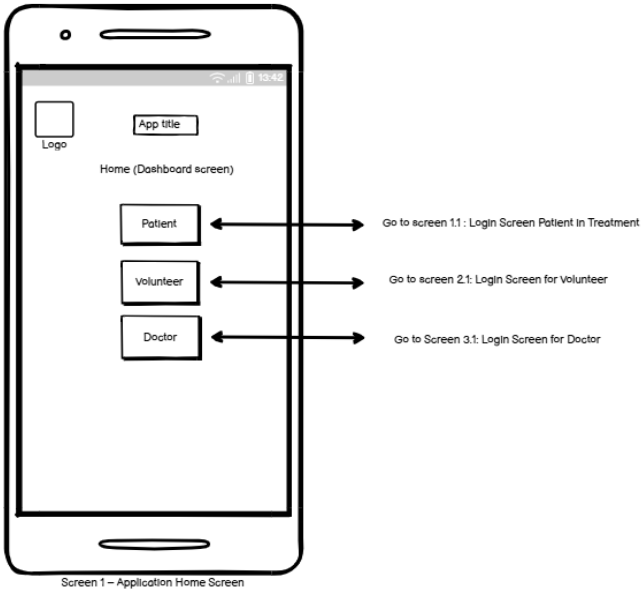


Figure 47: Wireframe of Application Screen 1 - Application Home Screen

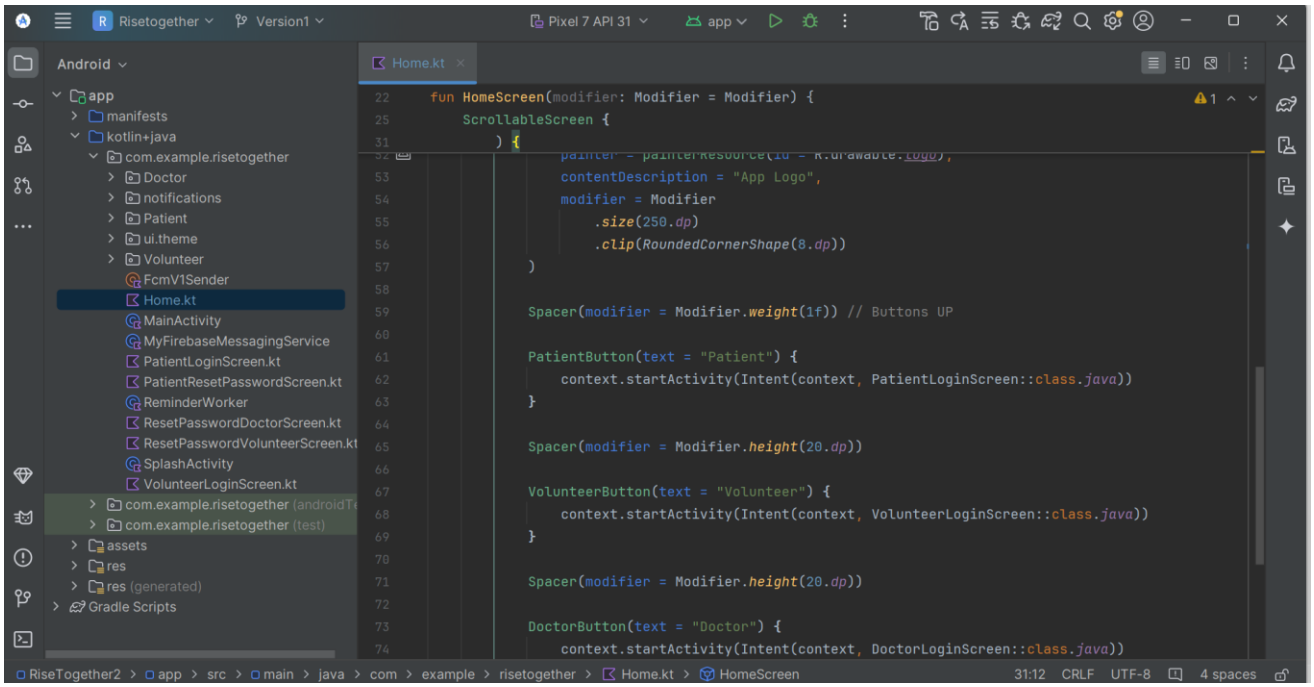


Figure 48: Snippet of Code for Application Screen 1 - Application Home Screen



Figure 49: Snapshot of the Actual Application Screen 1 - Application Home Screen

The decision was to have a unique color for each user type to improve their visual identity, and user recognition from the first interaction. In this initial screen, the developer chose a shade of blue for the Patient button, green for the Volunteer, and purple for the Doctor button, providing immediate visual distinction between roles. This color-coding strategy helps guide users intuitively and will be specially applied throughout the application for each type of user during each flow.

4.1.2 Doctor Interface: Application Screen 3.1 -Login Screen

As a doctor, the user can securely access the Rise Together app by logging in, using their email and password in the designated fields, which are clearly labelled on the interface, with icons to help the user better understand each section's purpose. Below these fields, the "Login" button is shown to begin the Login process, and a link is provided to Re-set/update passwords if necessary. Access to the platform is granted, based on the Login credentials configured during the offline Registration process for doctors. The app checks the credentials after the user inputs their username and password and clicks the *Login* button. If Login fails due to an incorrect email or password, presented is an error message "Login failed. Please check your credentials." This approach increases security by not disclosing which field contains the error.

The screen has a gradient background and a specific purple colour that aligns with the visual style selected for the *Doctor* user category across the platform, helping users recognize its interface.

Visual Presentation:

The wireframe for the application's *Doctor's Login Screen* is presented on Figure 50. A snippet of the code is presented on Figure 51, and snapshot of the actual application *Doctor's Login Screen* is presented on Figure 52.

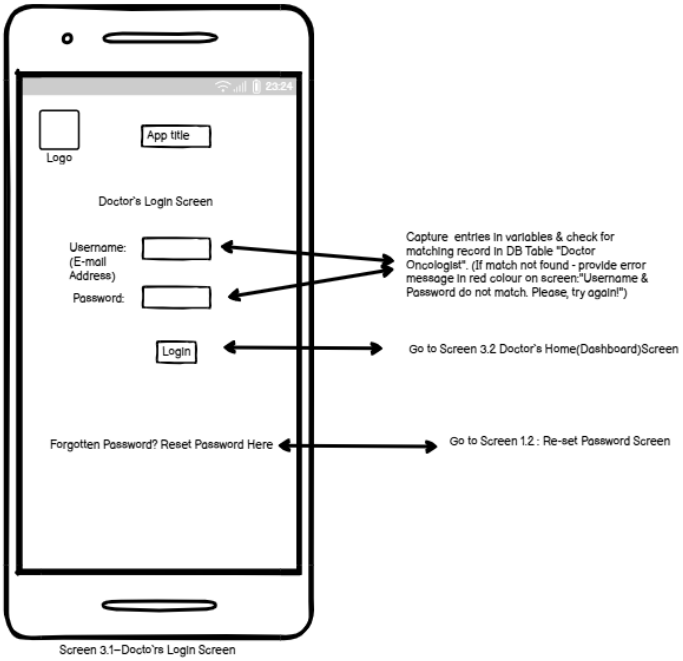


Figure 50: Wireframe of the Application Screen 3.1 - Doctor's Login Screen

```
fun DoctorLoginScreenContent() {
    ) {
        ) {
            ) {
                DoctorButton(
                    text = "Login"
                ) {
                    isLoading = true
                    errorMessage = null

                    auth.signInWithEmailAndPassword(email, password)
                        .addOnCompleteListener { task ->
                            if (task.isSuccessful) {
                                val userId = auth.currentUser?.uid ?: return@addOnCompleteListener

                                // Fetch from "Doctor" collection (uppercase)
                                db.collection( collectionPath: "Doctor").document(userId).get()
                                    .addOnSuccessListener { document ->
                                        if (document.exists()) {
                                            val role = document.getString( field: "role")
                                            if (role == "Doctor") {  // Ensure the role matches
                                                context.startActivity(Intent(context, DoctorDashboardScr
                                            } else {
                                                errorMessage = "Access Denied: Not authorized as a Docto
                                                auth.signOut()  // Logout unauthorized users
```

Figure 51: Snippet of Code for Screen 3.1 - Doctor's Login Screen



Figure 52: Snapshot of the Actual Application Screen 3.1- Doctor's Login Screen

4.1.3 Application Screen 1.2: Re-set Password Screen

Whenever a Doctor type user clicks on the "Forgotten Password? Reset Here" link, they are directed to the *Reset Password* Screen. ***This screen is shared by all user types in the Rise Together application, including Patients, Volunteers, and Doctors, and allows them to securely reset their password.*** In this application, Doctors are not allowed to self-register. Instead, the System Administrator adds them to the platform. Because of this, when a doctor logs in for the first time, they are required to reset the temporary password, initially set by the Admin and create a new one of their choice to ensure privacy and security. To reset their password, the Patient must enter the email address they were registered with and then tap the "Send Reset Password Link" button. The system automatically sends a reset link to that email, and once the link is opened, the Patient is redirected back to their dedicated Login screen. This same password reset process is available at any time to all user types. However, after completing the reset, each user is returned to their own Login screen, depending on their role in the system.

Figure 53 presents the wireframe of the Re-set Password Screen 1.2. Figure 54 shows snippet of the code used to develop the screen's functionality. Figure 55 displays a snapshot of the actual application screen. The Reset Password link sent to the user's registered email address is presented on Figure 56.

Visual Presentation:

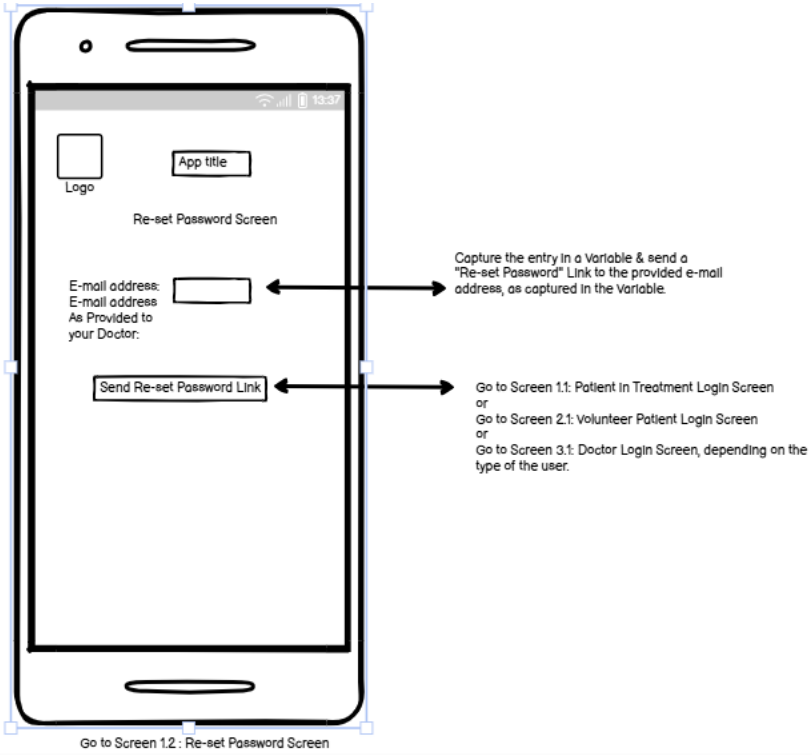


Figure 53: Wireframe of Application Screen 1.2 -Reset Password Screen

```kotlin
fun ResetPasswordDoctorScreenContent() {
    ) {
    // Reset Password Button
    DoctorButton(
        text = "Send Reset Password"
    ) {
        if (email.isNotEmpty()) {
            val auth = FirebaseAuth.getInstance()
            auth.sendPasswordResetEmail(email)
                .addOnCompleteListener { task ->
                    if (task.isSuccessful) {
                        message = "A reset password link has been sent to $email"
                        Toast.makeText(context, message, Toast.LENGTH_LONG).show()
                        context.startActivity(Intent(context, DoctorLoginScreen::class.java))
                    } else {
                        message = "Failed to send reset email. Please check the address."
                    }
                }
        } else {
            message = "Please enter a valid email address!"
        }
    }

    Spacer(modifier = Modifier.height(10.dp))
```

Figure 54: Snippet of Code for Screen 1.2 - Reset Password Screen



Figure 55: Snapshot of the Actual Application Screen 1.2 - Reset Password Screen

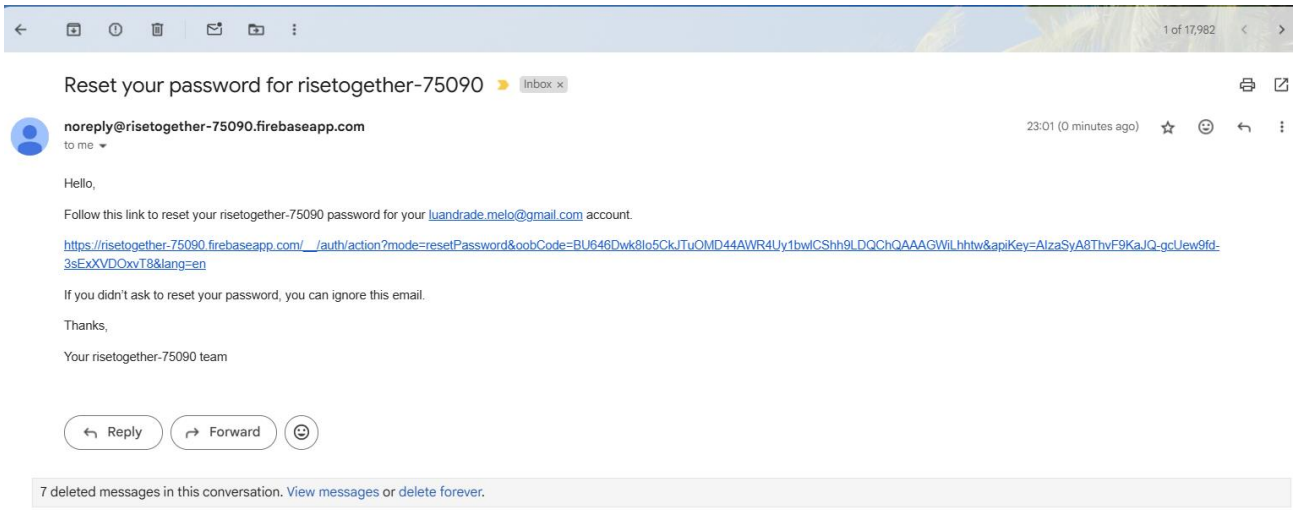


Figure 56: Snapshot of the email received with Reset Password Link, sent to the email address, provided by the User at registration point.

4.1.4: Application Screen 3.1: Doctor Home (Dashboard) Screen

When Doctor successfully logs into the application, they will land on the Doctor *Home* (Dashboard) screen, with two options: Registering Patients and Registering Volunteers. On this screen are two action buttons available for doctors to use; The first button is labelled "Register Patient in Treatment," which permits doctors to register new patients undergoing

cancer treatment. After the doctor puts the information on the screen and clicks on Sent, the system will automatically send an email to the patient with a unique link to set up their password and access the app securely, ensuring their privacy and controlled access to confidential data.

The next option available is the button labelled "Register Volunteering Cancer Survivor," which simplifies the registration process for volunteers who have successfully battled cancer in the past and are now fully recovered. Before proceeding with this registration step, a medical professional must validate eligibility requirements, like gender and type of cancer, and confirm that Garda Vetting procedures have been completed. Like with the registration procedures, a secure email containing a link to set up a password is sent to the Volunteer post-registration, to re-set the password. The Logout button at the right corner lets Doctors to securely end their session and return to the Login screen.
This screen layout's design stays visually cohesive with the application's look. Utilizing the purple gradient linked to the Doctor’s role, helps strengthen user recognition and maintain a sense of familiarity in navigation across different sessions.

Visual Presentation:

Figure 57: Presents the wireframe of Application Screen 3.2 Doctor Home (Dashboard) Screen.

Figure 58: Presents Snippet of Code for Application Screen 3.2

Figure 59: Presents Snapshot of the Actual Application Screen 3.2 Doctor Home (Dashboard) Screen.

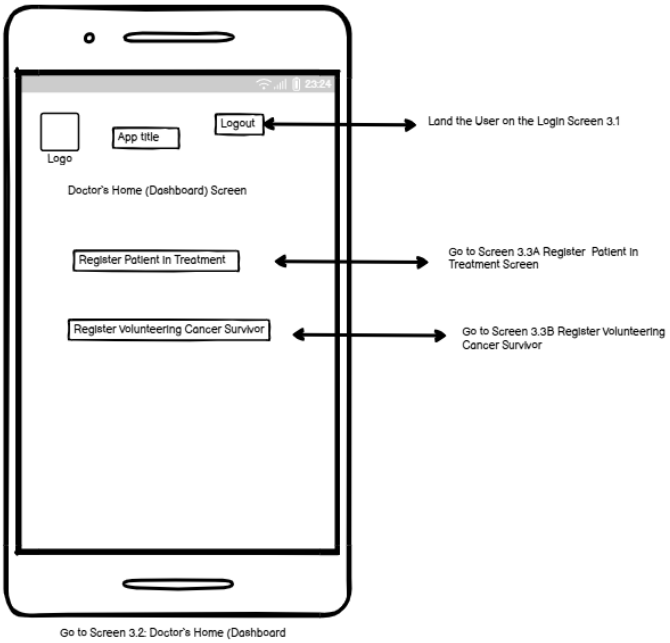


Figure 57: Wireframe for the Application Screen 3.2 - Doctor Dashboard Home Screen

DoctorDashboardScreen.kt

```
51      fun DoctorDashboardScreenContent() {
68          ) {
137             ) { paddingValues ->
146                 ) {
154                     ) {
187                         DoctorButton(
188                             text = "Register Patient in Treatment",
189                             modifier = Modifier
190                                 .fillMaxWidth()
191                                 .padding(2.dp) //
192                         ) {
193                             context.startActivity(Intent(context, RegisterPatientTreatment::class.java))
194
195                         }
196                         Spacer(modifier = Modifier.height(20.dp))
197
198                         DoctorButton(
199                             text = "Register Volunteering Cancer Survivor",
200
201                             modifier = Modifier
202                                 .fillMaxWidth()
203                                 .padding(2.dp)
204                         ) {
205                             context.startActivity(Intent(context, RegisterVolunteeringCancerSurvivor::class.java))
206                         }
```

Figure 58: Snippet of Code for Screen 3.2 - Doctor Dashboard Home Screen



Figure 59: Snapshot of the Actual Application Screen 3.2 - Doctor Dashboard Home Screen

4.1.5 Application Screen 3.3A - Register Patient in Treatment Screen

After a doctor selects the option “Register Patient in Treatment,” it enables the doctor to register new patients undergoing cancer treatment, ensuring that sensitive medical data is managed securely and efficiently. Once the doctor adds a new patient and clicks " Send " at the bottom of the screen, the system automatically sends a secure email to the patient, following the same process as explained for the screen before. The email contains a link that allows the patient to set up their password. This ensures privacy, individual access control, and compliance with data protection standards.

The interface is designed to be quick and straightforward, to guide the doctor through an easy, structured registration process. Key elements on the screen include text entry fields for entering the patient’s first and last name, a gender selection dropdown menu with options (Male, Female, Other), and a cancer type dropdown menu with options for some cancer types and the option other, in case specific option is not available there. Other critical fields include the mobile number field, which validates Irish mobile number formats (e.g., 083, 085 prefixes and 10-digit length), and the email address field, which checks for valid formatting with “@” and recognized domains, such as gmail.com or yahoo.com. The email also acts as a patient’s username.
The initial password field allows the doctor to assign a temporary password. Upon completion, the *Register Patient* button can be pressed to store the patient’s data in the system linked to the Doctor's user ID and automatically sends the patient a secure password reset link. Once the form is filled and the doctor clicks on the “Send” button, a confirmation message, “Cancer Patient Record Successfully Created,” is displayed on the screen.
Additionally, the *Send* button becomes inactive after being clicked, to prevent duplicate emails, while two navigation buttons, *Register Another Patient* and *Back to Dashboard*, allow the doctor to clear the form for a new entry, or return to the Dashboard screen.

Visual Presentation:

Figure 60: Wireframe of Application Screen 3.3A Register Patient in Treatment Screen

Figure 61: Snippet of Code for Application Screen 3.3A Register Patient in Treatment Screen.

Figure 62, 63, 64, 65, 66: Snapshot of the Actual Application Screen 3.3A Register Patient in Treatment Screen.

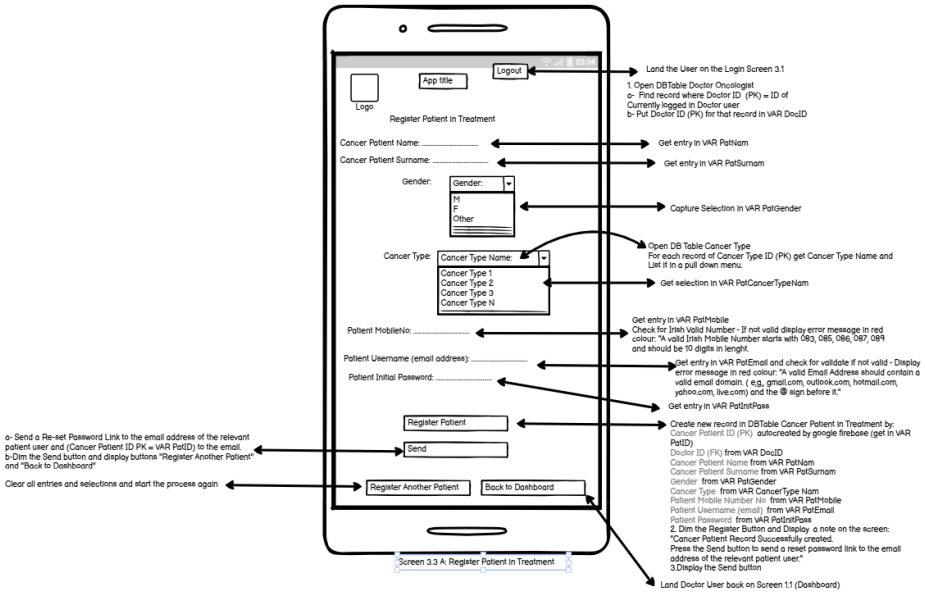


Figure 60: Wireframe of Application Screen 3.3A Register Patient in Treatment Screen.

RegisterPatientTreatment.kt

```
452     fun registerPatient(
466     ) {
467         auth.createUserWithEmailAndPassword(email, password)
468             .addOnSuccessListener { authResult ->
469                 val uid = authResult.user?.uid ?: UUID.randomUUID().toString()
470                 val patient = hashMapOf(
471                     "CancerPatientID" to uid,
472                     "CancerPatientName" to name,
473                     "CancerPatientSurname" to surname,
474                     "CancerPatientGender" to gender,
475                     "PatCancerType" to cancerType,
476                     "PatientMobileNo" to mobile,
477                     "PatientUsernameEmail" to email,
478                     "PatientPassword" to password,
479                     "DoctorID" to doctorId,
480                     "role" to "Patient"
481                 )
482                 db.collection( collectionPath: "PatientinTreatment").document(uid).set(patient)
483                     .addOnSuccessListener { onSuccess(uid) }
484                     .addOnFailureListener { onError("Failed to save patient to database.") }
485             }
486             .addOnFailureListener {
487                 onError("Failed to create user: ${it.localizedMessage}")
488             }
489     }
```

Figure 61: Snippet of Code for Application Screen 3.3A - Register Patient in Treatment Screen

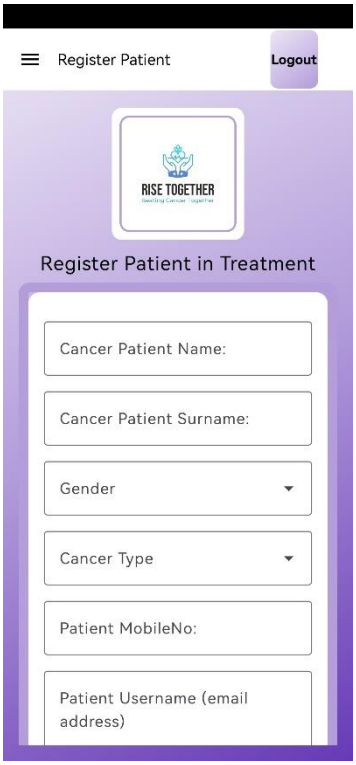


Figure 62: Snapshots of the Actual Application Screen 3.3A - Register Patient in Treatment Screen - 1

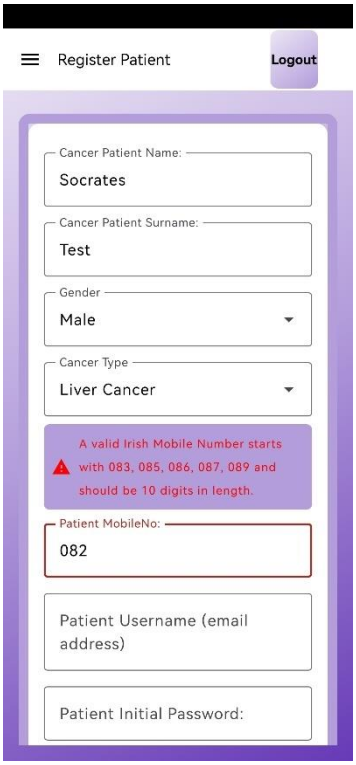


Figure 63: Snapshots of the Actual Application Screen 3.3A - Register Patient in Treatment Screen -2

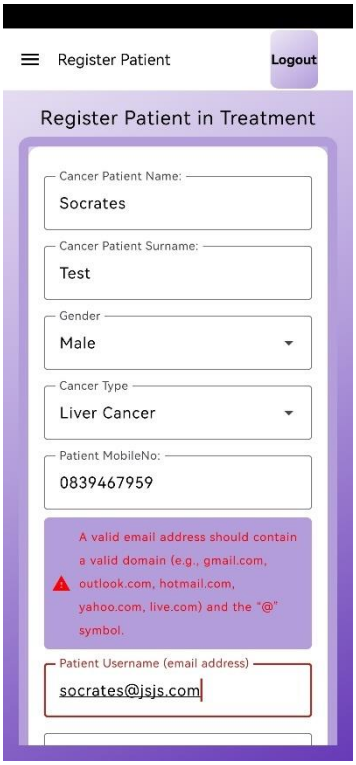


Figure 64: Snapshots of the Actual Application Screen 3.3A - Register Patient in Treatment Screen -3



Figure 65: Snapshots of the Actual Application Screen 3.3A - Register Patient in Treatment Screen - 4



Figure 66: Snapshots of the Actual Application Screen 3.3A - Register Patient in Treatment Screen -5

4.1.6 Application Screen 3.3B - Register Volunteering Cancer Survivor

The Register Volunteering Cancer Survivor screen has the same process as the Register Patient in Treatment flow. The main difference is that it supports the onboarding of volunteers who are fully recovered cancer survivors and wish to help current patients, undergoing treatment. This functionality is restricted to doctors logged into the system and is vital in ensuring a secure and ethical volunteer management process. However, the doctor must do some additional checks for this type of user, before starting the registration. For this process, doctors must verify that volunteers have completed Garda vetting and meet all eligibility criteria, including cancer type compatibility and gender match with already registered patients in the system The registration screen provides a step-by-step flow that guides doctors in collecting all essential information after checking the Garda Vetting, in the same way as for patients in treatment. It also includes personal details, such as the volunteer's full name, gender, cancer type, mobile number, and email address, all of which are validated to reduce entry errors. Real-time checks ensure the email format is correct and the phone number conforms to accepted formats.

Once the form is completed, the system creates a volunteer record linked to the doctor's ID. The volunteer uses the emailed by the Doctor re-set password link to set up their own password, and gain access to their interface within the application. Upon successful registration, a confirmation message is shown, and two options are presented to the doctor: *Register Another Volunteer*, which clears the form for a new entry, and *Back to Dashboard*, which redirects to the primary doctor's Dashboard screen.

All data entered during this process is handled securely by Google Firebase, and the system automates key tasks such as email dispatch, record creation, and user validation, minimizing manual steps and potential for error.

Visual Presentation:

Figure 67: Presents the Wireframe of Application Screen 3.3B - Register Volunteering Cancer Survivor Screen

Figure 68: Presents Snippet of Code for Application Screen 3.3B -Register Volunteering Cancer Survivor Screen

Figure 69: Presents Snapshot of the Actual Application Screen 3.3B - Register Volunteering Cancer Survivor Screen

Figure 70: Presents Snapshots of the email, sent by the doctor for the newly registered volunteer to change their passwords

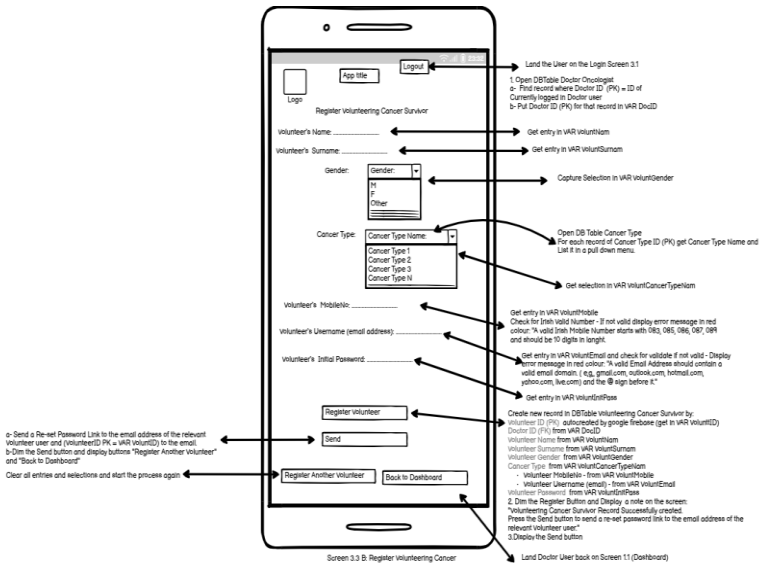


Figure 67: Wireframe of Application Screen 3.3B - Register Volunteering Cancer Survivor Screen

RegisterVolunteeringCancerSurvivor.kt

```
452     fun registerVolunteer(
466     ) {
467         auth.createUserWithEmailAndPassword(email, password)
468             .addOnSuccessListener { authResult ->
469                 val uid = authResult.user?.uid ?: UUID.randomUUID().toString()
470                 val volunteer = hashMapOf(
471                     "VolunteerID" to uid,
472                     "VolunteerName" to name,
473                     "VolunteerSurname" to surname,
474                     "VolunGender" to gender,
475                     "VolunCancerType" to cancerType,
476                     "VolunMobile" to mobile,
477                     "VolunteerUsername" to email,
478                     "VolunteerPassword" to password,
479                     "DoctorID" to doctorId,
480                     "role" to "Volunteer"
481                 )
482                 db.collection( collectionPath: "VolunteeringCancerSurvivor").document(uid).set(volunteer)
483                     .addOnSuccessListener { onSuccess(uid) }
484                     .addOnFailureListener { onError("Failed to save volunteer to database.") }
485             }
486             .addOnFailureListener {
487                 onError("Failed to create user: ${it.localizedMessage}")
488             }
489     }
```

Figure 68: Snippet of Code for Application Screen 3.3B -Register Volunteering Cancer Survivor Screen

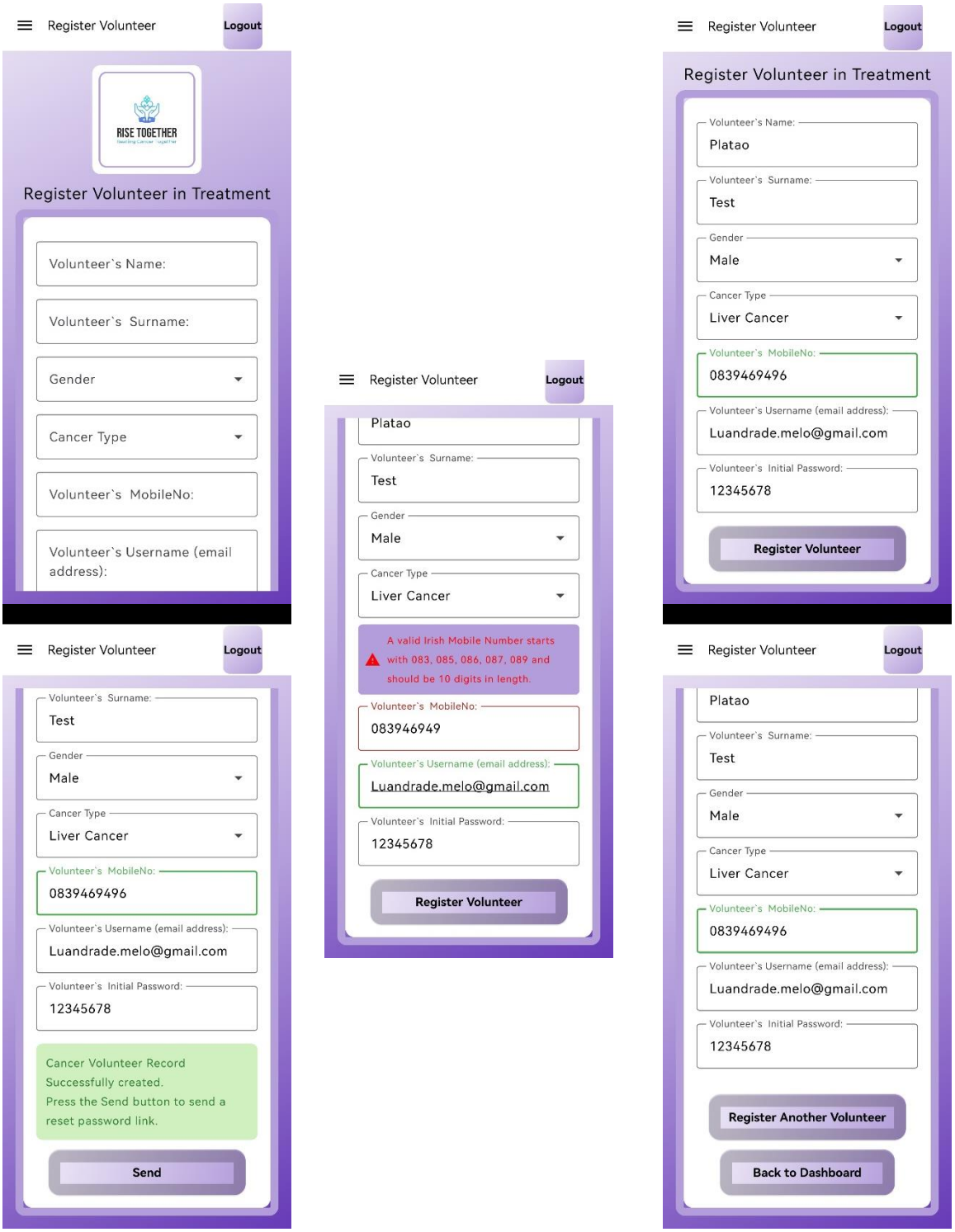


Figure 69: Snapshots of the Actual Application Screen 3.3B - Register Volunteering Cancer Survivor Screen

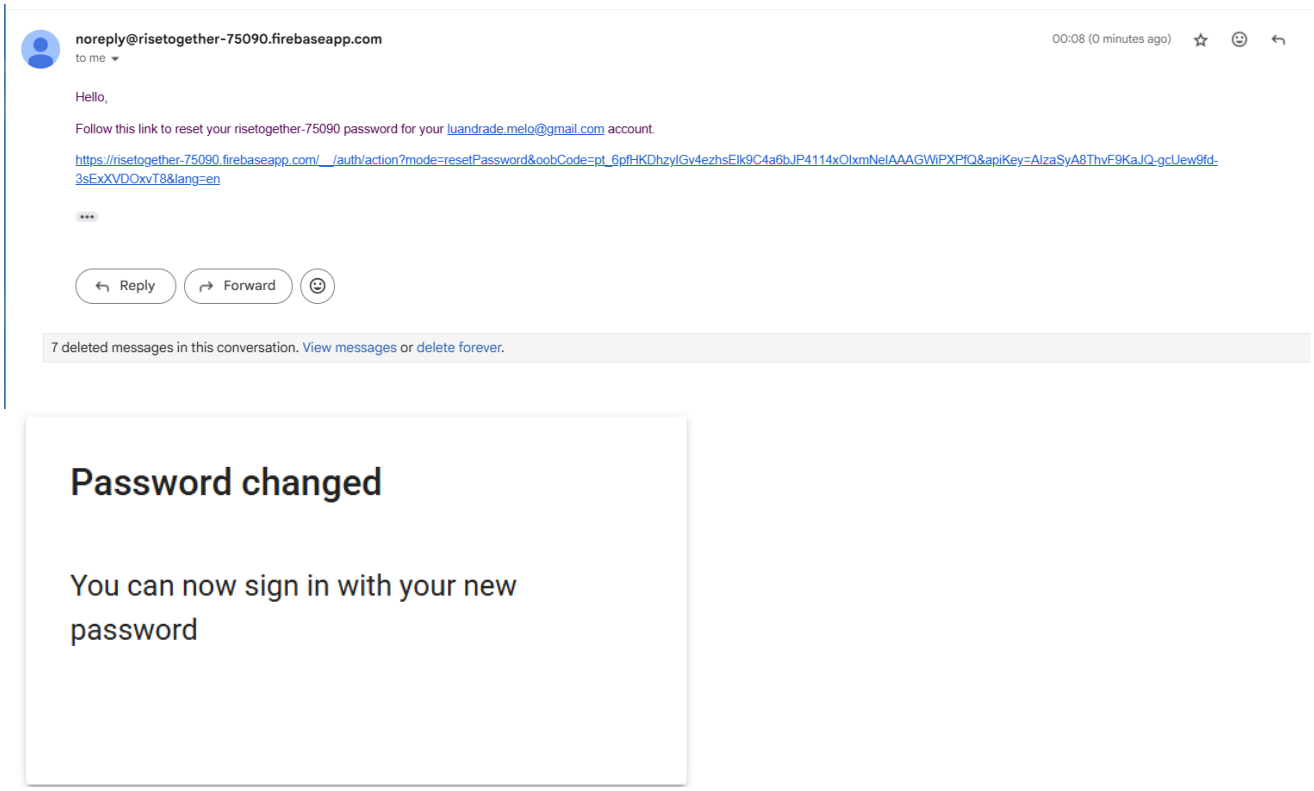


Figure 70: Snapshots of the email sent by the doctor for the newly registered volunteer to change their password

4.1.7 Application Screen 2.1 - Volunteering Cancer Survivor *Login* Screen

This screen is designed to allow volunteers to securely log into the application. If a volunteer forgets their password, they can quickly recover access, by clicking the "Reset Password Here" link available right below the Login form. As explained in the previous section, Volunteers do not create accounts themselves. Instead, the registration process is handled by their Doctor. Once registered, the volunteer receives an email, with a secure link to reset their password privately.

The login screen includes two main input fields: one for the email and one for the password, followed by a *Login* button. When the Volunteering user enters information or clicks on the Login button, the system proceeds to check against the stored records in the database. If the input by the volunteer is correct, the volunteer is landed on the Dashboard screen (Screen 2.2 - Volunteer Home). Otherwise, if the Login is not successful, the system presents error message in red colour to the user about the problem that has arisen. Inline validation was applied to the email input, the app verifies whether the format is correct (with a valid email domain and the “@” symbol). If the input is malformed, red warning text is presented below the input: "Please enter a valid email address." The system also verifies that the password field is not blank before attempting to sign in. If it is blank, the user is notified with: "Password cannot be empty."

Only when both fields are ok does the app proceed to authenticate with Firebase. If Firebase detects that the email or password is incorrect, it returns a specific error, and depending on the situation, the app may show: "Login Failed: The email address is badly formatted." (if the email has the wrong structure), or "Login Failed: The supplied auth credential is incorrect, malformed, or has expired." (if the credentials don’t match or are outdated). These error messages are dynamically displayed in red colour below the login form, helping the volunteer understand precisely what went wrong and how to correct it. Another point here is that in case

the volunteer has forgotten their password, they can click the "Reset Password Here" link, which navigates them to the Reset Password Screen (Screen 1.2). From there, they can change their password.

Visual Presentation:
The wireframe for the Volunteering Cancer Survivor Login Screen is presented on Figure 71, a snippet of code for the screen is shown in Figure 72, and a snapshot of the application's actual Volunteer Login Screen is displayed on Figure 73.

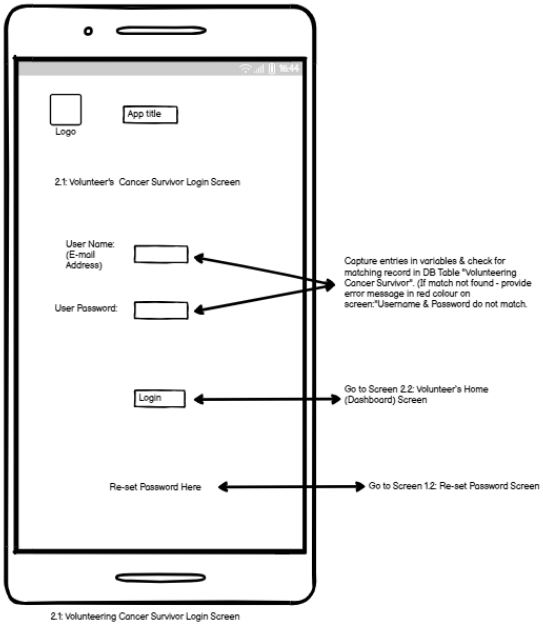


Figure 71: Wireframe of Application Screen 2.1 - Volunteering Cancer Survivor Login Screen

```
VolunteerLoginScreen.kt
53      fun VolunteerLoginScreenContent() {                                    4
70          ) {
109             ) {
113                 ) {
159                     VolunteerButton(text = "Login") {
168                     db.collection( collectionPath: "VolunteeringCancerSurvivor").document(userId)
169                         .get()
170                         .addOnSuccessListener { document ->
171                             isLoading = false
172                             if (document.exists()) {
173                                 val role = document.getString( field: "role") ?: "Unknown"
174
175                                 if (role.lowercase() == "volunteer") {
176                                     val volunteerID = document.getString( field: "VolunteerID") ?: ""
177                                     val volunteerGender = document.getString( field: "VolunGender") ?: ""
178                                     val volunteerCancerType = document.getString( field: "VolunCancerType") ?: ""
179
180                                     // Store Volunteer Data in SharedPreferences
181                                     val sharedPref = context.getSharedPreferences("VolunteerData", Context.MODE_PRIVATE)
182                                     with(sharedPref.edit()) {
183                                         putString("VolunteerID", volunteerID)
184                                         putString("VolunteerGender", volunteerGender)
185                                         putString("VolunCancerType", volunteerCancerType)
186                                         apply()
187                                     }
```

Figure 72: Snippet of Code for Application Screen 2.1 - Volunteering Cancer Survivor Login Screen

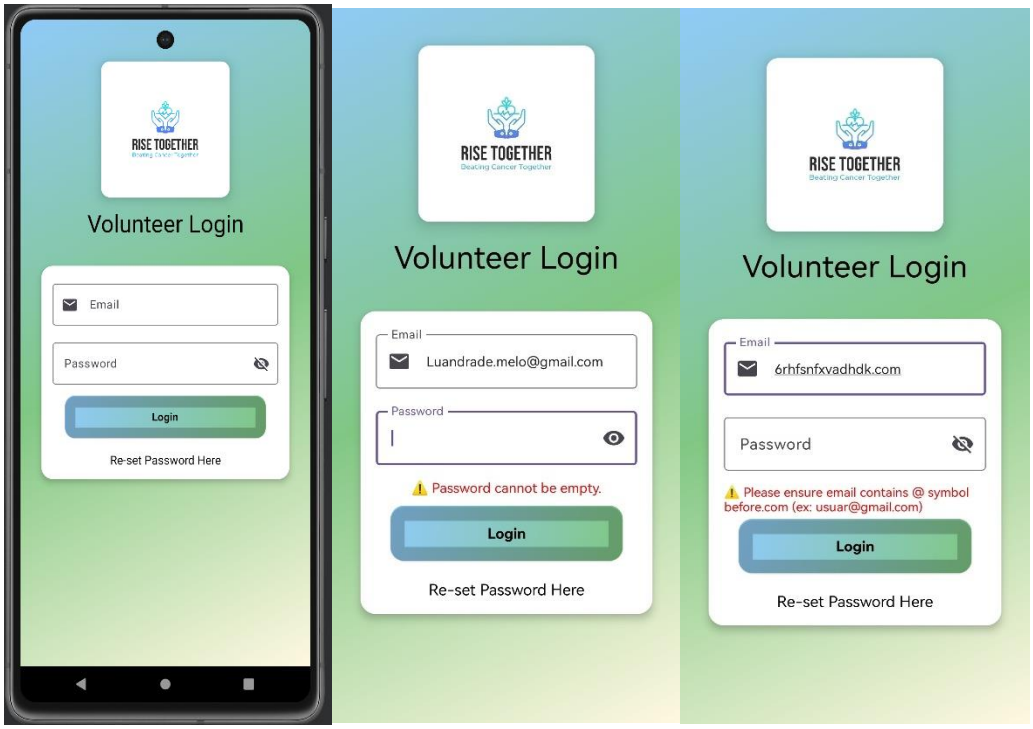


Figure 73: Snapshots of the Actual Application Screen 2.1 - Volunteering Cancer Survivor Login Screen

4.1.7.2. Screen 1.2: Reset Password Screen

The Reset Password Screen for volunteer work in a very similar way to the one used by doctors. However, there is a small but significant difference. Volunteer only use this functionality if they forget the password they created during their registration process. In contrast to doctors, who are required to reset their password when logging in for the first time, volunteers usually only need this screen in cases of forgotten credentials.

Just like in the doctors and patient flows, the volunteer needs to enter the email address they used during registration and then press the “Send Reset Password Link” button. The system will automatically generate and send a reset link to the provided email address. Once the volunteer clicks on the link in their inbox, they will be redirected back to their specific login screen, where they can enter the new password and access their dashboard with the password retyped in the link received in the email. This process ensures security, user independence, and consistency across the app. On the screen, there is a single input field where the user must enter their email. Below it, there is a clearly labelled button to send the reset link. The design is minimalist and direct, with a short explanatory text that reminds users they must use the email registered in the system. This simple layout supports a smooth and intuitive experience for users of all roles.

The password reset process is the same across all user types of patients, volunteers, and doctors but once completed, each user is directed to their corresponding login screen based on their role.

Visual Presentation:

Figure 74 presents the wireframe of the Reset Password Screen, showing the layout of the input field and the reset button.
Figure 75 includes a snippet of the code used to handle the reset request and validate the input.
Figure 76 displays a screenshot of the actual application interface showing the implemented Reset Password Screen.
Figure 77 shows an example of the reset password link received by the user in their email inbox.

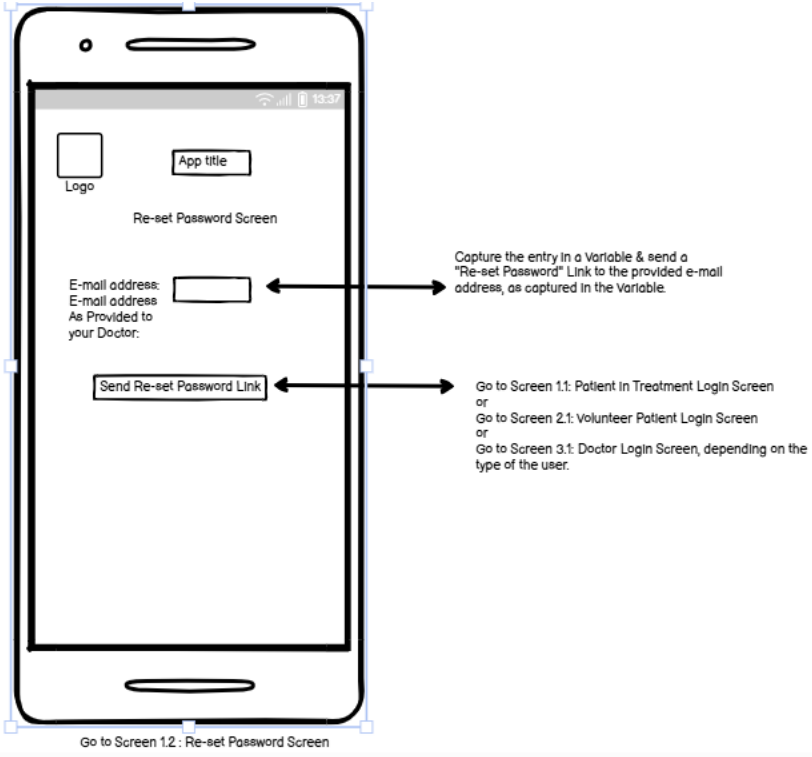


Figure 74: Wireframe of Application Screen 1.2 -Reset Password Screen

```kotlin
fun ResetPasswordVolunteerScreenContent() {
    ) {

    VolunteerButton(
        text = "Send Reset Password"
    ) {
        if (email.isNotEmpty()) {
            val auth = FirebaseAuth.getInstance()
            auth.sendPasswordResetEmail(email)
                .addOnCompleteListener { task ->
                    if (task.isSuccessful) {
                        message = "A reset password link has been sent to $email"
                        Toast.makeText(context, message, Toast.LENGTH_LONG).show()
                        context.startActivity(Intent(context, VolunteerLoginScreen::class.java))
                    } else {
                        message = "Failed to send reset email. Please check the email address."
                    }
                }
        } else {
            message = "Please enter a valid email address!"
        }
    }

    Spacer(modifier = Modifier.height(10.dp))
```

Figure 75: Snapshot of the code for Screen 1.2 - Reset Password Screen

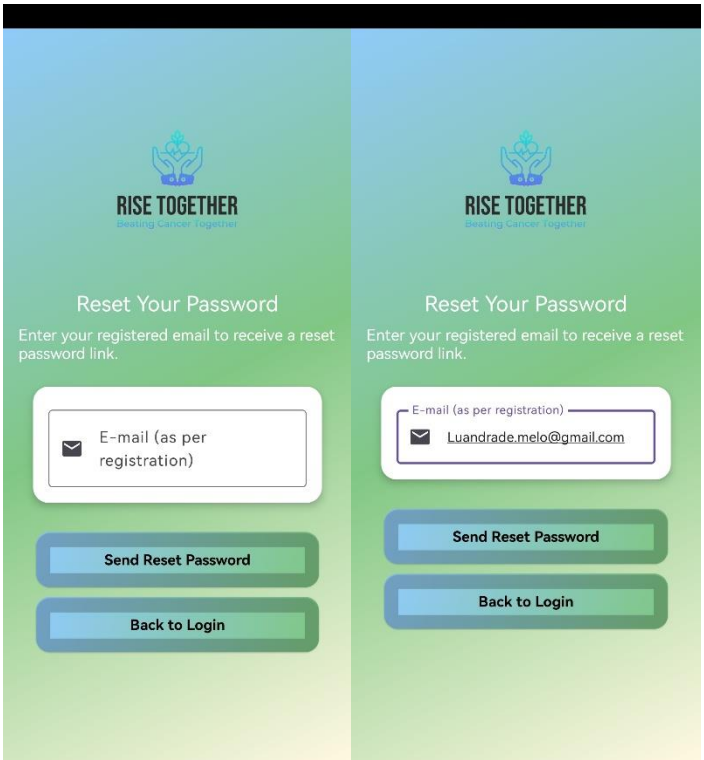


Figure 76: Snapshot of the Actual Application Screen 1.2 - Reset Password Screen

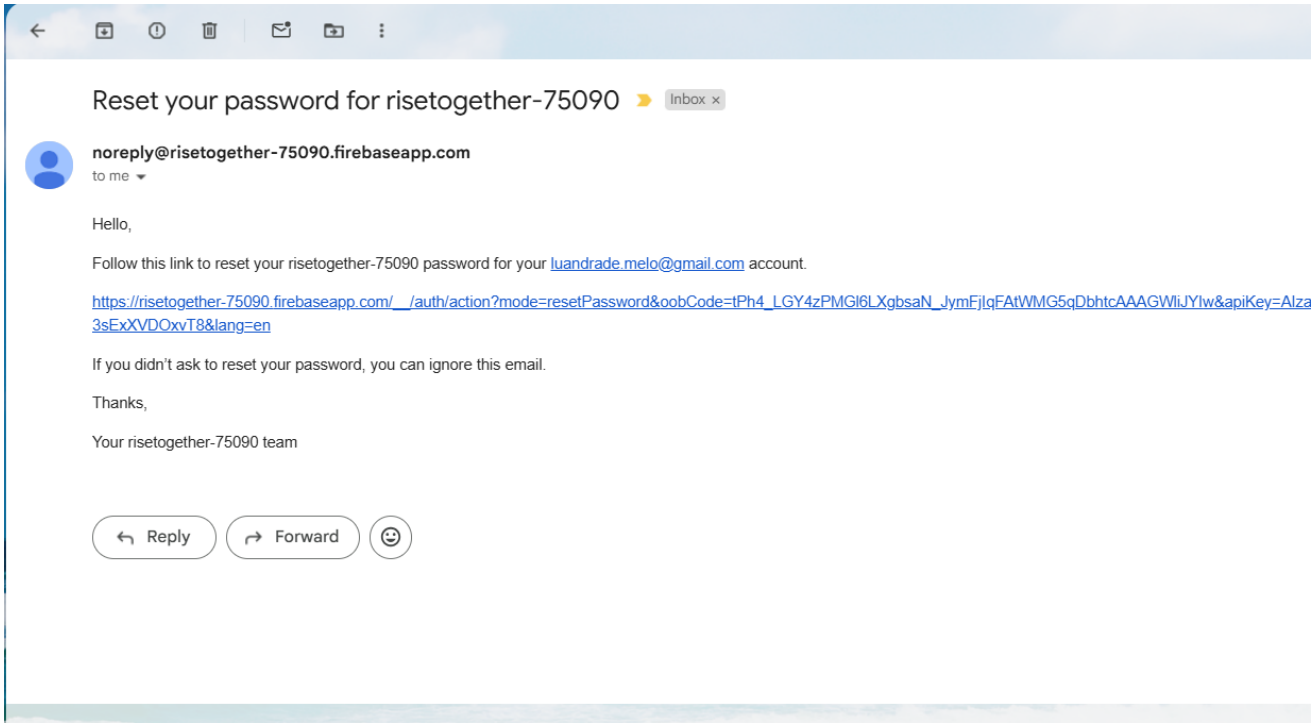


Figure 77: Reset Password Link, sent to the email address provided by the user on Application Screen 2.1

4.1.8 Application Screen 2.2 - Volunteer's Home (Dashboard) Screen

On successful login, Volunteering Cancer Survivor is taken to their Dashboard screen. It serves as a primary function interface for the volunteer to select an option to access relevant functions. The simplicity, clarity, and usability of the layout are highlighted to ensure that the system is easy to use by volunteers of all digital competencies. Once the dashboard is entered, three main areas of actions as options are shown to the user that are each associated with the phases of the volunteer involvement process.

The first option is the "Create Profile Record with Assistance Tasks & Category" button. This feature allows the volunteers to create own profile record with an assistance task from specific category, that they can help with (transportation assistance, household tasks, gardening). Volunteers specify task categorizations and availability, and the system uses these to intelligently match patient needs according to gender and cancer type. Selecting this button will land the Volunteer to Screen 2.2A - Create Volunteer's Profile Record.

The second option is " View List of Patient`s Requests for Assistance". Selecting this option, the Volunteer will be able to view List of assistance Requests, submitted by patients. The system sorts existing tasks by the volunteer's profile. The volunteer may accept or reject a request, based on their weekly availability. Selecting this button will land the volunteer on Screen 2.2B- View Patient`s Requests for Required Assistance.

Third is "View Feedback & Ratings for Assistance Provided" option, where the volunteers can view feedback and ratings provided by patients for obtained assistance. Selecting this button lands the Volunteer on Screen 2.2C - View List of Feedback and Ratings for Requested Tasks.

As with the Doctor's Dashboard screen, already discussed, the Logout button is in the top right corner of the Volunteer's Dashboard screen. Upon clicking, it logs out the user and redirects them to Screen 2.1 - Volunteer Login Screen.

Visual Presentation:

Figure 78 presents the wireframe of the application Screen 2.2: Volunteer`s Home (Dashboard) Screen, Figure 79 shows a snippet of code for the screen, while Figure 80 displays a snapshot of the actual application Screen 2.2: Volunteer`s Home (Dashboard) Screen.

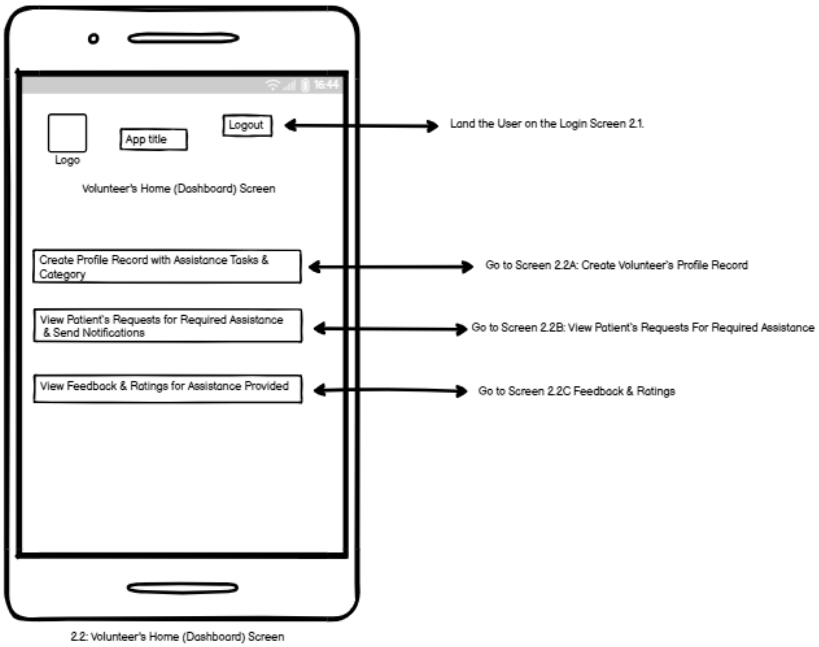


Figure 78: Wireframe of Application Screen 2.2 - Volunteer's Home (Dashboard) Screen

```
VolunteerDashboardScreen.kt
43      fun VolunteerDashboardScreenContent() {
59          ) {
128             ) { paddingValues ->
137                 ) {
145                     ) {
178                         VolunteerButton(
179                             text = "Create Profile Record with Assistance Tasks & Category",
180                             modifier = Modifier
181                                 .fillMaxWidth()
182                                 .padding(2.dp)
183                         ) {
184                             context.startActivity(Intent(context, VolunteerProfileScreen::class.java))
185                         }
186
187                         VolunteerButton(
188                             text = "View Patient`s Requests for Required Assistance\n" +
189                                     " & Send Notifications\n",
190                             modifier = Modifier
191                                 .fillMaxWidth()
192                                 .padding(2.dp)
193                         ) {
194                             context.startActivity(Intent(context, ViewPatientRequestsScreen::class.java))
195                         }
196
197                         VolunteerButton(
```

Figure 79: Snippet of Code for Application Screen 2.2 - Volunteer's Home (Dashboard) Screen



Figure 80: Snapshot of the Actual Application Screen 2.2 - Volunteer's Home (Dashboard) Screen

4.1.9 Application Screen 2.2A - Create Volunteer's Profile Record

Tapping on button "Create Profile Record from the Dashboard screen is landing the Volunteering User on the Create Volunteer's Profile Record screen. It enables Volunteers to specify the kind of assistance they are willing to provide and how frequently. This information is directly used in the system's matching process, so that patients are matched with those who are best suited to address their needs. The interactive and visually uncluttered design of the screen has been specifically created to be easy to use, leading the volunteer through the process smoothly, and capturing relevant data. Its usability and rational flow support users across different digital skill levels.

When the screen is loaded, the user is addressed with a customized welcoming message (e.g., "Hello, Ingrid Smith," the name of the user currently logged in) that lends a welcoming, personalized feel to the process. The first part of the form is about selection of task category, e.g., Household, Garden, or Transportation. Upon selection of a category, there is a subsequent dropdown menu, showing task options that are contextually related to the category that has been selected, e.g., choosing Gardening may display tasks like Hedge Trimming or Grass Mowing.

Then, the volunteers are requested to define their availability each week by selecting how many days per week they can assist (options 1 to 5 in a dropdown menu). Volunteers are also encouraged to upload a recent photo from their phone gallery. The photo is added to their profile and is meant to create trust and acquaintance between patients and volunteers.

When all fields are filled in, the Volunteer saves the Profile record by clicking the *Save* button to submit data. The system shows a loading indicator and indicates a temporary "Saving"

message to reflect progress. Upon successful recording of the data, a pop-up confirmation message is shown:
"Volunteer`s profile record with selected assistance task & category successfully created. Press button "Continue" to create a " Calendar schedule for that task".

From there, the user is presented with a few options to move on. They may press *Continue*, directing them to a Scheduling screen to confirm Weekdays and times of availability. After that, another message is shown saying “Record of Assistance Task & Category personal Calendar schedule successfully created”. You may either press "Another Assistance Task" button or "Back to Dashboard" button. Pressing Another Assistance Task will reset the existing choices and allow user to fill up another Task form of support. Or the *Back to Dashboard* link will bring the user back to their Dashboard screen (Screen 2.2).

Visual Presentation:

Figure 81 presents the wireframe of the application Screen 2.2A: Create Volunteer’s Profile Record. Figure 82 shows a snippet of code for the screen, while Figure 83 displays a snapshot of the actual application Screen 2.2A: Create Volunteer’s Profile Record.

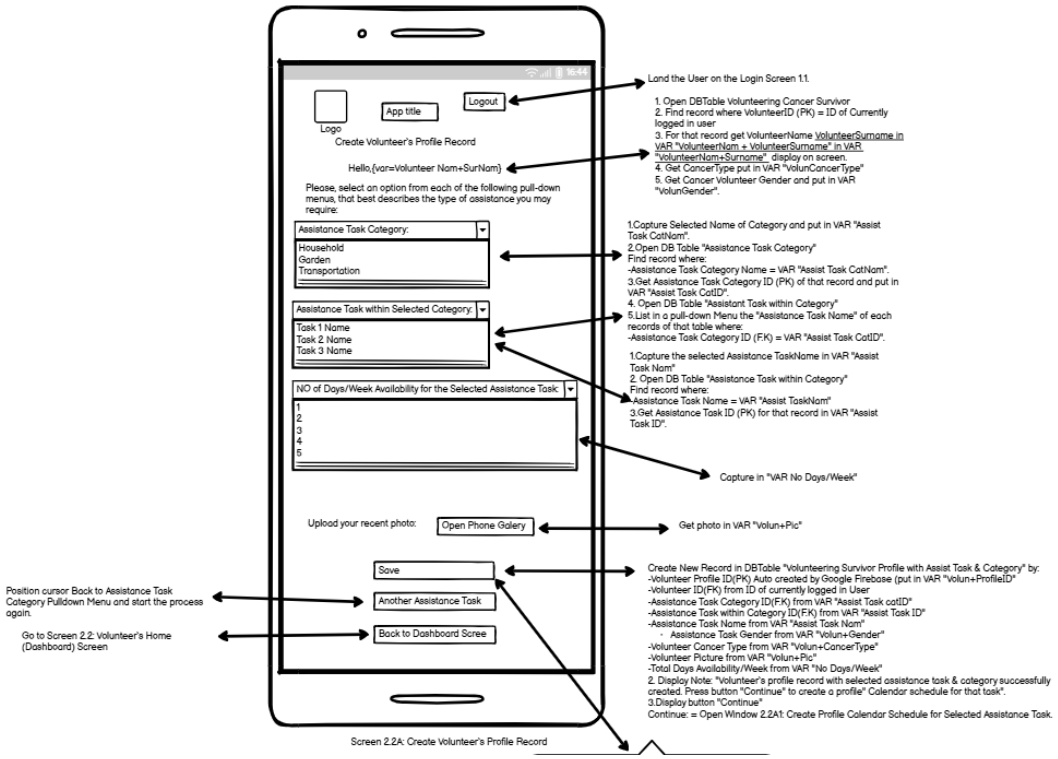


Figure 81: Wireframe of Application Screen 2.2A - Create Volunteer’s Profile Record

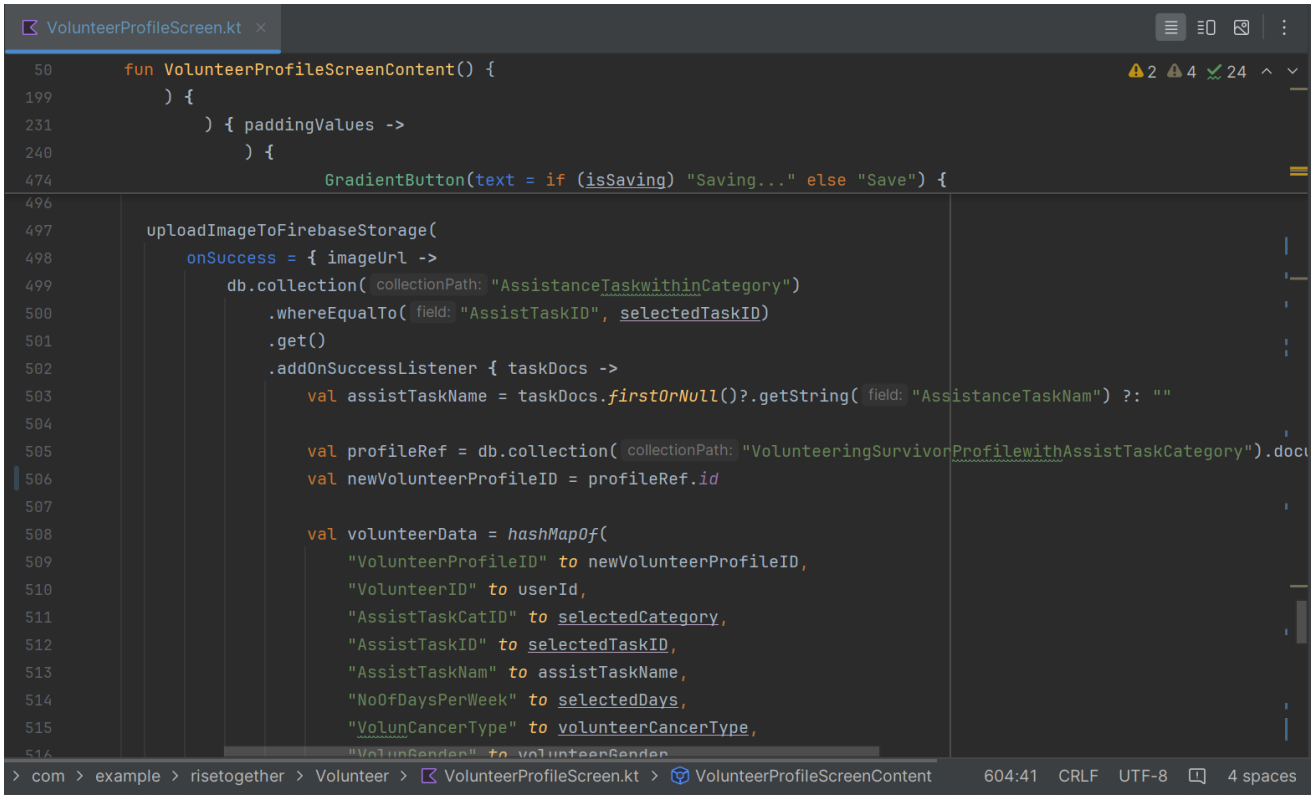


Figure 82: Snippet of Code for Application Screen 2.2A - Create Volunteer's Profile Record

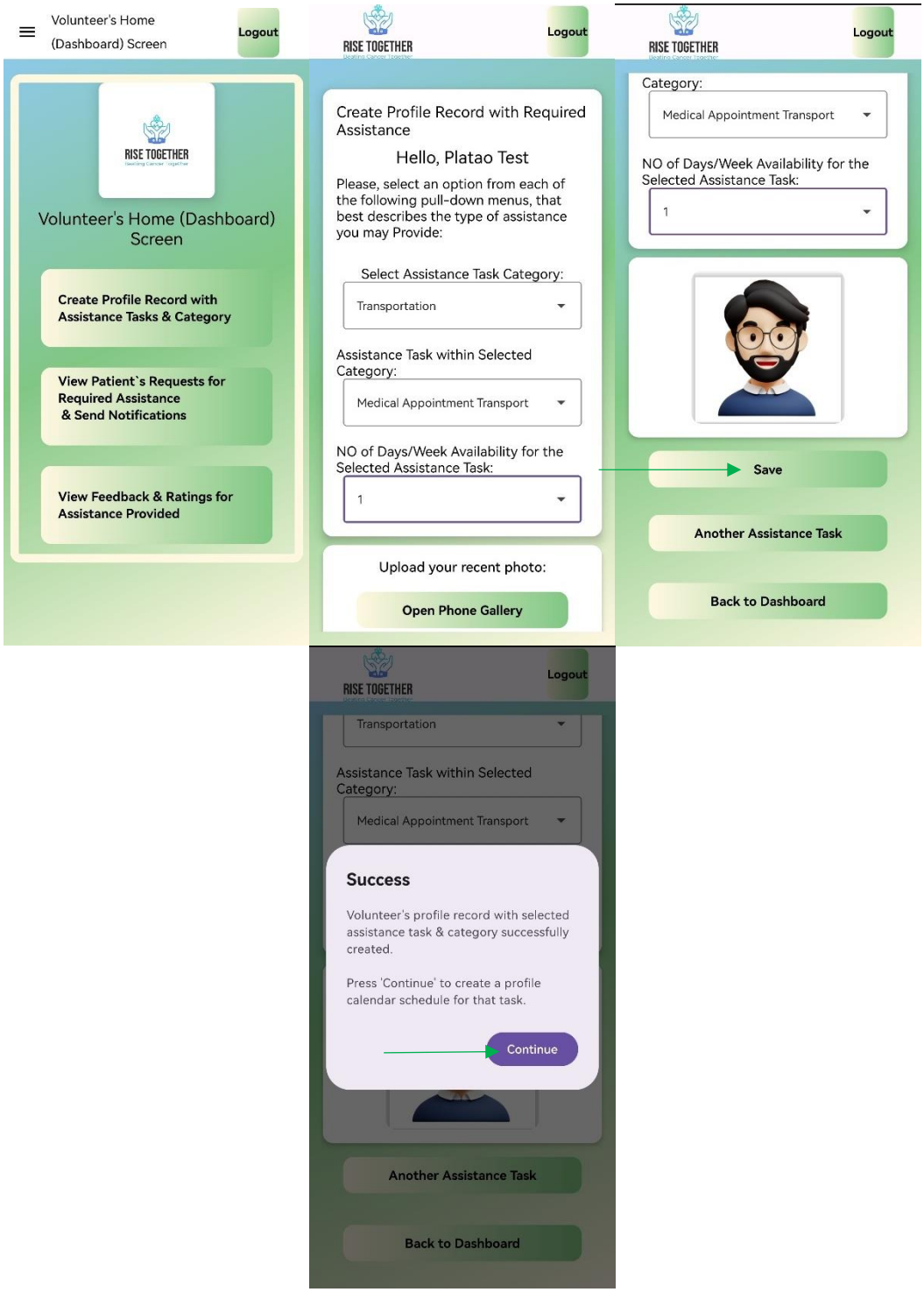


Figure 83: Snapshots of the Actual Application Screen 2.2A - Volunteer Profile Record Creation Process.

4.1.10 Application Pop-up Window 2.2A1 - Create Profile Calendar Schedule for the Selected Assistance Task

After the volunteer user creates a Profile record for an assistance task within specific selected category, a “Create Profile Calendar Schedule for Selected Assistance Task” Window pops-up. This window enables volunteers to specify the weekdays and time of the day (A.M./P.M.) they are available to provide the assistance task to patients.

The display is dynamic and interactive. According to the number of days chosen in the first step, the system automatically shows a list of day/time pickers. Volunteers may define both the AM and PM blocks of time for each weekday selected from predefined pulldown lists. They may choose weekday options (Monday to Saturday) and the accompanying time slots of 08:00-09:00, 09:00-10:00 until noon, and 14:00 -15:00, 3:00 - 4:00 p.m. until 18:00. Prior to presenting the schedule options to the user, the system automatically loads the previously chosen Task Name and Task Category, giving the user a reference as to what activity the current schedule is for. This contextual association makes users aware that every availability configuration is task specific.

When the volunteer has set their availability schedule, clicking on the *Save Schedule* button saves the selection in the database. A pop-up message appears saying: “Record of Assistance Task & Category personal Calendar schedule successfully created”. You may either press "Another Assistance Task" button or the "Back to Dashboard" button”. The button “*Another Assistance Task*” is used to reset the existing choices and allow them to fill different task form of support. Or the Back to Dashboard link brings the user back to their dashboard screen (Screen 2.2).

System Limitation - Prototype Version 1.0
It is important to note that there is a current restriction in the scheduling system. While the user interface supports selection of multiple different days of availability by a volunteer (up to five days of the week), there is a current back-end implementation (in prototype version 1.0) that only incorporates the first two chosen days into the logic of volunteer-to-patient matching. This implies that if the volunteer chooses availability across several days, such as choosing 4 or 5 between the options, only the first two chosen days will be utilized in suggesting availability of volunteers in responding to patient requests. The rest are stored but not used in active scheduling or matching algorithms, so in the volunteer details scheme, only the first two days are shown. This is because of the existing schema in the database and the matching engine. A high solution can incorporate full multi-day schedule support; in that case, the system will utilize all chosen availability towards better task assignment and match accuracy.

Visual Presentation:

Figure 84 presents the wireframe of the application Window 2.2A1 - Create Profile Calendar Schedule. Figure 85 shows a snippet of code for the window, while Figure 86 displays a snapshot of the actual application Window 2.2A1 - Create Profile Calendar Schedule.

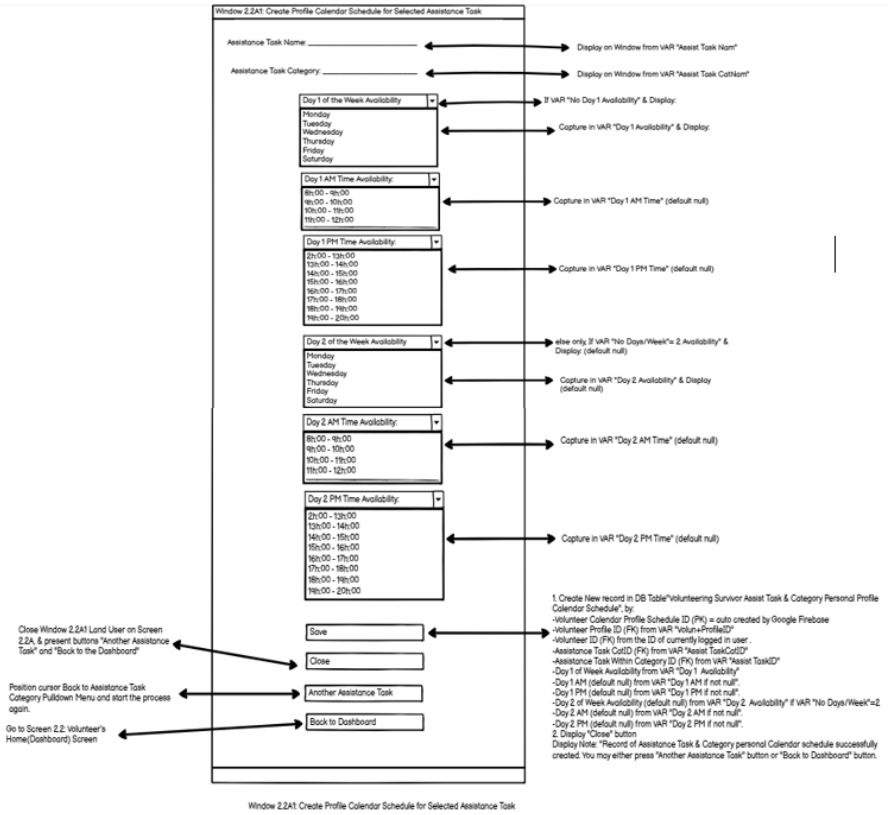


Figure 84: Wireframe of Application Window 2.2A1 - Create Profile Calendar Schedule

CreateProfileCalendarSchedule.kt

```
41    fun CreateProfileCalendarSchedule() {
100       ) {
105           ) {
198                   onClick = {
215                       "VolunteerID" to currentVolunteerID,
216                       "AssistTaskCatID" to assistTaskCatID,
217                       "AssistTaskID" to assistTaskID,
218                       "DaysSelected" to selectedDays,
219                   )
220
221                   schedule.forEach { (dayKey, triple) ->
222                       val dayNumber = dayKey.filter { it.isDigit() }
223                       scheduleData["Day$dayNumber Availability"] = triple.first?.takeIf { it.isNotBlank() } ?: null
224                       scheduleData["Day $dayNumber AM"] = triple.second?.takeIf { it.isNotBlank() } ?: null
225                       scheduleData["Day $dayNumber PM"] = triple.third?.takeIf { it.isNotBlank() } ?: null
226                   }
227                   isSaving = true
228                   db.collection( collectionPath: "VolunteeringSurvivorAssistTaskCategoryPersonalProfileCalendarSchedule")
229                       .add(scheduleData)
230                       .addOnSuccessListener { documentReference ->
231                           val generatedID = documentReference.id
232                           db.collection( collectionPath: "VolunteeringSurvivorAssistTaskCategoryPersonalProfileCalendarSchedul
233                               .document(generatedID)
234                               .update( field: "VolunteerCalendarProfileScheduleID", generatedID)
235                               .addOnSuccessListener {
```

Figure 85: Snippet of Code for Application Window 2.2A1 - Dynamic Field Generation and Data Saving Logic

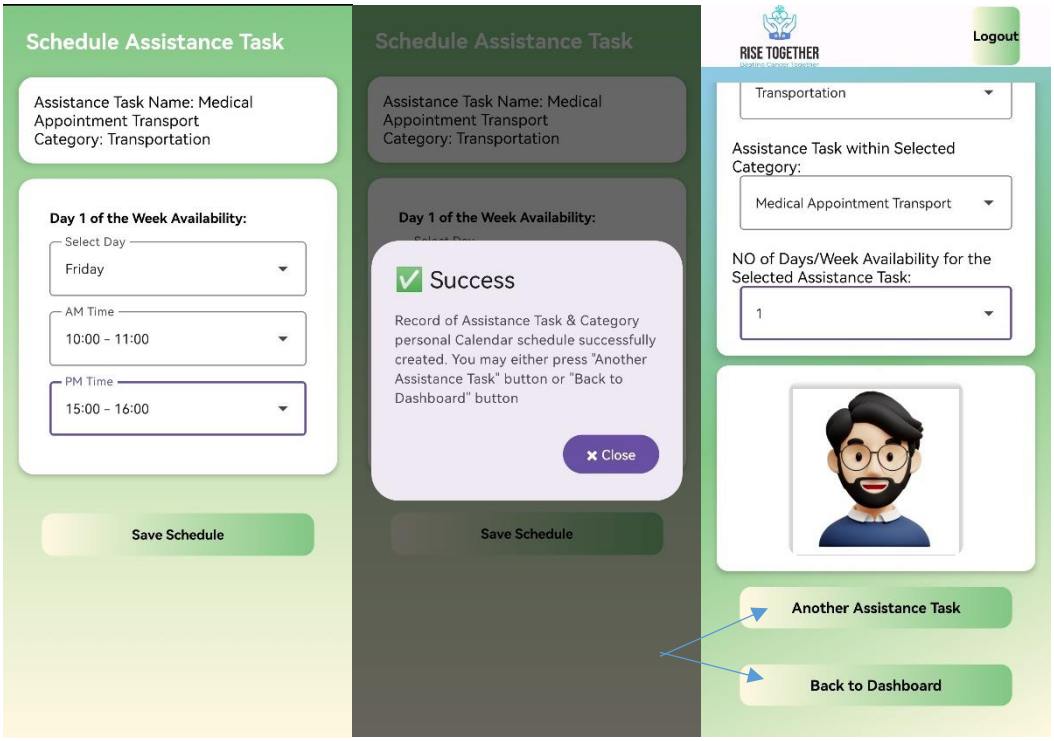


Figure 86: Snapshots of the Actual Application Window 2.2A1 - Calendar Schedule Selection and Confirmation

4.1.11 Application Screen 2.2B - View Patient's Requests for Required Assistance

The Volunteers are taken to this screen, where they can view List of requests made by patients. To begin, the volunteer chooses a week using a dropdown menu. After that, the system shows all the patient requests that are connected to that week and to that specific volunteer. These requests are displayed in a list, starting with the most recent one and ending with the oldest one. Each task request in the list includes the task name, the date it was requested, and the time. There is also a View button next to each task. When the volunteer clicks this button, a pop-up window opens where they can see full details about the request and decide to accept or reject it. The screen has a clean and simple design. It is easy to read and use. There are also two navigation buttons. One is Select "Another Week Commencing Date", which lets the volunteer choose a different week and refresh the request list. The other is Back to Dashboard, which takes them back to their home screen.

Visual Presentation: Figure 87 presents the wireframe of the application Screen 2.2B - *View Patient`s Requests for Required Assistance* Screen. Figure 4.40 shows a snippet of code for the screen, while Figure 88 displays a snapshot of the actual application Screen 2.2B - *View Patient`s Requests for Required Assistance.*

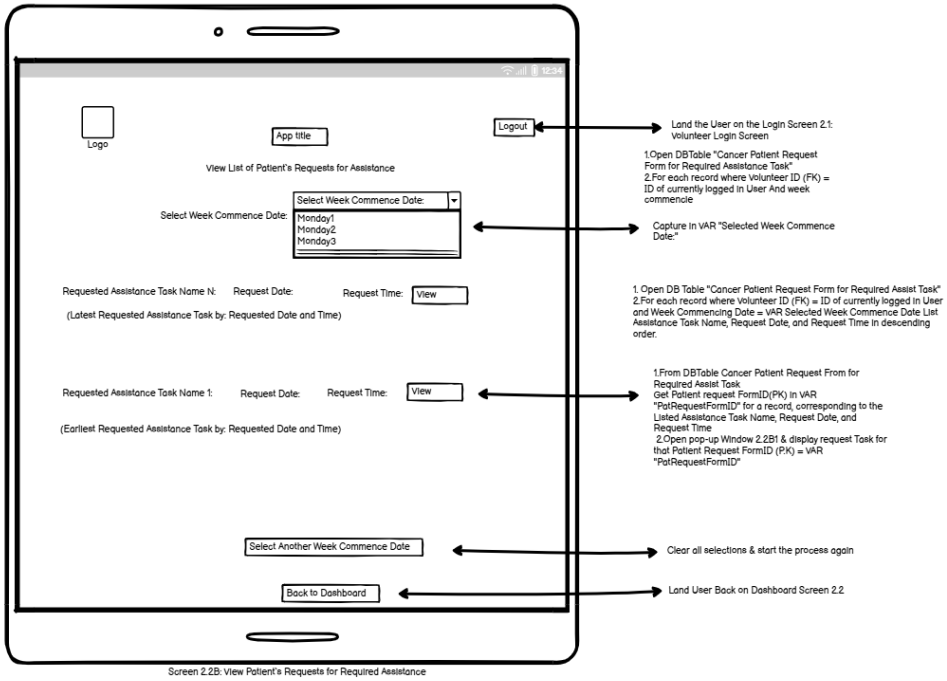


Figure 87: Wireframe of Application Screen 2.2B - View Patient's Requests for Required Assistance, showing the dropdown menu for week selection, list of requests, and navigation buttons.

```kotlin
fun ViewPatientRequestPopup(
    AlertDialog(
        onDismissRequest = onDismiss,
        title = { Text( text: "Patient Request Details") },
        text = {
            if (requestData == null || patientInfo == null) {
                CircularProgressIndicator()
            } else {
                val taskName = requestData?.get("AssistTaskName") ?: "N/A"
                val week = requestData?.get("WeekCommenceDate") ?: "N/A"
                val day1 = requestData?.get("Day1 Availability") as? String?
                val day2 = requestData?.get("Day2 Availability") as? String?
                val time1AM = requestData?.get("Day 1 AM") as? String?
                val time1PM = requestData?.get("Day 1 PM") as? String?
                val time2AM = requestData?.get("Day 2 AM") as? String?
                val time2PM = requestData?.get("Day 2 PM") as? String?

                val preferredDay = if (!day1.isNullOrEmpty()) day1 else day2
                val preferredTime = when {
                    !time1AM.isNullOrEmpty() -> time1AM
                    !time1PM.isNullOrEmpty() -> time1PM
                    !time2AM.isNullOrEmpty() -> time2AM
                    !time2PM.isNullOrEmpty() -> time2PM
                    else -> "N/A"
                }
```

Figure 88: Snippet of Code for Application Screen 2.2B - View Patient's Requests for Required Assistance

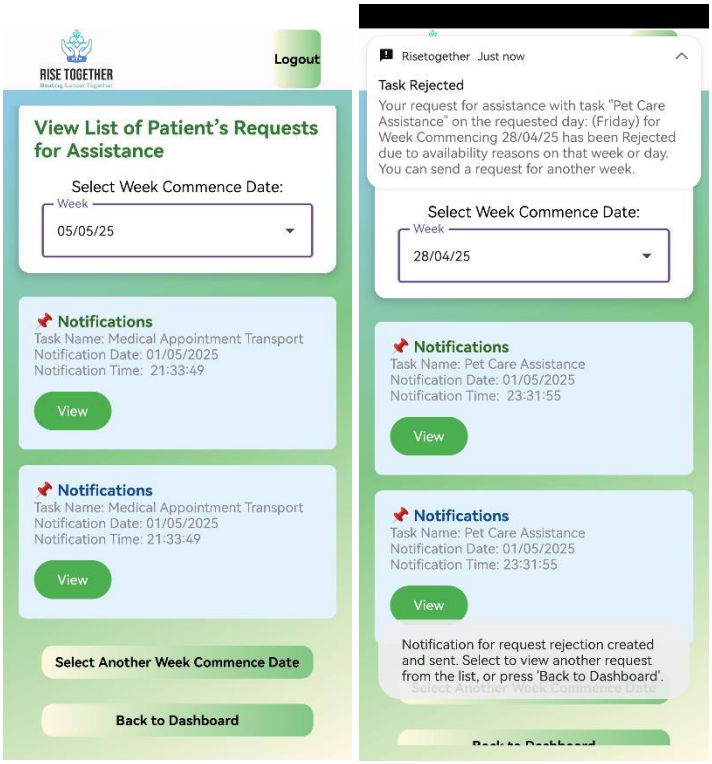


Figure 89: Snapshots of the Actual Application Screen - View Patient's Requests for Required Assistance.

4.1.12 Application Window 2.2B1 - View Selected Patient Request & Send Notification

When the volunteer taps on the "View" button from Screen 2.2B-View Requests for Task Assistance, the Window 2.2B1 opens, providing detailed information regarding the task request that a patient has made. It enables the volunteer to see exactly what is required. The information presented in this window is automatically created from the task that was chosen, the week the patient selected, and the actual days and time slots that the volunteer had indicated as available. This makes the request pertinent, with only information that corresponds with the volunteer's availability presented. The window's layout is like that of a message or e-mail. It begins with a brief, respectful introduction of the volunteer's name and patient's request text. It has the name of the assistance task, the selected week, the desired day and time, and patient contact information (name, email address, and phone number).

The volunteer has two options at the bottom of the window: a button to accept the request and a button to reject the request. If the volunteer selects "Accept", the system will launch another window with notification text to be send to the requesting patient. A different confirmation window will appear if the volunteer selects "Reject". The idea is to assist in keeping the process organized, efficient, and respectful for both the patient and the volunteer.

Visual Presentation:

Figure 90 presents the wireframe of the application Window 2.2B1 - *View Selected Patient`s Request for Assistance & Send Notification of Acceptance/Rejection*. Figure 91 shows a snippet of code for the window, while Figure 92 displays a snapshot of the actual application Window 2.2B1 - "*View Selected Patient`s Request for Assistance & Send Notification of Acceptance/Rejection".*

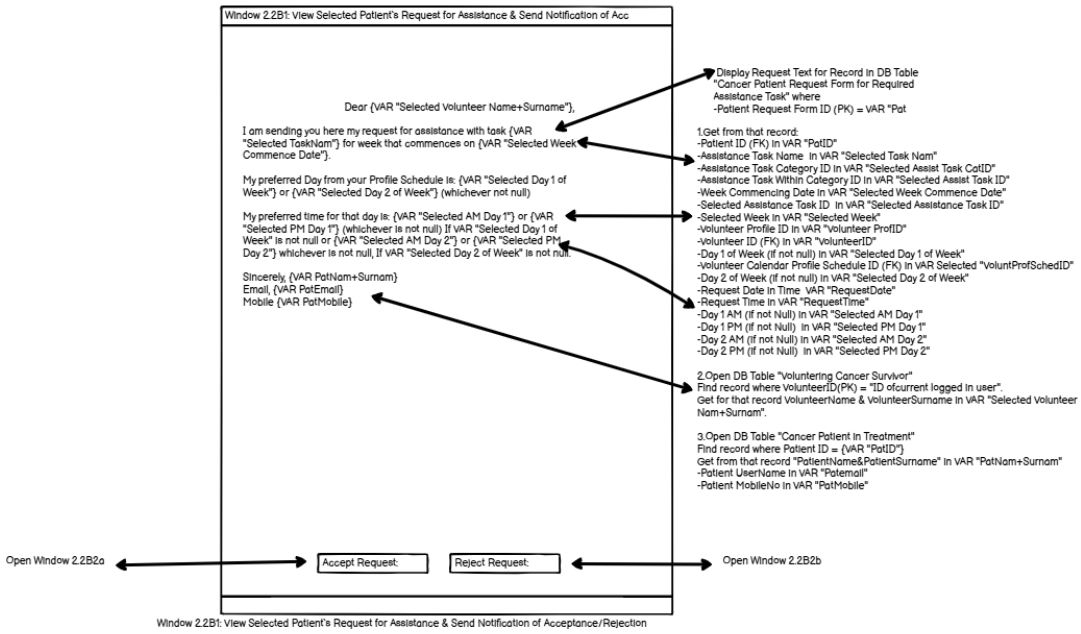


Figure 90: Wireframe of Application Window 2.2B1 - "View Selected Patient`s Request for Assistance & Send Notification of Acceptance/Rejection".

```
ViewPatientRequestsScreen.kt
364     fun ViewPatientRequestPopup(
419             text = {
435                 !time1PM.isNullOrEmpty() -> time1PM
436                 !time2AM.isNullOrEmpty() -> time2AM
437                 !time2PM.isNullOrEmpty() -> time2PM
438                 else -> "N/A"
439             }
440
441             requestData = requestData?.toMutableMap()?.apply {
442                 put("RequestedDay", preferredDay ?: "N/A")
443                 put("RequestedTime", preferredTime ?: "N/A")
444             }
445
446             Column(modifier = Modifier.verticalScroll(rememberScrollState())) {
447                 Text( text: "Dear, $volunteerName.", fontSize = 16.sp)
448                 Spacer(modifier = Modifier.height(8.dp))
449                 Text( text: "I am sending you here my request for assistance with the task \"$taskName\"
450                 Spacer(modifier = Modifier.height(8.dp))
451                 Text( text: "My preferred Day from your Profile Schedule is: $preferredDay", fontSize = 1
452                 Text( text: "My preferred Time for that day is: $preferredTime", fontSize = 14.sp)
453                 Spacer(modifier = Modifier.height(12.dp))
454                 Text( text: "Sincerely, ${patientInfo?.get("name")}")
455                 Text( text: "Email: ${patientInfo?.get("email")}")
456                 Text( text: "Mobile: ${patientInfo?.get("mobile")}")
457             }
458         }
nple > risetogether > Volunteer > ViewPatientRequestsScreen.kt > ViewPatientRequestPopup   388:60  CRLF  UTF-8  4 spaces
```

Figure 91: Snippet of Code for Window 2.2B1 "View Selected Patient`s Request for Assistance & Send Notification of Acceptance/Rejection".

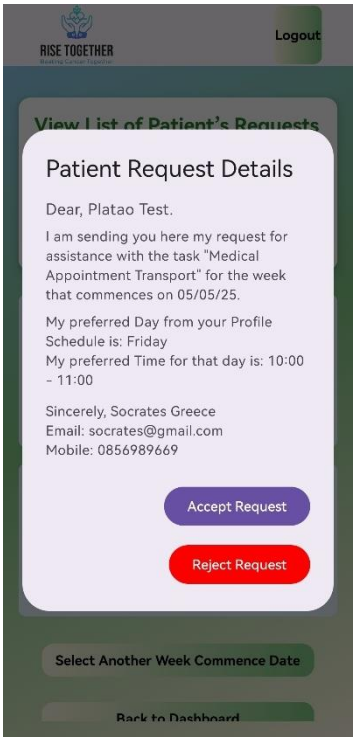


Figure 92: Snapshot of the Actual Application Window 2.2B1 "View Selected Patient`s Request for Assistance & Send Notification of Acceptance/Rejection".

4.1.13 Application Window 2.2B2a - Send Notification of Acceptance of Request for Assistance

Upon the volunteer's acceptance of the patient's request, there is a confirmation window. This window serves to complete the process since it enables the volunteer to check the information one last time and send off an official notification to the patient that the request has been accepted. The pre-defined text of the notification is already filled with the correct details. The volunteer must only verify that all is correct, including the name of the task, day, and week that the assistance will take place, and their contact details. The message is written in a light, polite style, with a sentence like "Will see you then!" to confirm the arrangement, and when the user presses button "Send", the notification will be sent to the requesting patient. Upon clicking this button, the system stores the notification in the database and puts in place a reminder in the patient's calendar. The second button closes the window and returns the user to the list of task requests.

This is intended to keep communication simple, fast, and transparent. It keeps the patient, and the volunteer informed and ensures that the agreement is registered in the system.

Visual Presentation:

Figure 93 presents the wireframe of the application Window 2.2B2a*: "Send Notification of Acceptance of Request for Assistance"*. Figure 94 shows a snippet of code for the screen, and Figure 95 displays a snapshot of the actual application Window 2.2B2a - "*Send Notification of Acceptance of Request for Assistance".*

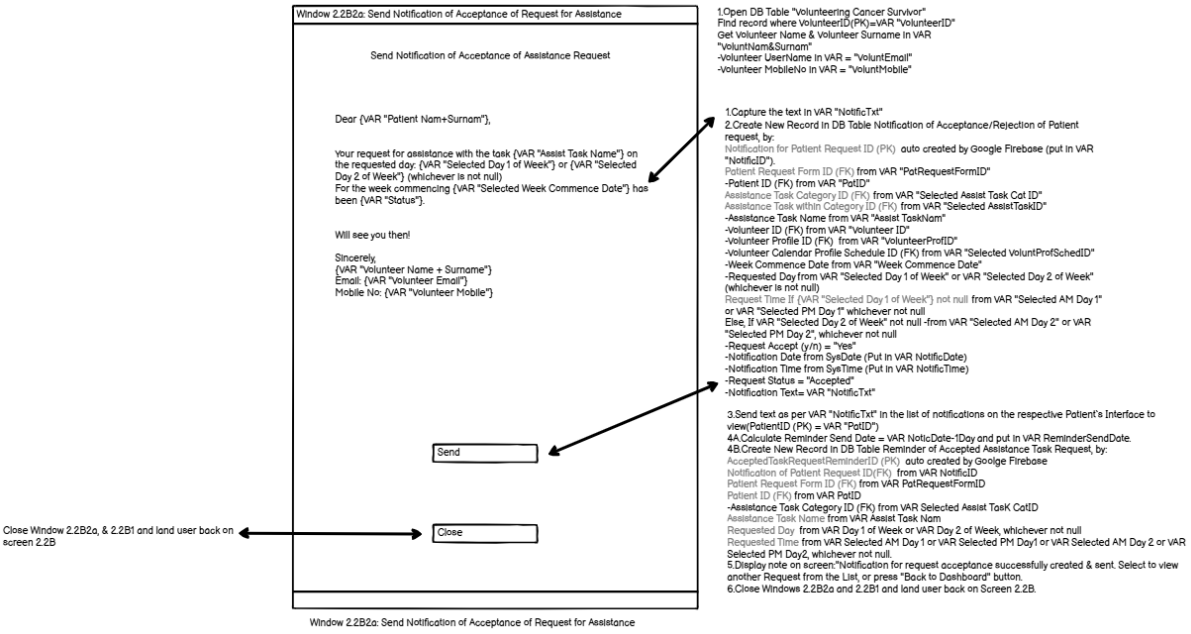


Figure 93: Wireframe of Application Window 2.2B2a - "Send Notification of Acceptance of Request for Assistance".

ViewPatientRequestsScreen.kt

```
42      fun ViewPatientRequestsContent() {
61          fun SmallGradientButton(text: String, onClick: () -> Unit) {
90      }
91      // Load weeks
92      LaunchedEffect(volunteerId) {
93          db.collection( collectionPath: "CancerPatientRequestFormRequiredAssistTask")
94              .whereEqualTo( field: "VolunteerID", volunteerId)
95              .get()
96              .addOnSuccessListener { snapshot ->
97                  val weeks = snapshot.documents.mapNotNull { it.getString( field: "WeekCommenceDate") }.
98                  availableWeeks = weeks.sorted()
99              }
100     }
101
102     fun fetchRequestsForWeek(week: String) {
103         db.collection( collectionPath: "CancerPatientRequestFormRequiredAssistTask")
104             .whereEqualTo( field: "VolunteerID", volunteerId)
105             .whereEqualTo( field: "WeekCommenceDate", week)
106             .get()
107             .addOnSuccessListener { documents ->
108                 val sorted = documents.sortedBy {
109                     val date = it.getString( field: "RequestDate") ?: ""
110                     val time = it.getString( field: "RequestTime") ?: ""
111                     "$date $time"
112                 }
```

Figure 94: Snippet of Code for Window 2.2B2a - "Send Notification of Acceptance of Request for Assistance".



Figure 95: Snapshot of the Actual Application Window 2.2B2a - "Send Notification of Acceptance of Request for Assistance".

4.1.14 Application Window 2.2B2b - Send Notification of Rejection of Request for Assistance

If the volunteer is unable to attend to the patient's request, because of availability issue on the requested day, this Window is used to send a polite decline notification. The message intends to inform the patient kindly and respectfully that the request cannot be accepted right now, and to advise the patient to place a request for another week and day. The message is in respectful and professional language. It contains the patient's name, the requested assistance task name, and the week in which the task was requested. It briefly explains that the reason behind the rejection is based on the volunteer's availability for the requested day and time. Like the acceptance notification, this one displays the volunteer's name, phone number, and e-mail address. This maintains the respectful and transparent exchange on both ends.

Two options are presented at the bottom of the window. Pressing the Send button, the volunteer can send the rejection notification, and after doing so, the system creates a record of the rejection in the database and notifies the patient. The second button closes the notification window and brings the volunteer back to the list of requests window.

Visual Presentation:

Figure 96 presents the wireframe of the application Window 2.2B2b: "*Send Notification of Rejection of Request for Assistance*". Figure 97 shows a snippet of code for the window, and Figure 98 displays a snapshot of the actual application Window 2.2B2b - "*Send Notification of Rejection of Request for Assistance".*

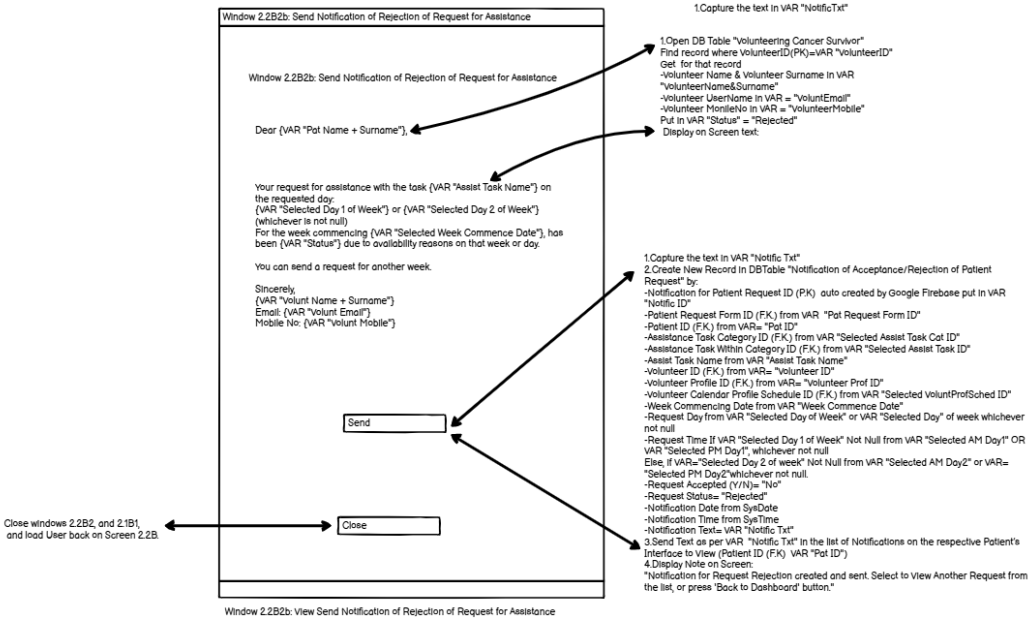


Figure 96: Wireframe of Application Window 2.2B2b: Send Notification of Rejection of Request for Assistance

NotificationRejectScreen.kt

```
19  fun NotificationRejectScreen(
41          ?: requestData["Day 2 AM"] as? String?
42          ?: requestData["Day 2 PM"] as? String?
43          ?: "N/A"
44
45      val status = "Rejected"
46      val volunteerId = requestData["VolunteerID"] as? String
47      val patientId = requestData["CancerPatientID"] as? String
48      var patientFcmToken by remember { mutableStateOf<String?>( value: null) }
49
50      // Load info
51      LaunchedEffect(requestData) {
52          if (volunteerId != null) {
53              db.collection( collectionPath: "VolunteeringCancerSurvivor").document(volunteerId)
54                  .get()
55                  .addOnSuccessListener { doc ->
56                      val name = doc.getString( field: "VolunteerName") ?: ""
57                      val surname = doc.getString( field: "VolunteerSurname") ?: ""
58                      val email = doc.getString( field: "VolunteerUsername") ?: "-"
59                      val phone = doc.getString( field: "VolunMobile") ?: "-"
60                      volunteerInfo = mapOf(
61                          "name" to "$name $surname",
62                          "email" to email,
63                          "mobile" to phone
64                      )
```

src > main > java > com > example > risetogether > Volunteer > NotificationRejectScreen.kt > NotificationRejectScreen

Figure 97: Snippet of Code for Window 2.2B2b: Send Notification of Rejection of Request for Assistance



Figure 98:Snapshot of the Actual Application Window 2.2B2b: Send Notification of Rejection of Request for Assistance

4.1.15 Application Screen 2.2C - View List of Feedback and Ratings for Requested Tasks

The volunteers utilize this screen to view feedback and reviews from patients after providing them with assistance. This is vital in completing the support loop, with the ability to reflect on what has been done and feel appreciated. It is also value-adding in the sense that it incentivizes improved service and improved communication between volunteers and patients. When selecting this screen, the volunteer first chooses a "Week Commencing Date" from a dropdown menu. From that choice, the application retrieves all feedback received for that week that is associated with the signed-in volunteer and displays it in a summary form, which consist of the name of the assistance task, the timestamp when the feedback was submitted, and a link to see the message in its entirety. The list of feedback is presented in reverse chronological order with the oldest feedback from that week at the bottom of the list. The design is easy and straightforward to navigate. They can switch the week by choosing a new week commencing date, and there is also a return button to the dashboard screen. To view the entire feedback and rating message, the volunteer must click on the "View" button beside each message in the list, which opens up a window with the detailed message.

Visual Presentation:
Figure 99 presents the wireframe of the application Screen 2.2C "*View List of Feedback and Ratings for Requested Tasks*" Screen. Figure 100 shows a snippet of code for the screen, while Figure 101 displays a snapshot of the actual application Screen 2.2C - "*View List of Feedback and Ratings for Requested Tasks".*

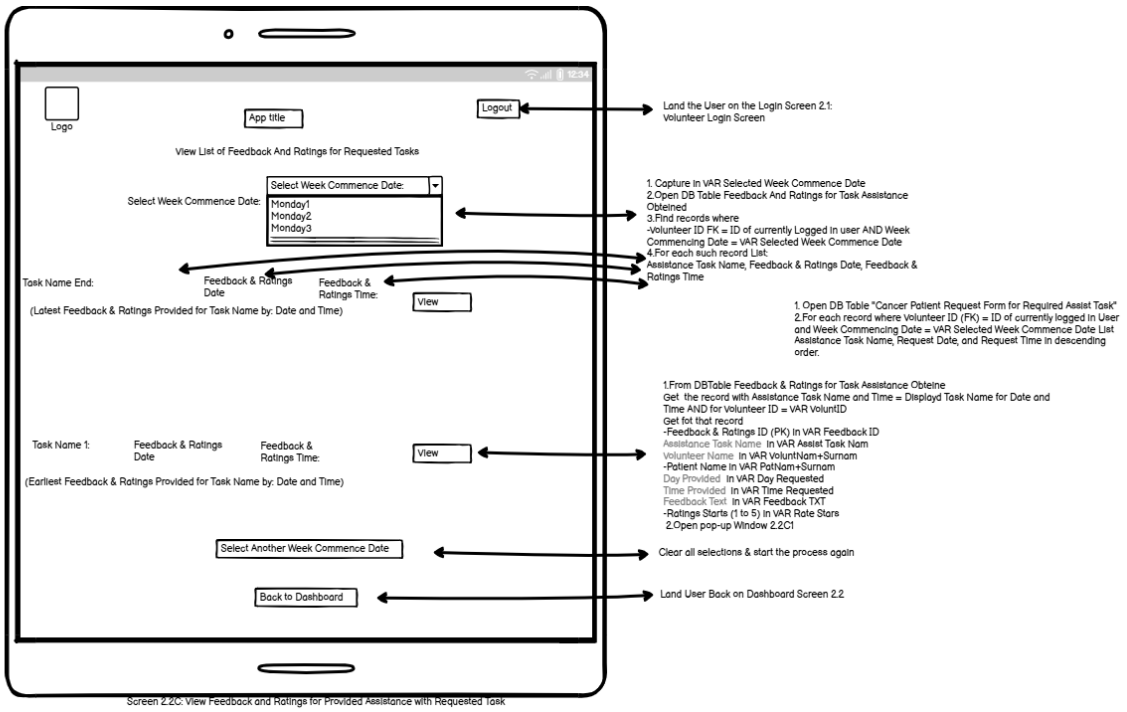


Figure 99: Wireframe of Application Screen 2.2C "View List of Feedback and Ratings for Requested Tasks"

ViewFeedbackRatingsScreen.kt

```
47      fun ViewFeedbackRatingsContent() {
48
49
50          LaunchedEffect(volunteerId) {
51              db.collection( collectionPath: "FeedbackRatings")
52                  .whereEqualTo( field: "VolunteerID", volunteerId)
53                  .get()
54                  .addOnSuccessListener { snapshot ->
55                      val weeks = snapshot.documents.mapNotNull {
56                          it.getString( field: "WeekCommenceDate")
57                      }.distinct()
58                      availableWeeks = weeks.sorted()
59                  }
60          }
61
62
63          fun fetchFeedbacksForWeek(week: String) {
64              db.collection( collectionPath: "FeedbackRatings")
65                  .whereEqualTo( field: "VolunteerID", volunteerId)
66                  .whereEqualTo( field: "WeekCommenceDate", week)
67                  .get()
68                  .addOnSuccessListener { docs ->
69                      val list = docs.map {
70                          mapOf(
71                              "id" to it.id,
```

main › java › com › example › risetogether › Volunteer › ViewFeedbackRatingsScreen.kt › ViewFeedbackRatingsContent

Figure 100: Snippet of Code for Application Screen 2.2C "View List of Feedback and Ratings for Requested Tasks"

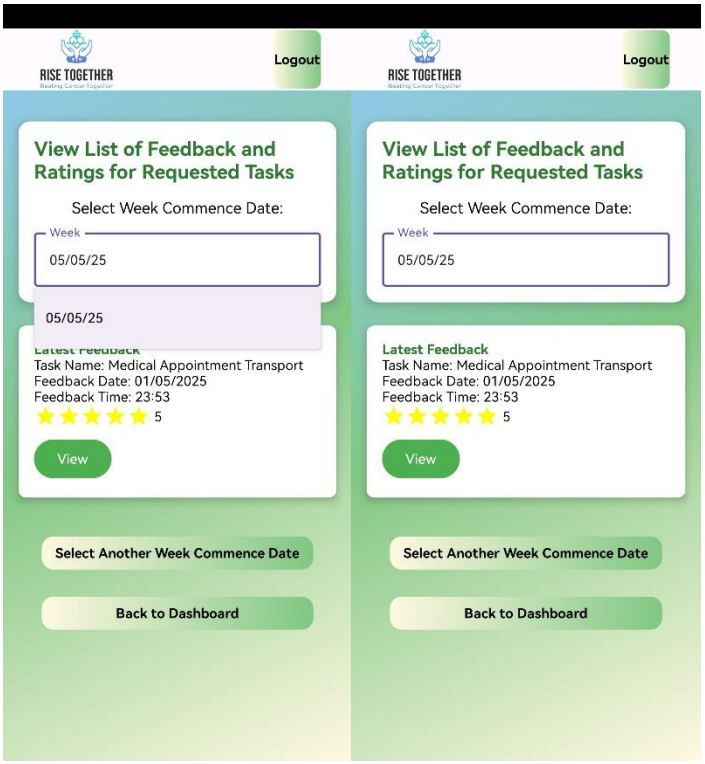


Figure 101: Snapshot of the Actual Application Screen 2.2C "View List of Feedback and Ratings for Requested Tasks".

4.1.16 Window 2.2C1 - Display Feedback & Ratings Details for Selected Assistance Task

When clicking on the "View" button in the feedback list screen, the volunteer is presented with a pop-up Window that contains complete information, regarding the feedback provided about the assistance task. This window presents both written comments and star ratings, making it easy to visualise how the patient perceived the help received. The information consists of the week that the assistance task was performed, the name of the task, the name of the volunteer, the day and time support was offered, the name of the patient, a numeric star rating (e.g., four stars out of five), and a brief patient written feedback comment. There is also a "Close" button, which closes the window and brings the volunteer back to the list of feedback entries screen.

Visual Presentation:
Figure 102 presents the wireframe of the application Window 2.2C1 *"Display Feedback & Ratings Details for Assistance of Selected Task"*. Figure 103 shows a snippet of code for the window, while Figure 104 displays a snapshot of the actual application Window 2.2C1 *"Display Feedback & Ratings Details for Assistance of Selected Task".*

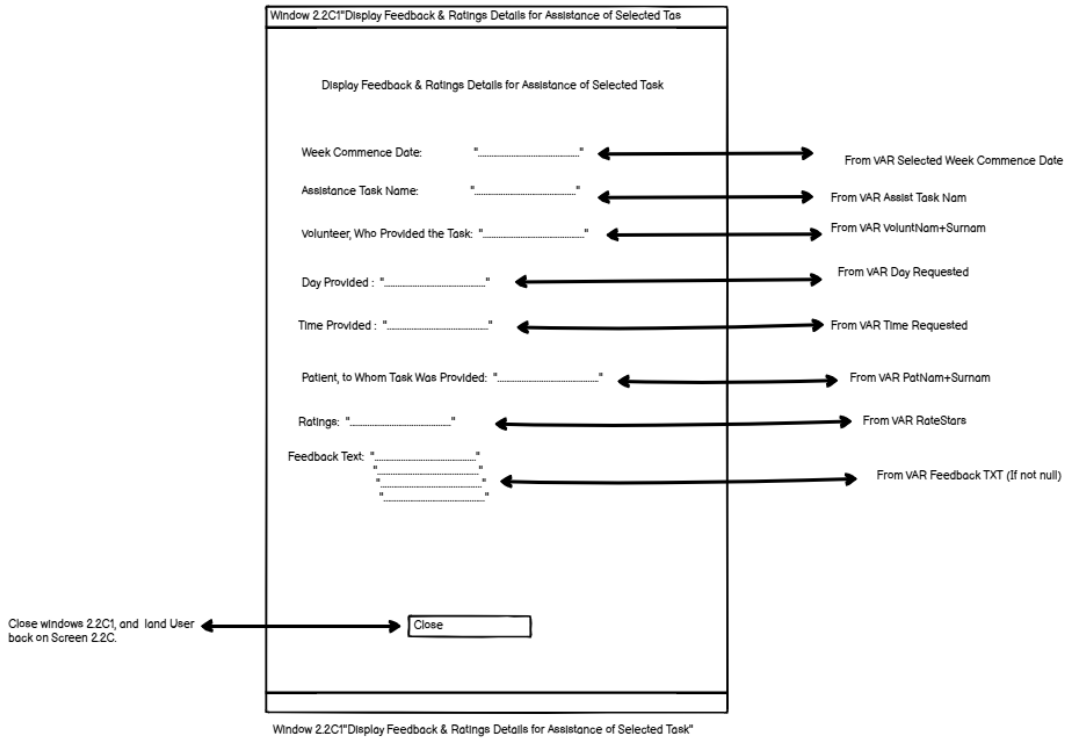


Figure 102: Wireframe of Application Window 2.2C1 "Display Feedback & Ratings Details for Assistance of Selected Task"

ViewFeedbackRatingsScreen.kt

```
47      fun ViewFeedbackRatingsContent() {
83          fun fetchFeedbacksForWeek(week: String) {
84              db.collection( collectionPath: "FeedbackRatings")
85                  .whereEqualTo( field: "VolunteerID", volunteerId)
86                  .whereEqualTo( field: "WeekCommenceDate", week)
87                  .get()
88                  .addOnSuccessListener { docs ->
89                      val list = docs.map {
90                          mapOf(
91                              "id" to it.id,
92                              "taskName" to (it.getString( field: "AssistTaskName") ?: ""),
93                              "date" to (formatDate(it.getTimestamp( field: "FeedbackDateTime"))),
94                              "time" to (formatTime(it.getTimestamp( field: "FeedbackDateTime"))),
95                              "rating" to it.getLong( field: "RatingStars")?.toString().orEmpty()
96                          )
97                      }
98                      feedbackList = list
99                  }
100         }
101         Box(
102             modifier = Modifier
103                 .fillMaxSize()
104                 .background(
105                     Brush.linearGradient(
106                         colors = listOf(Color( color: 0xFF90CAF9), Color( color: 0xFF81C784), Color( color: 0xFFFFF8E1))
```

main › java › com › example › risetogether › Volunteer › ViewFeedbackRatingsScreen.kt › ViewFeedbackRatingsContent 69:1

Figure 103: Snippet of Code for Window 2.2C1 "Display Feedback & Ratings Details for Assistance of Selected Task"

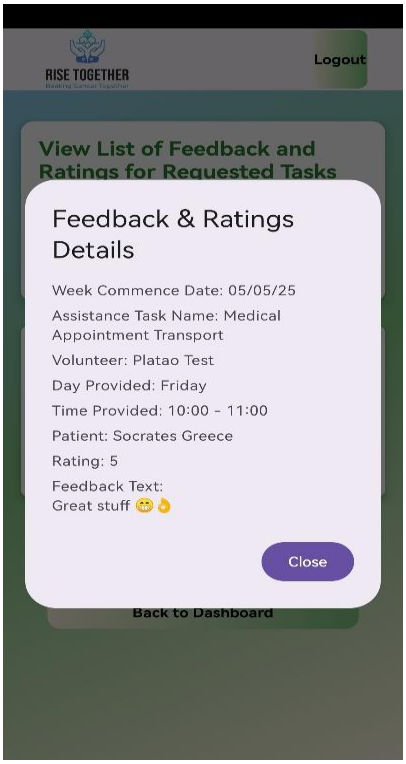


Figure 104: Snapshot of the Actual Application Window 2.2C1 "Display Feedback & Ratings Details for Assistance of Selected Task"

4.1.17 Application Screen 1.1 - Patient Login Screen

The patient's Login screen is where they can input their login credentials to access the system. Like with the Volunteers, Patients are registered by their Doctor. This keeps the system and process under control and secure, because the Doctor verifies the identity of the patient, before the process is initiated. Once the doctor has registered a patient, the system automatically sends an email with a secure link to the patient's email address to reset their password privately. When the patient enters the Login screen, they are prompted to input the email address as per their registration and their re-set password. Pressing the Login button, unless there is an issue, the app will land the patient on their Home (Dashboard) screen. An error message will be displayed if there is an issue, informing that the password and the email don't match record in the system.
If the patient forgets their password, there is also a simple "Forgot Password?" link on the Login screen that enables them to obtain a new reset link.

Visual Presentation:
Figure 105 presents the wireframe of the application Screen 1.1 - Patient in Treatment Login. Figure 106 shows a snippet of code for the screen, and Figure 107 displays a snapshot of the actual application Screen 1.1 - Patient in Treatment Login Screen.

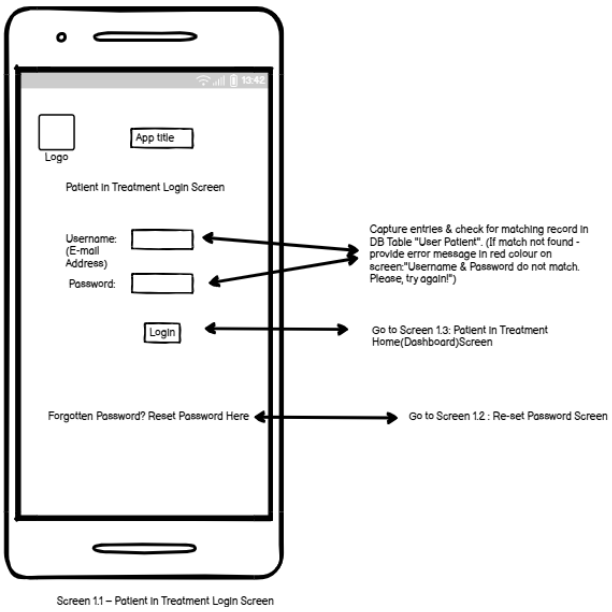


Figure 105: Wireframe of the Application Screen 1.1 - Patient Login Screen

PatientLoginScreen.kt

```
55    fun PatientLoginScreenContent() {
72        ) {
109           ) {
113               ) {
161                   ) {
172   auth.signInWithEmailAndPassword(email, password)
173       .addOnCompleteListener { task ->
174           if (task.isSuccessful) {
175               val userId = auth.currentUser?.uid ?: return@addOnCompleteListener
176
177               //Fetch the Patient record from Firestore
178               db.collection( collectionPath: "PatientinTreatment").document(userId).get()
179                   .addOnSuccessListener { document ->
180                       if (document.exists()) {
181                           val role = document.getString( field: "role") ?: "Unknown"
182
183                           if (role.lowercase() == "patient") {
184                               // 🔥 token FCM
185                               FirebaseMessaging.getInstance().token
186                                   .addOnCompleteListener { tokenTask ->
187                                       if (tokenTask.isSuccessful) {
188                                           val fcmToken = tokenTask.result
189
190                                           FirebaseFirestore.getInstance()
191                                               .collection( collectionPath: "PatientinTreatment")
```

Figure 106: Snippet of Code for Screen 1.1 - Patient Login Screen

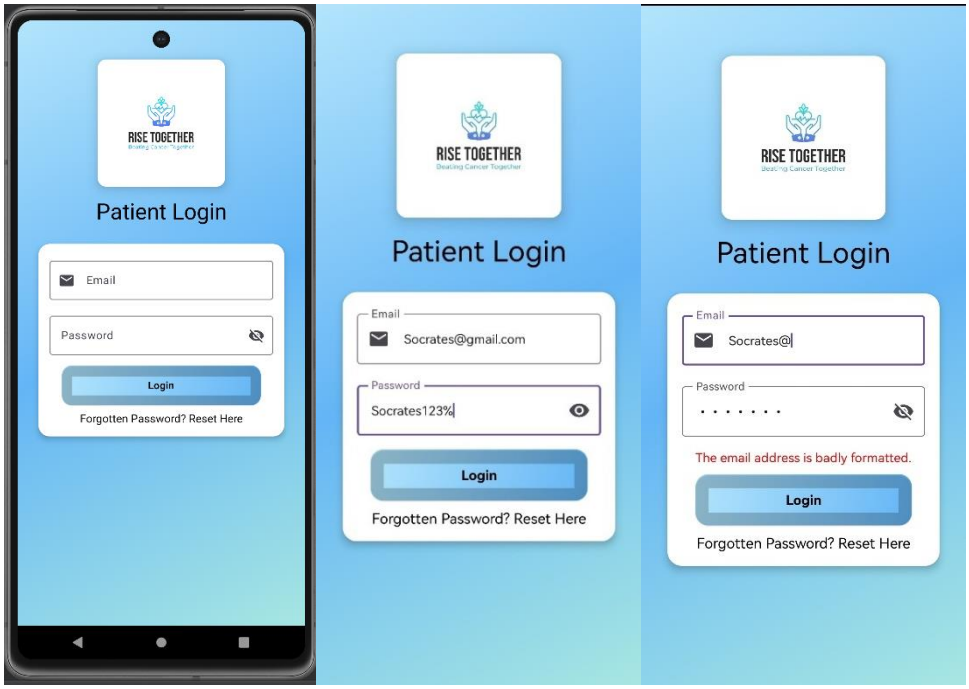


Figure 107: Snapshots of the Actual Application Screen 1.1 - Patient Login Screen

4.1.18 Application Screen 1.2 - Reset Password Screen

The same as in the other type of users, the password reset screen is where individuals can regain access to their account if they forget their password or want to change it. This screen is utilised by all users of the application, including doctors, volunteers, and patients.
As soon as the screen is opened, the user gets one input area in which they type the email address that the doctor used to register them to the system. Once the email is typed, the user clicks on the *send reset link* button. The app sends a reset link to that address securely. When the email is incorrect or not in the system, an error message will appear on the screen.

When the email is accepted, the application will redirect the user back to their login screen.

Visual Presentation:
Figure 108 presents the wireframe of the Reset Password Screen, showing the layout of the input field and the *reset password* button.
Figure 109 includes a snippet of code used to handle the reset request and validate the input.
Figure 110 displays a screenshot of the actual application screen showing the implemented Reset Password Screen.
Figure 111 shows an example of the reset password link received by the user in their email inbox.

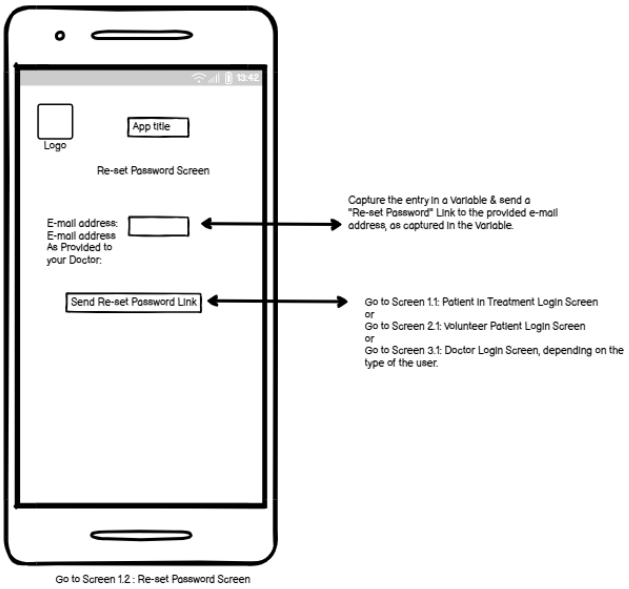


Figure 108: Wireframe of Application Screen 1.2 - Reset Password Screen

PatientResetPasswordScreen.kt

```
43      fun ResetPasswordScreenContent() {
54          ) {
106             // "Send Reset Password Link" Button
107             PatientButton(
108                 text = "Send Reset Password"
109             ) {
110                 if (email.isNotEmpty()) {
111                     val auth = FirebaseAuth.getInstance()
112                     auth.sendPasswordResetEmail(email)
113                         .addOnCompleteListener { task ->
114                             if (task.isSuccessful) {
115                                 message = "A reset password link has been sent to $email"
116                                 Toast.makeText(context, message, Toast.LENGTH_LONG).show()
117                                 context.startActivity(Intent(context, PatientLoginScreen::class.java))
118                             } else {
119                                 message = "Failed to send reset email. Check the email address."
120                             }
121                         }
122                 } else {
123                     message = "Please enter a valid email address!"
124                 }
125             }
126
127             Spacer(modifier = Modifier.height(10.dp))
128
```

app > src > main > java > com > example > risetogether > PatientResetPasswordScreen.kt > ResetPasswordScreenContent 106:12

Figure 109: Snippet of Code for Screen 1.2 - Email Reset Link

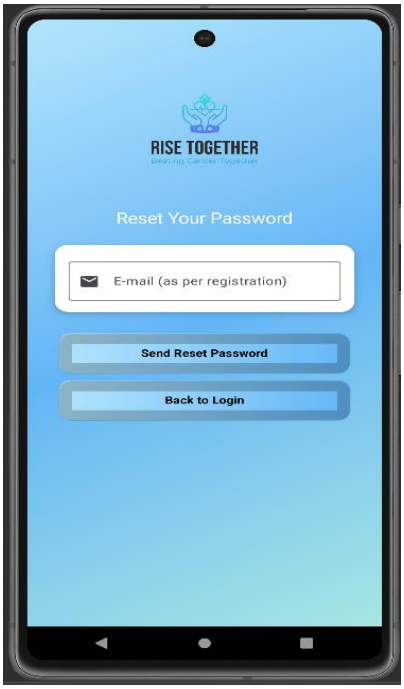


Figure 110: Snapshot of the Actual Application Screen 1.2 - Password Reset Screen

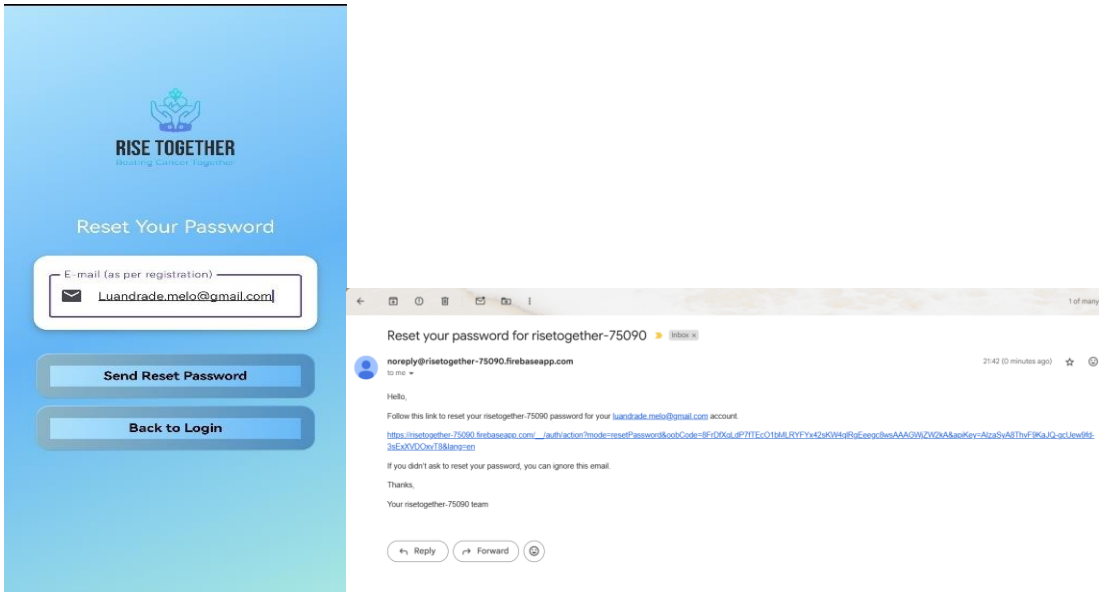


Figure 111: Snapshot of the email with the received link from screen 1.2 - Password Reset screen

4.1.20 Application Screen 1.3 - Patient in Treatment Home (Dashboard) Screen

After successful login, patients are landed on their Home screen. Four buttons represent all the key functions, and each one lets the patient navigate to a different section of the app, as follows:

- Create Profile with Required Assistance Tasks
- View Volunteers for Requirde Task and Send Request
- View Notifications & Reminders for Requested Task
- Provide Feedback & Ratings for Received Assistance

The first one enables the patient to set up their profile, where they can select the assistance category of tasks they require, such as household, garden, or transport. The second one leads

to the list of volunteers that match those requirements, and the patient can send them a request. The third one brings up the screen with all the notifications regarding the patient's requests, whether these were accepted or rejected, and reminders for accepted ones. The last button enables the patient to provide feedback and a rating to any assistance that they received from a volunteer. Also located in the top right corner of the screen is the Logout button, which will allow the patient to conclude their session and safely return to the Login screen.

Visual Presentation:
Figure 112 presents the wireframe of Screen 1.3: Patient in Treatment Home (Dashboard Screen). Figure 113 shows a snippet of code for screen 1.3
Figure 114 displays a screenshot of the actual application Screen 1.3: Patient in Treatment Home (Dashboard Screen)

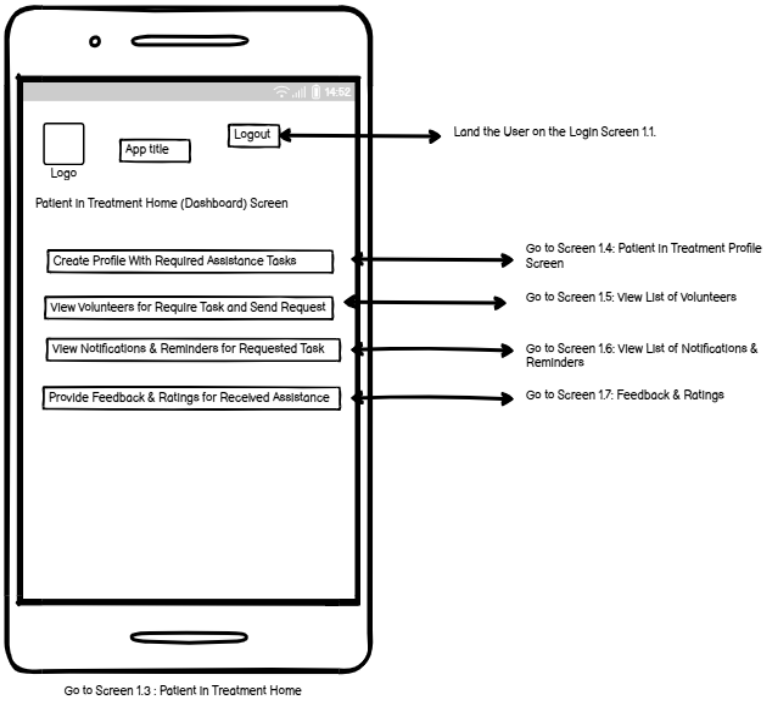


Figure 112: Wireframe of Application Screen 1.3: Patient in Treatment Home (Dashboard Screen)

PatientDashboardScreen.kt

```
42      fun PatientDashboardScreenContent() {
58          ) {
128             ) { paddingValues ->
137                 ) {
145                     ) {
168                         Text(
169                             text = "Patient in Treatment Home",
170                             fontSize = 20.sp,
171                             color = Color.Black,
172                             textAlign = TextAlign.Center
173                         )
174
175                         Spacer(modifier = Modifier.height(12.dp))
176
177                         //  Buttons (Using Standard PatientButton)
178                         PatientButton(
179                             text = "Create Profile With Required Assistance Tasks",
180                             modifier = Modifier
181                                 .fillMaxWidth()
182                                 .padding(2.dp)
183                         ) {
184                             context.startActivity(Intent(context, PatientinTreatmentProfile::class.java)
185                         }
186
187                         PatientButton(
```

Figure 113: Snippet of Code for Application Screen 1.3 - Navigation and Session Control



Figure 114: Snapshot of the Actual Application Screen 1.3 - Patient Dashboard Interface

4.1.21 Application Screen 1.4 - Create Patient Profile with Required Assistance Tasks

This is where the patient gets to determine what type of assistance they require, how often they need it, and the category under which the assistance will be classified. This will subsequently be utilized by the system to assist in linking them with suitable volunteers. The patient first chooses an assistance category from a drop-down menu, like domestic help, gardening, or transportation. Once one of these general support types is selected, there is a second dropdown menu with specific tasks from that selected category. For instance, if the patient chooses "Gardening" from the first list, the options may be hedge trimming, pressure washing, or grass mowing.

Once the assistance task is chosen, the patient determines the number of times each week that they will require the assistance; they may select from one to five days a week, as appropriate for their needs. When all options are filled in, the patient can tap on the *Save* button. The system saves their Profile assistance task record in the database. The patient can tap on "Another Assistance Task" button, which will reset the form and allow them to start the process again, or they can return on the main dashboard screen by tapping the "Back to Dashboard" button. The screen proceeds one task profile record at a time.

Visual Presentation:

Figure 115 presents the wireframe of Screen 1.4: Patient in Treatment Profile Screen.
Figure 116 includes a snippet of code for Screen 1.4
Figure 117 displays a screenshot of the actual application Screen 1.4: Patient in Treatment Profile Screen

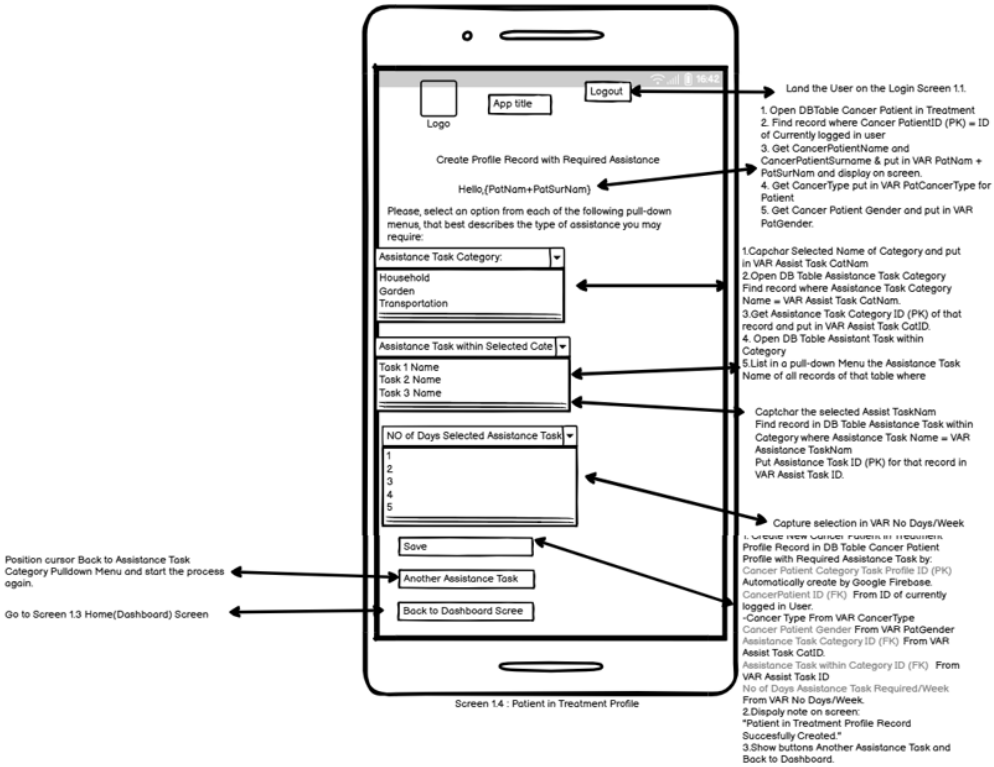


Figure 115: Wireframe of Application Screen 1.4 - Patient Profile with Required Assistance Tasks

```
PatientinTreatmentProfile.kt
44    fun PatientinTreatmentProfileContent() {
184       ) {
218           ) { paddingValues ->
227               ) {
401                   GradientButton(text = "Save", enabled = isSaveEnabled) {
402                       val userId = auth.currentUser?.uid ?: return@GradientButton
403
404
405                       if (selectedCategory.isBlank() || selectedTask.isBlank() || selectedDays.isBlank()) {
406                           errorMessage = "Please fill in all required fields before saving."
407                           return@GradientButton
408                       }
409
410
411                       val patientData = hashMapOf(
412                           "CancerPatientCategoryTaskProfileID" to db.collection( collectionPath: "CancerPatientProfile").document().id, //PK
413                           "CancerPatientID" to userId, // FK
414                           "PatCancerType" to patientCancerType,
415                           "CancerPatientGender" to patientGender,
416                           "AssistTaskCatID" to selectedCategory, // FK
417                           "AssistTaskID" to selectedTaskID,
418                           "NoOfDaysPerWeek" to selectedDays
419                       )
420
421                       db.collection( collectionPath: "CancerPatientProfile").add(patientData)
```

Unsupported API ⚠1 ⚠3 ✓21

Figure 116: Snippet of Code for Screen 1.4 - Saving Task and Schedule Preferences

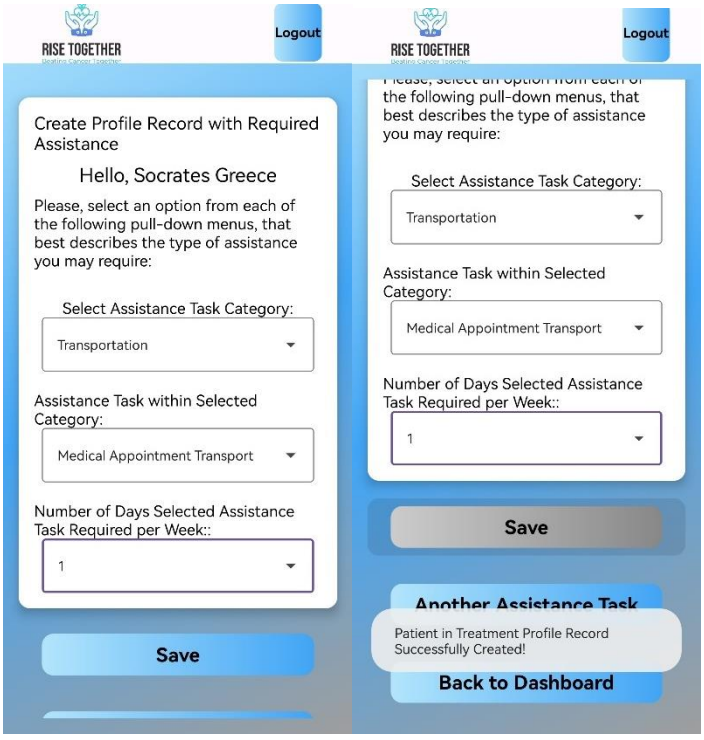


Figure 117: Snapshots of the Actual Application Screen 1.4 - Patient Profile with Required Assistance Tasks

4.1.22 Application Screen 1.5 - View List of Volunteers for Required Assistance Task

On this screen the patient selects general category of support they require from a drop-down menu (household assistance, garden work, or transportation), as registered in their profile. Once selected, a second dropdown displays list of assistance tasks, specific to that category. For instance, if the patient chooses category "household," possible options may include Cleaning & Tidying up, Meal preparation, Laundry & Ironing, or Pet Care Assistance. As soon as the patient selects the task, the system automatically finds in the database the volunteers, available for the category and task chosen and does a match in the database with gender and type of cancer. A list of volunteers, matching these criteria will appear on the screen. The patient can select a volunteer from the list by clicking on the Volunteer's name which will presents the details of the selected volunteer in a pop-up window. A button "Back to Dashboard" at the bottom of the screen, to bring patient back on the home screen.

Visual Presentation:

Figure 118 presents the wireframe of Screen 1.5: "*View List of Volunteers for Required Assistance Task".*

Figure 119 includes a snippet of code for Screen 1.5

Figure 120 displays a screenshot of the actual application Screen 1.5:" View List of Volunteers for Required Assistance Task."

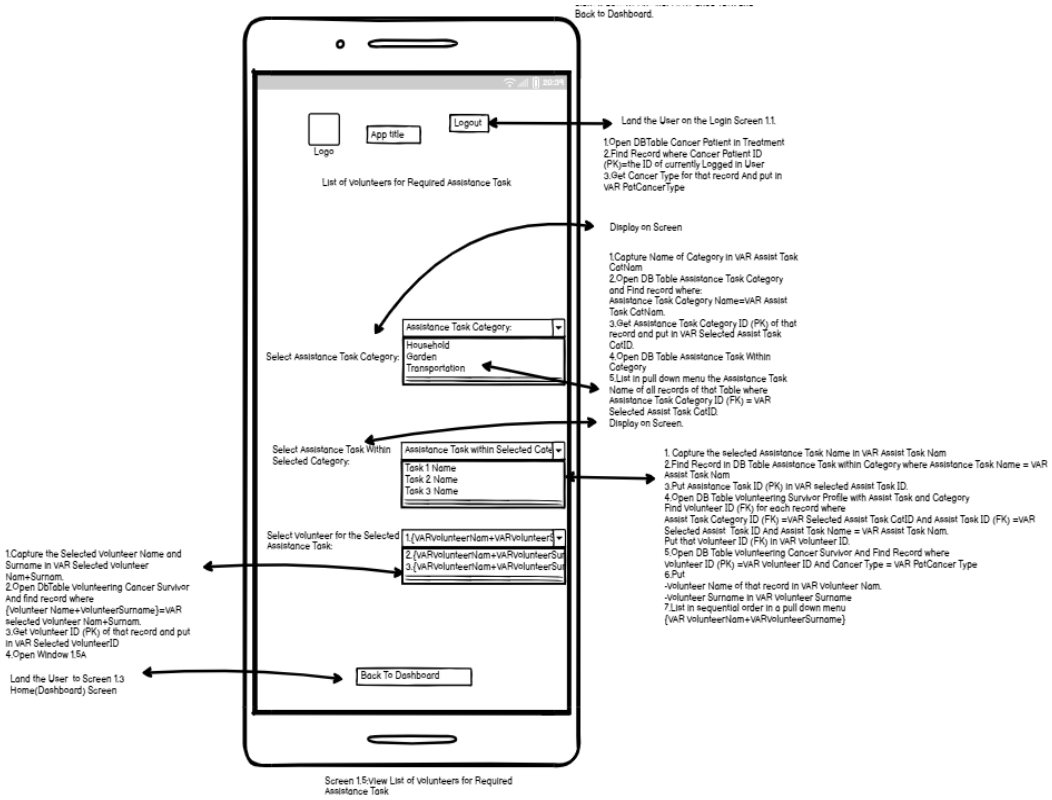


Figure 118: Wireframe of Application Screen 1.5: List of Volunteers for Required Assistance Task

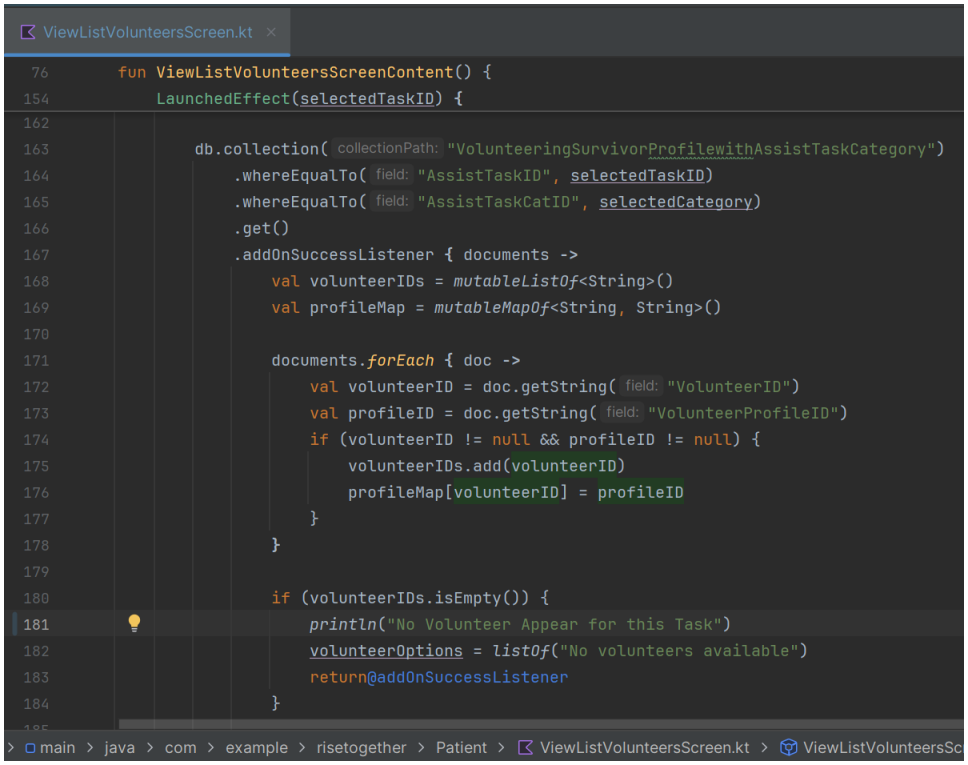


Figure 119: Snippet of Code for Screen 1.5

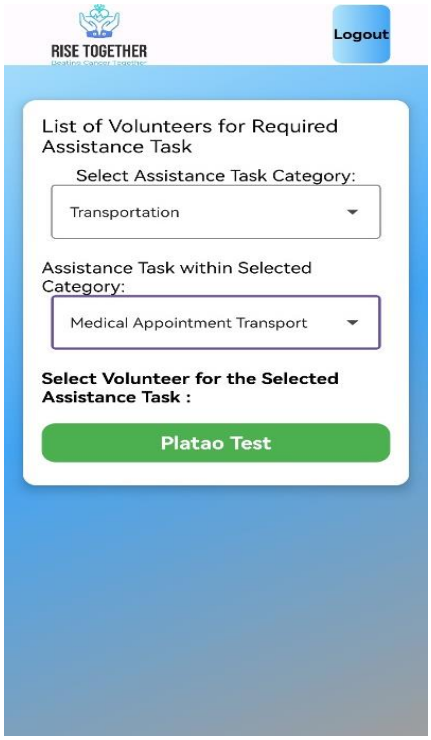


Figure 120: Snapshot of the Actual Application Screen 1.5 - Matched Volunteers by Task, Cancer Type, and Gender.

4.1.23 Application Window 1.5A - View Details of Selected Volunteer & Place Request

After the patient makes a choice of volunteer from the list, a pop-up window opens to present details and a picture of selected volunteer. The patient can view the volunteer's name, email address, mobile phone number, gender, cancer type, and what sort of task the volunteer is willing to assist with. The availability of the volunteer is also presented, and that is confined to max of two days chosen while volunteer created a profile record for that assistance task. The

patient has the option to select which of these two days they want to utilize and choose a time slot (morning or afternoon). Once an option is selected, the patient selects the week in which they prefer the assistance. From a dropdown menu for *Week Commencing Date*. User can then tap the button "Place Requests for Assistance" to send the request to the selected volunteer. The system automatically completes all the information for the request, using the volunteer's profile.

Please note that for the current prototype version of the app, while a volunteer may have registered availability for more than two days per week, this implementation only permits patient to choose from the first two. This restriction was put in place to keep the matching process simple and allow easy integration with the rest of the app.

Visual Presentation:
Figure 121 presents the wireframe of Window 1.5A: "*View Details of Selected Volunteer".*
Figure 122 includes a snippet of code for Window 1.5A
Figure 123 displays a screenshot of the actual application Window 1.5A: "*View Details of Selected Volunteer".*

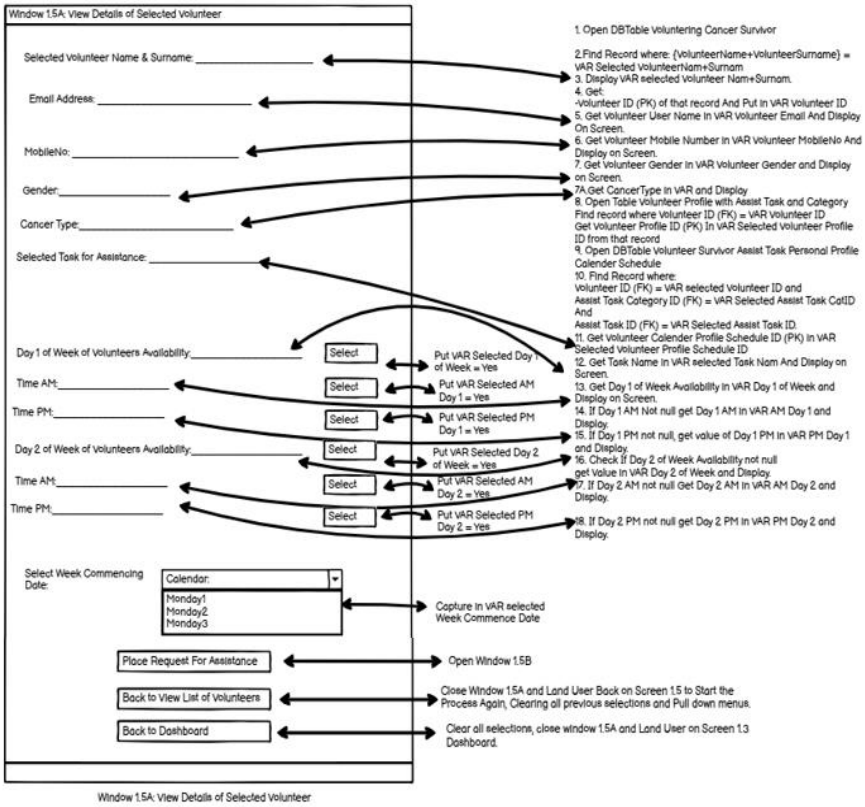


Figure 121: Wireframe of Application Window 1.5A: "View Details of Selected Volunteer".

```
ViewDetailsSelectedVolunteer.kt
117     fun ViewDetailsSelectedVolunteerScreen(
155         LaunchedEffect(volunteerID) {
160                     .addOnSuccessListener { documents ->
169     db.collection( collectionPath: "VolunteeringSurvivorProfilewithAssistTaskCategory")
170         .document(volunteerProfileID)
171         .get()
172         .addOnSuccessListener { doc ->
173             val url = doc.getString( field: "VolunteerProfilePic")
174             Log.d( tag: "IMAGE_DEBUG",  msg: "URL Image: $url")
175             profileImageUrl = url
176         }
177         .addOnFailureListener { e ->
178             Log.e( tag: "IMAGE_DEBUG",  msg: "Error to find image: ${e.message}")
179         }
180
181     db.collection( collectionPath: "AssistanceTaskwithinCategory")
182         .whereEqualTo( field: "AssistTaskID", selectedTaskID)
183         .get()
184         .addOnSuccessListener { taskDocs ->
185             selectedTask = taskDocs.firstOrNull()?.getString( field: "AssistanceTaskNam") ?: "Task Not Found"
186         }
187
188     db.collection( collectionPath: "VolunteeringSurvivorAssistTaskCategoryPersonalProfileCalendarSchedule")
189         .whereEqualTo( field: "VolunteerProfileID", volunteerProfileID)
190         .whereEqualTo( field: "AssistTaskID", selectedTaskID)
in > java > com > example > risetogether > Patient > ViewDetailsSelectedVolunteer.kt > ViewDetailsSelectedVolunteerScreen
```

Figure 122: Snippet of Code for Window 1.5A: "View Details of Selected Volunteer".

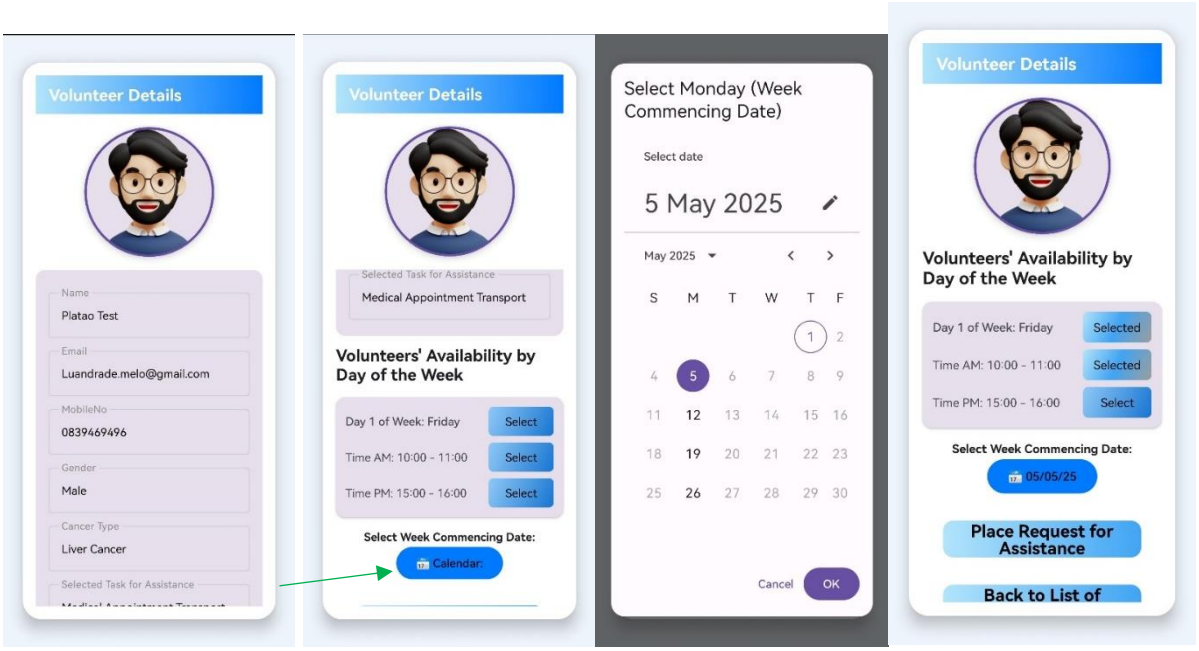


Figure 123: Snapshots of the Actual Application Window 1.5A: "View Details of Selected Volunteer".

4.1.24 Application Window 1.5B - Send Request for Selected Assistance Task

Once the volunteer is chosen, and the desired time and date are picked, the patient is brought to this window, which role is to send the request. It serves as the confirmation screen in that the patient sees the pre-populated request before submitting it. It serves to ensure all the information is accurate before the request is sent. The request text contents are automatically filled in by the system with the patient's information, the volunteer's free time slots, the chosen week for the task, and the name and category of the selected assistance task. The patient does not need to type anything; the request text is already pre-prepared and has all the information required. The patient's complete name, email address, and phone number are provided clearly along with the name of the volunteer, the requested task, the commencement of the week selected and the desired day and time, are all included in the request text. The language is professional and friendly. There is a "Send" button at the bottom of the window. Upon clicking on this button, the system saves the complete request in the database and notifies the chosen volunteer. A confirmation alert that the request had been sent successfully, appears on the screen. The patient is then able to return to the list of available volunteers if they wish to consider other options or return to the dashboard screen.

Visual Presentation:
Figure 124 presents the wireframe of Window 1.5B: "*Send Request for Selected Assistance Task".*
Figure 125 includes a snippet of code for Window 1.5B: "*Send Request for Selected Assistance Task".*
Figure 126 displays a screenshot of the actual application Window 1.5B: "*Send Request for Selected Assistance Task".*

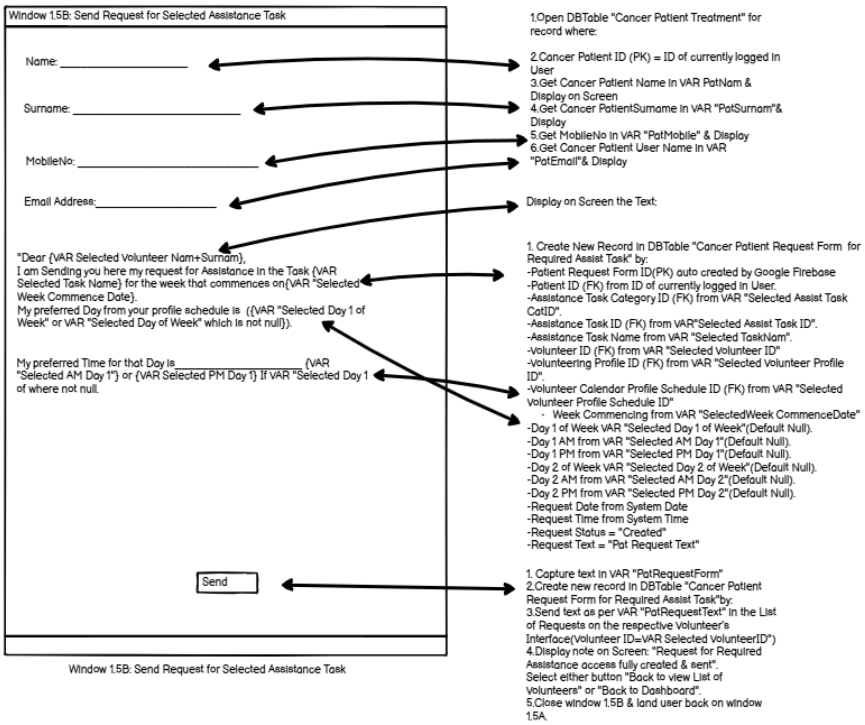


Figure 124: Wireframe of Application Window 1.5B - Send Request for Assistance Task

```
SendRequestSelectedAssistanceTask.kt
60  class SendRequestSelectedAssistanceTask : ComponentActivity() {
187
188     private fun sendRequestToFirestore(
189         patient: Patient,
190         taskData: TaskData,
191         onSuccess: () -> Unit
192     ) {
193         val db = FirebaseFirestore.getInstance()
194         val requestId = FirebaseFirestore.getInstance()
195             .collection( collectionPath: "CancerPatientRequestFormRequiredAssistTask")
196             .document().id
197
198         // Time and Date Right Place
199         val currentDateFormatted = SimpleDateFormat( pattern: "dd/MM/yyyy", Locale.getDefault()).format(Date())
200         val currentTimeFormatted = SimpleDateFormat( pattern: "HH:mm:ss", Locale.getDefault()).format(Date())
201
202         val patRequestText = """
203             Dear, ${taskData.volunteerName} ${taskData.volunteerSurname}.
204
205             I'm sending you here my request for assistance with the task "${taskData.taskName}"
206             for the week that commences on ${taskData.weekCommenceDate}.
207
208             My preferred day from your profile schedule is ${taskData.preferredDay1 ?: taskData.preferredDay2 ?: "N/A"}.
209             My preferred Time for that Day is ${
210             if (taskData.preferredDay1 != null) {
Unsupported API
ogether > Patient > SendRequestSelectedAssistanceTask.kt > SendRequestSelectedAssistanceTask > sendRequestToFirestore   198:37  CRLF  UTF
```

Figure 125: Snippet of Code for Window 1.5B

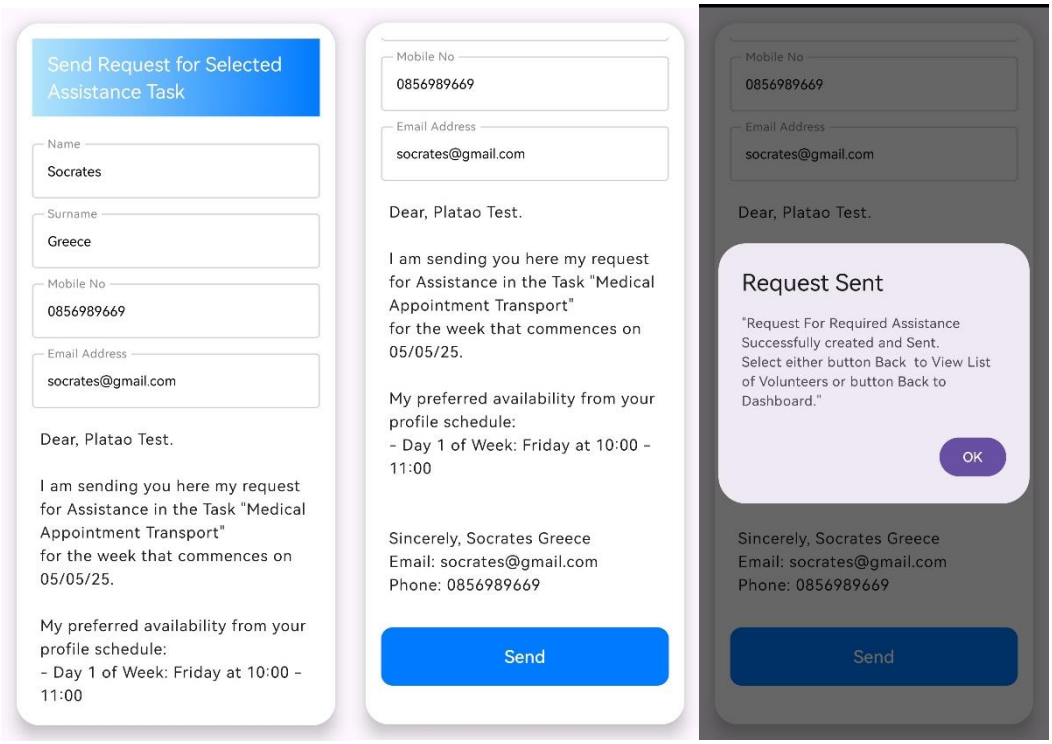


Figure 126: Snapshot of the Actual Application Window 1.5B - Request Submission

4.1.25 Application Screen 1.6 - View List of Notifications for Requested Tasks

This is the screen where patients can view List of responses to their requests for assistance. The system keeps track of whether each request was accepted or rejected every time a patient makes one to a volunteer and the volunteer answers it. This screen keeps the patient posted regarding the status of their request. To View the List of notifications, patient first selects “Week Commencement Date” from a dropdown menu. The system then lists only those notifications that match the week the request was made.

Every notification displays the name of the assistance task, the time and date the notification was sent, and has a "View" button. Upon clicking this button, the system verifies if the volunteer has accepted or rejected the request. On acceptance, a detailed confirmation window opens (Window 1.6A). Upon rejection, a different window is displayed (Window 1.6B), where the patient is notified of the status. There is also a “Back to Dashboard” button that allows the patient to leave the notification area and go back to the main menu screen. In the background, the system uses the patient ID and the week chosen to retrieve the proper data from the database. It also gathers additional information from the profile of the volunteer and the task history to display all details on the subsequent Window, after the "View" is chosen by the user..

Visual Presentation:

Figure 127 presents the wireframe of Screen 1.6 “View List of Notifications.”
Figure 128 includes a snippet of code for Screen 1.6: “View List of Notifications.”
Figure 129 displays a screenshot of the actual application Screen 1.6 “View List of Notifications”.

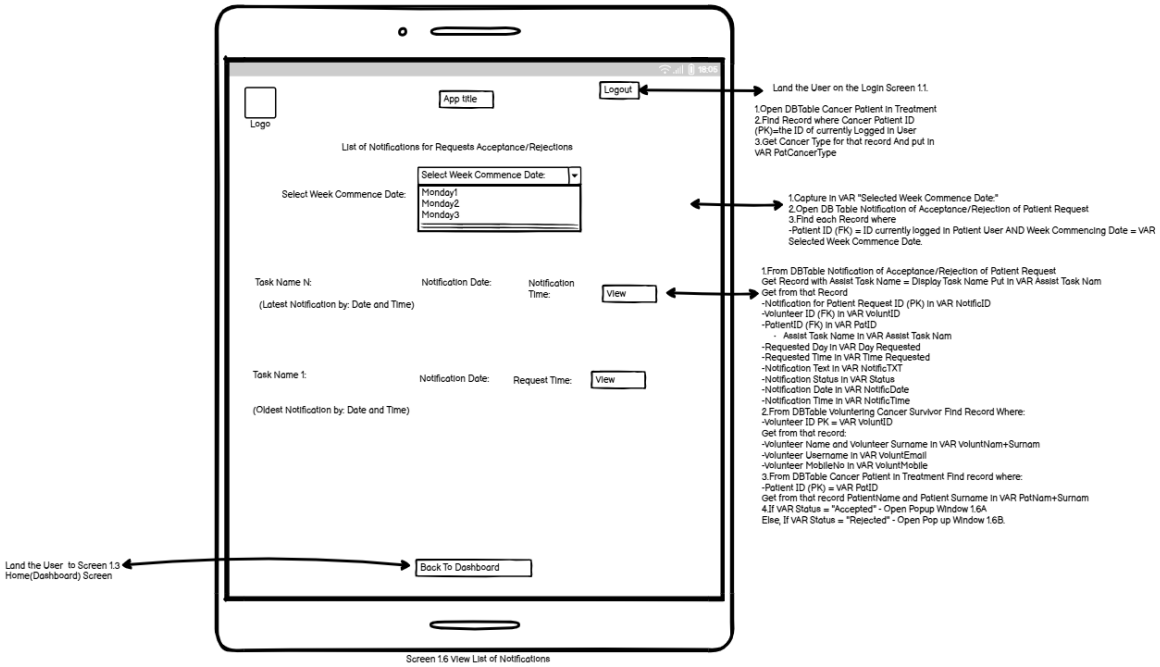


Figure 127: Wireframe of Application Screen 1.6 - List of Notifications for Requests Acceptance/Rejections.

ViewNotificationsReminders.kt

```
364         fun ViewPatientRequestPopup(
368             onReject: () -> Unit
369     ) {
370         val db = FirebaseFirestore.getInstance()
371         val context = LocalContext.current
372
373         var requestData by remember { mutableStateOf<Map<String, Any>?>( value: null) }
374
375         LaunchedEffect(requestId) {
376             db.collection( collectionPath: "NotificationOfPatientRequest")
377                 .document(requestId)
378                 .get()
379                 .addOnSuccessListener { doc ->
380                     if (doc.exists()) {
381                         val data = doc.data?.toMutableMap() ?: mutableMapOf()
382                         val volunteerId = doc.getString( field: "VolunteerID") ?: ""
383                         val patientId = doc.getString( field: "PatID") ?: ""
384
385                         db.collection( collectionPath: "VolunteeringCancerSurvivor")
386                             .document(volunteerId)
387                             .get()
388                             .addOnSuccessListener { vol ->
389                                 data["VoluntNam+Surnam"] = "${vol.getString( field: "VolunteerName")} ${
390                                 data["VoluntEmail"] = vol.getString( field: "VolunteerUsername") ?: ""
391                                 data["VoluntMobile"] = vol.getString( field: "VolunMobile") ?: ""
```

gether2 › app › src › main › java › com › example › risetogether › Patient › ViewNotificationsReminders.kt

Figure 128: Snippet of Code for Screen 1.6 - List of Notifications for Requests Acceptance/Rejections.

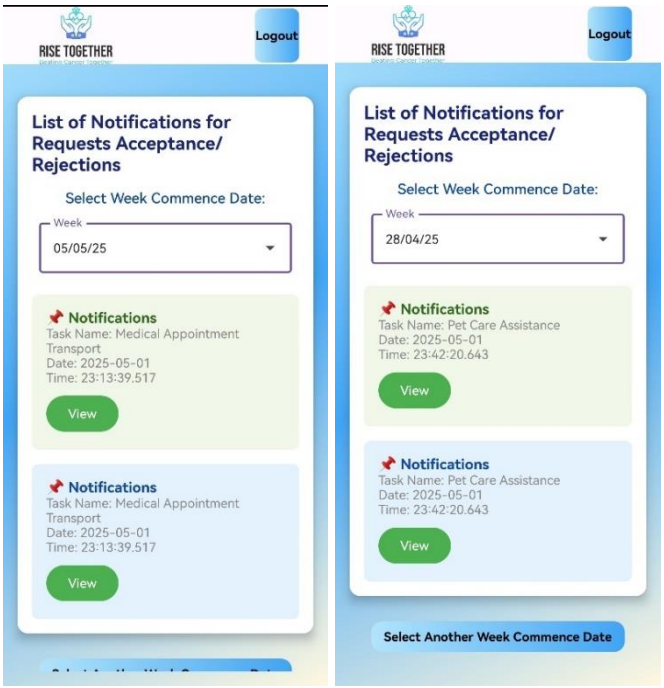


Figure 129: Snapshot of the Actual Application Screen 1.6 - List of Notifications for Requests Acceptance/Rejections.

4.1.26 Application Window 1.6A - Display Notification of Acceptance of Requested Task Assistance

When the patient clicks on the View button of selected Notification from the List, a pop-up window opens, presenting the notification text of accepted by volunteer request for assistance. The acceptance notification provides the volunteer's name, the assistance task's name, and the day and week the volunteer agrees to provide it. The message further displays the volunteer's phone number and email address so that the patient may contact the volunteer personally, if need be, or pose any questions. The window's design is simple. The message is fixed in the middle, while the contact details appear immediately beneath it. A "Close" button is placed in the window's bottom part. Once the patient clicks on it, the pop-up window closes and brings the user back onto the notification list screen (Screen 1.6). This window is important in the patient's journey because it confirms that the volunteer has agreed to assist and provides the patient with all the required information to be informed and reassured.

Visual Presentation:
Figure 130 presents the wireframe of Window 1.6A: "Display Notification of Acceptance of Requested Task Assistance".
Figure 131 includes a snippet of the code for Window 1.6A: "Display Notification of Acceptance of Requested Task Assistance."
Figure 132 displays a screenshot of the actual application Window 1.6A: "Display Notification of Acceptance of Requested Task Assistance".

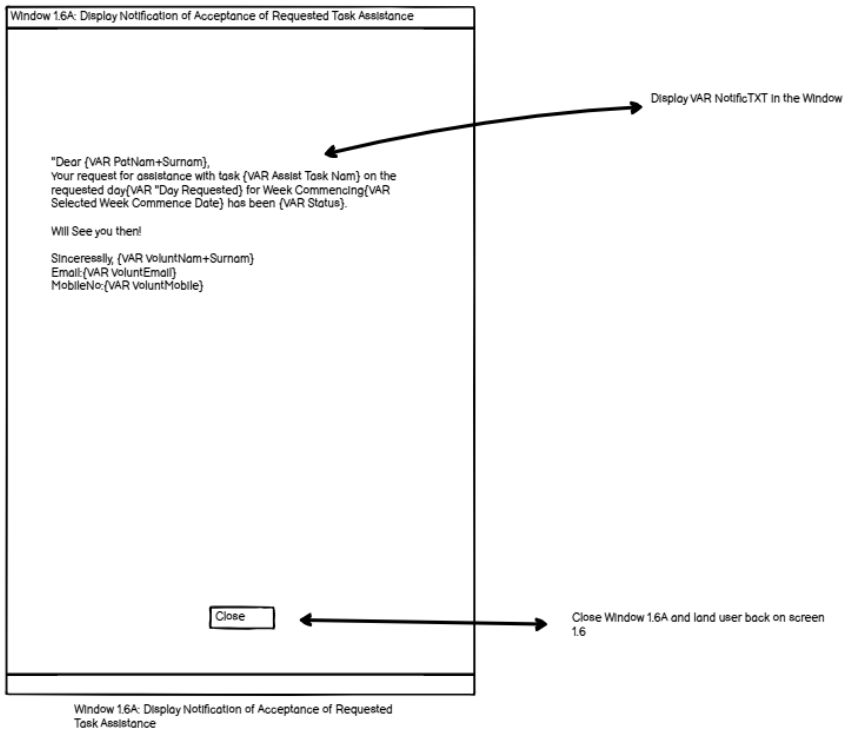


Figure 130: Wireframe of Window 1.6A: "Display Notification of Acceptance of Requested Task Assistance."

DisplayNotificationAcceptScreen.kt

```
10  import androidx.compose.ui.unit.sp
11
12  @Composable
13  fun DisplayNotificationAcceptScreen(
14      requestData: Map<String, Any>,
15      onClose: () -> Unit
16  ) {
17      val patientFullName = requestData["PatNam+Surnam"] ?: "Patient"
18      val assistTaskName = requestData["AssistTaskName"] ?: "N/A"
19      val weekCommenceDate = requestData["WeekCommenceDate"] ?: "N/A"
20      val requestedDay = requestData["RequestedDay"] ?: "N/A"
21      val status = requestData["RequestStatus"] ?: "Accepted"
22
23      val volunteerFullName = requestData["VoluntNam+Surnam"] ?: "Volunteer"
24      val volunteerEmail = requestData["VoluntEmail"] ?: "-"
25      val volunteerMobile = requestData["VoluntMobile"] ?: "-"
26
27      AlertDialog(
28          onDismissRequest = onClose,
29          title = {
30              Text( text: "Notification of Acceptance of Requested Task Assistance")
31          },
32          text = {
33              Column(modifier = Modifier
34                  .verticalScroll(rememberScrollState())
```

ether2 › app › src › main › java › com › example › risetogether › Patient › onClose

Figure 131: Snippet of Code for Window 1.6A: "Display Notification of Acceptance of Requested Task Assistance."

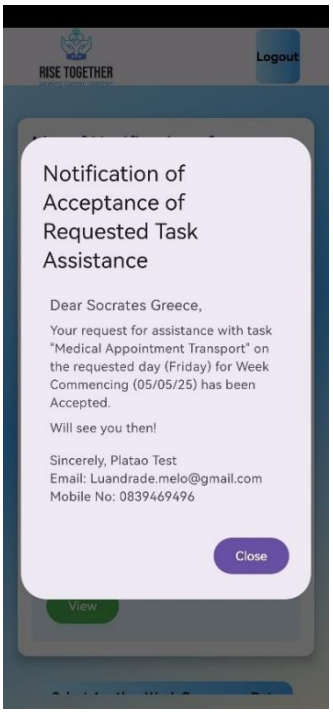


Figure 132: Snapshot of the Actual Application Window 1.6A: "Display Notification of Acceptance of Requested Task Assistance".

4.1.27 Application Window 1.6B - Display Notification of Rejection of Requested Task Assistance

This pop-up window appears when a volunteer has rejected a support request submitted by a patient. Just like the window shown before, the purpose of the notification is to clearly and respectfully inform the patient that the volunteer is not available on the selected day and week. Although the request was declined, the message is written in a polite and encouraging tone, helping to maintain a positive experience and motivating the patient to try again with a new request.
The message explains the situation in simple terms and confirms that assistance will not be available during the chosen day and time of the selected week. It also reminds the patient that they can submit another request for a different week, which keeps the process open and supportive. In addition to the rejection message text, the screen shows the volunteer's name, email address, and phone number. This gives patient the option to follow up directly if they have any questions or want to understand more about the reason for the rejection. The window layout is clean and simple. The main message appears at the top, and the contact details are displayed just below. At the bottom, there is a "Close" button. When clicked, this button returns the patient to the screen with List of Notifications, where they can view other notification or return to the dashboard. Even though the outcome is not what the patient hoped for, this screen helps keep communication respectful and ensures that the patient stays engaged and confident in using the system.

Visual Presentation:
Figure 133 presents the wireframe of Window 1.6A: "Display Notification of Rejection of Requested Task Assistance".
Figure 134 includes a snippet of code for Window 1.6A: "Display Notification of Rejection of Requested Task Assistance".
Figure 135 displays a screenshot of the actual application Window 1.6A: "Display Notification of Rejection of Requested Task Assistance".

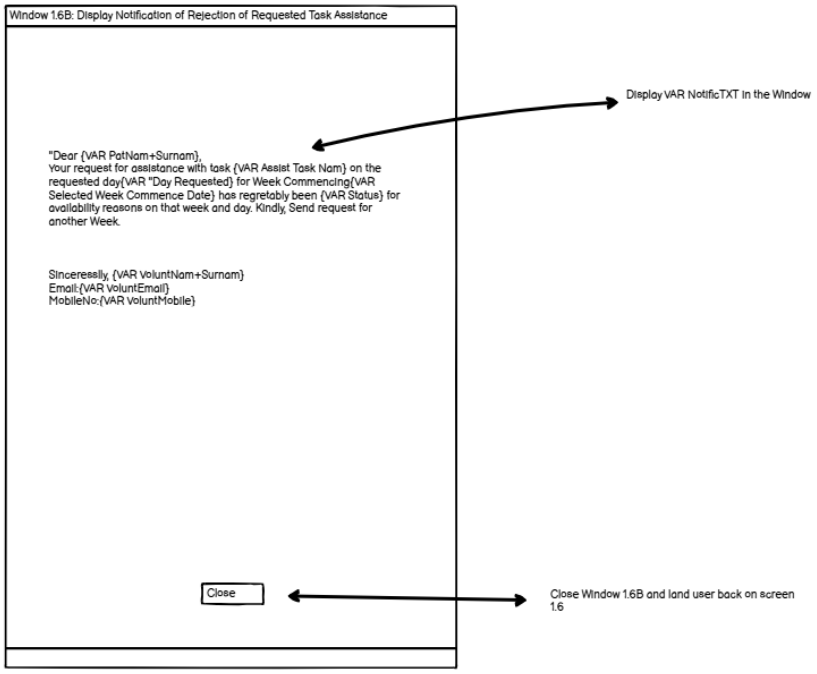


Figure 133: Wireframe of Window 1.6B: Display Notification of Rejection of Requested Task Assistance

```kotlin
import androidx.compose.ui.unit.sp

@Composable
fun DisplayNotificationRejectScreen(
    requestData: Map<String, Any>,
    onClose: () -> Unit
) {
    val patientFullName = requestData["PatNam+Surnam"] ?: "Patient"
    val assistTaskName = requestData["AssistTaskName"] ?: "N/A"
    val weekCommenceDate = requestData["WeekCommenceDate"] ?: "N/A"
    val requestedDay = requestData["RequestedDay"] ?: "N/A"
    val status = requestData["RequestStatus"] ?: "Rejected"

    val volunteerFullName = requestData["VoluntNam+Surnam"] ?: "Volunteer"
    val volunteerEmail = requestData["VoluntEmail"] ?: "-"
    val volunteerMobile = requestData["VoluntMobile"] ?: "-"

    AlertDialog(
        onDismissRequest = onClose,
        title = {
            Text( text: "Notification of Rejection of Requested Task Assistance")
        },
        text = {
            Column(
                modifier = Modifier
```

gether2 › app › src › main › java › com › example › risetogether › Patient › DisplayNotificationRejectScreen.kt

Figure 134: Snippet of Code for Window 1.6B: Display Notification of Rejection of Requested Task Assistance



Figure 135: Snapshot of the Actual Application Window 1.6B: Display Notification of Rejection of Requested Task Assistance.

4.1.28 Application Screen 1.7 - Provide Feedback and Ratings for Obtained Assistance

This screen allows patients to provide feedback and ratings for the assistance received from a volunteer. It is crucial to establish trust on the platform and ensure everyone is accountable for their actions. It also provides opportunities for the volunteers to self-reflect on how they have contributed and how they may do better in the future. The feedback process is only triggered by completed assistance tasks. To begin with, the patient chooses the Week Commencement Date from a dropdown menu. The system thereafter displays the tasks that were accomplished that week and qualify to receive feedback. For every task completed, the patient can view the name of the task, the weekday it was performed, and the time it was conducted. There is a "Select" button to the right of each task. Upon clicking on this button, the system will land the patient on a window to submit feedback, with both a star rating and optional written feedback comment, regarding the assistance provided by the volunteer who attended to them. If the patient wishes to view a different week, there is the “Select Another Week Commence Date” button. Once feedback is completed, the patient can return to the dashboard screen through the “Back to Dashboard” button.

Visual Presentation:
Figure 136 presents the wireframe of Screen 1.7: “Provide Feedback & Ratings for Task Assistance Obtained”.
Figure 137 includes a snippet of code for Screen 1.7.
Figure 138 displays a screenshot of the actual application Screen 1.7: “Provide Feedback & Ratings for Task Assistance Obtained”.

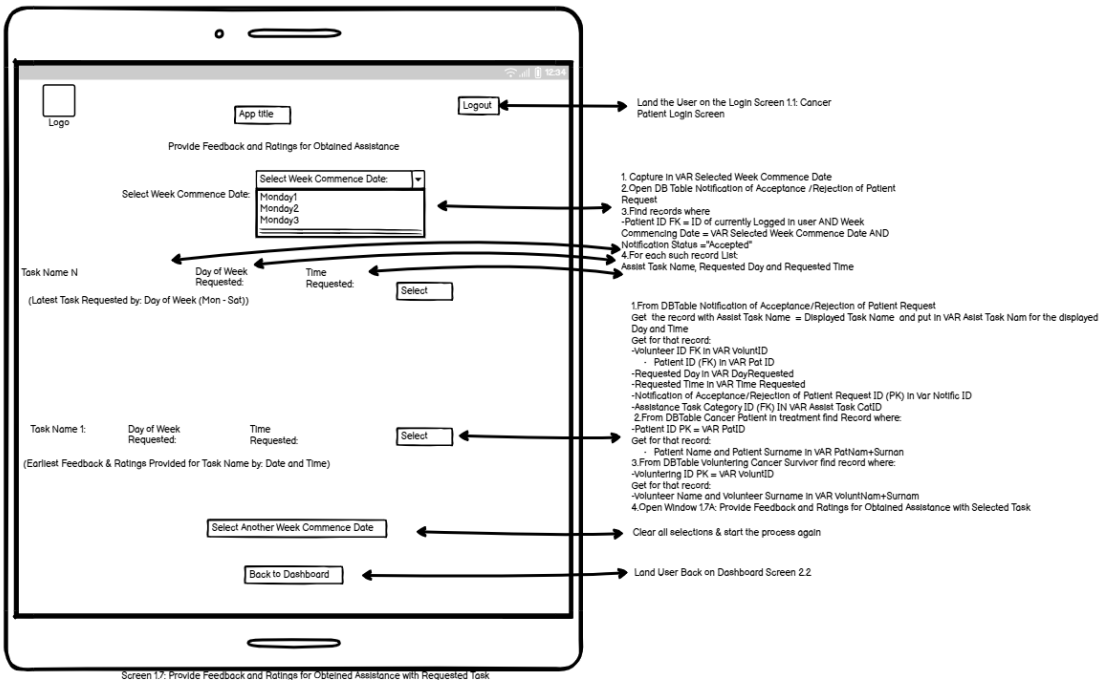


Figure 136: Wireframe of Application Screen 1.7: Provide Feedback and Ratings for Obtained Assistance with a Requested Task

```
ProvideFeedbackRatings.kt
43  fun ProvideFeedbackRatingsContent() {
56      // Fetch available weeks
57      LaunchedEffect(patientId) {
58          db.collection( collectionPath: "NotificationOfPatientRequest")
59              .whereEqualTo( field: "PatID", patientId)
60              .whereEqualTo( field: "RequestStatus", value: "Accepted") // use "RequestStatus" Firestore
61              .get()
62              .addOnSuccessListener { snapshot ->
63                  val weeks = snapshot.documents
64                      .mapNotNull { it.getString( field: "WeekCommenceDate") }
65                      .distinct()
66
67                  availableWeeks = weeks.sorted()
68              }
69      }
70
71      fun fetchAcceptedTasksForWeek(week: String) {
72          db.collection( collectionPath: "NotificationOfPatientRequest")
73              .whereEqualTo( field: "PatID", patientId)
74              .whereEqualTo( field: "WeekCommenceDate", week)
75              .whereEqualTo( field: "RequestStatus", value: "Accepted")
76              .get()
77              .addOnSuccessListener { documents ->
78                  val tasks = documents.map {
79                      mapOf(
src > main > java > com > example > risetogether > Patient > ProvideFeedbackRatings.kt > ProvideFeedbackRatingsContent
```

Figure 137: Snippet of Code for Screen 1.7: Provide Feedback and Ratings for Obtained Assistance with a Requested Task

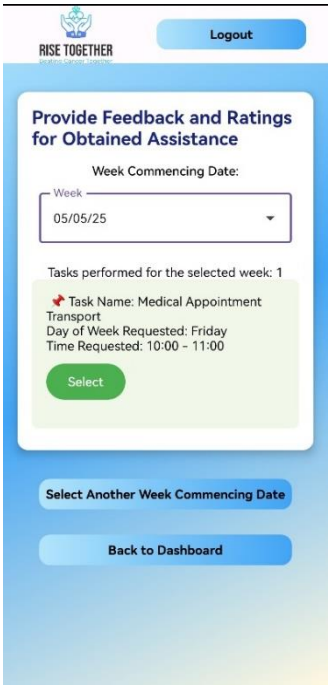


Figure 138: Snapshot of the Actual Application Screen 1.7: Provide Feedback and Ratings for Obtained Assistance with a Requested Task

4.1.29 Application Window 1.7A: Provide Feedback and Ratings for Obtained Assistance with Selected Task

This Window is shown after a patient chooses a task that has been completed from the list of Assistance tasks obtained. It enables them to leave a star rating and, optionally, to leave a feedback comment text on the assistance they received from the volunteer. It is significant in promoting good quality work, accountability, and respect across the platform.
The display has all the primary information regarding the task and the individuals involved to aid the patient. It shows what week the task was booked, the name of the task, what day and time the support was conducted, and the names of the patients and volunteers. These are presented in read-only mode, so that the patient may verify the information. Below this information, the patient may choose from one to five stars. This rating allows for assessment the volunteer's performance and inform subsequent patients what to expect. There is also an optional text box where the patient may describe the experience in a feedback text. This is the opportunity to comment positively, thank the volunteer, or recommend improvements.

Once the patient completes the feedback and rating process, they can click on the Submit button. Upon doing so, the system saves the feedback and displays an immediate confirmation message. All the information filled in, including the star rating and written feedback, is stored in the Feedback & Ratings database record. The record captures the patient's and volunteer's names and ID, the name of the task, the date and time the support was provided, the week the request was made, and the specific time that the feedback was submitted.

Subsequently, this feedback may be accessed from the volunteer's profile as part of their history. This is done to record every contribution and its value and utilize it to aid continuous quality and betterment on the platform.

Visual Presentation:

Figure 139 presents the wireframe of Window 1.7A: Provide Feedback and Ratings for Obtained Assistance with Selected Task

Figure 140 includes a snippet of code for Window 1.7A

Figure 141 displays a screenshot of the actual application Window 1.7A: Provide Feedback and Ratings for Obtained Assistance with Selected Task

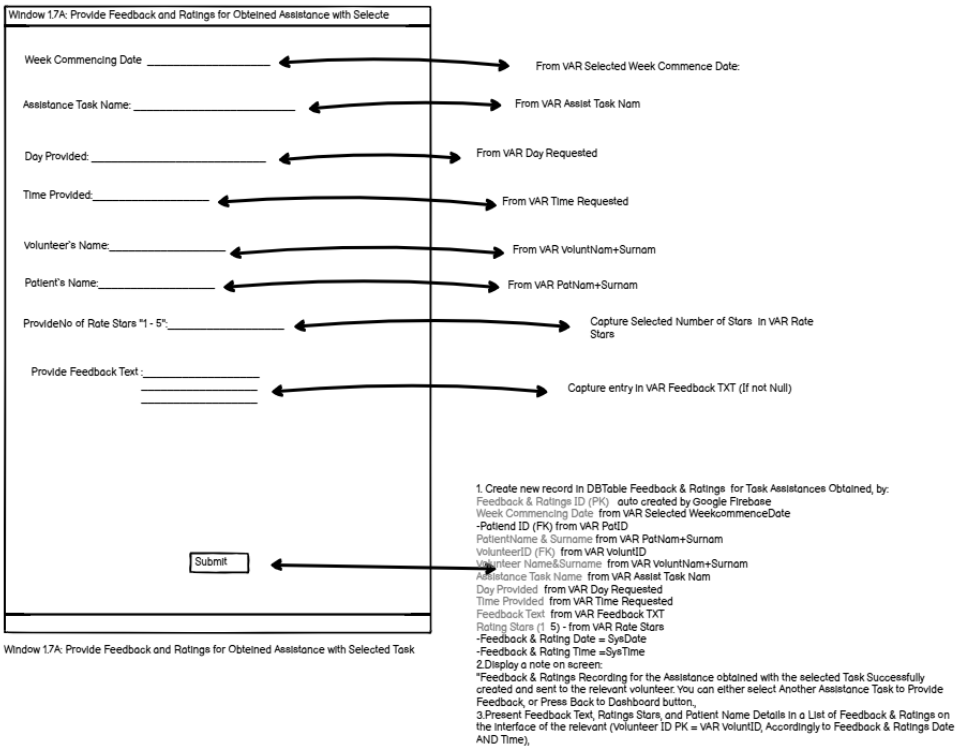


Figure 139: Wireframe of Window 1.7A: Provide Feedback and Ratings for Obtained Assistance with Selected Task

ProvideFeedbackRatingsDetailScreen.kt

```
41      fun ProvideFeedbackRatingsDetailContent(requestId: String) {
89
90          fun submitFeedback() {
91              if (rating in 1 ≤ .. ≤ 5 && feedbackText.isNotBlank()) {
92                  val feedback = hashMapOf(
93                      "WeekCommenceDate" to taskData["WeekCommenceDate"],
94                      "PatientID" to taskData["PatientID"],
95                      "PatientName" to patientName,
96                      "VolunteerID" to taskData["VolunteerID"],
97                      "VolunteerName" to volunteerName,
98                      "AssistTaskName" to taskData["AssistTaskName"],
99                      "DayProvided" to taskData["DayRequested"],
100                     "TimeProvided" to taskData["TimeRequested"],
101                     "RatingStars" to rating,
102                     "FeedbackText" to feedbackText,
103                     "FeedbackDateTime" to Timestamp.now()
104                 )
105
106                 db.collection( collectionPath: "FeedbackRatings").add(feedback)
107                     .addOnSuccessListener {
108                         Toast.makeText(context, text: "Feedback sent successfully!", Toast.LENGTH_LONG).show()
109                         context.startActivity(Intent(context, PatientDashboardScreen::class.java))
110                     }
111                     .addOnFailureListener {
112                         Toast.makeText(context, text: "Error when send feedback.", Toast.LENGTH_LONG).show()
```

va > com > example > risetogether > Patient > ProvideFeedbackRatingsDetailScreen.kt > ProvideFeedbackRatingsDetailContent

Figure 140: Snippet of Code for Window 1.7A: Provide Feedback and Ratings for Obtained Assistance with Selected Task

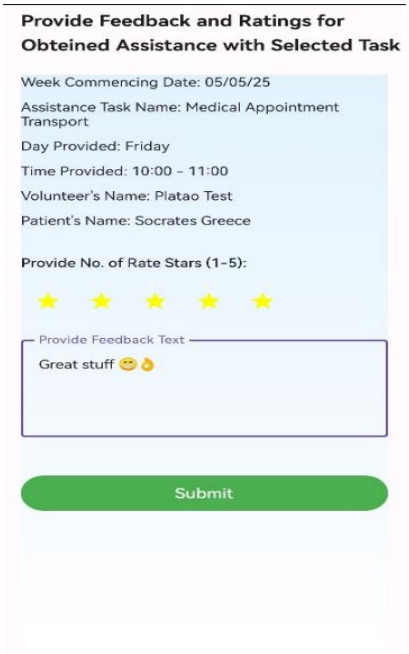


Figure 141: Snapshot of the Actual Application Window 1.7A: Provide Feedback and Ratings for Obtained Assistance with Selected Task

4.2 Application's Database Design and Connectivity

Every Software Application System is comprised of two essential elements: Front-end and Back-end.

The database design of the present project and making it accessible to the application users were accomplished with Google's Firebase Cloud Firestore as a Database Management System in conjunction with Android Studio and Java+Kotlin.

Front-End: The Front-End is the application's presentation tier that delivers the application's Graphical User Interface (GUI). The primary role of the User Interface is to convert the backend outputs into what users can view and interpret. The presentation tier of the current application is intended to show data in an interactive and user-friendly format.

Back-End: The Back end involves all activities that are not readily apparent to the application user. Each screen in the application interacts with the database to read or write information. For the current application, for example, the login process confirms the authenticity of the user with the data stored in the database and grants access to the succeeding screens. The Registration screen establishes a new record for the user in the database. All the actions associated with the database come after this, so the application can perform specific actions depending on the context (read or write operations accordingly to their rules). (Firebase 2024a).

Main Characteristics of Google's Firebase Cloud Firestore:

The application uses Firebase Cloud Firestore as its core database solution. Firestore is a flexible and scalable server, web, and mobile app-friendly database. Data is stored using documents that are organized into collections. There is a series of key-value pairs in each document. Firestore is fit to handle the varied tables utilized in the application because it is capable of handling complex data structures, objects, and arrays that are nested. User profiles, health data, and any additional organized data required by the program are stored in this manner. Firestore provides real-time synchronization, offline support, and rich querying (Firebase, 2024b).

o Firebase Authentication

This feature grants access to the application only to authorised users. It supports login through third-party APIs like Gmail and GitHub and allows developers to implement customised methods of authentication. Firebase has support for default functions, like password forgetting and user verification through the Firebase console, and has a simple API to support user login or sign-in with the help of third-party APIs (Firebase, 2024c)

o Firebase Cloud Storage

Firebase provides storage of files like videos, photos, and audio. For this project, it is used for pictures, and the data is stored and accessed from the client SDK directly. Uploads and downloads will happen in the background, and only authenticated users are permitted to view what has been stored. Firebase SDKs support seamless operations even in the event of loss of connectivity and will pick up from where it was interrupted as soon as the connectivity is re-established. (Google Cloud, 2021).

o Firebase Cloud Functions

Enable developers to control fewer configurations, including application updates and security for Google services and Firebase products. A Cloud Function is linked to the Firebase application and runs on the Google server, avoiding duplicate coding by supplying an admin SDK and encapsulating third-party services. This command automatically scales up design usage as data computing rises without the need for maintenance in cases of upgraded configurations, credentials, or decommissioning. (Firebase, 2024b).

Firebase Cloud Functions are utilized in the *Rise Together* app to execute background operations like pushing notifications, data processing, and responding to user actions without the need for a traditional server. They are run on Google Cloud, automatically scale, and enhance security by executing backend logic in isolation. They are activated by occurrences, such as database updates, changes in authentication, or HTTP requests, and are directly linked to Firebase services. This simplifies the code in the app, eliminates duplication, and keeps updates or modifications from impacting app performance.

With the new version (Cloud Functions v2), the platform also supports improved performance, more flexible configuration, and broader integration with Google Cloud using Eventarc.

Firebase Cloud Functions enables significant functionalities in the app, including emailing users after account registration or updating the database after patient or volunteer activity, all without having to manually manage servers.

4.2.1 Implementation Path

Gradle is an efficient build automation tool, primarily utilized for Java, Kotlin, and Android development. Gradle performs build actions like code compilation, dependency resolution, testing, and app deployment. Gradle utilizes build scripts to specify what must be executed and how. The scripts can be written either in Groovy (build. Gradle) or in Kotlin (build.gradle.kts), the Kotlin DSL being the most used of these today in Android apps. There are three significant steps in the Gradle build process. The first is the initialization step, in which Gradle locates all participating projects (which are only valuable for multi-module environments). The second step is the Configuration step, in which Gradle loads and executes the build scripts to establish all the tasks. Lastly, in the Execution step, Gradle executes the engaged activities, like code compilation or APK generation.

A standard configuration. Gradle. The kts file contains a plugins block of plugins that must be included, an Android block for Android functionality, and a dependencies block where project dependencies are declared. For instance, the SDK is placed in the dependencies section to utilize an SDK like Firebase Firestore. Gradle will download and integrate the library upon project build. One of Gradle's prime advantages is its extensibility and versatility. Developers can create custom tasks, utilize the code conditionally, and improve build performance, supporting small-scale as well as large-scale projects. Firestore dependency in the dependencies section of the build. Gradle.kts file. On Figure 142, the gradle file is presented.

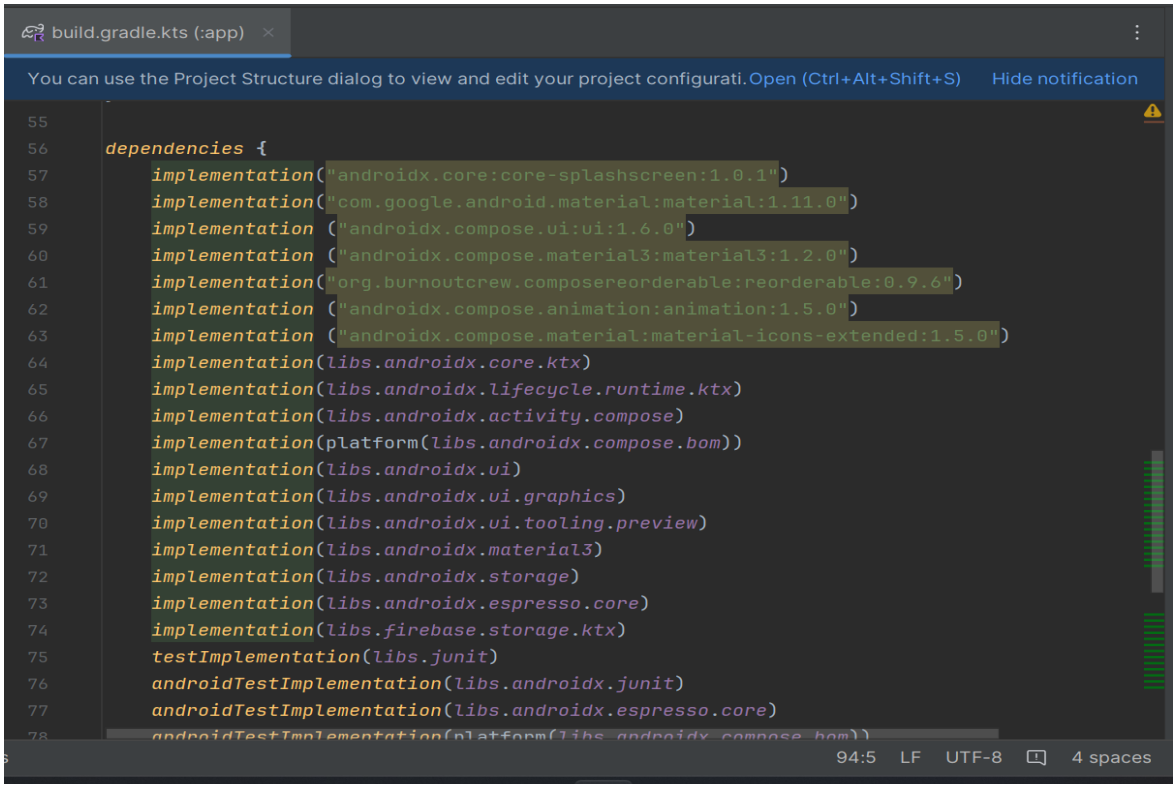


Figure 142:Gradle.kts file and dependencies

4.2.3 Connectivity

How do the notifications work in this application.

The *Rise Together* app employs Firebase Cloud Messaging (FCM) version 1 (v1) of the FCM API to deliver real-time push notifications to users. FCM is a robust and scalable messaging service from Google that enables messages to be sent to user devices directly by the app. As evidenced by the Firebase configuration file, the FCM v1 API is active and in use and is the standard setup used in most contemporary projects (Firebase, 2024d). The legacy API is deactivated and scheduled to be de-emphasized in official usage by June 2024, so the project is already in accordance with up-to-date best practices. The Kotlin class FCM V1Sender is used to handle notification logic. When the send Notification () function is triggered, it is executed in the background by Kotlin coroutines (Dispatchers.IO) so as not to block the main thread. The function first refers to the Firebase Admin SDK JSON key, saved in the app's assets folder. This key is used to safely authenticate with Firebase services. After being authenticated, the app obtains an OAuth 2.0 access token, scoped to Firebase Messaging, and includes it in the HTTP request header. A JSON payload is constructed with the recipient's FCM token and the notification title and body. A POST request is sent to the endpoint by the app using the OkHttp library: https://fcm.googleapis.com/v1/projects/risetogether-75090/messages:send. This request has the correct header that includes the token and the content type. If the request is successful, the app will write a success message. A failure will write an error message with the HTTP status code (OkHttp, 2024). For security and performance, the notifications are sent only to authenticated devices that possess valid FCM tokens.

These are not exposed and are safely stored. Due to the infrastructure of Google Cloud, the system scales automatically with load. The notifications in the *Rise Together* app may also be customized by user role, like Patients or Volunteers, based on application logic, when the volunteer accepts or rejects a request. When the volunteer agrees with a request for tomorrow, today+1, the reminders of the task in *Rise Together* work as if a volunteer accepts the patient’s request for assistance, the app inserts a reminder entry in the Firestore collection AcceptedTaskRequestReminder. Concurrently, a message is stored in the patient’s own Notifications subcollection in a category named Patient in Treatment. If the activity is booked for tomorrow and the patient possesses a valid FCM token, automatically after a one-minute delay (1 minute) the push message is sent using

the FcmV1Sender class. The push notification serves as a reminder to the patient, regarding the scheduled task. It is just a way to show as a test; in the real system, it should be implemented for all the tasks checking day+1. And send a reminder to all the users who have a request the day after. The reminder encompasses:

- The Message Text
- The date and time today
- A push notification with the subject heading "Task Reminder".

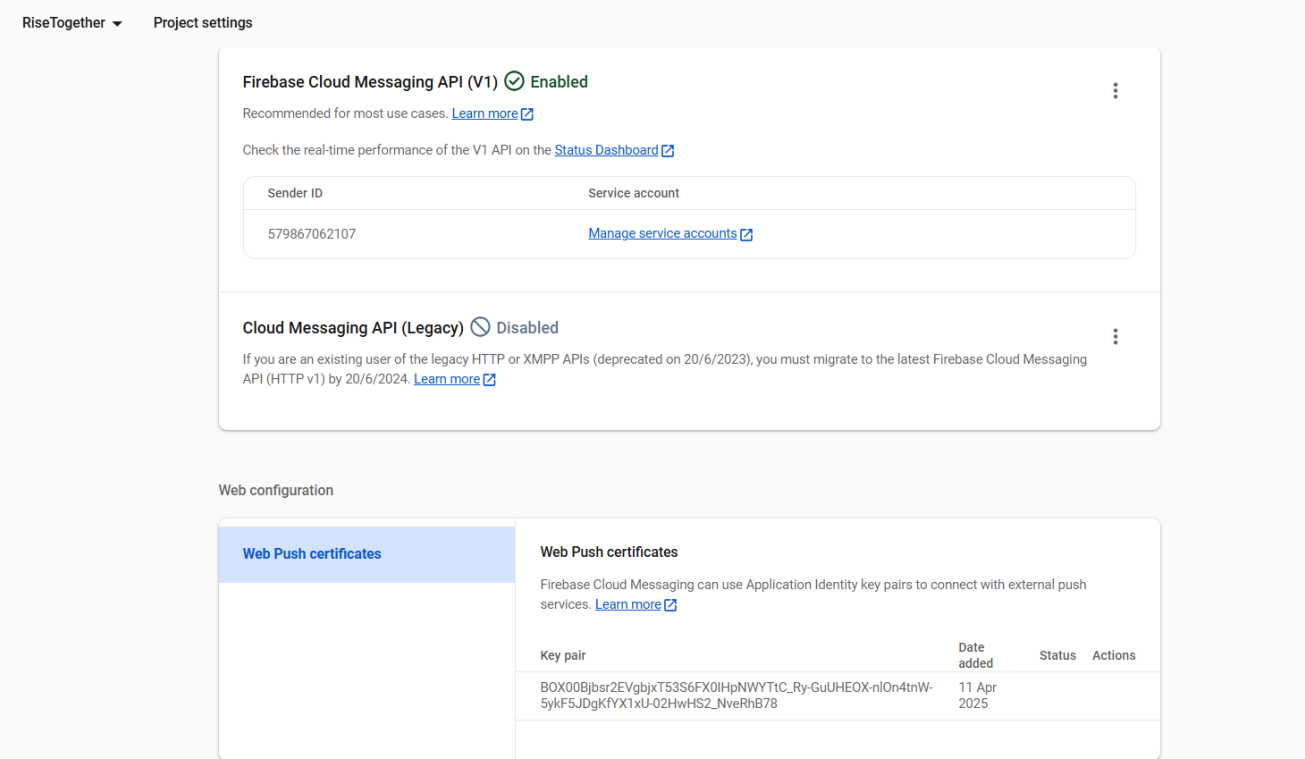


Figure 143: Screenshot of the Rise Together connecting Application's real-time Database for notifications. -1

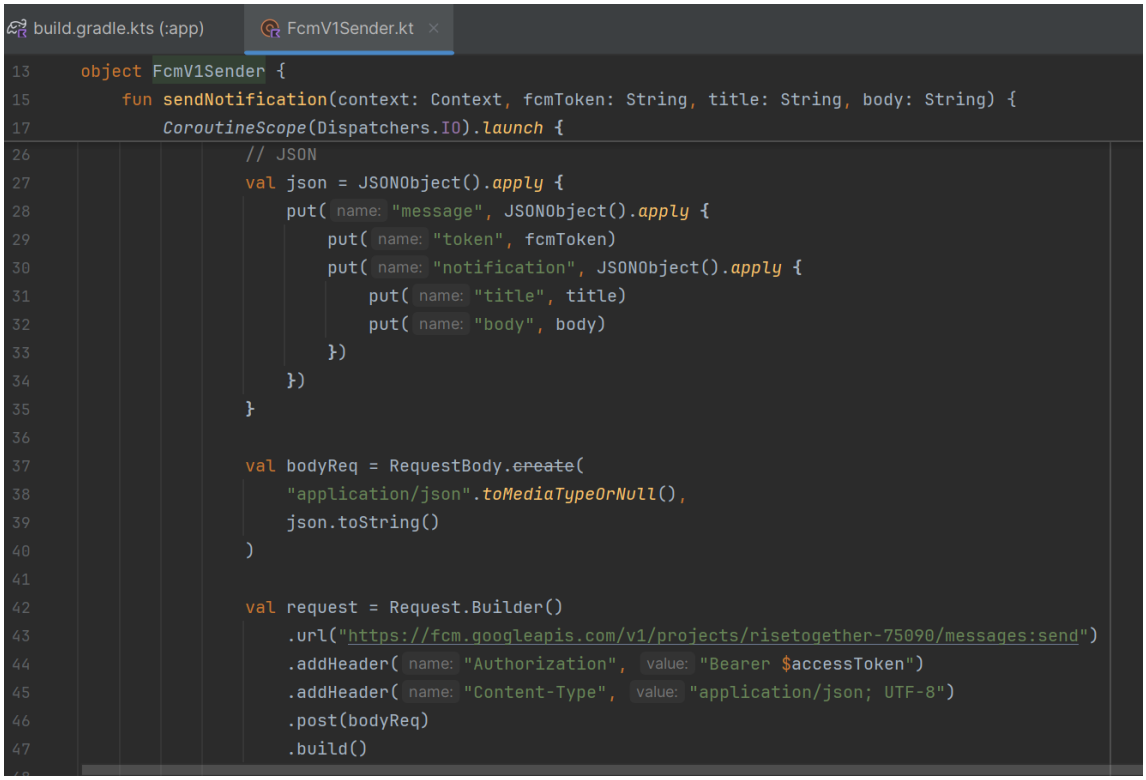


Figure 144: Screenshot of the Rise Together connecting Application's real-time Database for notifications.

4.3 Database Connectivity

The activity involved in transforming an application-domain concept schema to a schema of the data model from which a specific database management system (DBMS), like the relational or Object-Oriented database design, is derived is termed the process of logical database design. The database is one of the most critical elements of the overall application. Guaranteeing the flexibility and scalability of the database to grow or reduce in size without any errors is vital. Tables and attributes that represent the structure of the database of this application were created based on the Object Class Diagram, described in Chapter 3. Google Firebase has been utilized to build the application database (please refer to Figures 145 & 146).

Doctor`s Collection

Figure 145 shows the significant attributes of the "Doctors" collection in the Database, with an example document (record) of Doctor Liam O`Brien. The attribute-value pairs within the document are as follows: name with value 'Liam' (String), surname with value 'O`Brien' (String), email with value Liam@gmail.com (String), and DoctorID (Primary key attribute) with an encrypted value.

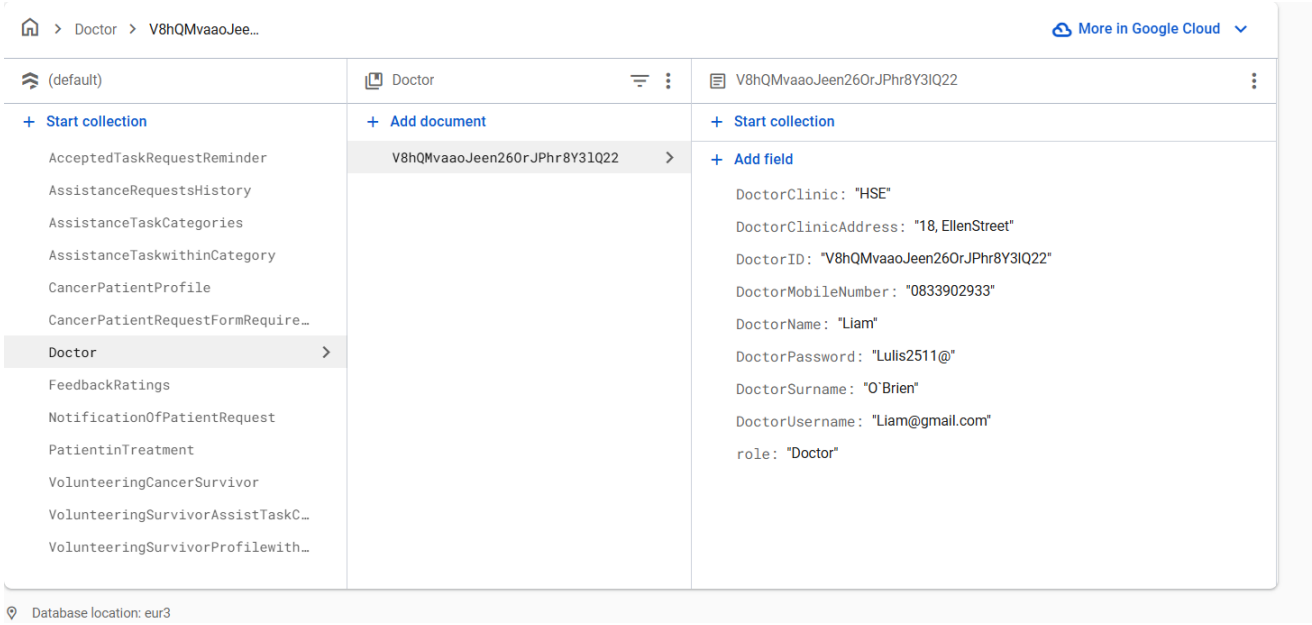


Figure 145: Table (Collection) Doctor with UNIQUE ID, encrypted for each record.

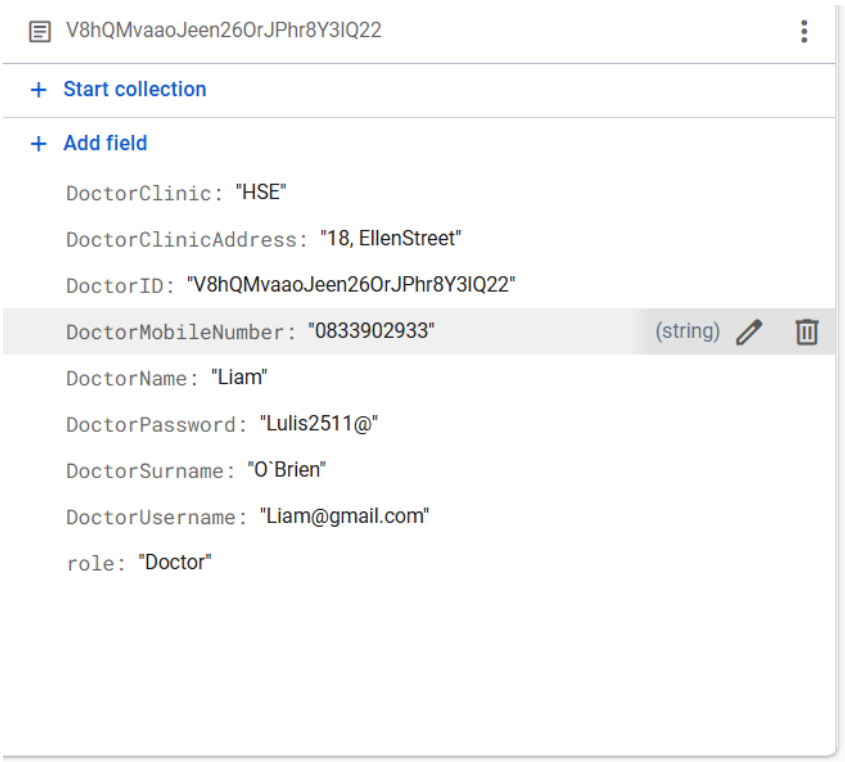


Figure 146: Doctors' Collection with UNIQUE encrypted IDs.

Patients' Collection:

Figure 147 "Patients" collection with document for Patient Socrates Greece. The main fields include name, surname, gender, mobile, role, email, PatCancerType, DoctorID

is a foreign key, while PatientID is the Primary key. Values are all Strings, and the FCM Token is used to find a specific user and send a notification.

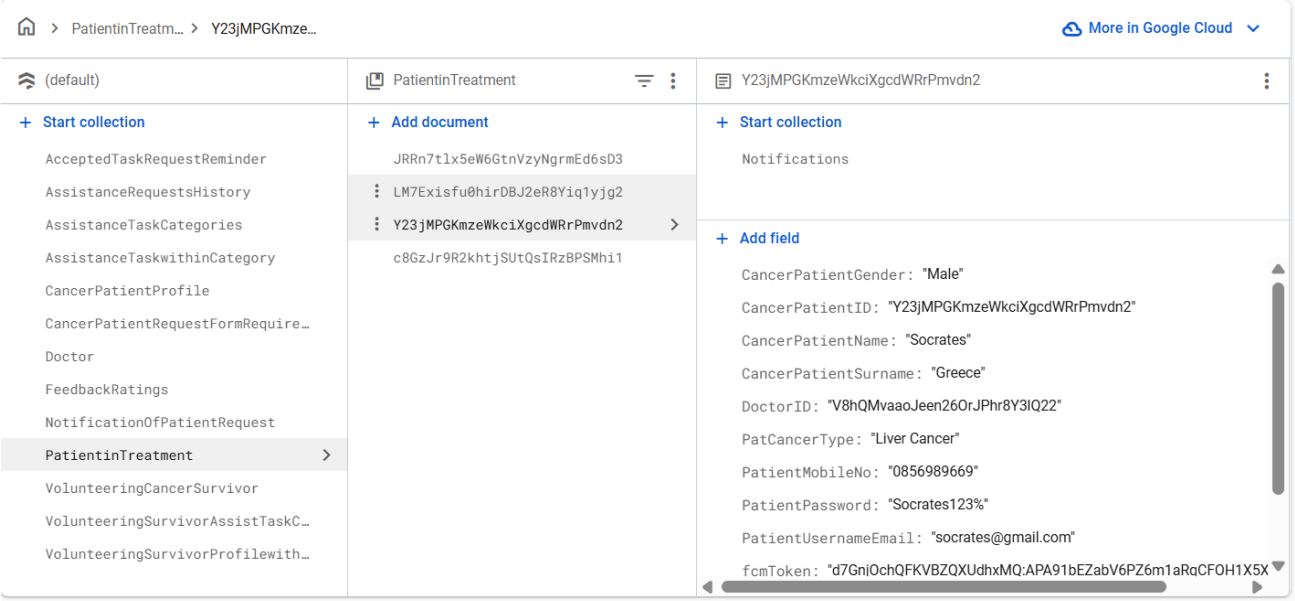


Figure 147: Patient details for an individual Patient ID.

Volunteering Cancer Survivor Collection:
Figure 148 presents "VolunteeringCancerSurvivor" collection with document for Volunteer Platão Melo.

The main fields include VolunteerName, VolunteerSurname, VolunGender, VolunMobile, role, VolunteerUsername, VolunCancerType. DoctorID is a foreign key, while VolunteerID is the Primary key and the role to identify. Values are all Strings, and the collection stores survivors of cancer who participate in volunteer activities within the application.

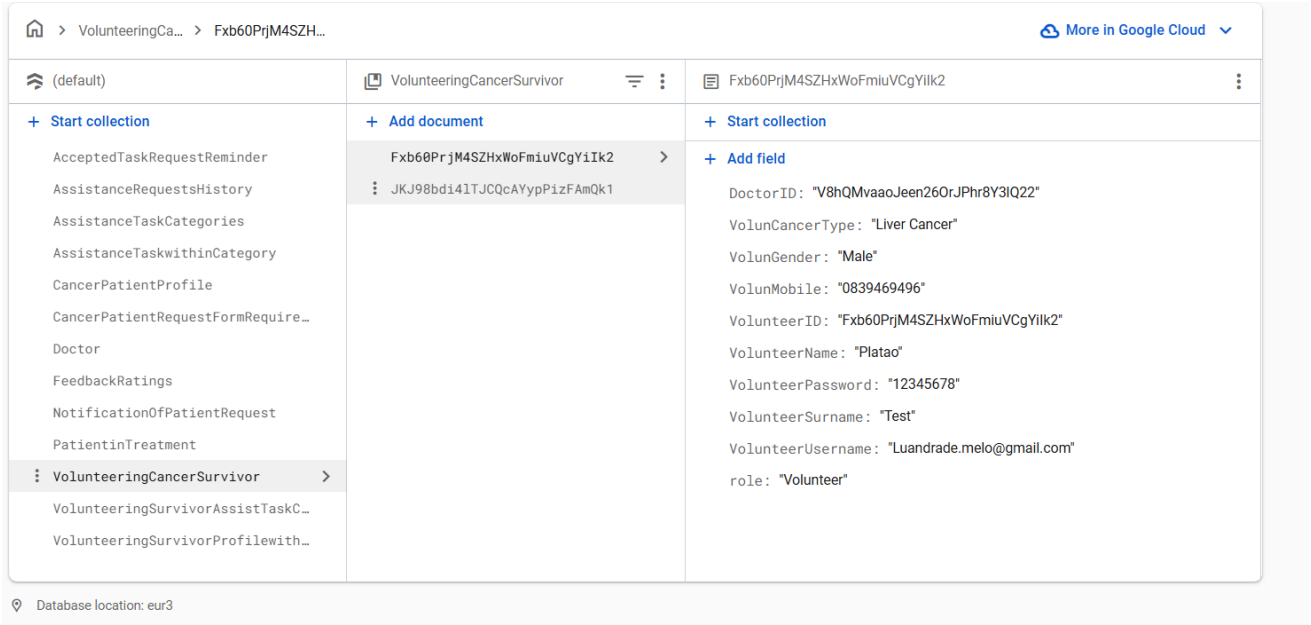


Figure 148: DATA Selection for a specific Volunteer ID record.

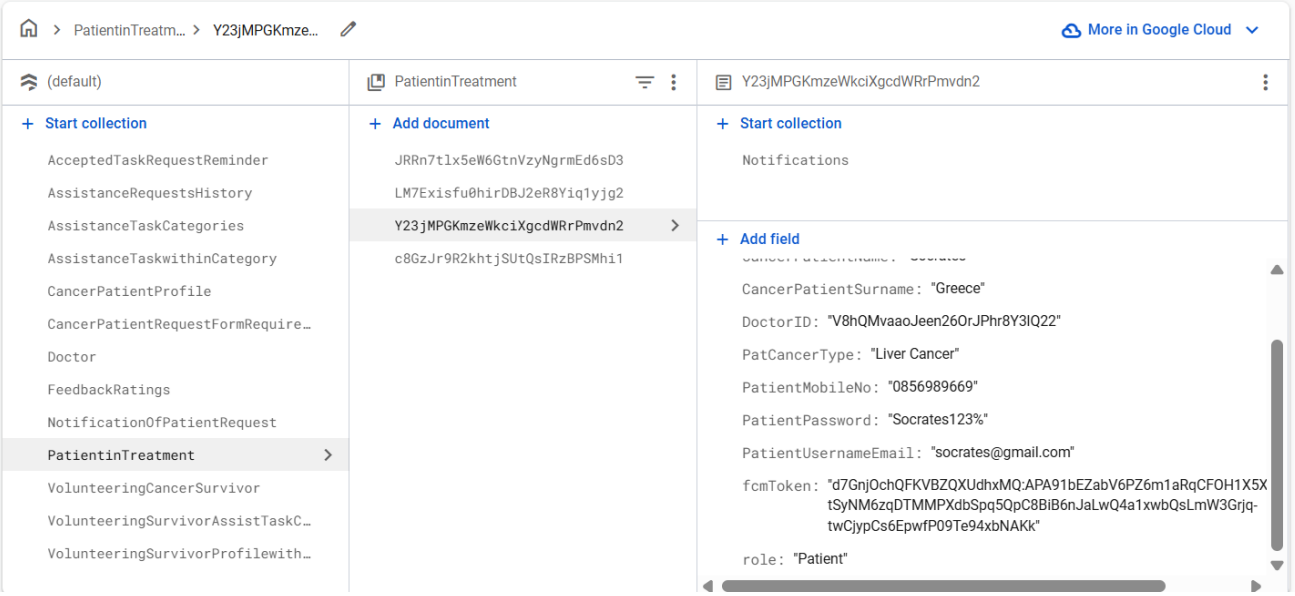


Figure 149: User Tokens for Volunteer`s REQUEST collection with UNIQUE ID, encrypted for each notification record.

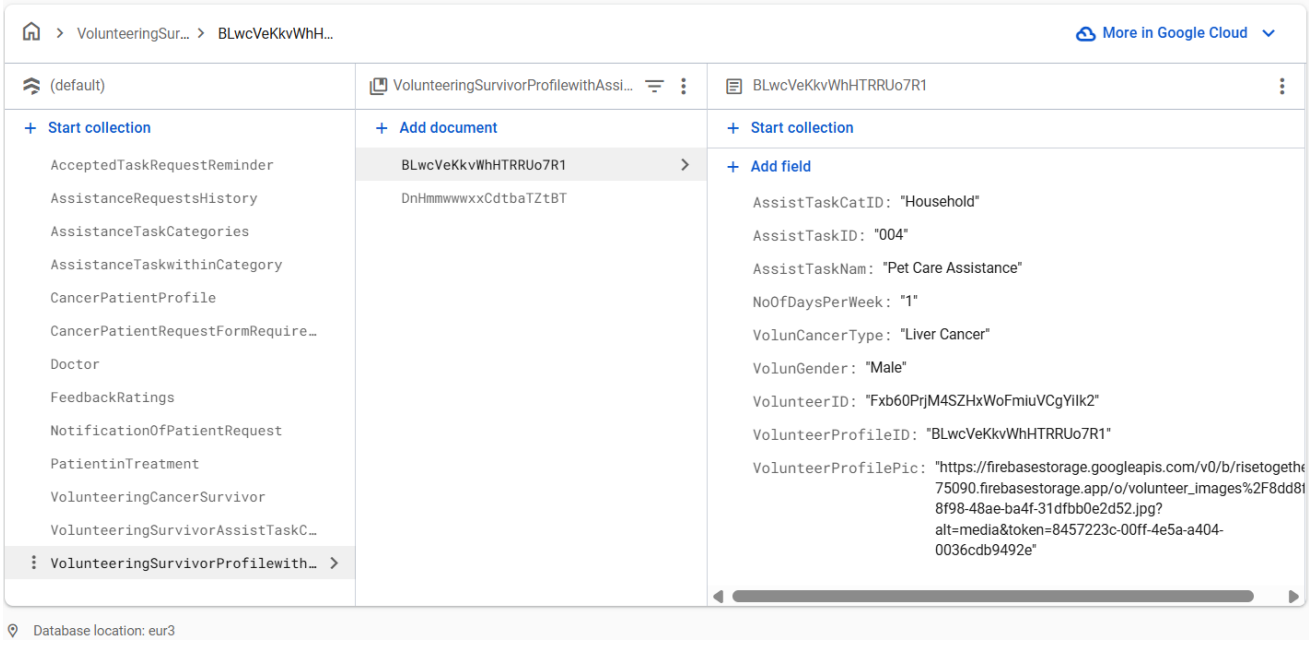


Figure 150: Volunteer Profile ID - to bring all the information and details on the screen.

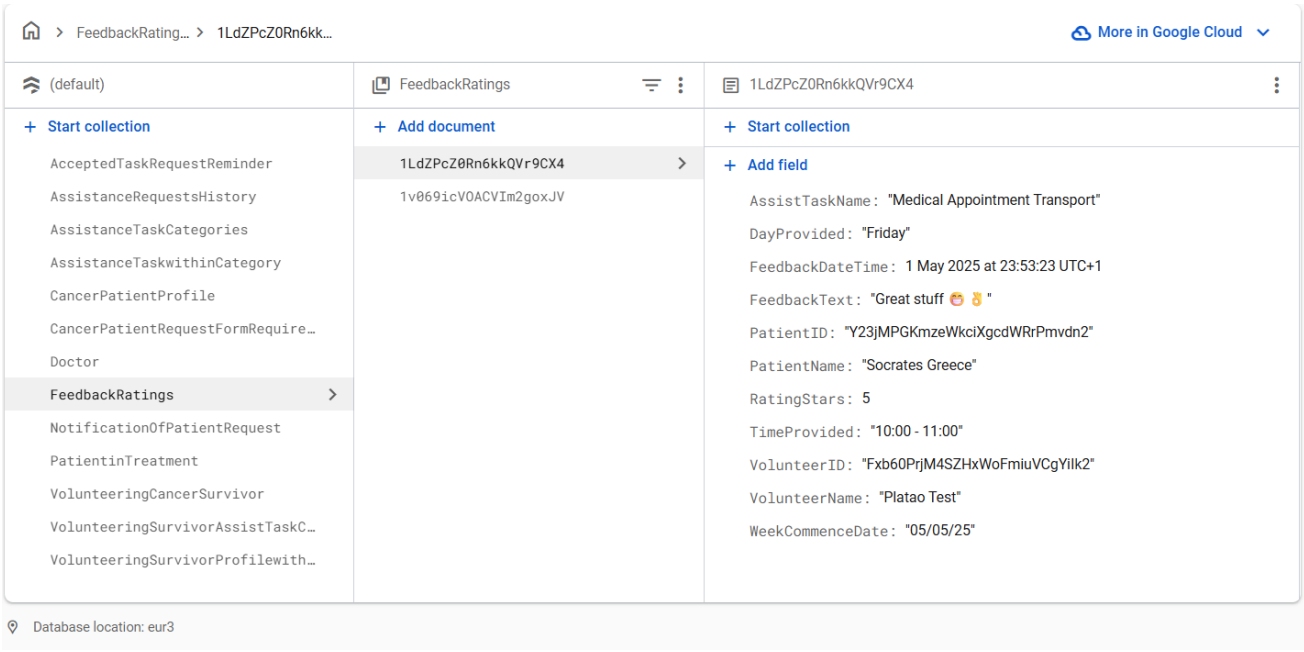


Figure 151: Feedback - to bring all the information and details to the screen through their ID (Patient and Volunteer) taken from the task provided.

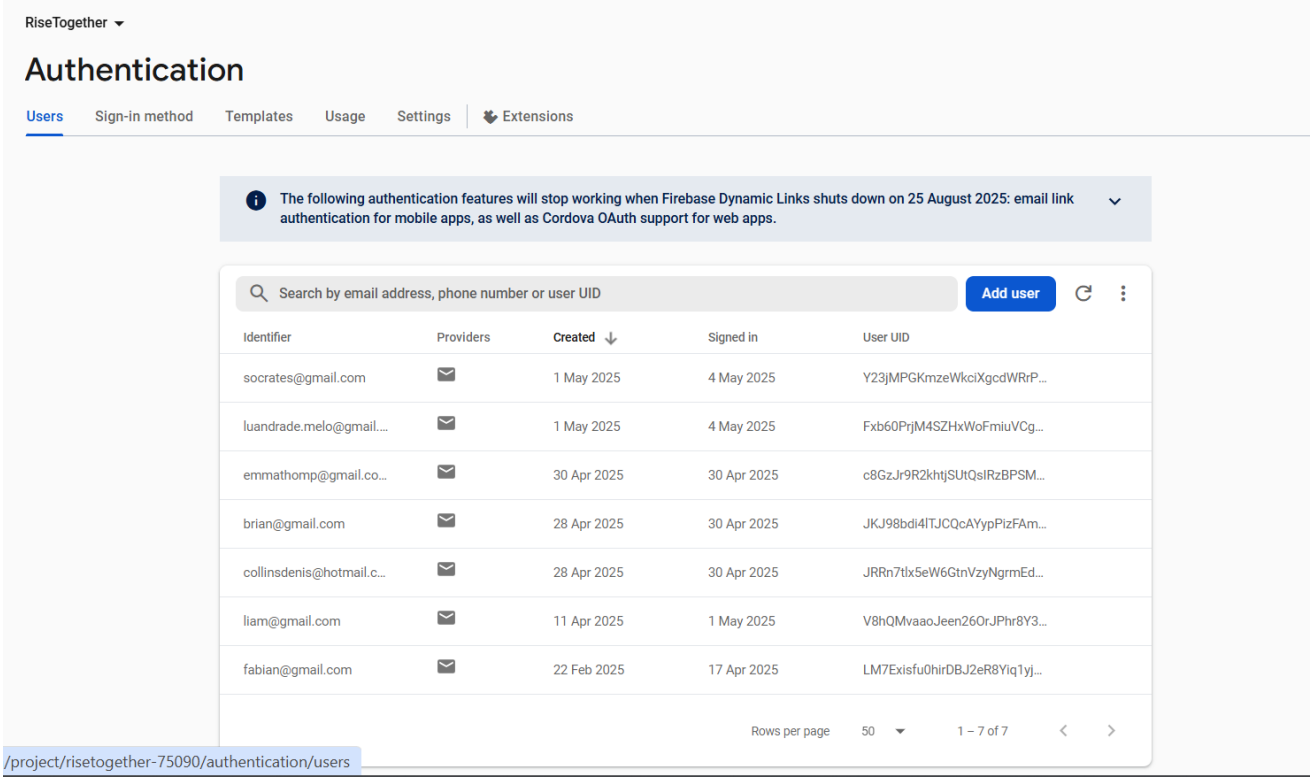


Figure 152:Usernames (Emails) and IDs created and encrypted by Firebase for each User at Registration Point

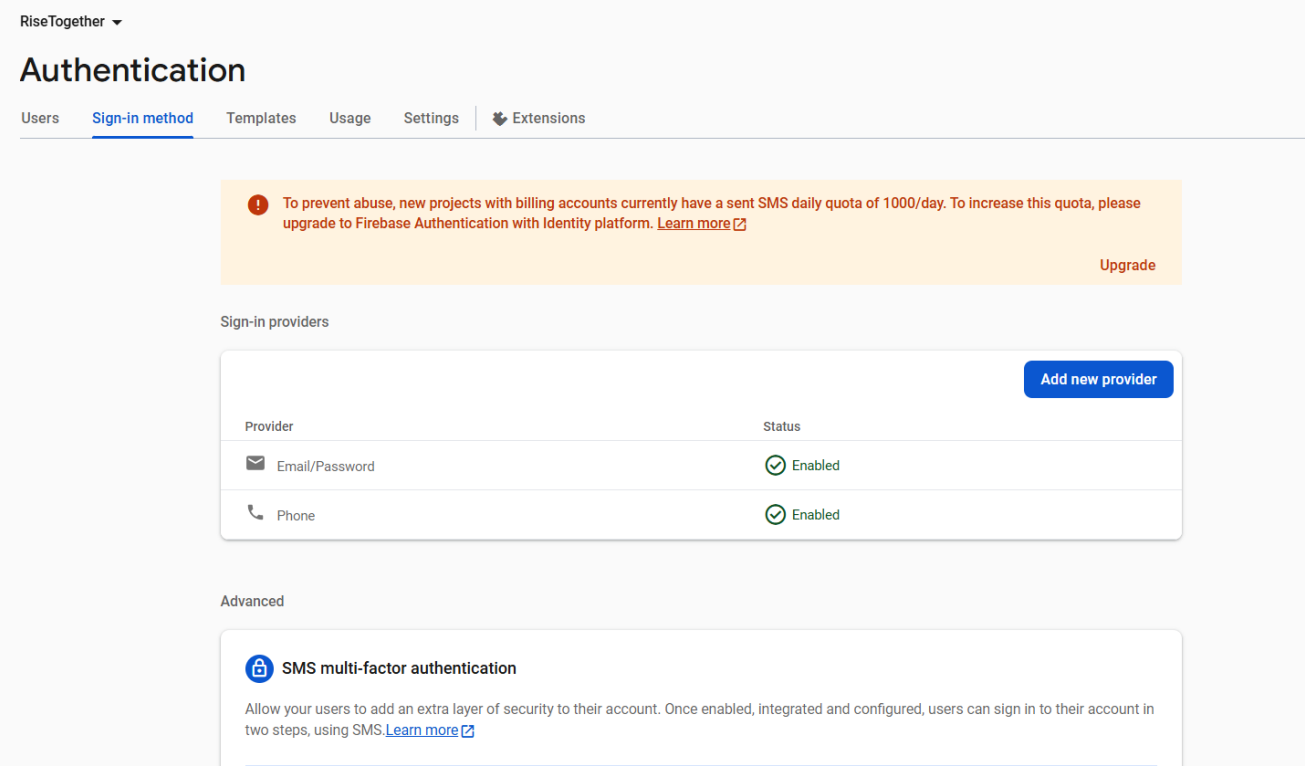


Figure 153: Login Authentication Method in the Google Firebase

4.3.1 Software Architecture

A Three-Tier Architecture is an architecture of three hierarchical tiers of logical computation, which is made up of the First tier, the Second tier, and the Third tier (IBM Knowledge Center, 2020). These are the *Presentation* Layer, the *Application* Layer, and the *Data* Layer.

Presentation layer - is on the first tier of architecture and is responsible for presenting information relating to the services offered in the program in the shape of a Graphical User Interface (GUI). This is the front-end layer of the application with which end-users will interact, usually developed with HTML/CSS/JavaScript/TypeScript.

Application Tier - also known as the middle tier. It is the layer that deals with the main functionality of the application. It is usually developed using programming languages such as Java, Kotlin, Python, C++, Perl, PHP, etc. Through extensive processing, it regulates the core functionalities of the application.

Data Layer - includes the database of the System, created with the help of Database Management System (DBMS), like MongoDB, Oracle, MySQL, Microsoft SQL Server,

Google Firebase, etc., wherein all the details of the application are stored as well as retrieved as and when needed, with actions being regulated through the middle tier.

Software Architecture for the Current Project

However, the current Project's application has two tiers; the developers can update or modify an existing tier with utmost ease, without the need to rewrite the complete application. It offers greater flexibility in the stages of Design and Coding. Yet, for the present Project, Two-Tier Architecture seemed more appropriate, as Google Firebase facilitates direct interaction capability through its provided Client-side SDK. Firebase offers the SDK on the client side, which directly talks with its back-end services. This type of architecture makes the system simple with fewer requirements for an intermediary server, thereby offering improvement in performance as well as maintainability.

Two-Tier Architecture for the Project:

1. Presentation Layer (First Tier):
Development Environment: Android Native UI Framework
Development Tools: Android Studio (Ladybug 2024.2.2 Version)
Technologies Used: Kotlin and Java
Function: This tier provides the graphical user interface (GUI) through which users interact with the system. It handles user input, presents data, and communicates with backend services using Firebase's client SDK.

2. Data Layer (Backend Services):
Technology Used: Google Firebase

Firebase Cloud Messaging (FCM): Enables real-time notifications and data sync.
Firebase Firestore: Manages structured data storage and operations.

Storage Services: Handles the storage and retrieval of images and other media. (Figure 154 presents the storage bucket for this project) and (Figure 155, the Two-tier Architecture of the Project's Application).

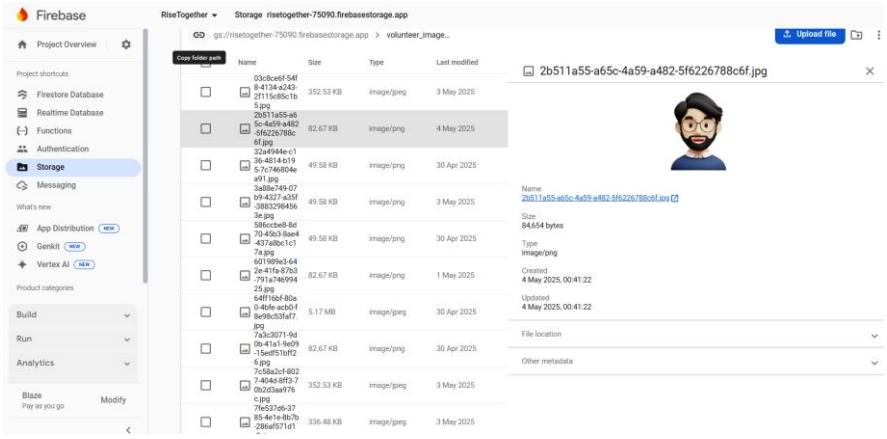


Figure 154: Snippet of the database storage with the pictures stored in Google Firebase



Figure 155: The Two-tier Architecture of the Project's Application

Implementation Details:

- **Frontend Development:**

To ensure robust performance and scalability, this project uses a design for implementing Kotlin in an *Android Native* mobile application development environment, providing robust performance and scalability (JetBrains, 2024).

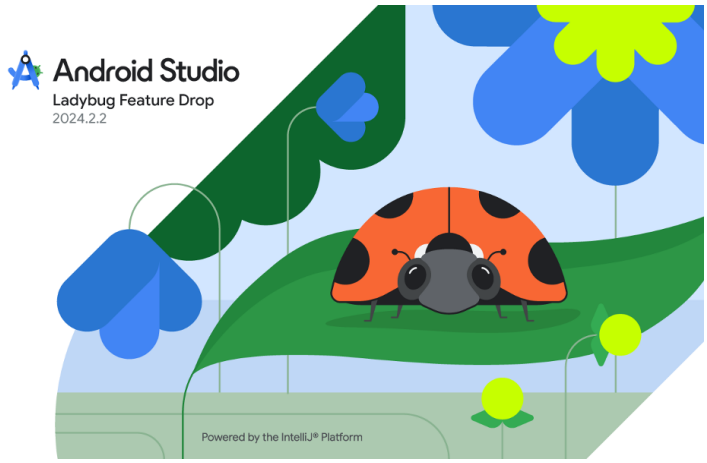


Figure 156: The Use of Android Studio (Ladybug 2024.2.2 Version), in building the Curren Application.

Key Features of the current Project's Application Architecture:

• Real-Time Data

Firebase Real-time Database allows for immediate synchronization of all user devices with the data, thereby improving the user experience.

Scalability and Reliability: Firebase Cloud Firestore offers scalable storage and management of data, enabling the app to scale and cope with rising volumes of data without performance degradation. (Firebase, 2024e). To see the example of an e*fficient Data Management*, please, see Figure 157.

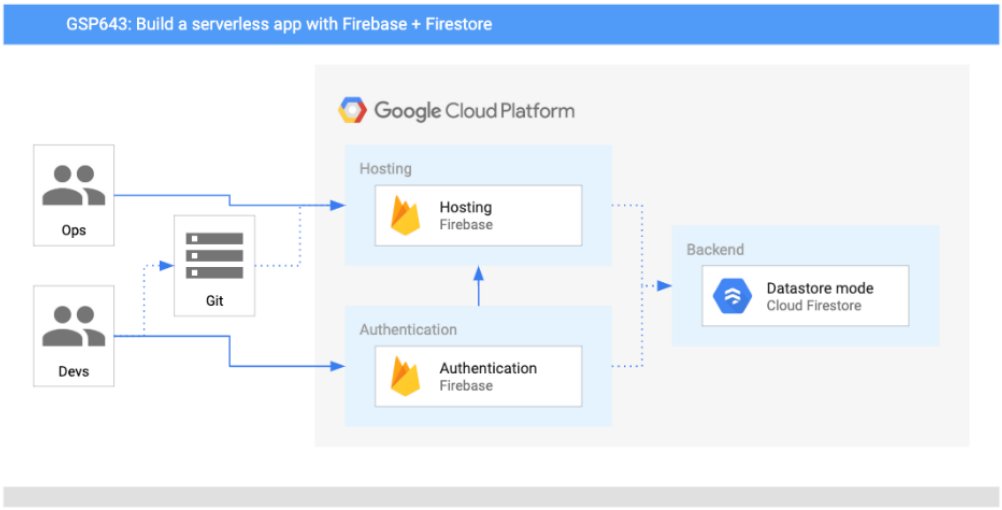


Figure 157: The Use of Google's Firebase and Cloud Firestore in building the Backend of the current Application.

4.4 Chapter Summary:

This chapter documented the essential steps involved in the *Rise Together* application to design and develop an easy-to-use interface. The use of Wireframe Design within the Storyboarding stage helped to establish the structure, navigation, and role-based functionality of the app. These initial design steps acted as a visual roadmap and functional blueprint for the Coding stage. The wireframes built with Balsamiq (Balsamiq, 2024) were crucial to designing the application quickly. Usability, accessibility, and user data privacy were prioritized throughout development, given the use of sensitive health-related user data. Significant backend integrations like Firebase Authentication, Firestore, Cloud Functions, and FCM were included for managing login flows, data storage, secure communication, and real-time notifications. The Coding stage involved additional system reliability enhancements and user experience for different user types, such as Patients, Volunteers, and Doctors. The resultant developed prototype adequately addresses the core goals of the project and provides an operational system, facilitating cancer care collaboration.

The subsequent chapter will detail the Testing phase of the app development, describing the testing techniques utilized, as well as the approach used for ensuring functionality, protection, and system reliability in all user interfaces.

Chapter 5: Application Testing Phase

Software testing is an essential stage of the Software Development Life Cycle (SDLC), whereby the system operates as specified, is secure, and provides a consistent user experience under various conditions. Software testing is the process of testing a system or its components with the aim of discovering faults, omissions, or incomplete requirements as compared to the requirements, as per the International Software Testing Qualifications Board (ISTQB). (ISTQB, 2024).

In today's software development practices, testing is not just an activity of validation, but rather a strategic quality assurance process, supported by international standards, like ISO/IEC 25010, which outline quality characteristics such as functionality, reliability, usability, performance efficiency, as well as maintainability. With these ideas, testing allows for the detection of defects early on, saves costs, and increases confidence in deployed systems. With usage trending from legacy systems towards systems, deployed on the Cloud, testing methods also must adapt in the face of challenges, such as scalability, systems being distributed, and risks of attacks. Under such conditions, the integration of automated testing, unit testing, and Continuous Integration/Continuous Deployment (CI/CD) pipelines come to the forefront. With automation, Test Cases could be run faster, repeatedly, and consistently, thus increasing system reliability, as well as enabling frequent code releases with no compromise in stability (ISO/IEC, 2011).

The Cost of Poor Testing:

A Real-World Example. In 2012, Knight Capital Group suffered a loss of $440 million within 45 minutes, due to the deployment of outdated, untested code, causing the trading system to execute thousands of faulty transactions. The incident highlighted the dangers of poor regression testing and insufficient deployment controls in real-time financial systems. The example demonstrates that an error in testing could lead to catastrophic results, especially in systems with sensitive data or real-time functionalities. Today, mistakes for such risks can be countered with automated unit tests, regression tests, and test orchestration in the Cloud that facilitate early detection of bugs before release (SEC, 2013).

5.1 Testing Methods

Software testing is an expansive and dynamic field, that can be done in several ways, based on the development phase, complexity of the system, and the purpose of testing. Figure 158 provides an image with easy visual breakdown of the various types of software testing, as well as their methods, bifurcating them mainly as *manual* testing and *automation* testing, further grouping into other types like *white-box*, *black-box*, and *grey-box* testing. At its core, testing can either be dynamic or static. *Dynamic* testing is done by running the system or application to observe how it performs in reality; that is what the term "testing" is typically understood. *Static* testing is independent of system execution. Rather, it involves *inspecting* documents such as requirements, user stories, or source code to identify defects early. Figure 5.1 depicts all these types of testing.

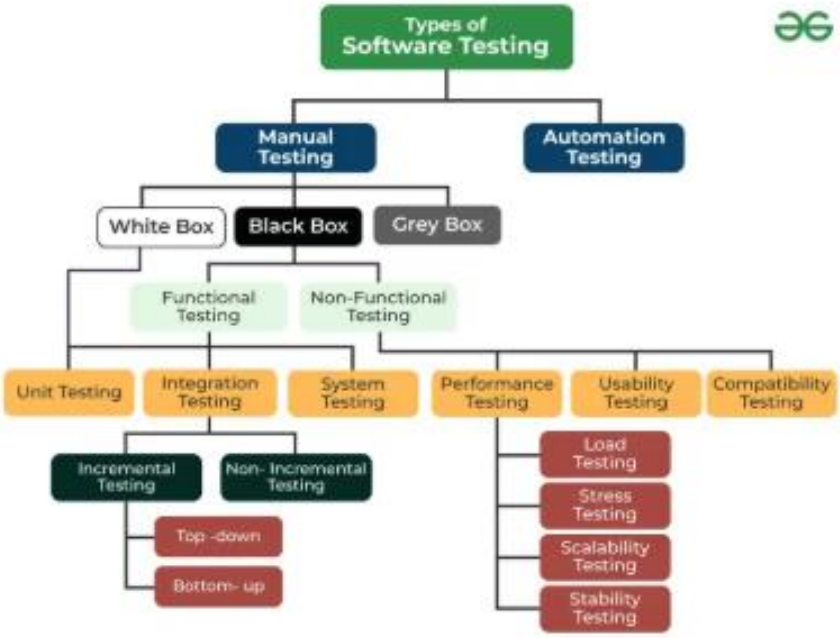


Figure 158: Types of Software Testing by GeeksforGeeks (2025)

The lower section of the diagram represents deeper subdivisions, including incremental vs. non-incremental integration. Top-down vs. Bottom-up testing strategies and specialized types, such as load, stress, scalability, and usability testing, which are critical when staging an app for real user loads, as well as diverse devices. Testing is not only about verifying that something functions, it is about verifying that something functions correctly, efficiently, safely, and in such a way as to satisfy the needs of live users. Using the appropriate kind of test at the proper time, developers can create more dependable, maintainable, and high-quality code.

According to Miller (2012), "*Software testing is a foundational component of system development that ensures software behaves as expected under defined conditions. It provides*

developers and testers with a systematic approach to identify, isolate, and resolve defects, before deployment."

5.2 Testing Methods

This section outlines the three primary software testing methods, widely used in today's industry: *Black Box*, *White Box*, and *Grey Box* testing. Each technique offers unique advantages, depending on the testing objectives and the level of system visibility.

5.2.1 Black Box Testing

Black box testing is one of the most popular methods, used in the field of software testing. It is as simple, as it is effective in its approach: testing the program with no knowledge of what goes on inside it. The test engineer is not privy to the source code or system design; they use the program in the same way as an end-user would, inputting and receiving outputs. Black-box testing, also known as *specification-based* or *opaque-box* testing, focuses on verifying the software's functionality against its requirements, without any knowledge of the internal code or logic (ISTQB, 2024). *Black box* testing is mainly utilized for testers to verify if the system performs as anticipated, according to functional as well as non-functional requirements. An interesting point about Black box testing is that it is *independent* of the code; in other words, it can be used for testing during several stages:
The method is usually applied in the *Integration Testing*, *System Testing*, and *User Acceptance Testing* (UAT) phase, when test users interact with the interfaces of the system and verify output for diverse inputs. While not usually applied for *unit* testing, *Black box* techniques can be used when public APIs or third-party components are being tested with no source code available. (Myers, Sandler, and Badgett, 2011).

5.2.2. Techniques of Black Box Testing:

Equivalence Partitioning: Partitioning of the input data into valid and invalid. Only a single test value is used for an entire set, with the expectation that the test value will be equivalent for the set.
As an example, when testing for a field for which the users must be 18 years of age or older, input values can be classified as either valid (age>=18) or invalid (age<18). It is tested with one sample from either class.

Boundary Value Analysis: targets testing the values on the edges of input boundaries, since that is where systems tend to fail. For instance, if an application asks users' age to be 18 or older, then test for values precisely equal to 17, 18, and 19 years. These values are just on the border of acceptability as well as rejection and will be the maximum likelihood of exposing latent bugs.

Decision table testing: Excellent for business rules. It involves testing input combinations with their anticipated actions. State Transition Testing: Applied when the system responds in different manners, depending on its current state. Typical in workflows or in navigation steps.
Use Case Testing: Constructs Test Cases from usage paths in real-life scenarios, typically used in acceptance testing and for UI verifications.

How the Black Box Testing Process Works (Manual Perspective):
A simplified, human-oriented step-by-step description of how *Black box* testing is generally performed in manual test environments, is as follows: Understand the specifications: Go through user stories, functional specs, and acceptance criteria to comprehend what the system should do.

Design the Tests: Determine what is being tested, how, who is performing the tests, and in what environment. Determine the scope and objectives.

Design Test Cases: Provide step-by-step test steps, input values, desirable output, and validation checkpoints for every situation.

Prepare Test data: Generate test data sets that mimic real-world usage in terms of both valid, invalid, and edge-case inputs.

Run the Tests: Manually interact with the app - press the buttons, fill in the forms, load files, and verify results against expectations.

Defects in Log: Report defects clearly with steps to reproduce, screenshots, and results observed.

Do Regression Testing: After fixing bugs, test the areas affected once again, to ensure nothing else is broken.

Results Report: Report test coverage, passed/failed cases, and open bugs.
Test Closure: Close out the process by examining documentation, test evidence, and results.

Continuous Improvement: Provide feedback for developers and recommend enhancements in the test process for future releases.

Figure 159: provides a visual representation of the Black Box Testing, a method where inputs are tested against expected outputs without any knowledge of the internal system logic. This approach focuses solely on the functionality from the user's perspective.

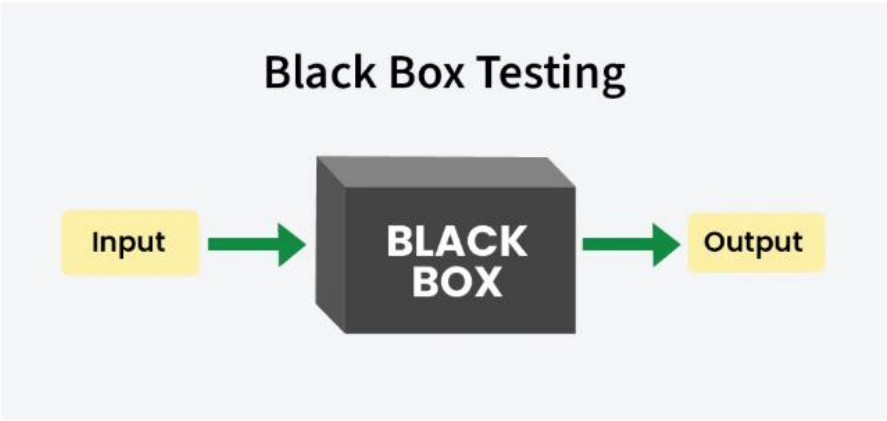


Figure 159: Black Box Testing by: (GeeksforGeeks, 2024a).

Advantages of Black Box Testing:

- Testers don't have to program or be system experts to execute it.
- Ideal for large systems from an end-user perspective.
- Test cases can be readily executed repeatedly.
- Aids in clarifying ambiguous or contradictory requirements in the specs.

Disadvantages of Black Box Testing:

- Tests can be duplicated inadvertently.
- Test cases are complex and require an ambiguous design.
- It can be difficult and time-consuming to manage complex or diverse inputs.
- Failures can sometimes be difficult to trace, and errors in internal logic can be unnoticed.

5.2.2 White Box Testing

White Box Testing, or *structural*, or *code-based* testing, is an approach that features direct access to and examination of a program's source code. In contrast with *Black box* testing, which can only check outputs based on inputs, *White box* testing enables the testing of internal logic, control flows, and code structure, to guarantee correctness, efficiency, and security. (GeeksforGeeks,2024b).

Integration Testing: Emphasizes internal module interaction. White box *integration* testing verifies that the data passed between units is accurate and that internal APIs behave correctly.

Code Coverage Analysis: Although not technically "a test," it is used to measure how much of the code is being tested. It aids in test optimization and makes sure key paths aren't overlooked.

Loop Testing: Primarily used to test for looping constructs in code (for example, for, while). It checks that the loops go through the right number of iterations and process boundary conditions correctly.

Path testing: Pertains specifically to testing all the logical routes through code, such as if-else constructs, nested conditions, and loops. Typically used in mission-critical applications.

Data Flow Testing traces variables as they are declared, used, and modified in the code. It highlights issues such as unnecessary variables or improper data overwrites. Tip: These can be performed manually, but they're optimized when automated, such as in CI/CD pipelines.

White box testing is beneficial when used in combination with code quality analysis, vulnerability detection, or secure coding compliance tools.

It is ideal for initial testing in the SDLC, particularly the development, as it allows for early detection of defects and optimization of code and enables security evaluations with techniques like static code analysis. Benefits are code coverage, early bug detection, and optimization. It is, however, based on detailed knowledge of coding, can miss issues in the user interface, and can cause bias from familiarity with the system's implementation.

Main White Box Testing Techniques:

One of the most extensive techniques used for White Box Testing, is the *code coverage* technique, used for determining how much of the code is covered through testing. The following are the techniques that are of utmost importance as per ISTQB, as well as standard industry practice.

Coverage statement: Ensures that all lines of code are executed at least once when being tested. This guarantees detection of useless or "dead" code segments that would otherwise be

overlooked. Example: Testing of a function used to compute the area of a triangle, ensuring that every assignment, condition, and return statement is executed.

Decision Coverage (Branch Testing): Checks that all decision points (for example, conditions such as if, switch, or while) are both true and false at least once. This guarantees that all logical paths have been tested. For example, testing the Login function with valid and invalid credentials ensures both the path of success and failure are called during the test. (GeeksforGeeks, 2024b).

Figure 160 represents the process of the *White Box* Testing, in which the inner structure, logic, and code of the program are all accessible to the tester. The illustration depicts how Test Cases are entered directly into the program code to test their behavior through unit testing, integration testing, and system testing. The testers or developers full-stack check code paths, conditions, and branches of logic as they trace from input to output. With this approach, thorough code correctness, performance, and security can be analysed from inside out. (Miller, 2012).

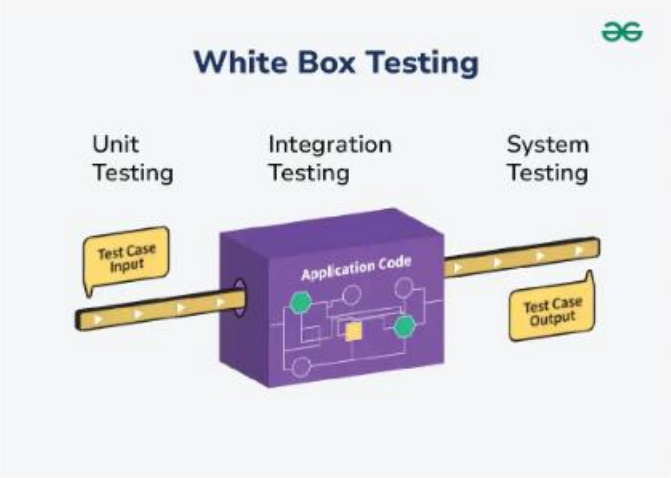


Figure 160: White Box Testing (GeeksforGeeks, 2024b)

Advantages of White Box Testing:

- It provides rich insight into the code's internal structure and reasoning, rendering it very useful in identifying intricate bugs that are hard to detect.
- Enables the detection of bugs early on by identifying lines of erroneous reasoning or uncovered code paths.
- Easily integrates with CI/CD pipelines and automated unit test tools for increased development effectiveness.
- Enabling strongly personalized tests, particularly for intricate conditions, boundary cases, and security testing.

Disadvantages of White Box Testing:

- Needs source code access as well as extensive programming knowledge, something that is not always available within some teams.
- It may be time-consuming to apply, especially in large or old codebases.
- May miss usability, functionality, or visual design-related issues as it is not concerned with user interaction.

5.2.3 Grey Box Testing

Gray Box Testing is the blend of both *White Box* and *Black Box* Testing, which utilizes the strengths of both. *Gray Box* Testing offers the test engineer partial knowledge regarding the internal workings of the system to develop improved Test Cases with minimum access to the source code. *Gray Box* Testing is best suited for testing user-facing functions, data flow, system functions under different conditions, and system functions behind the API. Practically implemented, *Gray Box* Testing allows testing of real-world conditions with visibility of the backend logic. Matrix testing, state transition testing, and testing of the API are among the techniques involved. It reveals bugs in every description, ranging from logic bugs to bugs of integration, and is easily implemented on the web as well as in mobile apps. One of its greatest advantages is that it is balanced: no deep programming knowledge is needed by the testers, but still, the testers can gain knowledge of the internal mechanics of the system. This leads to having a closer focus on the tests, better communication between the developers and the testers, and the overall quality of the product is enhanced. This testing is aided by tools, such as Selenium, Postman, and Burp Suite. It does not offer full code access, as is the case with *White Box* Testing, and is also not good enough to be implemented for thorough testing of the algorithm. Nonetheless, it is good enough for *integration*, as well as for *functional* testing from the *user* point of view. It is effective, flexible, and good enough to detect problems that might otherwise not be found through solely *internal* or *external* testing techniques. (GeekforGeeks, 2024c).

Common Grey Box Testing Types:
API Testing: Ensures that the external applications or frontends will work with the system. These test methods utilize applications such as Postman or REST-assured to check the request/response functionality, headers, status codes, and data handling.

Database Testing: Guarantees data reliability, compatibility, and protection by confirming how the app writes to or reads from a database. The test cases tend to check for queries, indexes, and triggers even when no full DB access is available.

Security Testing: (Penetration or Vulnerability): Examines access control techniques, input checks, and authentication flows based on partial knowledge of their internal implementation.

Risk-based testing: Prioritizes testing according to risk thresholds, incorporating *external behavioral* checks with *internal architectural* knowledge to concentrate on the highest-risk or critical areas.

Functional Testing with Internal Acuteness: Grey box testers verify anticipated system functionality when they test backend logic like business rules, workflows, or internal error handling.

Model-based testing: Utilizes system models or system diagrams (such as UML) for generating Test Cases. Best when an internal partial design is only revealed to the testers.

Primary Techniques for Grey Box Testing:

The main techniques used for *Grey box* testing include functional testing, which is combined with structural testing, and checks the system operation with confirmation of internal arguments and data processing. Testing both the login screen and the processing of login requests in the backend database. Partial Code Access or Logic Awareness Aids assist test writers in building more efficient Test Cases through component or business logic identification. A test checks internally how a payment module operates before testing actual transactions. Interface as well as integration testing, check how components interact (e.g., frontend, backend, service, and database). Testing that an API endpoint returns the correct user profile based on the current database. Figure 161 presents the *Grey box* testing.



Figure 161: Grey Box Testing (GeeksforGeeks, 2024c)

Advantages of Grey Box Testing:

- Balanced Perspective: Combines the strengths of both Black Box and White Box testing, allowing testers to validate internal processes while still focusing on user-facing functionality.
- Improved Collaboration: Acts as a bridge between testers and developers by providing enough insight into the system architecture for meaningful communication.
- Efficient Defect Detection: Helps catch both logic and integration bugs by targeting areas often missed in purely Black or White Box approaches.
- No Deep Coding Knowledge Required: Testers don't need full access to source code or advanced programming skills.

Disadvantages of Grey Box Testing:

- Limited Internal Access: Testers only have partial visibility into the system's internals, which may limit test depth.
- Not Ideal for Algorithm Testing: In-depth algorithm validation often requires full access to code, which Gray Box doesn't always provide.
- Complex Test Case Design: Requires a careful balance between functional behavior and internal structures, making test design more challenging.
- Harder Defect Tracing: Sometimes, it's difficult to trace the exact root cause of a defect, especially in distributed systems.

5.3 Chosen Test Method for the Current Project's Application

For the *Rise Together* mobile application, *Grey Box* Testing was selected as the most appropriate approach, due to its hybrid nature, which combines both functional (*Black Box*) and structural (*White Box*) testing perspectives. This method allowed to validate not only the app's user-facing features, but also its backend logic, including database operations and authentication flows. Given the app's Cloud-based architecture (Firebase SDK with Firestore),

limited development time, and the need for secure and accurate data handling, *Grey Box* Testing enabled focused testing on complex areas, such as:

- ✓ Matching patients with volunteers
- ✓ Login and access controls
- ✓ Form validations
- ✓ Data flow and database integrity

This approach aligns with current industry practices for mobile and Cloud-integrated apps and ensures both quality and security, prior to deployment (TestMatick, 2020).

Testing approach used for the developed application:

1. Planning Phase
   - ✓ Defined what features would be tested and the goals of testing
   - ✓ Chosen appropriate testing types: functional, regression, security, etc.

2. Design Phase
   - ✓ Created detailed Test Cases for each app feature

3. Execution Phase
   - ✓ Ran tests, logged defects, and performed regression testing
   - ✓ Verified that bug fixes didn't introduce new issues

4. Closure Phase
   - ✓ Documented test results and defect summaries (IEEE, 2014)

5.3.1 Initial Errors Encountered During the Application Coding Process

Although the development of the *Rise Together* application followed a clear architecture and well-structured coding practices, several challenges and implementation issues emerged throughout the coding and integration phases. These were identified during testing and progressively resolved, before the final deployment. A summary of the most significant errors encountered, and how they were addressed during the Coding process is provided as follows:

1. Password Field Without Validation

In the initial version of the Login screen, when the user leaves the *password* field empty and clicks on *Login* button, the app shows a white screen and crashes. This was rectified by adding real-time field validation through the implementation of Text Watcher and conditional validation to check that the password field cannot be left empty before initiating Firebase Authentication.

2. Volunteer Details Missing Date and Time

In the Volunteer profiles that were shown to the patients, the listed tasks were visible, but the date and time of availability were not shown. Following the investigation, it was realized that the app wasn't correctly binding Firestore fields into the RecyclerView layout. A solution was achieved through changing the Firestore query and including all calendar schedule fields and mapping them to the related UI elements.

3. Matching logic disregarding gender and cancer type

At the beginning, the matching system was not screening the volunteers, based on both gender and type of cancer. While the matching system was fetching the volunteers, it wasn't using combined filters on these two fields. This is because there was a Firestore query that employed one filter at a time. The query was then rectified using chained whereEqualTo () clauses to apply the two conditions together, and that corrected the inconsistency.

4. Gradle Build Configuration Errors

Various Gradle synchronization and Firebase dependency issues were encountered during project configuration. The build.gradle.kts file at one time contained out-of-date plugin versions and was resulting in failing builds within Android Studio (Version Ladybug 2024.2.2). The Google Services: The JSON file was improperly set and referenced, resulting in build failures during Firebase Authentication or Firestore usage. These were rectified by scrutinizing the Gradle configuration and updating dependencies to the most recent Firebase SDK and Kotlin.

5. Notification Failures with Firebase Cloud Messaging (FCM)

Push notifications didn't get delivered to the user devices at all, despite the logic being correct. Upon testing, it was found that the problem was due to missing FCM tokens and wrong authorization scope, set in the configuration of the Firebase Admin SDK. The FCM V1Sender class was modified to retrieve and store the device tokens securely at the Login step, and the OAuth 2.0 scope was also set to include https://www.googleapis.com/auth/firebase.messaging. The legacy FCM API was also manually disabled to make the app use the FCM V1 endpoint.

6. Form Reset and Button Behavior Issues

Certain types (for example, Register Patient or Register Volunteer) were not properly resetting after submission. This created double entries in case of the doctor clicking "*Send*" button twice. In the same vein, submission feedback forms contained buttons that were still active once the form was saved. Buttons disabled on click has rectified these issues and offered clear success messages and optional reset or navigation.

7. Authentication Redirect Loop

One of the subtle bugs was related to the Login and Password Reset screens. The users were not correctly redirected to the role-specific login screen, after resetting the password. Sometimes, the app defaults to the doctor login screen, regardless of the user type. This occurred because of the absence of session data and the lack of proper Intent handling. The redirection logic was revised to save the *role* of the user during the *reset* and redirect them accordingly once the action was completed.

5.4 Application Testing Process with Test Cases

During the actual Testing phase, the *Grey box* testing method was utilized with a developed set of Testing Cases, and a thorough examination of all the application's features took place for validation and verification. These actions were taken to guarantee that all project requirements and functional criteria were met precisely, as outlined in the Project's objectives. The primary purpose was to confirm that the application operates in accordance with the defined requirements, specified in the current Project's Use Case Diagram. The Testing process, along

with the detailed Testing Cases, is documented in Tables 5.1 to 5.3, ensuring comprehensive testing of the application.

Table 4: Fixed Errors during the Testing Process with Testing Cases

Test Case	Expected Result	Actual Result	Fixed?
"Recover the *forgotten* password "	The system sends a *Reset Password* link to the registered email.	The system fails to send the email; the user receives no reset instructions.	✓
"Register user with an invalid Irish phone number."	The system should *reject* invalid numbers and show an error.	The user is registered even with an invalid number.	✓
“Register a user with a mismatched password confirmation”.	The system should show: *“Passwords do not match”* error message and block the registration.	The system showed a white screen, and the interface was closed.	✓
“Log in with *empty* password field”.	“The system should prompt the user to fill in the password”.	The app crashes or shows a blank screen.	✓

5.4.1 Functional Testing

This employs Black box testing methodology, focusing on evaluating software functional requirements.

Table 5: Functional Testing Process with Testing Cases

*Function: **Doctor** - Login/Register Patient and Volunteer.*

Test Case	Expected Result	Pass	Fail	Actual Result
Access the 'Doctor' functionality from the app's Home screen	Redirected to the '*Doctor*' Login screen.	**Pass**		Redirected to the 'Doctor' *Login* screen.
Register a patient with a phone number, that is not in valid Irish format (10 digits in the field domain)	*Error message* appears, indicating invalid phone number format.	**Pass**		An error message, indicating an invalid phone number format, is displayed.
Register a patient with an *invalid* Irish mobile *number*	Error message appears, indicating invalid Irish mobile number.	**Pass**		Error: "Invalid Irish number".
Register the patient with an *invalid* email format	Error message appears, indicating invalid email format.	**Pass**		An error message indicating an invalid email format is displayed.
Register the patient with a password of fewer than 8 characters in length and composition (upper/lower	Error message appears, indicating weak password.	**Pass**		An error message indicating a weak password is displayed.

letters/digits/special character).				
Register the patient with an *invalid* matching gender/cancer type	When registering a patient with a different type of cancer for a volunteer match.	**Pass**		The system does not show a volunteer with the same type of tasks as expected.
Register the patient with *all valid* data.	Success message appears, indicating successful registration.	**Pass**		A success message, indicating successful registration, is displayed.
Check email sent after clicking *send* in the registration process.	A link to re-set password has been sent.	**Pass**		Link Received
Register the patient with a phone number that is *not* in the Irish *format* (10 digits in the field domain)	Error message appears, indicating invalid phone number format.	**Pass**		An error message, indicating an invalid phone *number format*, is displayed.
Register a patient with an *invalid* Irish mobile *number*	Error message indicating invalid phone number.	Pass		Error displayed: "A valid Irish mobile number starts with 083,085,086,087,089 and should be 10 digits long"
Register a Patient with **cancer type as "Breast Cancer"**	Cancer type "Breast Cancer" saved and linked to patient profile.	**Pass**		Cancer type "Breast Cancer" saved and linked to patient profile.
Register a volunteer with cancer type **"Breast Cancer."**	Cancer type "Breast Cancer" correctly displayed in patient record.	**Pass**		Cancer type "Breast Cancer" saved and linked to volunteer profile.

Register a patient with **cancer type as "Other"**	Cancer type "Other" accepted and saved correctly.	**Pass**		Cancer type "Other" accepted and reflected in patient matching logic.
Register a volunteer with **cancer type as "Other"**	Volunteer successfully registered with cancer type "Other".	**Pass**		Cancer type "Other" accepted and reflected in volunteer matching logic.
Register a patient with Gender **"Male"**	Patient successfully registered with gender "Male".	**Pass**		Patient successfully registered with gender "Male".
Register a patient with Gender **"Female"**	Patient successfully registered with gender "Female".	**Pass**		Patient successfully registered with gender "Female".
Register a patient with Gender **"Other"**	Patient successfully registered with gender "Other".	**Pass**		Patient successfully registered with gender "Other".
Register a patient with the field "Cancer Patient Name" *empty*.	An error message is displayed indicating that this is a mandatory field and must be completed before submission.	**Pass**		Error message indicating required field is displayed.
Register a patient with the field "Cancer Patient Surname" *empty*.	An error message is displayed indicating that this is a mandatory field and must be completed before submission.	**Pass**		Error message indicating required field is displayed.
Register a patient with the field "Gender" as *empty*.	An error message is displayed indicating that this is a mandatory field and	**Pass**		Error message indicating required field is displayed.

	must be completed before submission.			
Register a patient with the field "Initial Password" *empty*.	An error message is displayed indicating that this is a mandatory field and must be completed before submission.	**Pass**		Error message indicating required is displayed.
Register a volunteer with the field "Volunteer`s Name" empty.	An error message is displayed indicating that this is a mandatory field and must be completed before submission.	**Pass**		Error message indicating required field is displayed.
Register a volunteer with the field "Volunteer`s Surname" *empty*. Register a volunteer with the field "Gender" as *empty*.	error messages are displayed indicating that these are mandatory fields and must be completed before submission.	**Pass**		Error messages indicating required fields, is displayed.
Register a volunteer using the field "Initial Password" *empty*.	An error message is displayed indicating that this is a mandatory field and must be completed before submission.	**Pass**		Error message indicating required field, is displayed.
Register a volunteer with the field "Cancer Type" empty.	An error message is displayed indicating that this is a mandatory field and	**Pass**		Error message indicating required field is displayed.

	must be completed before submission.			
Click on Logout	Redirects user to the Login screen.	**Pass**		User landed on the Doctor Login screen
*Function: **Volunteer** - Login/Create Profile and Task Availability/ Volunteer - Feedback & Ratings*				
Access the 'Volunteer' functionality from the app's *Home* screen	Redirected to the 'Volunteer' Login screen.	**Pass**		Redirected to the 'Volunteer' Login screen.
Log in with invalid password.	Error message indicating invalid password.	**Pass**		Login fails, error message displayed.
Log in with a *valid* username and password	Success message indicating successful login.	**Pass**		A success message indicating a successful login is displayed, and the user is landed on their Dashboard successfully.
Reset password for registered user.	Success message indicating password reset email is sent.	**Pass**		A success message indicating a password reset email has been sent.
Reset password for a *non-registered* user.	Error message indicating user not found.	**Pass**		An error message indicating the user is not found is displayed.
Log in with an *invalid password.*	Error message indicating invalid password.	**Pass**		Login failed; error message displayed.
Create a volunteer profile by selecting a valid category and task.	The volunteer profile with selected task and category has been successfully created.	**Pass**		The system shows a message: "*The volunteer profile with the task and category has been successfully created*".

Create a profile with *no* category selected.	An error message appears indicating that a category is required.	**Pass**		Error message indicating required field is displayed.
Create a profile with *no* task selected.	An error message appears indicating that the task is required.	**Pass**		Error message indicating required field, is displayed.
Create a profile with *valid* availability days and time slots.	Calendar schedule created successfully.	**Pass**		The system shows a message: *Calendar schedule created successfully*.
Create a profile *without* uploading a picture.	An error message is displayed, indicating that this is a mandatory field and must be completed before submission.	**Pass**		Error message indicating required field, is displayed.
Create a profile by uploading a picture.	The photo is uploaded and attached to the profile.	**Pass**		The photo is uploaded and displayed in the volunteer's profile.
Create a profile with the field Number of Days/Week Availability for the Selected Assistance Task *empty*.	An error message is displayed, indicating that this is a mandatory field and must be completed before submission.	**Pass**		Error message indicating required field is displayed.
Click on *another Assistance Task* or *go back to the Dashboard*.	The system cleans the fields to restart the process when another assistance task is selected. Or go back to the Dashboard by	**Pass**		The system cleans the fields to restart the process when another assistance task is selected. Or lands user back on the Dashboard if button "Back to Dashboard" is clicked.

	selecting the button “Back to Dashboard.			
View patient requests for the current week	List of requests for matching selected week is displayed.	**Pass**		All relevant requests shown for the selected week.
Select specific request and view details	Details of the selected request are shown.	**Pass**		Full patient request information is displayed.
Accept request	Notification is sent to the patient, and the request is marked as *accepted*.	**Pass**		Confirmation message displayed and notification sent.
Reject request	A rejection message is sent to the patient.	**Pass**		Rejection message successfully displayed and Notification is sent.
Access the feedback screen and select a week.	Feedback list for selected week is displayed.	**Pass**		Feedback entries shown in reverse chronological order. (most recent at the top)
Click "View" on the feedback entry.	Full feedback text and rating details are displayed.	**Pass**		Star rating and written comment are displayed.

Log out of the current section.	Redirected to the Login screen.	**Pass**		The system redirects the user back to the Doctor Login screen.
*Function: Login/**Patient** - Create Profile with Required Assistance Tasks/ Patient - View Details & Send Request to Volunteer/ Patient - Provide Feedback and Ratings*				
Tap “Patient” on the Home Screen	Redirected to the 'Patient' Login screen.	**Pass**		Redirected to the 'Patient' Login screen
Log in with invalid password.	Error message indicating invalid password.	**Pass**		Login failed; error message presented.
Log in with a *valid* username and password	Success message indicating successful login.	**Pass**		A success message indicating a successful login is displayed, and the user is redirected to their Dashboard successfully.
Reset password for registered user.	Success message indicating password reset email is sent.	**Pass**		A success message indicating a password reset email has been sent.
Reset password for a *non-registered* user.	Error message indicating user not found.	**Pass**		An error message indicating the user is not found is displayed.
Log in with an invalid password.	Error message indicating invalid password.	**Pass**		Login failed; error message displayed.
Create an assistance task Profile record selecting *valid* task category and *task*	Profile record is created and saved successfully.	**Pass**		Profile with required assistance task saved successfully.

Submit form without selecting category	Error message appears indicating category is required.	**Pass**		Error message displayed: “Please select a support category”.
Submit form *without* selecting task.	Error message appears indicating task is required.	**Pass**		Error message displayed: “Please select a task”.
Submit a profile with *all valid* fields.	The system saves the record and shows a success message.	**Pass**		Confirmation message displayed and Profile data saved.
View volunteers for selected task	List of volunteers, matching gender, cancer type, and task is displayed.	**Pass**		List of Volunteers correctly filtered and displayed.
View list with *no* volunteers available	Message indicating no volunteers available is shown.	**Pass**		Message: “No volunteers found for this task”.
Click on volunteer *name*	Volunteer details screen is shown.	**Pass**		Volunteer information displayed successfully.
Select available day/time and week and press *submit* request.	Request sent successfully to volunteer.	**Pass**		Request recorded and notification sent.

Submit request *without* selecting day or time.	Error message indicating day/time is required.	**Pass**		Error: “Please select a valid day and time”.
Choose a *week* and then *cancel* request.	Returns to volunteer list without saving data.	**Pass**		Navigation works correctly; no data saved.
View notification list by week.	All notifications for selected week are displayed.	Pass		Notifications shown in order (newest on top).
Click “View” on accepted request.	Detailed information with volunteer data appears.	Pass		Confirmation window for accepte shown with Volunteer’s contact ir
Click “View” on rejected request.	Rejection message shown with polite notice.	Pass		Rejection message correctly displayed.
Click “Back to Dashboard”.	User is redirected to patient *dashboard*.	Pass		Navigation successful.
Select week and view completed tasks.	Completed tasks shown with option to leave feedback.	Pass		List of completed tasks shown with “Select” button.
Click “Select” and give a star rating only.	Selected task is shown with the option to leave feedback.	Pass		Rating stored without written comment.
Give rating + optional comment and submit.	Feedback saved successfully.	Pass		Success message displayed; record saved.

Submit feedback with *no* rating.	An error message is shown indicating that a rating is required.	Pass		"Please give at least a star rating" message displayed.

5.4.2 Performance Testing:

Performance testing is key to verifying that the application provides consistent speed, response, and stability to all the user flows, such as login, registration, navigation, and volunteer-patient request interactions. Rather than testing for functional bugs, the process confirms system scalability, data syncing, and transitions between users.

Table 6: Performance Testing Process

Test Case	**Expected Result**	**Pass**	**Fail**	**Actual Result**
Evaluate Login and Logout Speed for all User Roles	Login and Logout actions respond promptly for Patients, Volunteers, and Doctors.	✓		Logins and logouts are executed within 1 second per action.
Validate Navigation Speed Between Dashboard Options	Transitioning between screens (e.g., Dashboard > Profile > Back) is smooth and without lag.	✓		Screens load within 1 second and navigate without delay.
Verify Record Save Speed for Patient and Volunteer Profiles, Assess Notification Delivery Time	When a patient or volunteer profile is created, the data is stored and reflected immediately.	✓		Records are saved and available for retrieval instantly
Assess System Stability During Multiple User Operations	The app remains stable during continuous actions, like profile creation, navigation, and request management.	✓		The application remains stable with no crashes or memory overload.

5.4.3 Usability Testing:

Usability testing within this application evaluates the intuitiveness, accessibility, and readability of the interface to Patients, Doctors, and Volunteers. It makes sure that all pages are simple to navigate and that primary functions are discoverable and functional on all digital literacy levels.

Table 7: Usability Testing Process with Testing Cases for the User Interface

Test Case	**Expected Result**	**Pass**	**Fail**	**Actual Result**
Check Home Screen Buttons for Role Selection	Clear and colour-coded role buttons (Patient, Volunteer, Doctor) are accessible and responsive.	✓		Each role redirects user to correct login screen.
Test "Forgot Password?" Functionality	Link redirects to password reset screen; reset instructions are emailed.	✓		Users can reset passwords and receive reset link via email.
Verify Simplicity of Volunteer Profile Creation.	Volunteers can easily navigate dropdowns to choose category, task, and schedule.	✓		Volunteers complete profile creation with minimal guidance.
Validate Patient Request Flow.	Patients can select tasks, choose a volunteer, and send a request in under five steps.	✓		Flow is intuitive and completed successfully.
Evaluate Dashboard Layout and Clarity.	Each role-based dashboard shows relevant actions with clear labels.	✓		Buttons and options are clear, visible, and easy to understand.

5.4.4 Security Testing:

Security testing confirms that sensitive information within the Rise Together app, including emails, passwords, user roles, and profile information, is secured through encryption and proper session handling. The tests also check that users are not able to access data belonging to others or stay logged in without authorization.

Table 8: Security Testing Process with Testing Cases

Test Case	**Expected Result**	**Pass**	**Fail**	**Actual Result**
Verify Session Termination After Logout	After logout, the user cannot access any screen without logging in again.	✓		The session is cleared immediately; access is denied after logout.
Check Role-Based Access Control	Users cannot access dashboards or data of other roles (e.g., Patient accessing the Doctor area)	✓		Navigation is restricted based on role, redirection is in place.
Verify Reset Password Email Privacy	Only the registered email receives the reset link; unregistered emails are rejected.	✓		Only valid users receive password reset instructions.

5.5 Chapter summary

This chapter described the primary testing techniques, employed during the development of this app. Three types of testing were discussed: Black Box, White Box, and Grey Box. Black Box Testing considers what the app does externally, without inspecting the code. White Box Testing is the opposite: it tests the internal code and logic directly. Grey Box Testing, which is the middle ground option between the two, was the best fit for the project, since it allowed to simulate how the app is being used yet ensured partial internal framework examination. During the developmental phase, the entire set of tests was conducted to find and fix defects in the early stages. There were a few bugs in the early stages, including Gradle setup errors, delayed notifications, and form validation bugs. These were easily eliminated using debugging and modifications during the codding process. Overall, a comprehensive and well-balanced testing ensured that the app runs smoothly, performs fast, and protects users' data. The result of the

User Evaluation process, what was done in the project, and what can be done better in the future, will be disclosed in next and final chapter.

CHAPTER SIX: Application Evaluation, Results Analysis, Project Achievements, Conclusions, and Scope for Future Work

6.1 Application Evaluation

The User Evaluation process holds a vital role in assessing how practical the developed application is and how satisfied users are with it. This entails testing by real users is gathering data through an Evaluation Survey, to measure system's success in meeting user expectations and functional objectives. It facilitates the identification of areas of improvement and makes the solution user-centric and evidence-guided. Different factors, like user interface design, usability, navigation, role-specific functionality, stability, and overall user satisfaction, are used to gauge the project's application strengths and areas of possible enhancement.

In the context of the current Project, a mobile application was designed to connect Patients in Treatment, Volunteering Cancer Survivors, and Doctors. The Evaluation process was central to understand how well the app performs its key tasks, such as request handling, user onboarding, feedback exchange, and security assurance. To facilitate this evaluation, a *Survey Questionnaire* was created, using *Google Forms* online tool, and shared with 15 participants, including users from all three target user groups. The aim was to assess whether the application features align with the distinct needs of each user group, and to gather insights regarding interface appeal, ease of use, and functionality satisfaction. The survey covered a wide range of questions, tailored to the different types of users. It yielded valuable feedback, which formed the foundation for analysing the overall application effectiveness and user satisfaction with the system.

6.2 Analysis of the Evaluation Survey Results

The User Testing process plays a critical role in identifying whether the developed software is practical and to what extent users are satisfied with it. This involves testing, using real users, as well as data collection using a structured questionnaire to gauge how successfully the system accomplishes user expectations and functional requirements. It facilitates the identification of areas where there is a room for improvement, and ensures the solution is user-centered and evidence-driven. Aspects, like user interface design, usability, navigation, role-specific functionality, stability, and overall user satisfaction, are examined to identify the strengths and areas where the application might improve.

A thorough Evaluation Survey was created and sent to people involved with the test of the application in April 2025 through the Google Forms tool, to gather structured feedback about the developed application. All 26 questions in the questionnaire were strategically spread across various types of users, such as Doctors, Volunteering Cancer Survivors, and In-Treatment Patients, and were aimed at assessing multiple aspects of the system, including functionality, ease of use, design attractiveness, role-specific operations, and built-in security elements. A total of 15 participants filled out the Survey, all of whom had direct access to the application. All users were provided time to discover and use the application before returning their response. All collected feedback enabled the developer to identify how well the application addressed the needs and expectations of its target users (please, see *Appendix A8*). An analysis of the outcome offers insight into the users' level of satisfaction, confirms the tested-by-practicality implemented features, and emphasizes areas to apply improvements in the future. Furthermore, the results serve as proof of the empirical success of the application and, as such, contribute directly to the fulfilment of this Project by confirming its applicability, technical validity, and practicality in real-life scenarios. As in any user-oriented system, feedback is essential and, in this instance, confirms the well-received nature and viability of the developed solution.

Questions for All Users:

Q1: *How user-friendly did you find the application to use*? Of the15 respondents to the question, 53.3% (8 persons) selected option *Very Friendly* as an answer, with the remaining 46.7% (7 persons) selecting option *Friendly.* No one (0%) has selected option *Not friendly at all* as an answer to the question.

Q2: *How appealing did you find the overall design of the application (colour scheme, font size, design features)?* Out of 15 respondents, 93.3% (14 participants) found it *Very appealing,* 6.7% (1 person) found it *Appealing,* and none (0%) found it *Not appealing at all.*

Q3: *How did you find the navigation through the application?* Out of 15 respondents, 86.7% (13 respondents) chose option *Very easy* as an answer to the question, with the remaining 13.3% (2 respondents) selecting option *Easy* as an answer.

Q4: *Did you find the Login and Registration process clear and simple*? 93.3% (14 people) responded with *Yes*, and only 1 person (6.7%) responded with *No.*

Q5: *Did you find the Reset Password process clear and simple*? **All** 15 respondents (100%) answered with an affirmative *Yes.*

Q6-Q7 incl. to be answered only by Doctor type of Survey participants.

Q6: *If you are a Doctor, how satisfied are you with the functionality to Register New Patient in Treatment, by providing details of name, surname, mobile number, type of cancer, gender, email address (Username), and Initial Password with access link to create own password and log into the app?* 3 responses were recorded, with **all** respondents (100%) indicating to be *Very satisfied.*

Q7: *If you are a Doctor, how satisfied are you with the functionality to Register New Volunteering Cancer Survivor, by providing details of name, surname, mobile number, type of cancer, gender, email address (Username), and Initial Password with access link to create own password and log into the app?* Again, **all** three respondents (100%) were *Very satisfied.*

Q8-Q12 incl. to be answered only by Volunteering Cancer Survivor type of Survey participants.

Q8: *If you are a Volunteer, how satisfied are you with the functionality to Create a record in your Profile with details about Assistance Task (select task category, task name from lists & provide own photo, details of number of days/week availability, day of the week availability, and time slots (A.M./P.M.) availability)?* Of the 6 respondents to the question, 66.7% (4 persons) were *Very satisfied,* and 33.3% (2 people) were *Satisfied.*

Q9: *If you are a Volunteer, how satisfied are you with the functionality to View List of Requests, sent from Patients?* Out of the 6 respondents, 83.3% (5 persons) were *Very satisfied*, and 16.7% (1 person) - *Satisfied.*

Q10: *If you are a Volunteer, how satisfied are you with the functionality to Select a specific Request and Accept/Reject it?* Of the 6 respondents to the question, 66.7% (4 persons) were *Very satisfied,* and 33.3% (2 people) were *Satisfied.*

Q11: *If you are a Volunteer, how satisfied are you with the functionality to Receive a Reminder for an Accepted Request to provide an Assistance, a day before the required Assistance must take place?* There were 6 respondents to the question, with 50% (3 persons) being *Very satisfied,* and the remaining 50% (3 persons) being *Satisfied.*

Q12: *How satisfied are you with the functionality to View Feedback & Ratings from a Patient for the Assistance you have provided*? Out of the 6 respondents to the

question, 66.7% (4 persons) were *Very satisfied,* with the remaining 33.3% (2 persons) being *Satisfied.*

Q13-Q18 incl. to be answered only by a Patient in Treatment type of Survey participants.

Q13: *If you are a Patient, how satisfied are you with the functionality to Create a record in your own Profile with details about Assistance Task (select task category, task name from lists, and details of number of days/weeks you will require the Assistance)?* There were 6 responses to the question in total. 83.3% of the respondents (5 persons) were *Very satisfied,* and 16.7% (1 person) - *Satisfied.*

Q14: *If you are a Patient, how satisfied are you with the functionality to View List of Volunteers, matching your gender and cancer type, who can provide an Assistance with a required by you Task?* Out of the 6 respondents to the question, 66.7% (4 persons) selected option *Very satisfied* as an answer, 16.7% (1 person) selected option *Satisfied,* and the remaining 16.7% (1 person) selected option *Not satisfied at all* as an answer to the question.

Q15: *If you are a Patient, how satisfied are you with the functionality to Select specific Volunteer from the List to reveal their personal details (name, photo, email, mobile number) and select specific day of the week, and time slot (A.M./P.M). of their availability, to Send a Request for selected week from a list (by its commencing date) to the relevant Volunteer?* From the 6 respondents to the question, 83.3% (5 persons) were *Very satisfied,* with the remaining16.7% (1 person) being *Satisfied.*

Q16: *If you are a Patient, how satisfied are you with the functionality to Receive and View Notification of Acceptance/Rejection of your Request for an Assistance with required Task?* From the 6 respondents to the question, 50% (3 persons) were *Very satisfied,* with the remaining 50% (3 persons) being *Satisfied.*

Q17: *If you are a Patient, how satisfied are you with the functionality to Receive a Reminder for an Accepted Request for Assistance with a specific Task, a day before the required Assistance must take place?* There were 6 respondents to the question. 83.3% (5 persons) were *Very satisfied,* and 16.7% (1 person) being *Satisfied.*

Q18: *If you are a Patient, how satisfied are you with the functionality to Send Feedback & Ratings to a Volunteer, who has provided you with an Assistance you have required with a specific Task?* From the 6 respondents to the question, 66.7% (4 persons) were *Very satisfied,* with the remaining 33.3% (2 persons) being *Satisfied.*

Q19-Q26 incl. to be answered by all Survey participants. (Security and Usability).

Q19: *How satisfied are you with the knowledge that this application uses the latest strong Encryption Standards (256 AES & Hashing) to secure sensitive user information in the application database, keeping it in scrambled, unreadable format?* There were 15 responses in total. 93.3% of the respondents (14 persons) were *Very satisfied,* with the remaining 6.7% (1 person) being *Satisfied.*

Q20: *How satisfied are you with the knowledge that this application uses data verification, user authentication, and embedded authorization rules as security measures to prevent unauthorized access to the application's data?* Out of the 15 respondents to the question, 80% (12 persons) were *Very satisfied,* with the remaining 20% (3 persons) being *Satisfied.*

Q21: *Did you encounter any technical problems while using the application (crashing, not opening the intended screen, or not responding)?* **All** 15 respondents to the question (100%) answered *No.*

Q22: *If you have answered "Yes" to Q21, please state what technical problem did you encounter while using the application (crashing, not opening the intended screen, or not responding) and at what circumstances it has happened*. There were no respondents (0%) to the question, since no one answered "*Yes*" to the previous question.

Q23: *Do you have any suggestions to add/remove/improve functionalities of the application?* From the 15 respondents to the question, 20% (3 persons) answered *"Yes"*, with the remaining 80% answering "*No*".

Q24: *If you have answered "Yes" to Q23, please state what is your suggestion to add/remove/improve functionalities of the application*. There were 3 respondents to the question, who provided the following suggestions:

- *Implement maps to see volunteers nearby*
- *Add a map to see volunteers nearby*
- *Share the feedback with all users*

Q25: *Overall, how satisfied are you with the application?* 15 responses were obtained in total. 93.3% of the respondents were *Very satisfied,* with the remaining 6.7% being *Satisfied.*

Q26: *Would you recommend this application to friends/colleagues to use?* From the 15 respondents to the question, 93.3% answered with affirmative *"Yes",* while 6.7% - with *Maybe.*

6.3 Project Achievements

As presented in Chapter 1, the Project set well-defined goals, all of which were attained within structured planning, incremental development, and user-focused methodology:

Project Proposal: Initial research has been performed to outline the context for the project in a proposal, highlighting the social and health-related issues encountered by cancer patients and their careers in Ireland, and around the world.

Primary and Secondary Research: Secondary research mapped the existing condition of patient-volunteer support systems and mHealth apps. The primary research performed has followed a mixed research methodology, which incorporated structured surveys and interviews. Information collected from sample of Irish patients, caregivers, and healthcare professionals identified key concerns of patient isolation, emotional support needs, and logistical challenges, immediately informing the app's mission.

Requirements Analysis: An explicit conceptual model was developed with UML diagrams (Use Case, Sequence, and Class Diagram) to structure the requirements for the proposed application, based on the research findings. This model informed the logic, functionalities and interactions of user roles with the application, and its data structures.

App Design, Coding and Testing: The overall design and development process was completed within the timeframe, using mobile-first design methodology, Firebase cloud technology, and incremental testing. Functional as well as non-functional testing, like usability, performance, and security, made the app strong, usable, and user needs aligned.

Real-Time Interaction and User Matching: Another of the app's innovations was its volunteer-patient matching tool, based on type of cancer, gender, and task availability. Real-time communication was facilitated by in-app notifications, enabling live updates, confirmations of assistance task requests, and feedback loops among users.

Cloud Integration: The app integrated Firebase services (Authentication, Cloud Firestore, Messaging), providing scalability, data integrity, and role-based secure access. This

deployment made it easy to provide access to all three user roles: Patients, Doctors, and Volunteers.
User Evaluation was performed post deployment. In real-life scenarios, doctors, patients and volunteers used the system, and the feedback was collected using Google Forms online tool. The feedback was then used to confirm primary functionalities and areas to fine-tune.
Thesis completion: The Project work culminated in an extensive Thesis document, which reports each phase of the projects in detail. It also discusses opportunities for future development.

6.4 Conclusion

This project succeeded in its intended purpose, achieving all its objectives, by being able to deliver a stable and secure mobile application that is user-friendly, aesthetically pleasing, usable and well-rated by users. The very positive reviews received during the Evaluation survey process reflect the system's success in its intended role of establishing a support network among patients, survivors, and Doctors. Having separate functionalities for every role has been effective in serving diverse user needs. Incorporation of contemporary security features has further enhanced trust and ease of use. By and large, the project has not only achieved its technical and scholarly goals, but is also poised to have a positive real-world impact within healthcare support networks.

6.5 Scope for Future Work

While the application successfully incorporated the intended features, some user recommendations and potential enhancements emerged. Future development may consider the following improvements:

- Integration of Google Maps, enabling patients to see available volunteers in their immediate vicinity.
- Increased visibility of public feedback, where all users get to see reviews and ratings provided for volunteers.
- Enhanced interactive elements, like in-app direct messaging among patients and volunteers.
- More filtering and matching criteria, like availability, urgency, or similar experiences.

The infrastructure developed in this work is well-suited to expand the system even further to maximize user engagement, reach, and impact in community-based cancer care support.

References

Afshar, Z.M., Hosseinzadeh, R., Barary, M., Ebrahimpour, S., Alijanpour, A., Sayad, B., Hosseinzadeh, D., and Miri, S.R. (2021). Challenges posed by COVID-19 in cancer patients: A narrative review. *Cancer Medicine*, [Online] 23 December. Available at: https://onlinelibrary.wiley.com/doi/full/10.1002/cam4.4519 (Accessed 11 Oct. 2024).

Afshar, Z.M., Hosseinzadeh, R., Barary, M., Ebrahimpour, S., Alijanpour, A., Sayad, B., Hosseinzadeh, D., and Miri, S.R. (2023). The long-term psychological impact of COVID-19 on cancer survivors. *Wiley Online Library*. [Online] Available at: https://onlinelibrary.wiley.com/doi/full/10.1002/cam4.4519 (Accessed 11 Oct 2024).

Ambler, S. W. (2004). *The Object Primer: Agile Model-Driven Development with UML 2.0*. Cambridge: Cambridge University Press.

American Association for Cancer Research (2024). *Cancer Progress Report Highlights*. [Online] Available at: https://www.aacr.org/about-the-aacr/newsroom/news-releases/aacr-cancer-progress-report-highlights-innovative-research-novel-treatments-and-powerful-patient-stories/ (Accessed 9 Oct. 2024).

American Cancer Society (2024a). *What is Cancer?* [Online] Available at: https://www.cancer.org/cancer/understanding-cancer/what-is-cancer.html (Accessed 10 Oct. 2024).

American Cancer Society (2024b). *Managing cancer-related side effects: Emotional and mood changes*. [Online] Available at: https://www.cancer.org/cancer/managing-cancer/side-effects/emotional-mood-changes.html (Accessed 10 Oct. 2024).

American Cancer Society (2024c). *Social Isolation and Loneliness*. [Online] Available at: https://www.cancer.org/cancer/managing-cancer/side-effects/emotional-mood-changes/social-isolation-loneliness.html (Accessed 28 Oct 2024).

American Cancer Society (2024d). *Coping with hair loss*. [Online] Available at: https://www.cancer.org/cancer/managing-cancer/side-effects/hair-skin-nails/hair-loss/coping-with-hair-loss.html (Accessed 26 Oct 2024).

American Psychological Association (2024). *Psychosocial impacts of cancer: Addressing isolation and the importance of support networks*. [Online] Available at: https://www.apa.org/monitor/2024/10/cancer-psychosocial-support (Accessed November 2024).

Android Developers (2024) *Android Studio User Guide: Ladybug 2024.2.2 Version*. [Online] Available at: https://developer.android.com/studio (Accessed 2 May 2025).
AppAdvice.com. *My Cancer Coach App*. [Online] Available at: https://appadvice.com/app/my-cancer-coach/991912414 (Accessed 23 Oct. 2024).

Badr, H. and Acitelli, L.K. (2019). Role dynamics in cancer caregiving and the need for support. *Health Psychology*, 38(4), pp.341-352.
Balsamiq (2024) *Balsamiq Wireframes*. [Online] Available at: https://balsamiq.com (Accessed 5 May 2025).

Beck, K. (2001). *The Agile Manifesto*. [Online], Available at: https://agilemanifesto.org. [Accessed 22 Nov. 2024].

Berkman, L.F. and Syme, S.L. (1979). Social networks, host resistance, and mortality: A nine-year follow-up study of Alameda County residents. *American Journal of Epidemiology*, 109(2), pp.186-204.

Bocij, P., Greasley, A., and Hickie, S. (2008). *Business Information Systems: Technology, Development & Management*. London: Pearson Education.

Booch, G., Rumbaugh, J., & Jacobson, I. (1999). *The Unified Modeling Language User Guide*. Reading: Addison-Wesley Professional.

Boucher, J., Ketcher, D. and Fell, K. (2018). Practical assistance for cancer patients and its impact on treatment adherence. *Journal of Oncology Practice*, 14(2), pp.98-104.

Bower, J.E. (2015). Cancer-related fatigue: Mechanisms, risk factors, and treatments. *Nature Reviews Clinical Oncology*, 11(10), pp.597-609. doi:10.1038/nrclinonc.2014.127.

Brennan, J. and Cameron, M. (2005). *Cancer in context: A practical guide to supportive care*. New York: Oxford University Press.

Brennan, J. and Moynihan, C. (2005). *Cancer in context: a practical guide to supportive care*. New York: Oxford University Press.

BreastCancerNow.org. *Becca*. [Online]. Available at: https://breastcancernow.org/support-for-you/becca/ (Accessed 1 Nov 2024).

Buck, A. (2024). Android vs iOS Market Share: Most Popular Mobile OS in 2024. *MobiLoud*. [Online] Available at: https://www.mobiloud.com/blog/android-vs-ios-market-share (Accessed 1 Nov 2024).

Butow, P., Havard, P.E., Butt, Z., Juraskova, I., Sharpe, L., Dhillon, H., Beatty, L., Beale, P., Cigolini, M., Kelly, B., Chan, R.J., Kirsten, L., Best, M. and Shaw, J. (2023). The impact of peer support on cancer patients. *Journal of Cancer Support*, 30(4), pp.3-4. [Online] Available at: https://pmc.ncbi.nlm.nih.gov/articles/PMC8801619/ (Accessed 28 Oct 2024).

Callaham, J. (2017). From Android Market to Google Play: A Brief History of the Play Store, *Android Authority*. [Online]. Available at: https://www.androidauthority.com/android-market google-play-history-754989/ (Accessed: 22 November 2024).

Callaham, J. (2020). The History of Android: The Evolution of the Biggest Mobile OS in the World, *Android Authority*. [Online]. Available at: https://www.androidauthority.com/history-android-os-name-789433/ (Accessed: 23 November 2024).

CareClinic.io (2024). *CareClinic App Features*. [Online] Available at: https://careclinic.io/features/ (Accessed 23 Oct. 2024).
CareZone.com (2024). *CareZone App Features*. [Online] Available at: https://carezone.com/ (Accessed 23 Oct. 2024).

Ceci, L. (2024). Number of apps available in leading app stores 2024. *Statista*. [Online] Available at: https://www.statista.com/statistics/276623/number-of-apps-available-in-leading-app-stores/ (Accessed 28 Oct 2024).

Central Statistics Office (2019a), *Information Society Statistics - Households 2019: Frequency of Internet Usage*. [Online]. Available at: https://www.cso.ie/en/releasesandpublications/ep/p-isshh/informationsocietystatistics-households2019/frequencyofinternetusage (Accessed: 23 November 2024).

Central Statistics Office (2019b), *Information Society Statistics - Households 2019: Type of Internet Activities*. [Online]. Available at: https://www.cso.ie/en/releasesandpublications/ep/p-isshh/informationsocietystatistics-households2019/typeofinternetactivities/ (Accessed: 23 November 2024).

Citizens Information (2018). *Overview of the General Data Protection Regulation (GDPR)*. [Online] Available at: https://www.citizensinformation.ie/en/government_in_ireland/data_protection/overview_of_general_data_protection_regulation.html (Accessed 24 Nov 2024).

Citizens Information (2020). *Data Protection Rights*. [Online] Available at: https://www.citizensinformation.ie/en/government_in_ireland/national_government/standards_and_accountability/data_protection.html (Accessed 24 Nov 2024).

Cockburn, A. (2001). *Writing Effective Use Cases*. Addison-Wesley Professional.
Colgan, M. (2019). mHealth apps: Market potential in Ireland and globally. *Tapadoo*. [Online] Available at: https://tapadoo.com/blog/posts/mhealth-apps-market-potential-ireland-globally/ (Accessed 28 Oct 2024).

Creswell, J.W. (2012). *Educational Research: Planning, Conducting, and Evaluating Quantitative and Qualitative Research*. 4th ed. University of Nebraska-Lincoln.
Data Protection Commission (2019a). *Data Protection Basics: Commonly Asked Questions About the Basics of Data Protection*. [Online] Available at: https://www.dataprotection.ie/sites/default/files/uploads/2019-07/190710%20Data%20Protection%20Basics.pdf (Accessed 24 Nov 2024).

Data Protection Commission (2019b). *Principles of Data Protection*. [Online] Available at: https://www.dataprotection.ie/en/individuals/data-protection-basics/principles-data-protection (Accessed 24 Nov 2024).

Doroshow, J.H. (2020). *NCI's COVID-19 in cancer patients study aims to answer key questions*. National Cancer Institute. [Online] Available at: https://www.cancer.gov/news-events/cancer-currents-blog/2020/nci-covid-19-cancer-nccaps-study (Accessed 10 Oct. 2024).

Elad, B. (2024). *Android Statistics*. Enterprise Apps Today. [Online] Available at: https://www.enterpriseappstoday.com/stats/android-statistics.html (Accessed 1 Nov 2024).
Fabbietti, P. (2023). Social support score in patients with malignant diseases—with sociodemographic and medical characteristics. [Online] Available at: https://pmc.ncbi.nlm.nih.gov/articles/PMC10267316/ (Accessed 28 Oct 2024).

Ferlay, J., Ervik, M., Lam, F., Colombet, M., Mery, L., Piñeros, M. (2021). *Global Cancer Observatory: Cancer Today*. Lyon: International Agency for Research on Cancer. [Online] Available at: https://www.who.int/news-room/fact-sheets/detail/cancer (Accessed 10 Oct. 2024).

Ferlay, J., Ervik, M., Lam, F., Colombet, M., Mery, L., Piñeros, M., Znaor, A., Soerjomataram, I. and Bray, F. (2021). *World Health Organization*. [Online] Available at: https://www.who.int/news-room/fact-sheets/detail/cancer (Accessed 10 Oct 2024).

Firebase (2024a). *Cloud Firestore Documentation*. [Online] Available at: https://firebase.google.com/docs/firestore (Accessed 1 May 2025).

Firebase (2024b). *Cloud Functions for Firebase*. [Online] Available at: https://firebase.google.com/docs/functions (Accessed 6 May 2025).

Firebase (2024c). *Firebase Authentication*. [Online] Available at: https://firebase.google.com/docs/auth (Accessed 3 May 2025).

Firebase (2024d). *Firebase Cloud Messaging (FCM)*. [Online] Available at: https://firebase.google.com/docs/cloud-messaging (Accessed 25 April 2025).

Firebase (2024e). *Firebase Storage*. [Online] Available at: https://firebase.google.com/docs/storage (Accessed 5 May 2025).

Firebase Documentation (2020a). *Firebase Security Rules*. [Online]. Available at: https://firebase.google.com/docs/rules (Accessed: 24 November 2024).

Firebase Documentation (2020b). *Get Started with Firebase Security Rules*. [Online]. Available at: https://firebase.google.com/docs/database/security/get-started (Accessed: 24 November 2024).

Firebase Support (2020). *Privacy and Security in Firebase*. [Online]. Available at: https://firebase.google.com/support/privacy (Accessed: 24 November 2024).

Fitch, M.I., Nicoll, I., Newton, L., and Strohschein, F.J. (2022). Challenges of survivorship for older adults diagnosed with cancer. *Topical Collection on Geriatric Oncology*, 14 March, p. 766.

Fowler, M. (2004). *UML Distilled: A Brief Guide to the Standard Object Modeling Language*. Boston: Addison-Wesley Professional.

GeeksforGeeks (2024). *What is GUI?* [Online] Available at: https://www.geeksforgeeks.org/what-is-graphical-user-interface/ (Accessed 4 May 2025).

GeeksforGeeks (2024a). *Black Box Testing in Software Engineering*. [Online] Available at: https://www.geeksforgeeks.org/software-engineering-black-box-testing/ (Accessed 4 May 2025).

GeeksforGeeks (2024b). *White Box Testing in Software Engineering*. [Online] Available at: https://www.geeksforgeeks.org/software-engineering-white-box-testing/ (Accessed 4 May 2025).

GeeksforGeeks (2024c). *Gray Box Testing in Software Testing*. [Online] Available at: https://www.geeksforgeeks.org/gray-box-testing-software-testing/ (Accessed 4 May 2025).

GeeksforGeeks (2025). *Types of Testing in Software Engineering*. [Online] Available at: https://www.geeksforgeeks.org/ (Accessed 4 May 2025).

GeeksforGeeks (2024). *What is Agile Methodology?* [Online], Available at: https://www.geeksforgeeks.org (Accessed 22 Nov. 2024).

Given, B., Sherwood, P. and Given, C.W. (2017). Support for caregivers of cancer patients. *CA: A Cancer Journal for Clinicians*, 67(3), pp.177-193. doi:10.3322/caac.21387.

Google Cloud (2021). *Firestore Server-side Encryption*. [Online] Available at: https://cloud.google.com/firestore/docs/server-side-encryption (Accessed 26 April 2025).

Google Play (2024a). *BECCA [mobile app]*. Available at: https://breastcancernow.org/support-for-you/becca/ (Accessed 1 Nov 2024).

Google Play (2024b). *Belong Beating Cancer Together [mobile app]*. Available at: https://play.google.com/store/apps/details?id=com.belongtail.belong&hl=en_IE (Accessed 1 Nov 2024).

Google Play (2024c). *MySunrise [mobile app]*. Available at: https://www.mysunrise.co.uk/ (Accessed 1 Nov 2024).

Henderson-Sellers, B. (2005). *UML 2.0: A Practical Guide to the Standard Object Modeling Language*. Reading: Addison-Wesley Professional.

Highsmith, J. & Cockburn, A. (2005). *PM Declaration of Interdependence*. Agile Alliance. [Online], Available at: https://agilealliance.org [Accessed 22 Nov. 2024].

Howard, K. (2020). *Managing cancer-related side effects: Emotional and mood changes*. American Cancer Society. [Online] Available at: https://www.cancer.org/cancer/managing-cancer/side-effects/emotional-mood-changes.html (Accessed 10 Oct. 2024).

IBM Knowledge Center (2020). *Three-tier architectures*. [Online] Available at: https://www.ibm.com/docs/en/was/8.5.5?topic=overview-three-tier-architectures (Accessed 10 May 2025).

Illanes, M.H. (2024). Towards quality of life: Experiences and health-behavior change in chronic and oncological patients in virtual communities. *Journal of Psychological and Educational Research*, 32(1), pp.123-144. [Online] Available at: https://www.marianjournals.com/files/JPER_articles/Hurtado_JPER_2024_32_1_123_144.pdf (Accessed 27 Oct 2024).

Itransition (2024). *Software Development Methodologies*. [Online], Available at: https://www.itransition.com/software-development/methodologies (Accessed 18 Nov. 2024).
ISTQB. (2024). *Standard Glossary of Terms Used in Software Testing*. International Software Testing Qualifications Board. [Online] Available at: https://www.istqb.org/downloads/glossary (Accessed 4 May 2025).

Jacobson, I. (1992). *Object-Oriented Software Engineering: A Use Case-Driven Approach*. Reading: Addison-Wesley.

JavaTpoint (2019a). *UML Sequence Diagram*. [Online], Available at: https://www.javatpoint.com/uml-sequence-diagram (Accessed 21 Nov. 2024).

JavaTpoint (2019b). *UML Object Diagram*. [Online], Available at: https://www.javatpoint.com/uml-object-diagram (Accessed 21 Nov. 2024).

JetBrains (2024). *Kotlin Programming Language Documentation*. [Online] Available at: https://kotlinlang.org/docs/home.html (Accessed 28 April 2025). Latest stable version: 2.1.20.

Kiemen, A., Czornik, M., and Weis, J. (2023). How effective is peer-to-peer support in cancer patients and survivors? A systematic review. *Journal of Cancer Research and Clinical Oncology*, 149(11), pp.9461-9485. [Online] Available at: https://www.ncbi.nlm.nih.gov/pmc/articles/PMC10374798/ (Accessed 10 Oct. 2024).

Kranenburg, L. and Shen, N. (2023). *National Health Interview Survey (2022)*. [Online] Available at: https://pmc.ncbi.nlm.nih.gov/articles/PMC10316283/#B27 (Accessed 16 Nov. 2024).

Lien-Gieschen, T. (2009). The concept of social isolation in cancer patients: A review. *Journal of Psychosocial Oncology*, 27(1), pp.1-18. doi:10.1080/07347330903018449.
LivingWith.health (2024). *LivingWith Mobile Application*. [Online] Available at: https://livingwith.health/ (Accessed 23 Oct. 2024).

Medaris, A. (2024). As patients develop cancer at younger ages, psycho-oncologists are studying the mental health impact. [Online] Available at: https://www.apa.org/monitor/2024/10/psychological-toll-early-onset-cancer (Accessed Jan. 2024).

Meyer, A. and Katz, A. (2007). One-to-one peer support in cancer care: A review of scholarship. *European Journal of Cancer Care*, 24(4), pp.299-312. doi:10.1111/j.1365-2354.2006.00745.x.

Meyer, A., Coroiu, A., and Körner, A. (2014). One-to-one peer support in cancer care: a review of scholarship. *European Journal of Cancer Care*, 24, pp.299-312.

Miliard, M. (2020). What to expect in 2021 and beyond? IDC offers 10 healthcare predictions. *Healthcare IT News*, 28 December. [Online] Available at: https://www.healthcareitnews.com/news/what-expect-2021-and-beyond-idc-offers-10-healthcare-predictions (Accessed 16 Nov. 2024).

Miller, E. (2012). *Software Testing Techniques*. 2nd ed. Cambridge: MIT Press.

Mullins, T. (2000). *A First Course in Programming with Java*. Ireland: ColourBooks Ltd.
Myers, G.J., Sandler, C., and Badgett, T. (2011). *The Art of Software Testing*. 3rd ed. New York: John Wiley & Sons.

MySunrise.co.uk. *MySunrise*. [Online], Available at: https://www.mysunrise.co.uk/ (Accessed 1 Nov 2024).

NASA. (2023). *Design Solution Definition Process*. [Online], Available at: https://nen.nasa.gov/web/se/doc-repository (Accessed 21 Nov. 2024).

National Cancer Institute (2022). *When someone you love is being treated for cancer: Support for caregivers*. [Online] Available at: https://www.cancer.gov/about-cancer/coping/caregiver-support (Accessed 27 Oct 2024).

National Comprehensive Cancer Network (NCCN) (2023a). *Distress management: NCCN guidelines for patients*. [Online] Available at: https://www.nccn.org (Accessed 10 Oct 2024).
National Comprehensive Cancer Network (NCCN) (2023b).

NCCN Guidelines for Supportive Care in Oncology. [Online] Available at: https://www.nccn.org/patients/guidelines/content/PDF/palliative-patient.pdf (Accessed 27 Oct 2024).

National Library of Medicine (2023). *Cancer and mental health: Addressing depression, anxiety, and social isolation*. [Online] Available at: https://pmc.ncbi.nlm.nih.gov/articles/PMC10316283/ (Accessed 26 Oct 2024).

Nash, M. (2003). *Java Frameworks and Components: Accelerate Your Web Application Development*. UK: Cambridge University Press.

Nucleus AI (2024). Why Android Versions Are Named After Desserts: The Sweet Truth. *YourStory*. [online] Available at: https://yourstory.com/2024/06/android-dessert-naming-tradition (Accessed 1 Nov 2024).

Object Management Group (OMG). (2017). *Unified Modeling Language (UML) Specification, Version 2.5.1*. [Online], Available at: https://www.omg.org/spec/UML/2.5.1/ (Accessed 20 Nov. 2024).

OkHttp (2024). *Square OkHttp Documentation*. [Online] Available at: https://square.github.io/okhttp/ (Accessed 5 May 2025).

Pollock, A., Campbell, P., Cheyne, J., Cowie, J., Davis, B., McCallum, J., McGill, K., Elders, A., Hagen, S., McClurg, D., Torrens, C. and Maxwell, M. (2020). Interventions to support the resilience and mental health of frontline health and social care professionals during and after a disease outbreak, epidemic, or pandemic: A mixed methods systematic review. *PMC PubMed Central*. [Online] Available at: https://pmc.ncbi.nlm.nih.gov/articles/PMC8226433/ (Accessed 27 Oct 2024).

ResearchGate (2024). *Worldwide Smartphone Shipment OS Market Share Forecast*. [Online] Available at: https://www.researchgate.net/figure/Worldwide-Smartphone-Shipment-OS-Market-Share-Forecast-18_tbl2_340686449 (Accessed 28 Oct 2024).
Richmond, S. (2019). Who Invented Apps for Smart Phones? *Investopedia*. [Online]. Available at: https://www.investopedia.com/articles/personal-finance/062215/who-invented-apps-smart phones.asp (Accessed: 22 November 2024).

Roemmele, B. (2017). Steve Jobs 1983 Gives A Talk About the Future. *YouTube [Online video]* Available at: https://www.youtube.com/watch?v=1LLbCU6QxoM (Accessed: 21 November 2024).

Royal Phillips (2021). People may expect telehealth foundations to have been laid three years from now, which would require less spending. [Online] Available at: https://www.philips.com (Accessed 23 Oct. 2024).

SEC (U.S. Securities and Exchange Commission). (2013). *Order Instituting Administrative and Cease-and-Desist Proceedings: Knight Capital Americas LLC*. [Online] Available at: https://www.sec.gov/litigation/admin/2013/34-70694.pdf (Accessed 4 May 2025).

Schneider, G., & Winters, J. (1998). *Software Engineering: A Practitioner's Approach*. New York: McGraw-Hill.

StatCounter Global Stats (2024). *Mobile Operating System Market Share Worldwide*. [Online] Available at: https://gs.statcounter.com/os-market-share/mobile/worldwide (Accessed 28 Oct 2024).

StatCounter Global Stats (2024a). *Mobile Operating System Market Share in Ireland*. [Online] Available at: https://gs.statcounter.com/os-market-share/mobile/ireland (Accessed 28 Oct 2024).

StatCounter Global Stats (2024b). *Mobile Android Version Market Share in Ireland*. [online] Available at: https://gs.statcounter.com/android-version-market-share/mobile/ireland (Accessed 28 Oct 2024).

Stein, K.D. (2020). *Physical and psychological long-term and late effects of cancer*. [Online] Available at: https://pmc.ncbi.nlm.nih.gov/articles/PMC7047657/#R5 (Accessed 28 Oct 2024).

Strain, M. (2015). '1983 to Today: A History of Mobile Apps', *The Guardian*, 13 February [Online]. Available at: https://www.theguardian.com/media-network/2015/feb/13/ history mobile-apps-future-interactive-timeline (Accessed: 22 November 2024).

TestMatick. (2020). *Gray Box Testing Definition and Advantages*. [Online] Available at: https://testmatick.com/gray-box-testing-definition-and-advantages/ (Accessed 4 May 2025).

Tiongson, J. (2015). *Mobile App Marketing Insights: How Consumers Really Find and Use your Apps*. [Online]. Available at: https://www.thinkwithgoogle.com/marketing-strategies/app-and-mobile/mobile-app-marketing-insights/ (Accessed: 22 November 2024).

Van Puffelen, A.J., van der Sar, L.J., Moerman, F., Eicher, M., and Oldenmenger, W.H. (2023). Cancer care during the Covid-19 pandemic from the perspective of patients and their relatives: A qualitative study. *Heliyon*, 1(CePress), May, p. 5.
Visual Paradigm. (2024). *What is a Use Case Diagram?* [Online], Available at: https://www.visual-paradigm.com/guide/uml-unified-modeling-language/what-is-use-case-diagram/ (Accessed 20 Nov. 2024).

Wang, C., Xie, Q., Xu, Y., Jiang, M. and Qian, L. (2024). Factors influencing social isolation among cancer patients: A systematic review. *PMC PubMed Central*. [Online] Available at: https://pmc.ncbi.nlm.nih.gov/articles/PMC11120751/ (Accessed May 2024).

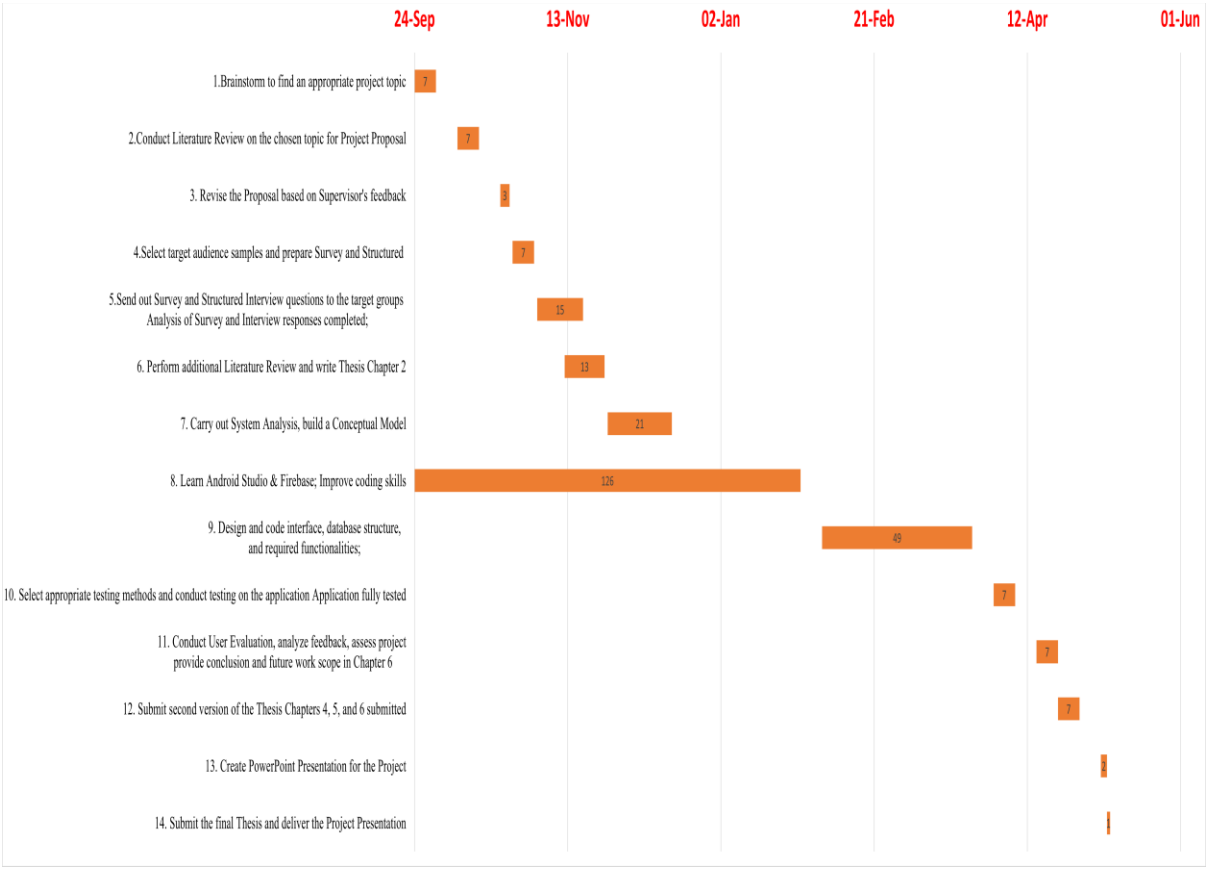
Willige, A. (2021). *World Economic Forum*. [Online] Available at: https://www.weforum.org/agenda/2021/05/digital-health-telemedicine-telehealth-ai-digitalize-health-services/ (Accessed 17 Oct. 2024).

World Health Organization (WHO) (2022). *Cancer Fact Sheet*. [Online] Available at: https://www.who.int/news-room/fact-sheets/detail/cancer (Accessed 26 Oct 2024).

World Health Organization (WHO) (2023). *Depression and other common mental disorders: Global health estimates*. [Online] Available at: https://www.who.int/mental_health/management/depression/en/ (Accessed 10 Oct 2024).

Yao, Z. (2024). *Peer support in online communities*. Carnegie Mellon University. [Online] Available at: http://reports-archive.adm.cs.cmu.edu/anon/hcii/CMU-HCII-24-107.pdf (Accessed 27 Oct 2024).

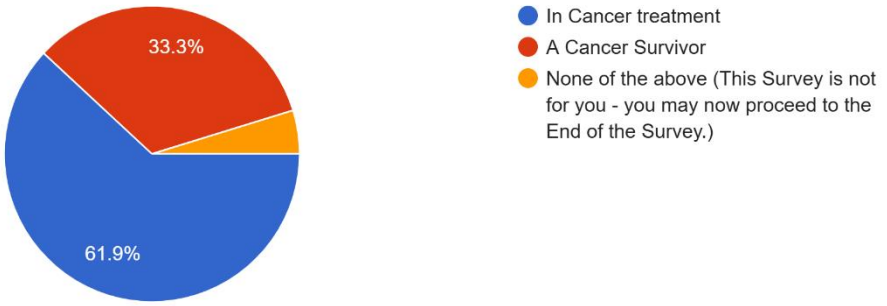
APPENDIX A1: Project's *Gantt Chart*



APPENDIX A2: Graphical Representation of the General Population Survey Results

Q1. Are you currently a cancer patient, who is:

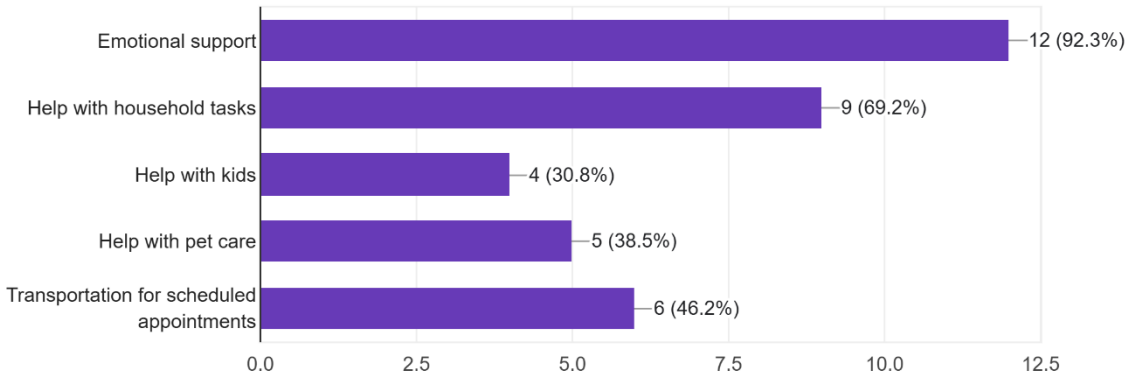
21 responses



Questions Q2 to Q6 incl. to be answered by patients in current Cancer Treatment:

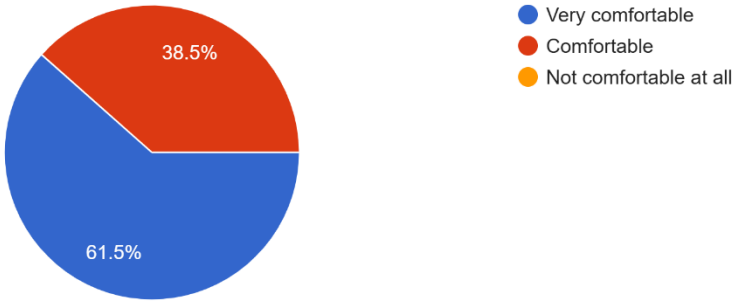
Q2. What type of assistance and support do you find most helpful in your daily life with cancer? (Please, select all that applies.)

13 responses



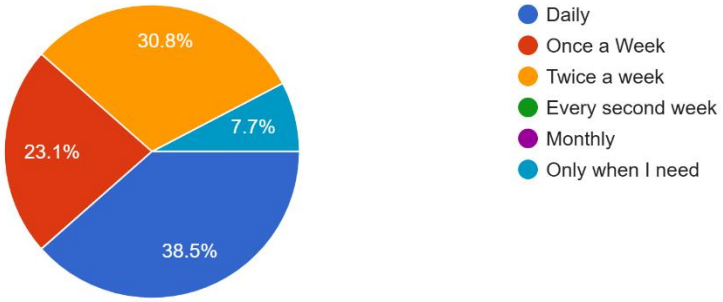
Q3. How would you feel about receiving support from someone, who has gone through a similar cancer experience and would volunteer to assist others?

13 responses



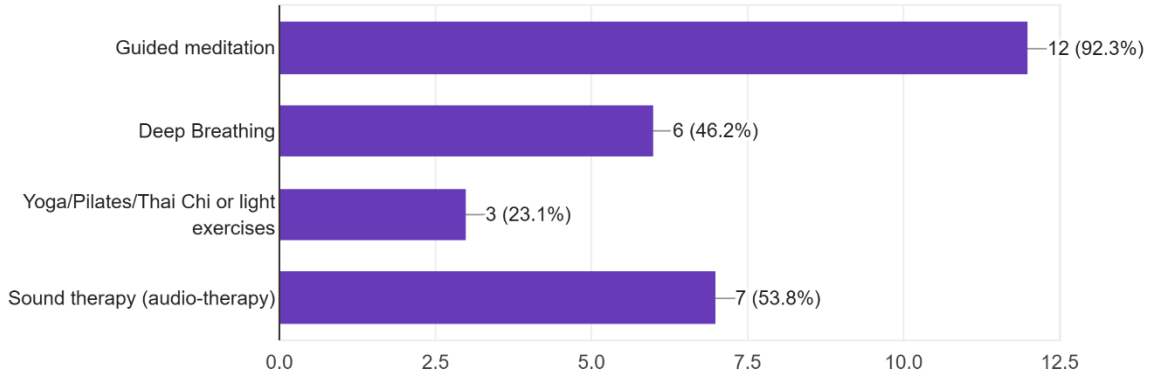
Q4. How often would you prefer a volunteer to check in on your well-being?

13 responses



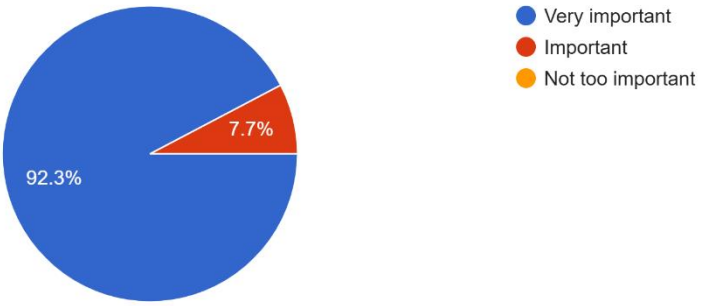
Q5. Which wellness activities are most important to you? (Please, select all that applies)

13 responses



Q6. How important to you is having an emotional support?

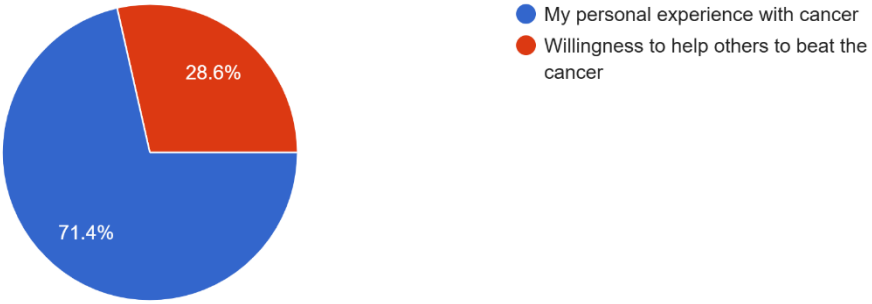
13 responses



Questions Q7 to Q10 incl. to be answered by Cancer Survivor Patients, volunteering as Mentors to Cancer Patients in Treatment:

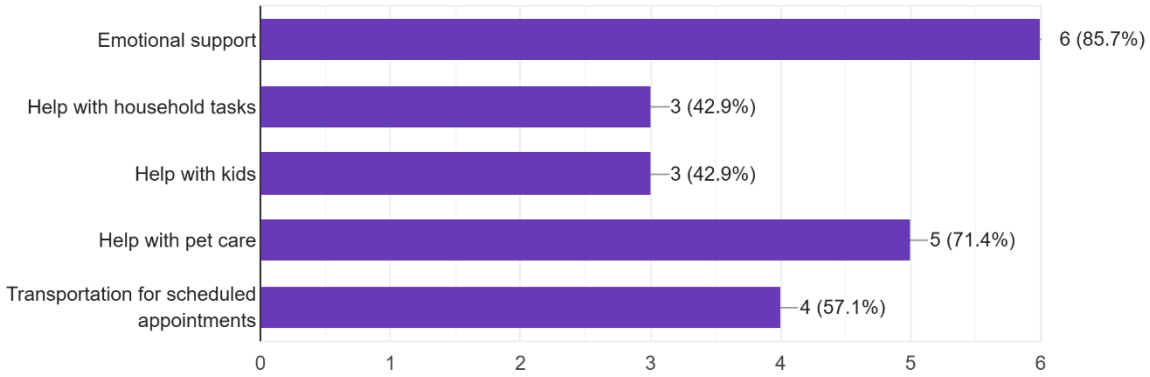
Q7. What motivates you to volunteer as a Mentor to Cancer patients in treatment?

7 responses



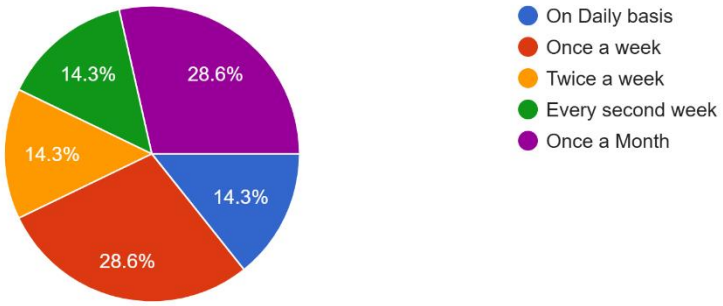
Q8. Being a Cancer Survivor, what type of support would you like to offer to patients in current cancer treatment? (Please, select all that applies)

7 responses



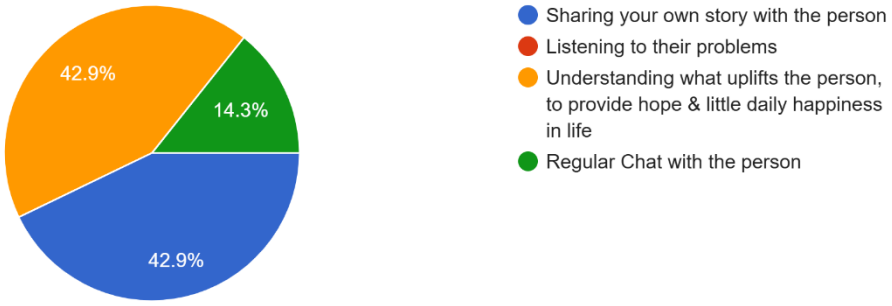
Q9. How often would you be available to provide such support?

7 responses



Q10. In your personal opinion as a Cancer Survivor, what type of emotional support is most needed by patients in treatment?

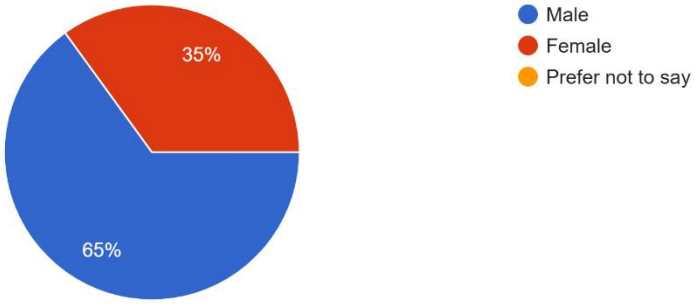
7 responses



Questions to be answered by *all* Survey Participants:

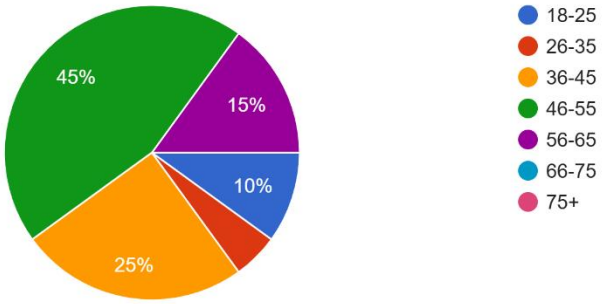
Q11. What is your gender?

20 responses



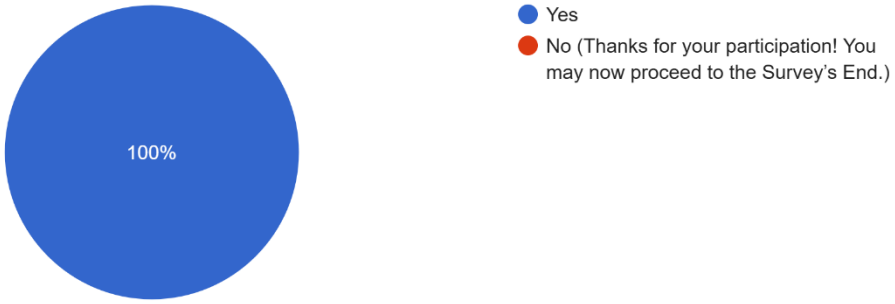
Q12. What is your age group?

20 responses



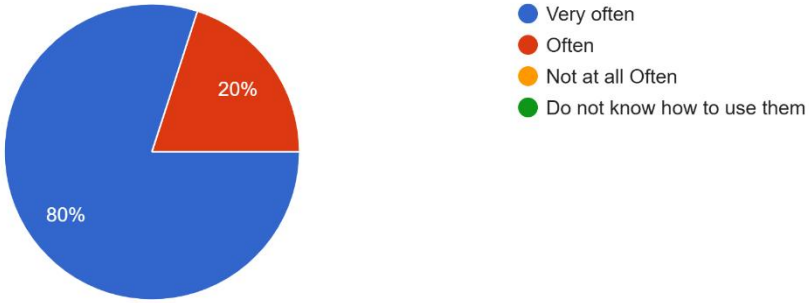
Q13. Do you have a smartphone or tablet?

20 responses



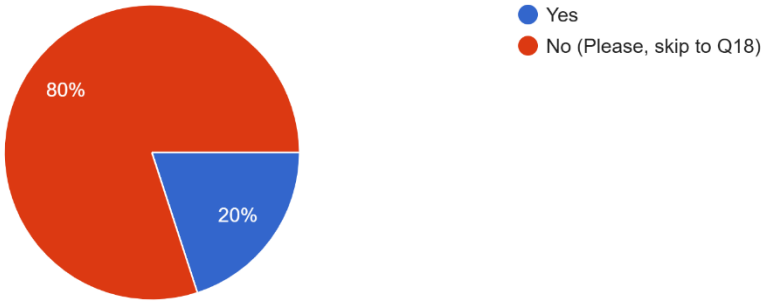
Q14. How often do you use mobile applications on your device?

20 responses



Q15. Are you familiar with apps, designed to help your overall improvement and well-being, while in cancer treatment/recovery?

20 responses



Q16. If you have answered "*Yes*" to Q15, please state the name of such an application.

2 responses

My Cancer Couch

My wellbeing

Q17. If you have answered "*Yes*" to Q15, please state what were the features that you most liked in such an application?

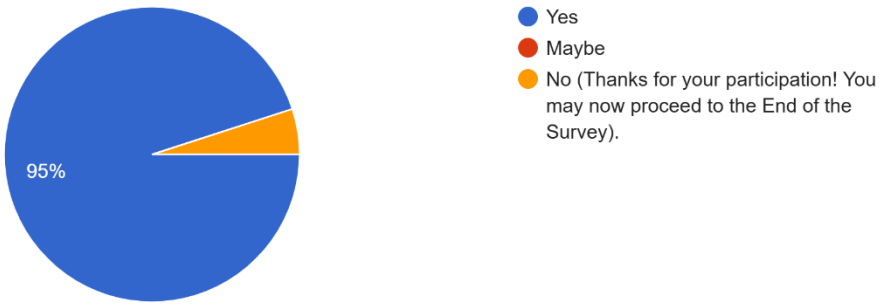
2 responses

Appointments and reminders to the calendar

Easy Navegation and features

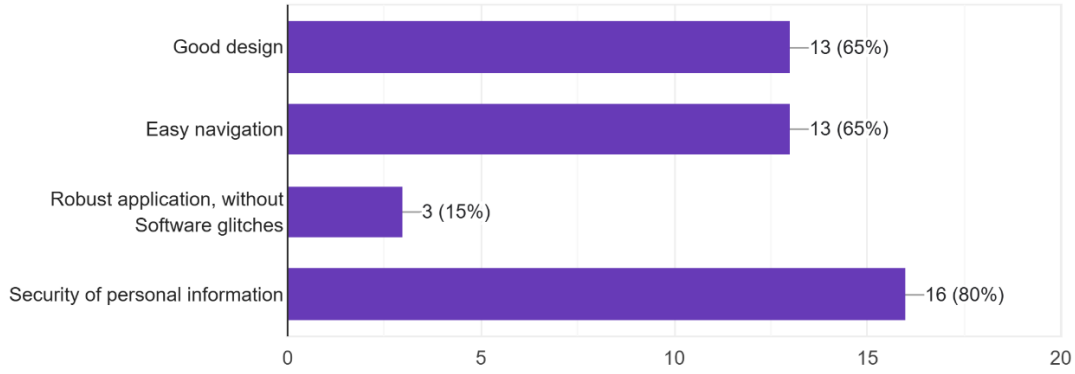
Q18. Would you be interested in using a specially designed Android Mobile application, as a platform to provide emotional and practical support...tients, currently in cancer treatment and recovery?

20 responses



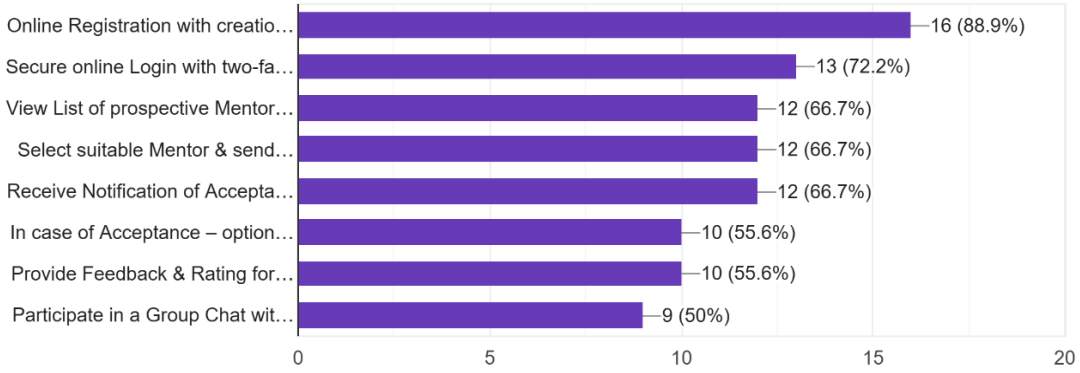
Q19. If you have answered “Yes” or “Maybe” to Q18, what features would you like such an application to have? (Please, select all that applies)

20 responses



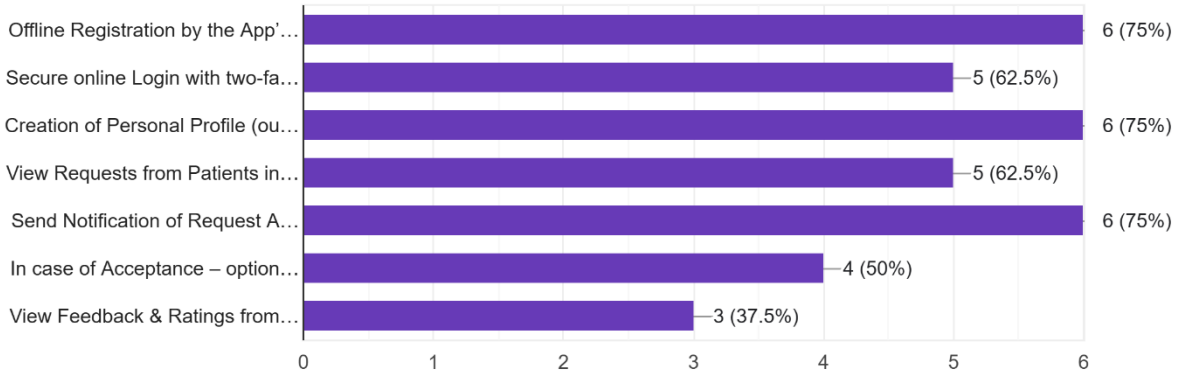
Q20. If you have answered “Yes” or “Maybe” to Q18, what functionalities would you like such an application to have? (Please, select all that applies) For Patients in Cancer Treatment:

18 responses



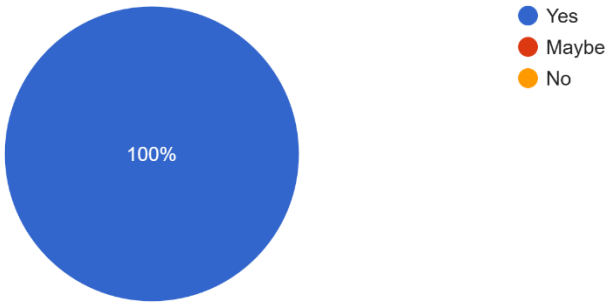
For vetted Cancer Survivors, volunteering as Mentors:

8 responses



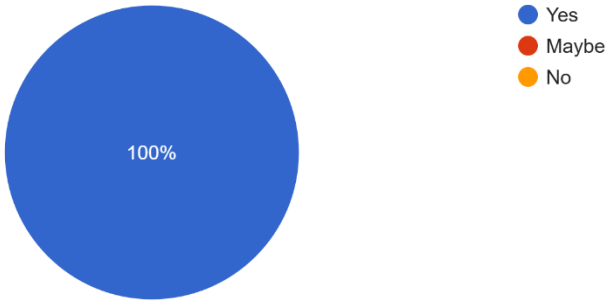
Q21. Would you recommend such an app, when being developed, to other Cancer Patients to use?

20 responses



Q22. In your opinion, would such an application, when developed, help to enhance Patient' personal well-being and speed-up their recovery process?

20 responses



APPENDIX A3: Structured Interview Questions for Doctors.

Q1. Please, state your Name and Surname:

3 responses

Eric Masterson

Alexander Fraser

Dr Emmet Kerin

Q2. Please, state the name of the organization you work for:

3 responses

Bon Secours Limerick

HSE

Gentle Procedures Ireland

Q3. Please, state your position at this organization:

3 responses

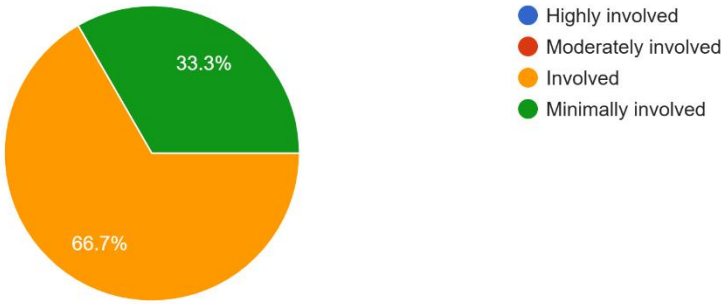
Orthopaedic Surgeon

Consultant Rheumatologist

Doctor

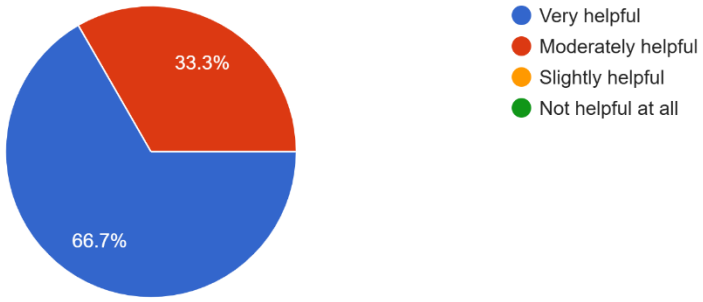
Q4. What is your level of involvement in cancer patient care?

3 responses



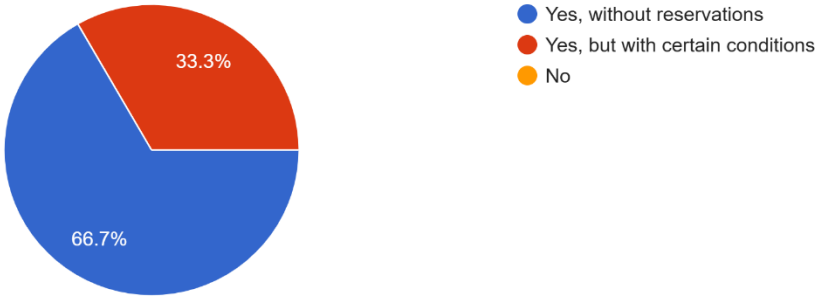
Q5. In your professional opinion, how helpful would be a peer mentorship system between cancer survivors and patients in treatment?

3 responses



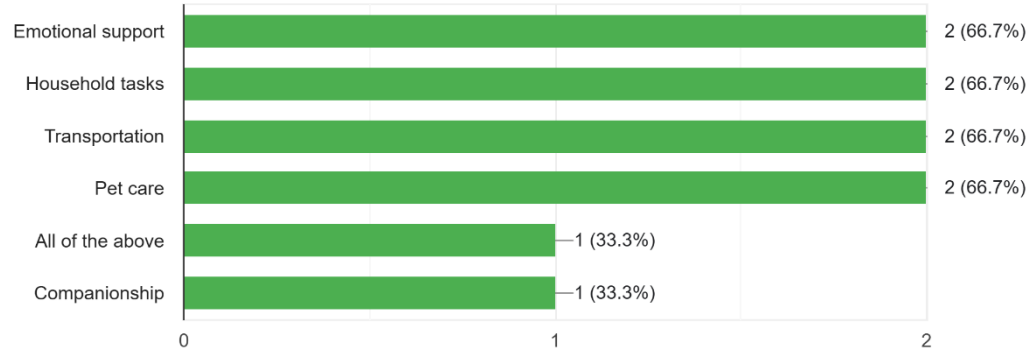
Q6. Would you feel comfortable approving cancer survivors, that you have treated, as mentor volunteers (after being thoroughly vetted), to your current cancer patients in treatment?

3 responses



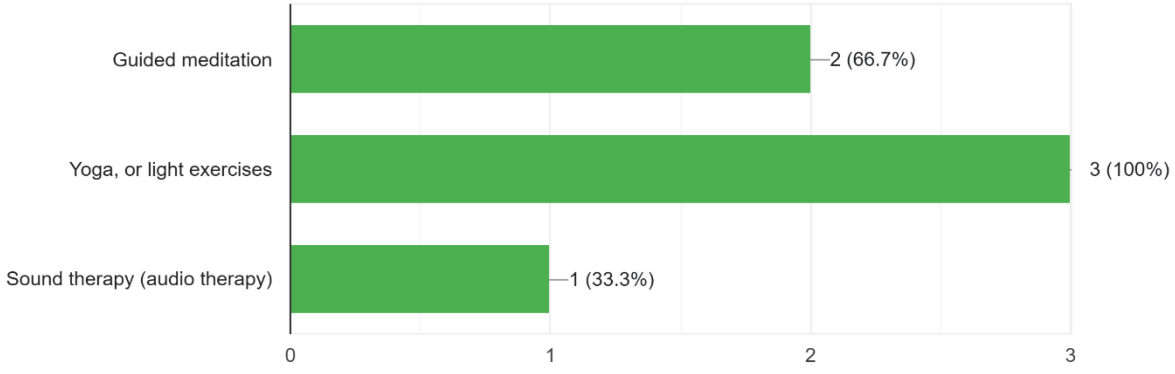
Q7. In your opinion, what types of support would be most important to assist with cancer patients in treatment? Please, select all that applies.

3 responses



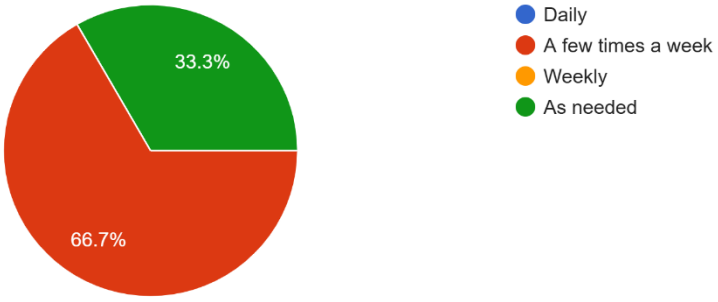
Q8. What wellness activities would you recommend to your patients? Please, select all that applies.

3 responses



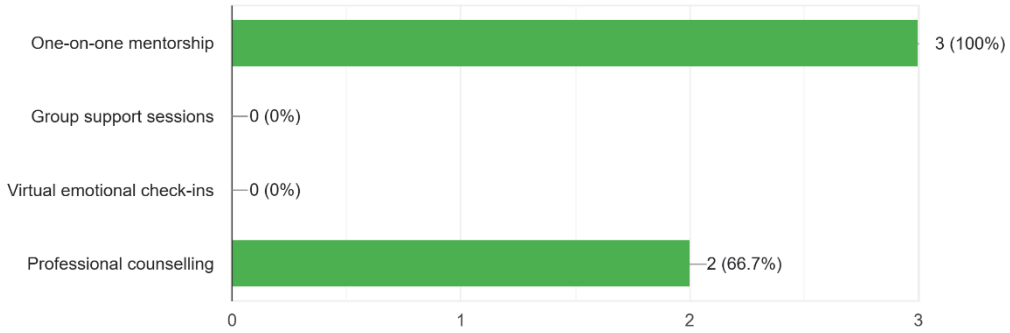
Q9. How often would you recommend cancer patients to engage in mindfulness, or audio therapy activities (such as meditation, or light exercises)?

3 responses



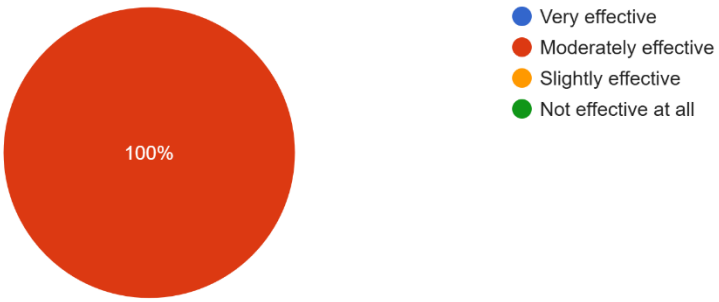
Q10. In your professional opinion, what type of emotional support has the most positive impact on patients during treatment? Please, select all that applies.

3 responses



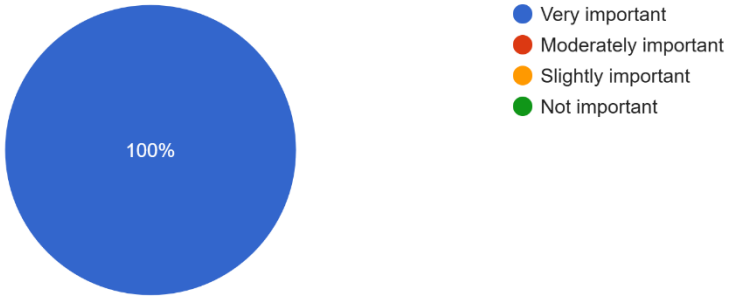
Q11. In your opinion, how effective an emotional support via digital means (e.g., use of specially designed mobile application) would be for cancer patients?

3 responses



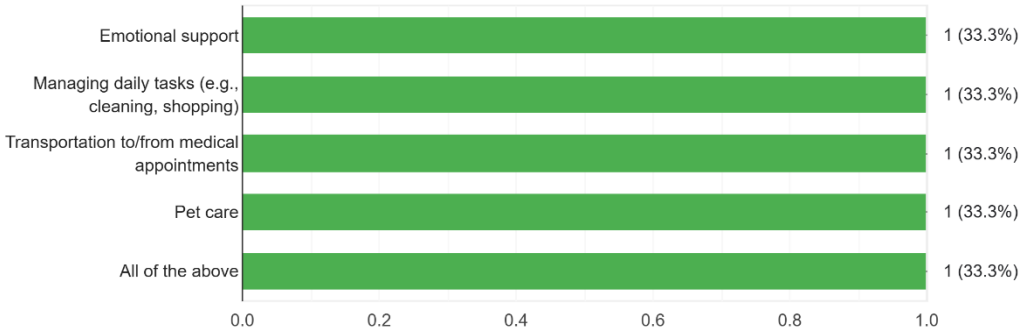
Q12. In your professional opinion, how important a peer support (mentorship relationship between cancer survivors and patients currently in treatment) is in improving patient's outcomes?

3 responses



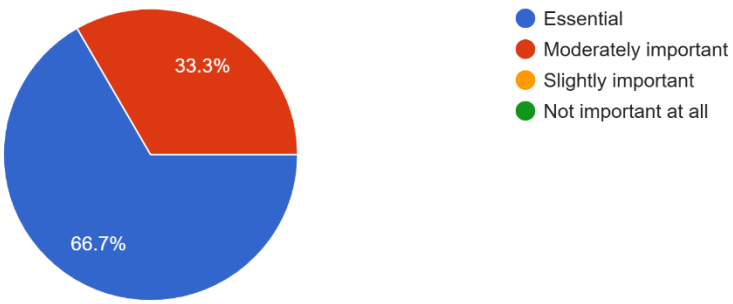
Q13. In your opinion, what are the biggest challenges that patients in treatment face, apart from the medical treatment itself? Please, select all that applies.

3 responses



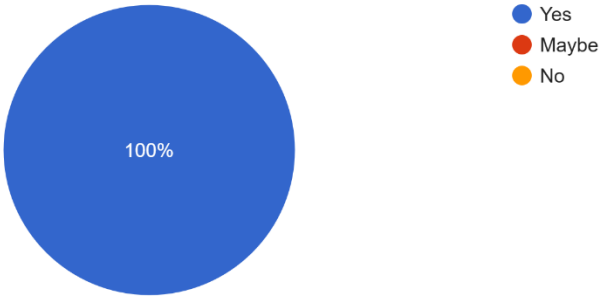
Q14. In your opinion, how important a volunteer system for practical tasks would be for your patients?

3 responses



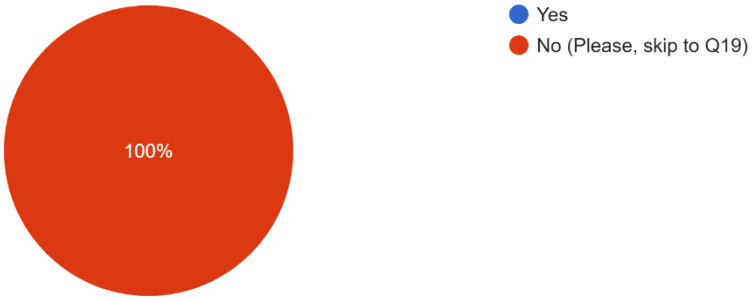
Q15. Would you encourage cancer patients in treatment to use a specially designed app, that connects them to cancer survivors as volunteering mentors for support?

3 responses



Q16. Are you familiar with apps, designed to help cancer patients' overall improvement and well-being, while in cancer treatment/recovery?

3 responses



Q17. If you have answered "*Yes*" to Q16, please state the name of such an application.

0 responses

No responses yet for this question.

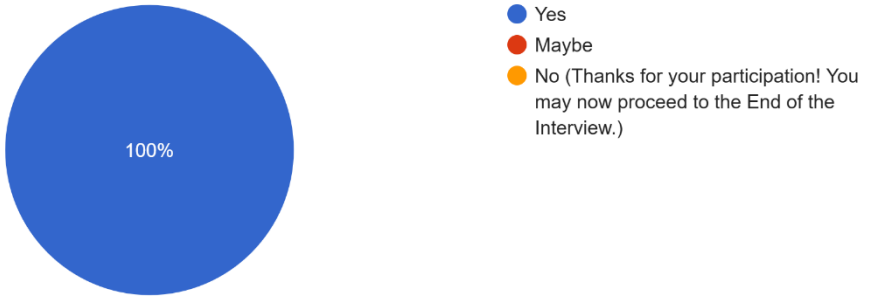
Q18. If you have answered "*Yes*" to Q16, please state what were the features that you most liked in such an application?

0 responses

No responses yet for this question.

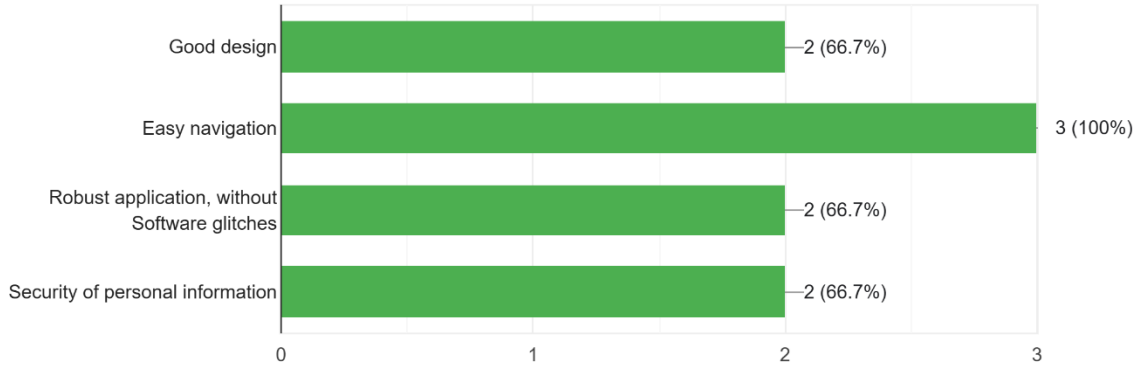
Q19. Would you be interested in using (and recommending your cancer patients to use) a specially designed Android Mobile application, as a platform ...tients, currently in cancer treatment and recovery?

3 responses



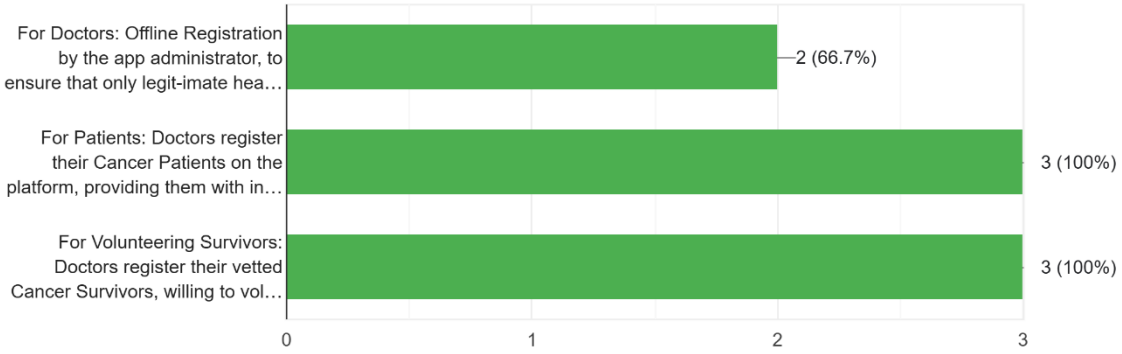
Q20. If you have answered “Yes” or “Maybe” to Q19, what features would you like such an application to have? (Please, select all that applies)

3 responses



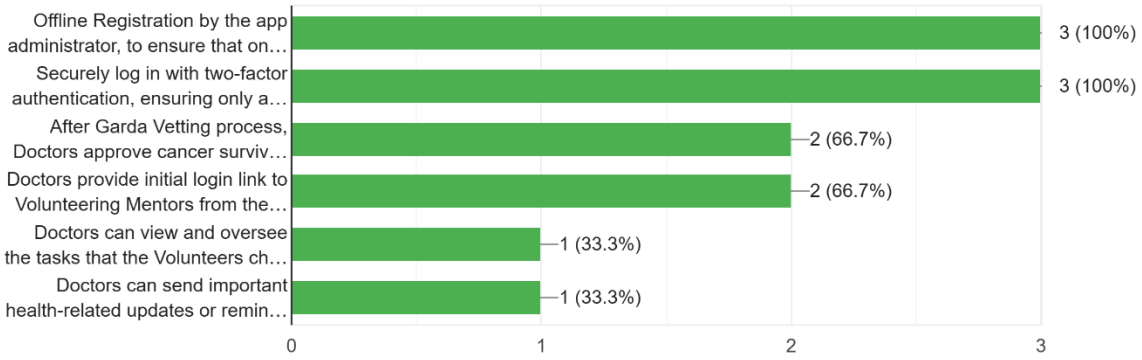
Registration and Profile Creation:

3 responses



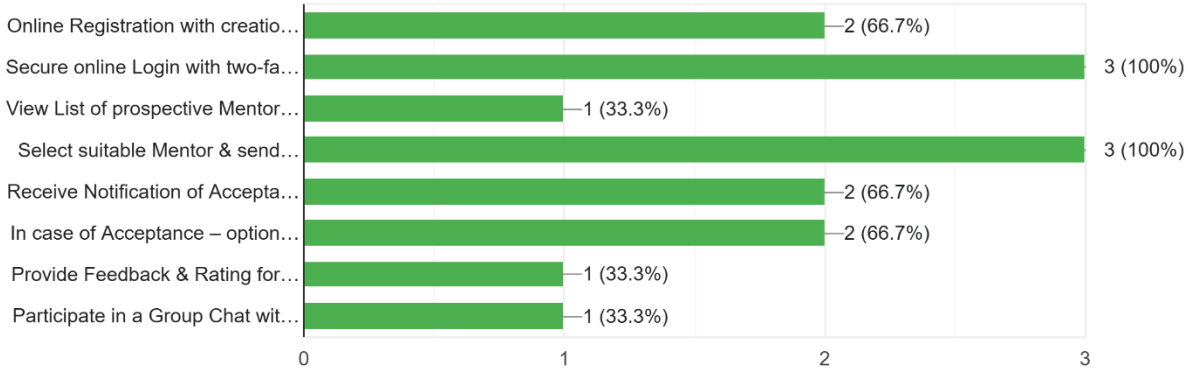
For Doctors:

3 responses



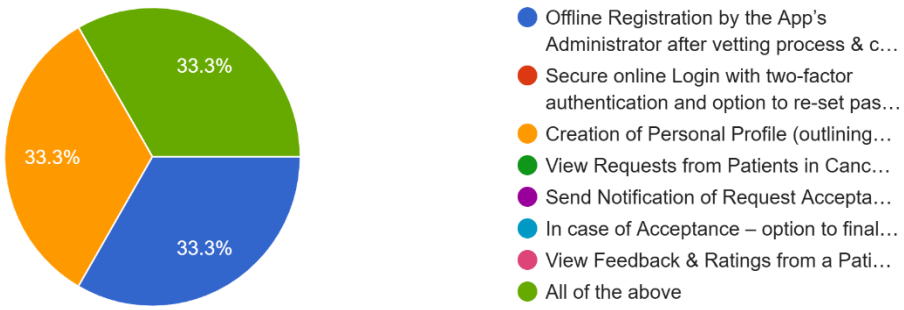
For Patients in Cancer Treatment:

3 responses



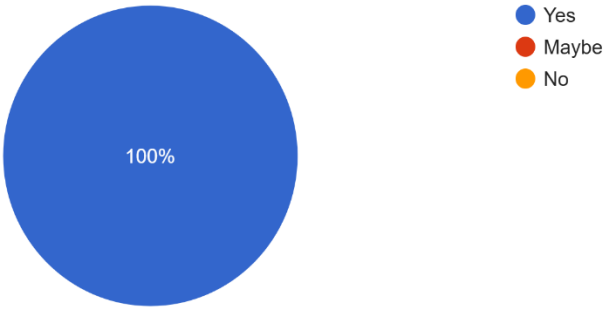
For vetted Cancer Survivors, volunteering as Mentors:

3 responses



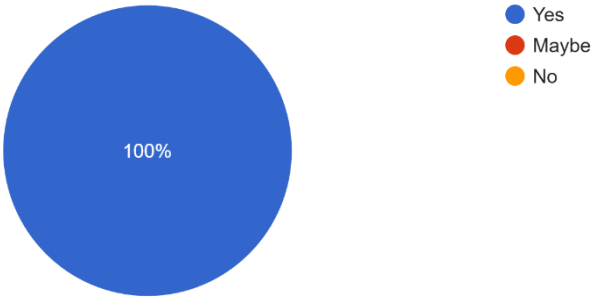
Q22. Would you recommend such an app, when being developed, to your Cancer Patients to use?

3 responses



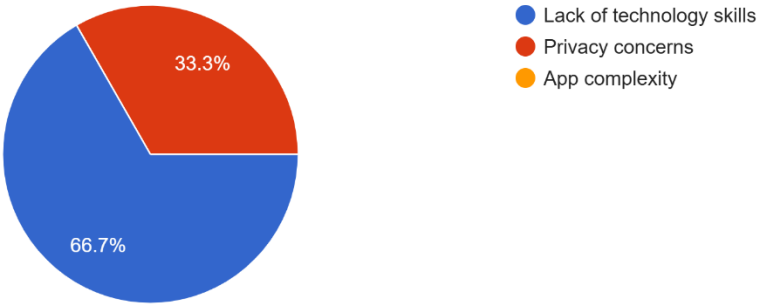
Q23. In your opinion, would such an application, when developed, help to enhance Patients' personal well-being and speed-up their recovery process?

2 responses

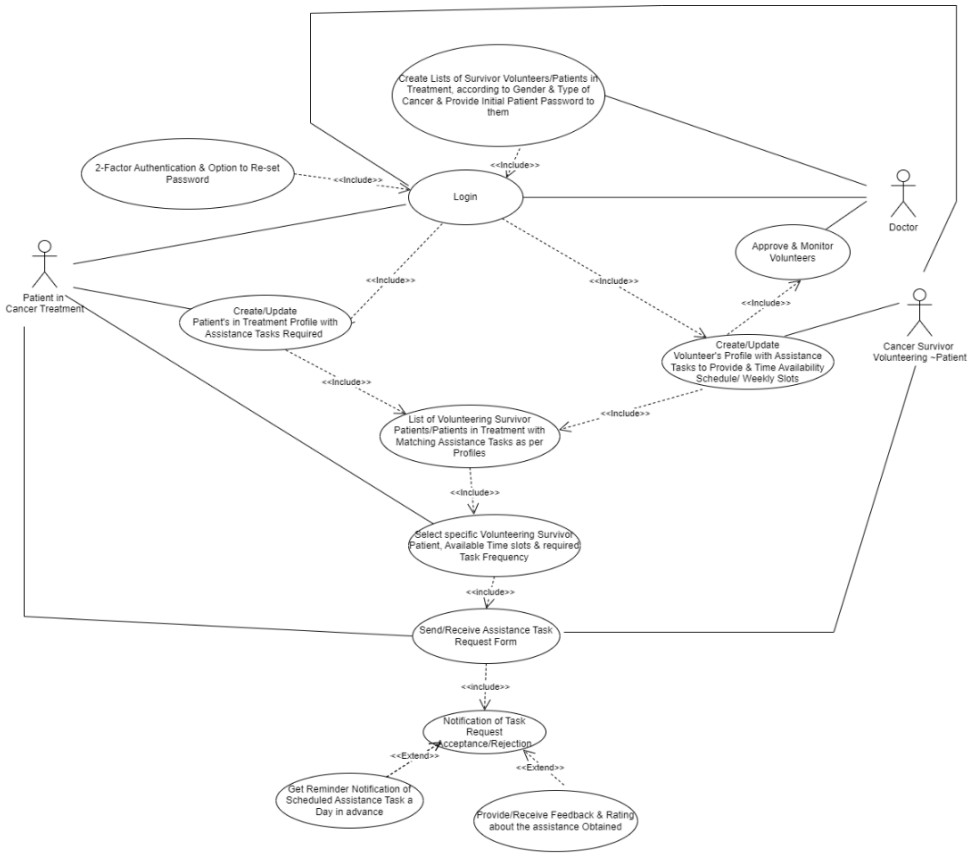


Q24. In your professional opinion, what might prevent patients from using such an app?

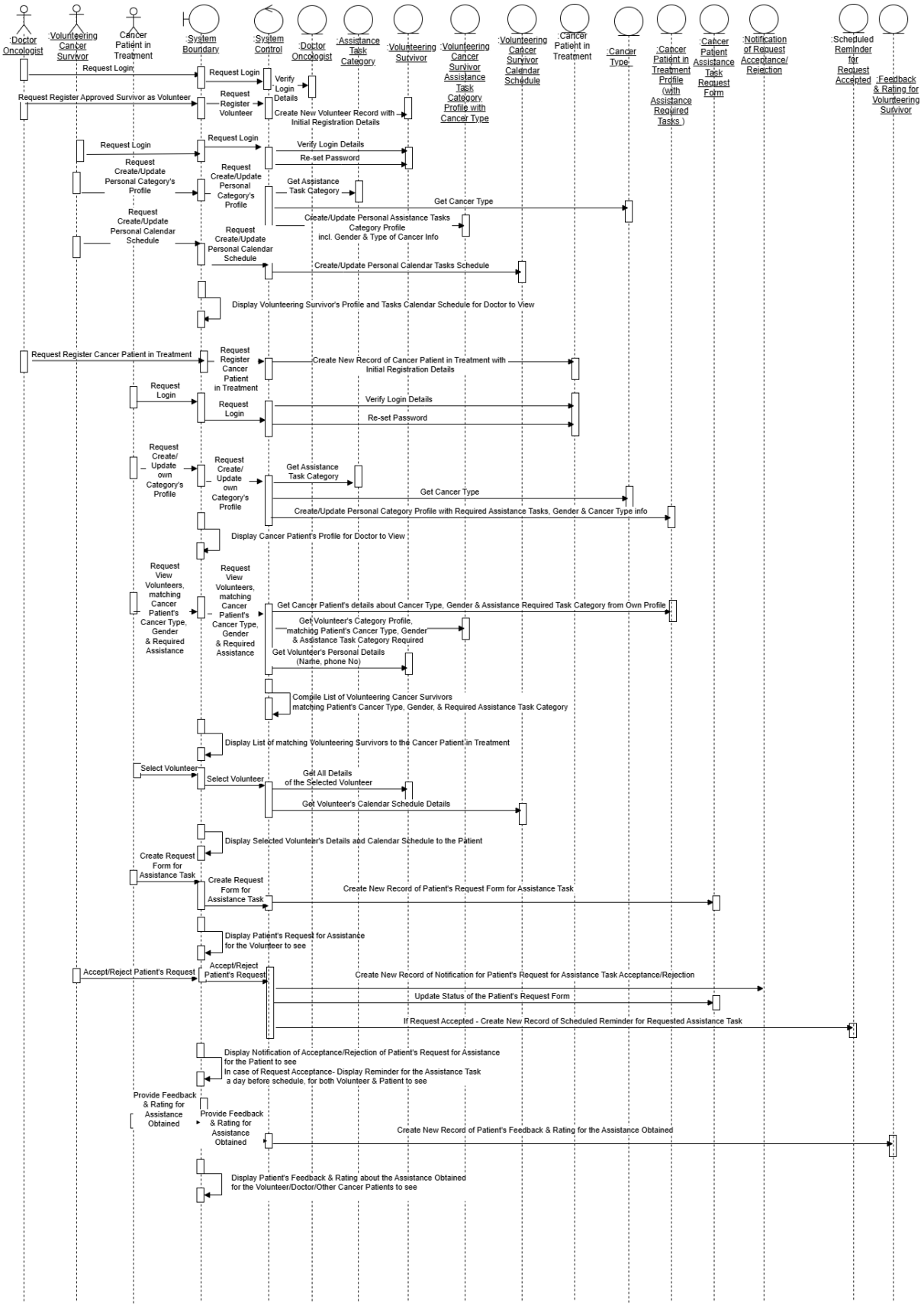
3 responses



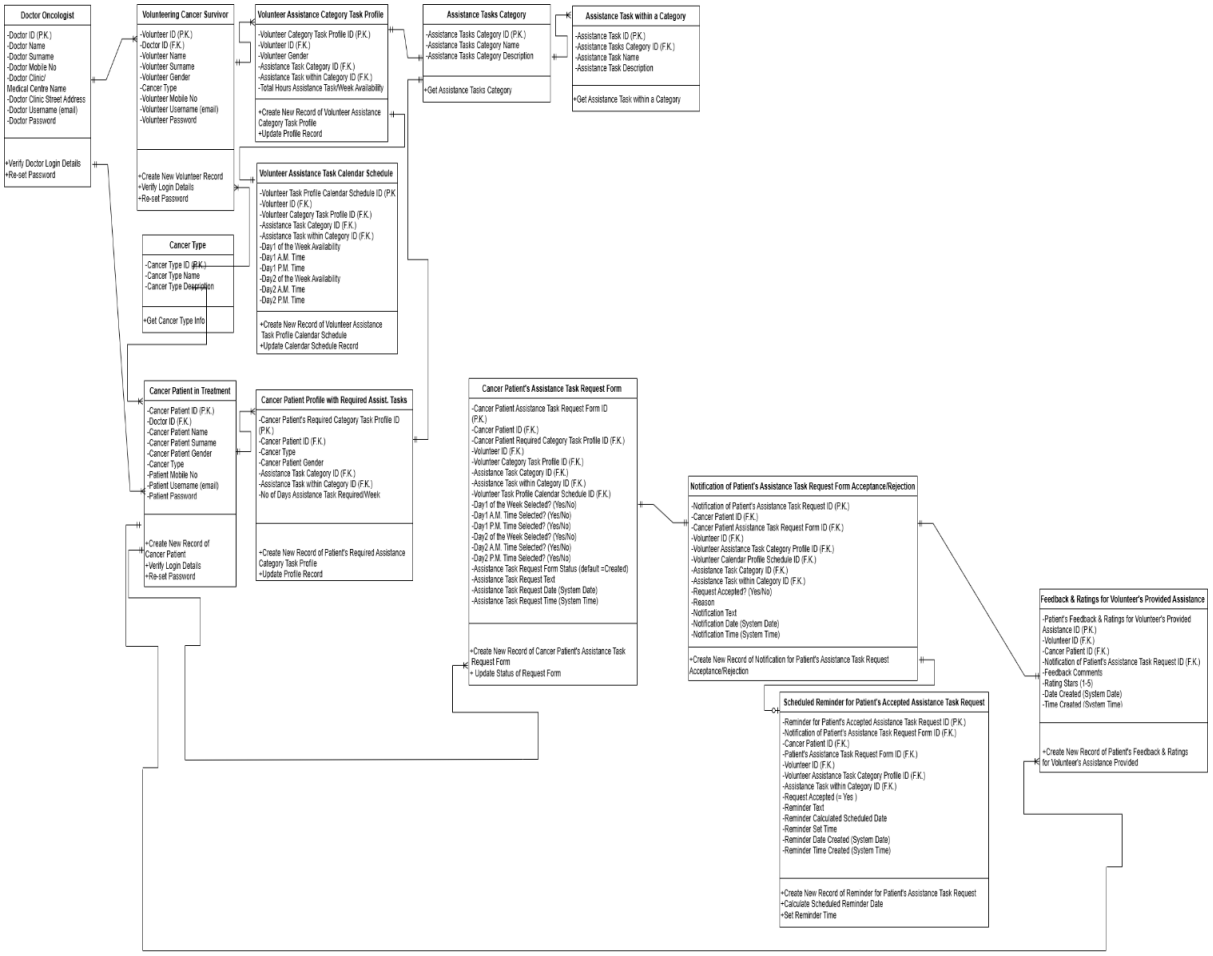
Appendix A4: Project's Use Case Diagram



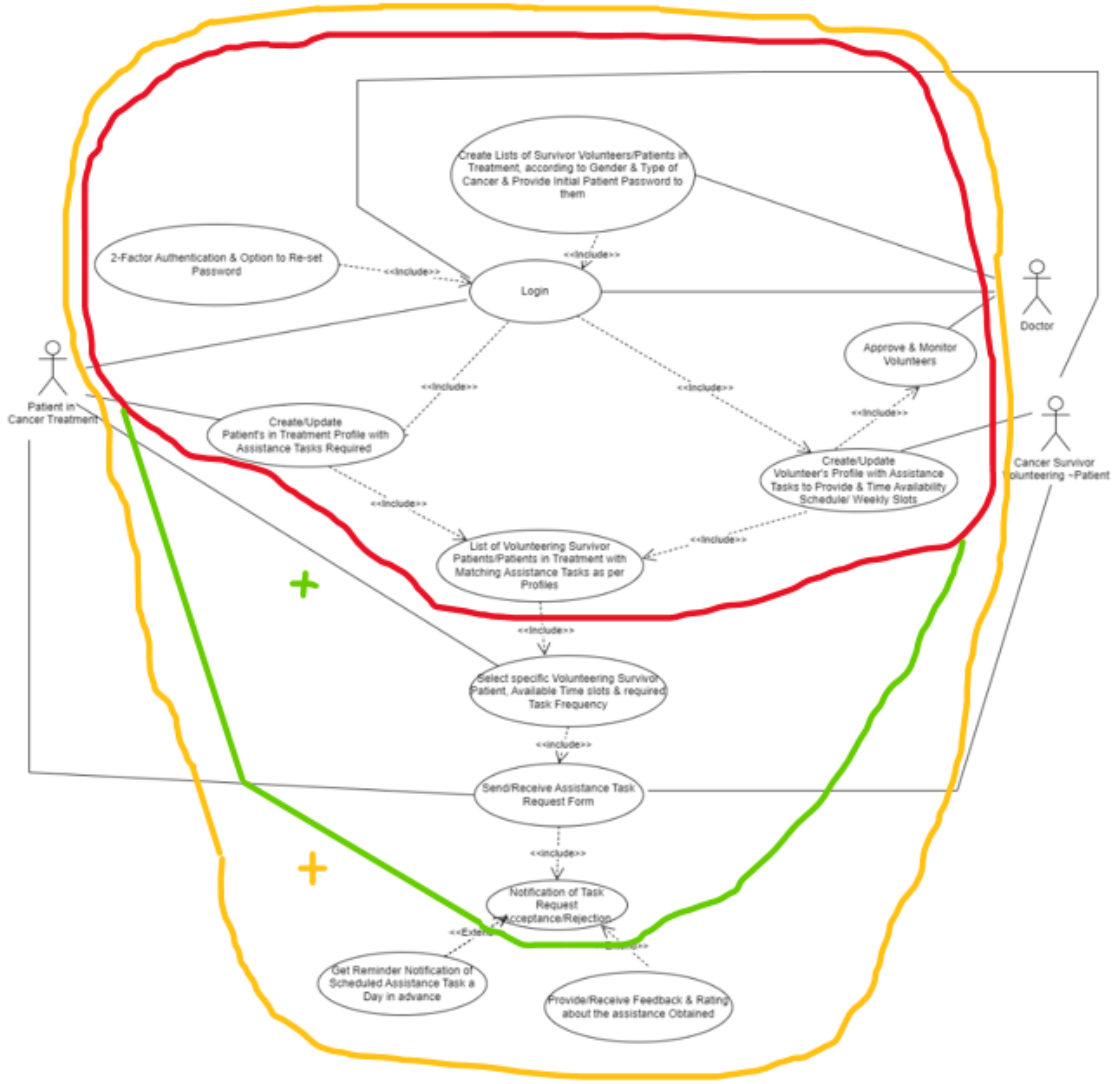
Appendix A5: Project's Sequence Diagram



Appendix A6: Project's Object Class Diagram



Appendix A7: Project's Three Alternative Design Solutions

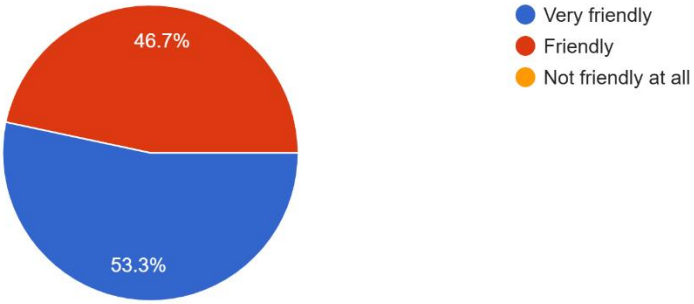


Low-End Solution:
Mid-Range Solution:
High-End Solution:

Appendix A8: Graphical Representation of the Evaluation Survey Results

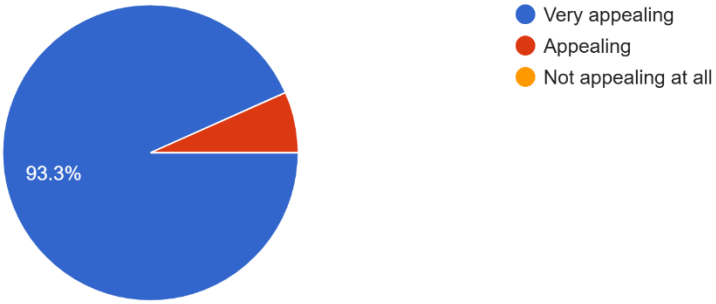
Q1 How user-friendly did you find the application to use?

15 responses



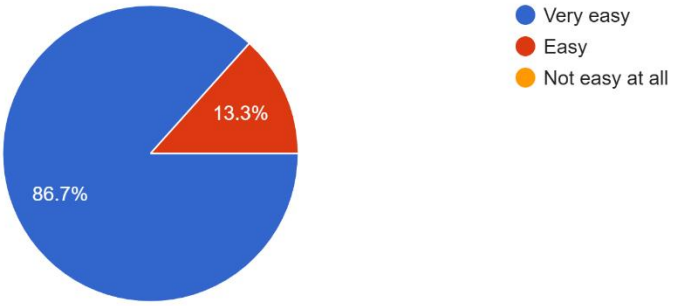
Q2 How appealing did you find the overall design of the application (colour scheme, font size, design features)?

15 responses



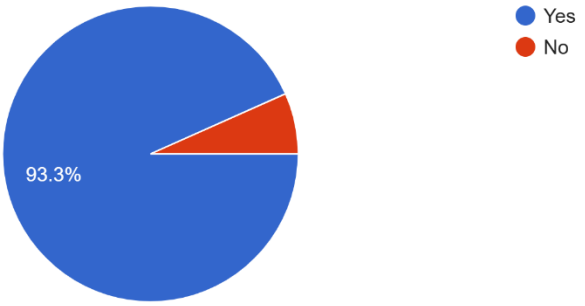
Q3 How did you find the navigation through the application?

15 responses



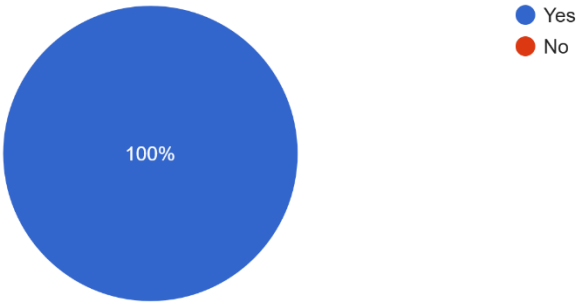
Q4 Did you find the Login and Registration process clear and simple?

15 responses



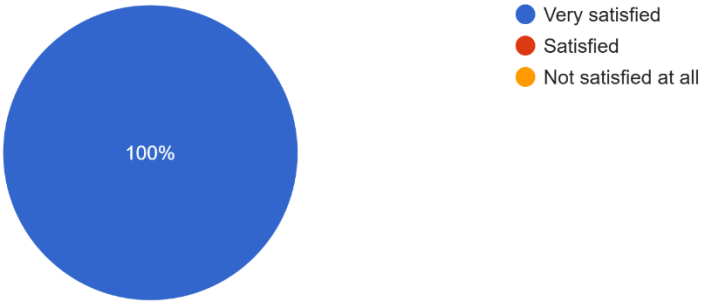
Q5 Did you find the Reset Password process clear and simple?

15 responses



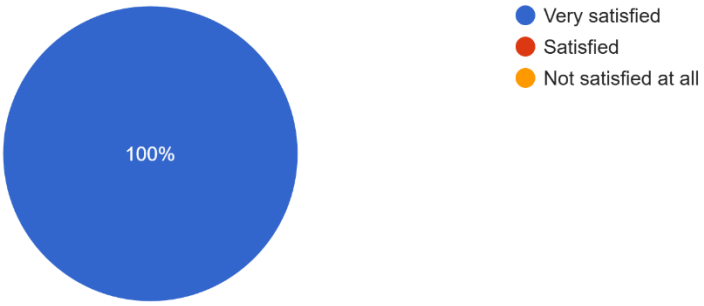
Q6 If you are a Doctor, how satisfied are you with the functionality to Register New Patient in Treatment, by providing details of name, surname, ... link to create own password and log into the app?

3 responses



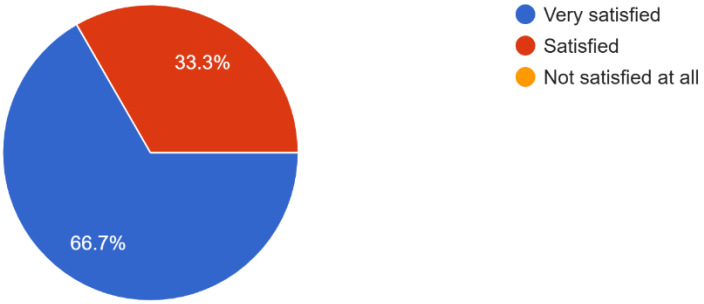
Q7 If you are a Doctor, how satisfied are you with the functionality to Register New Volunteering Cancer Survivor, by providing details of name, surn... link to create own password and log into the app?

3 responses



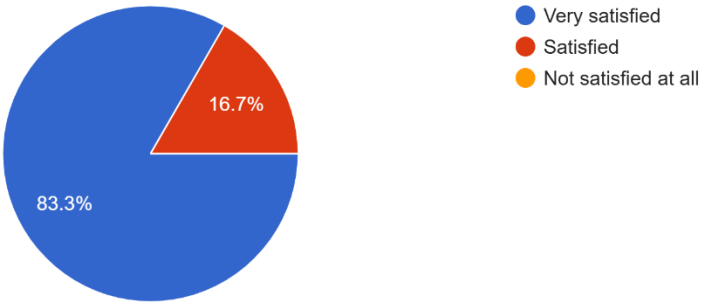
Q8 If you are a Volunteer, how satisfied are you with the functionality to Create a record in your own Profile with details about Assistance Task (select tas...ailability, and time slots (A.M./P.M.) availability)?

6 responses



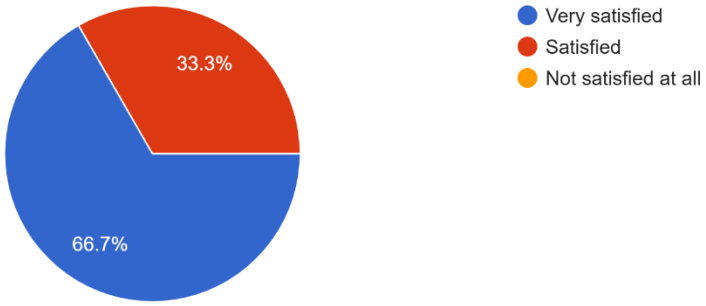
Q9 If you are a Volunteer, how satisfied are you with the functionality to View List of Requests, sent from Patients, for Tasks Assistance?

6 responses



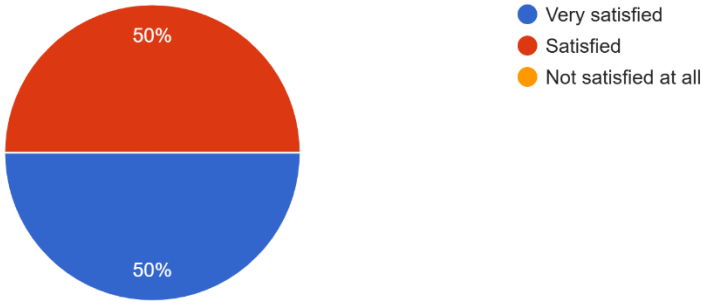
Q10 If you are a Volunteer, how satisfied are you with the functionality to Select a specific Request from the List to reveal its details, and Send Notific...k to the relevant Patient, who has sent the Request?

6 responses



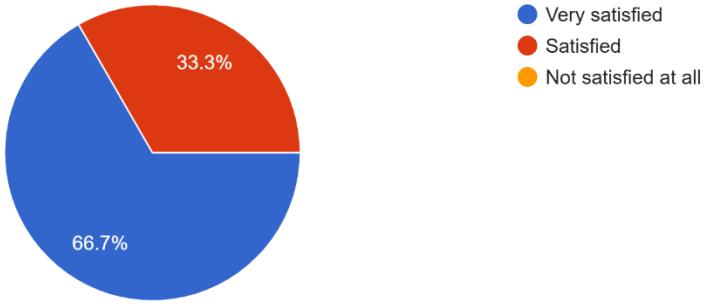
Q11 If you are a Volunteer, how satisfied are you with the functionality to Receive a Reminder for an Accepted Request to provide an Assistance, a day before the required Assistance must take place?

6 responses



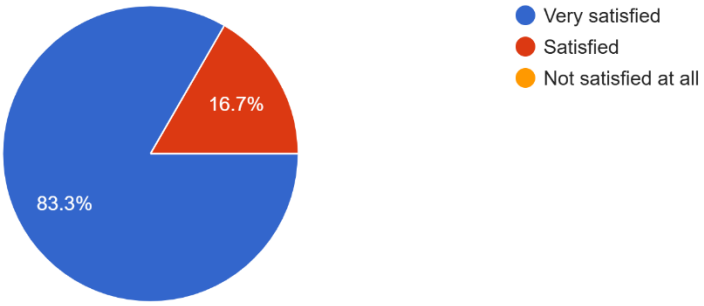
Q12 If you are a Volunteer, how satisfied are you with the functionality to View a Received Feedback & Rating from a Patient for the Assistance you have provided?

6 responses



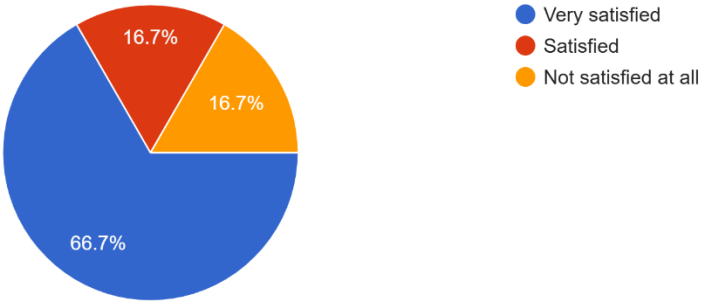
Q13 If you are a Patient, how satisfied are you with the functionality to Create a record in your own Profile with details about Assistance Task (select ...ber of days/week you will require the Assistance)?

6 responses



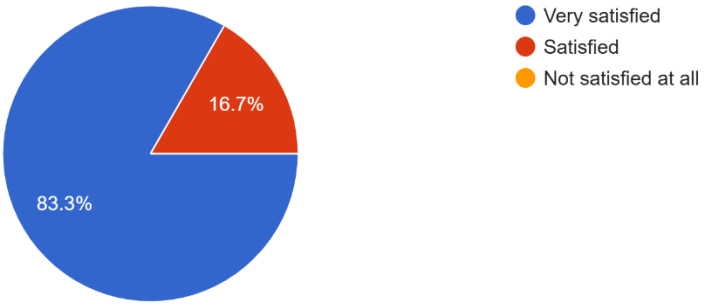
Q14 If you are a Patient, how satisfied are you with the functionality to View List of Volunteers, matching your gender and cancer type, ...rovide an Assistance with a required by you Task?

6 responses



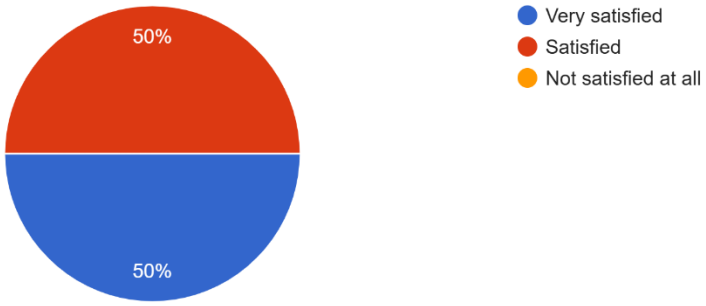
Q15 If you are a Patient, how satisfied are you with the functionality to Select specific Volunteer from the List to reveal their personal de...(by its commencing date) to the relevant Volunteer?

6 responses



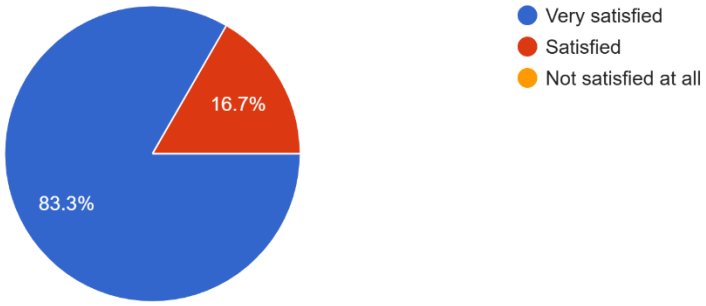
Q16 If you are a Patient, how satisfied are you with the functionality to Receive and View Notification of Acceptance/Rejection of your Request for an Assistance with required Task?

6 responses



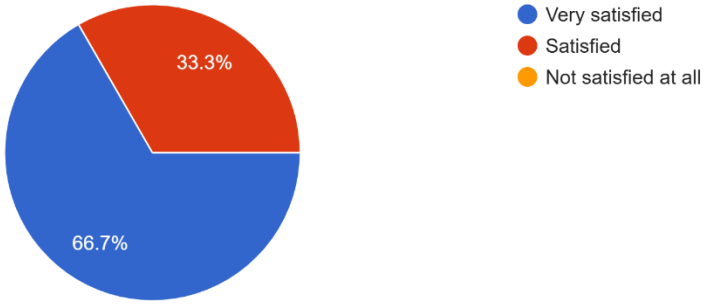
Q17 If you are a Patient, how satisfied are you with the functionality to Receive a Reminder for an Accepted Request for Assistance with a specific...y before the required Assistance must take place?

6 responses



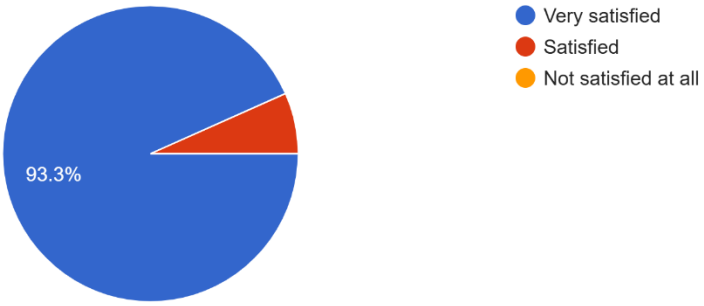
Q18 If you are a Patient, how satisfied are you with the functionality to Send Feedback & Ratings to a Volunteer, who has provided you with ...Assistance you have required with a specific Task?

6 responses



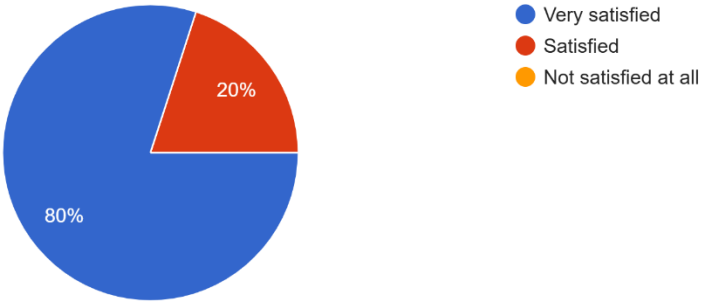
Q19 How satisfied are you with the knowledge that this application uses the latest strong Encryption Standards (256 AES & Hashin...se, keeping it in scrambled, unreadable format?

15 responses



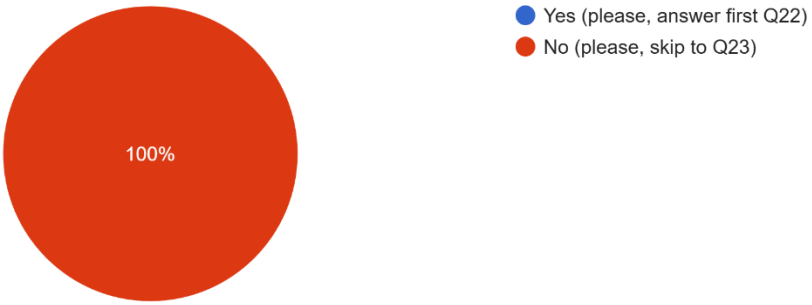
Q20 How satisfied are you with the knowledge that this application uses data verification, user authentication, and embedded authorization ru...nt unauthorized access to the application's data?

15 responses



Q21 Did you encounter any technical problems, while using the application (crashing, not opening the intended screen, or not responding)?

15 responses



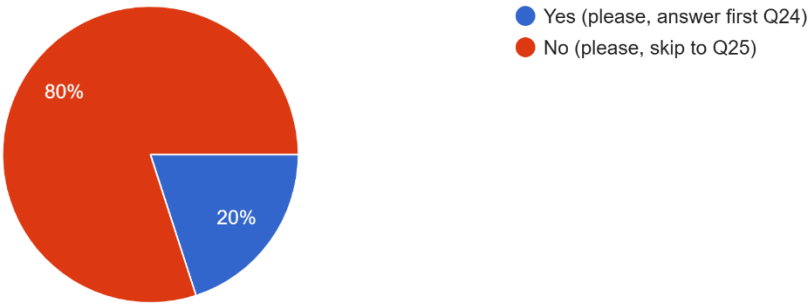
Q22 If you have answered “Yes” to Q21, please state what technical problem did you encounter while using the application (crashing, not opening the intended screen, or not responding) and at what circumstances it has happened.

0 responses

No responses yet for this question.

Q23 Do you have any suggestions to add/remove/improve functionalities of the application?

15 responses



Q24 If you have answered “Yes” to Q23, please state what is your suggestion to add/remove/improve functionalities of the application.

3 responses

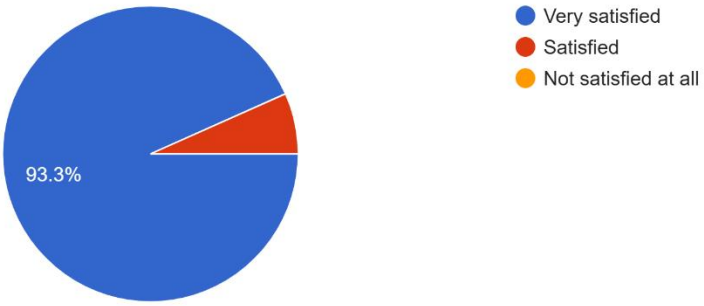
Implement maps to see volunteers nearby

Add a map to see volunteers nearby

Share the feedback with all users

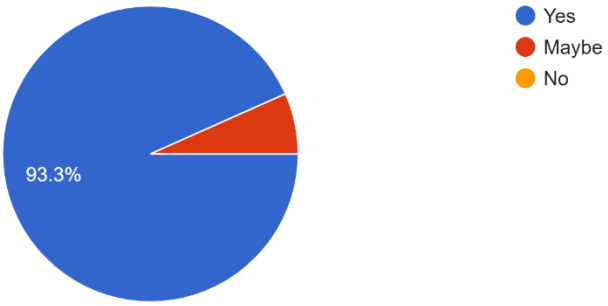
Q25 Overall, how satisfied are you with the application?

15 responses



Q26 Would you recommend this application to friends/colleagues to use?

15 responses



Appendix B: User Manual for Rise Together Cancer Support Mobile Application.

The objective of the User Manual for the Cancer Support mobile application is to provide clear and comprehensive guidance to all user types: Doctor, Volunteering Cancer Survivor, and Patient in Treatment, to effectively utilize the application. This document acts as a support resource, offering step-by-step instructions for account access, feature navigation, and key operational flows. The application is designed for Android users and can be accessed via installation from the provided distribution platform.

Main Application Home Screen:

Upon launching the application, users are directed to the Main *Home* Screen, which presents buttons to select the appropriate user type: Doctor, Volunteering Cancer Survivor, or Patient in Treatment. Upon selecting their role, users are redirected to the corresponding *Login* Screen. For first-time users:

- ✓ Doctors will receive credentials from the System Administrator.
- ✓ Volunteers and Patients will be registered by a Doctor, and receive initial password via a link in the email registered by the doctor to change their password, completing their profile during their first login.

Reset Password (All User Types):

If any user forgets their password, they can reset it by following the steps below:

1. Tap on the "*Forgotten Password? Reset Password Here*" link on the *Login* screen.
2. Enter the registered email address.
3. Tap the "*Send Reset Link*" button.

A *password reset* link will be emailed to the user. Follow the instructions in the email to set a new password and then Log into the app.

Doctor User Interface:

Doctors are responsible for registering new users into the system. Upon Login, the Doctor has access to the following functionalities:

1. Register New Patient/Volunteer by:
    - entering patient's/volunteer's full name, mobile number, gender, type of cancer, email address (username), and an initial password.
    - Then tap on "Send a link", which emails access link for new Patients/Volunteers
2. The patient/volunteer receives the access link via email to create a custom password and access their profile.

Logout

To end the session, tap the *Logout* button. The user is returned to the main *Home* Screen.

Volunteering Cancer Survivor User Interface:

After logging in, the Volunteer can manage their availability and respond to requests from Patients. The following functionalities are available:

- ✓ Create Availability Profile

- ✓ Access your profile to add Assistance Tasks.
- ✓ Select a Task Category and Task Name.
- ✓ Upload a profile photo, and provide:
  - o Number of available days per week
  - o Specific days and time slots (AM/PM)

View Requests from Patients:

- ✓ View a list of task requests, sent by Patients.
- ✓ Select a specific request to view its details.
- ✓ Accept or Reject the request, based on your availability, by sending a Notification.

Receive Reminder notification:

- ✓ Receive Reminders
  - o A reminder notification is sent one day before the scheduled task to ensure commitment.

View Feedback & Ratings:

- ✓ Volunteers can view ratings and feedback, provided by Patients, after completing assistance tasks.

Logout:

To log out of the application, tap the *Logout* button.

Patient in Treatment User Interface:

Upon successful login, the Patient has access to the following features:

- ✓ Navigate to your profile to add a new required assistance task.
  - o Select the Task Category and Task Name.
  - o Enter the number of days per week assistance is required.
- ✓ Send Assistance Task Request

View List of Matching Volunteers:

1. Based on gender and cancer type, the app filters and displays a list of suitable Volunteers.
2. Click on a Volunteer to view their name, photo, email, and mobile number.
3. Choose the desired time slot (AM/PM) and day of the week,
4. Then send your request for assistance.
5. Contact Volunteer.

Receive Notifications:

- ✓ Receive real-time Notifications on the Acceptance or Rejection of your request.
- ✓ If the request is accepted, a Reminder notification is sent one day prior to the scheduled assistance.

Send Feedback & Ratings:

- ✓ After the assistance is provided, you can submit feedback and ratings for the Volunteer who helped you.

Logout:
To end your session, tap the *Logout* button to return to the *Home* Screen.